

Decentralized Message Transfer & Access Protocol (DMTAP)

draft-paruk-dmtap-00

Abstract

DMTAP is an open protocol for sovereign, metadata-private communication — mail, chat, and files — and decentralized identity/login, over a peer-to-peer mesh, with an optional bridge to legacy email. It unifies message *transfer* (the role of SMTP) and message *access* (the role of IMAP) into one decentralized, encrypted protocol, and extends the same keypair identity into decentralized web login. A user's identity is a keypair; a human name (an 8-word key-derived name, an @handle, or a domain) is a replaceable pointer to it. Content, authenticity, and the social graph are protected: messages are end-to-end encrypted and signed, and who-talks-to-whom is hidden from a global passive observer. DMTAP composes existing standards (MLS, HPKE, JMAP, libp2p, DNS, key transparency, Privacy Pass, WebAuthn, OAuth, DID); the novelty is the composition and transport, not new cryptography.

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of an open, royalty-free specification. Internet-Drafts are working documents; this document is a Experimental proposal and is *not* an RFC — an RFC number is assigned only upon publication by the relevant standards body. It is valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use this document as reference material or to cite it other than as “work in progress.” This draft will expire on January 2027.

Requirements Language

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY**, and **OPTIONAL** in this document are to be interpreted as described in RFC 2119 and RFC 8174 when, and only when, they appear in all capitals.

Copyright Notice

Copyright © 2026 Imran Paruk and the DMTAP contributors. This specification is licensed under Creative Commons Attribution 4.0 International (CC BY 4.0); reference implementations are licensed separately under MIT.

Reference implementation: *Envoir* (MIT). Message object: *MOTE*.

Table of Contents

0. Overview & Architecture	8
0.1 Goals	8
0.2 The two components	8
0.3 The three layers of indirection	9
0.4 Message-flow summary	9
0.5 Where state lives	10
0.6 Privacy posture (summary; full model in §6)	10
0.7 Non-goals	10
1. Identity Lifecycle	10
1.1 Algorithm suites & crypto-agility	11
1.2 Keys and the identity hierarchy	11
1.3 The Identity object (published)	12
1.4 Recovery policy (and rotating the recovery methods)	12
1.5 Key rotation	15
1.6 Identity (name) migration — never losing your address	15
1.7 Object summary	16
2. The MOTE Object	16
2.1 Layered structure	16
2.2 Envelope (CBOR)	16
2.3 Message kinds	17
2.4 Payload (CBOR, encrypted)	18
2.5 Attachments and files	18
2.6 Delivery semantics	19
2.7 Validation (recipient MUST)	19
2.8 Why this shape	20
3. Naming & Directory (name → key)	20
3.1 Roles (who holds what)	21
3.2 DNS records	21
3.3 Resolution	21
3.4 Trust on first use (TOFU) + pinning	22
3.5 Key transparency (KT)	22
3.6 Optional: self-sovereign naming (un-loseable addresses)	23
3.7 Private lookups	23
3.8 Onboarding tiers (day-one setup)	23
3.9 Names (one identity, many pointers)	24
4. Transport: Mesh, Mixnet, Delivery	26
4.1 Substrate: libp2p	26
4.2 The key → location record (DHT)	26
4.3 Reachability ladder	27
4.4 The mixnet (metadata privacy)	27
4.5 Bulk / file transfer	28
4.6 Privacy tiers	28
4.7 Delivery state machine (sender)	28
4.8 Local, isolated, and delay-tolerant networks	28

5. Messaging & Files (the unified substrate)	29
5.1 Why MLS everywhere	29
5.2 Forward secrecy & (non-)deniability	30
5.3 Async session initiation (MLS-native)	31
5.4 Message kinds in context	31
5.5 Files: content-addressed, chunked, any size	31
5.6 Multi-device (the personal cluster)	32
5.7 Three products, shared components	32
5.8 Groups as addressable identities	32
6. Privacy & Threat Model	34
6.1 Adversary model	34
6.2 What is protected, and how	34
6.3 Mixnet, cover traffic, padding	35
6.4 Why we avoid PIR (honest framing)	35
6.5 Privacy tiers (and where each product sits)	35
6.6 Honest boundaries (what we do NOT claim)	36
6.7 Data at rest	37
7. The Legacy Gateway (optional)	37
7.1 Responsibilities	37
7.2 Inbound	37
7.3 Outbound & DKIM delegation	38
7.4 Statelessness & durability	38
7.5 Decentralization & economics (summary; anti-abuse in §9)	38
7.6 Dual-stack addressing	38
7.7 Fairness, self-host backstop & non-lock-in	39
8. Client Access	39
8.1 JMAP (native)	40
8.2 IMAP / POP / SMTP-submission (compatibility)	40
8.3 Multi-device	40
8.4 Calendar & contacts (first-class, decentralized)	40
8.5 The decentralization invariant (all data classes)	41
9. Anti-Abuse & Postage	41
9.1 Principles	41
9.2 Recipient policy	41
9.3 Anonymous rate-limit tokens (the core mechanism)	41
9.4 Proof-of-work (fallback)	42
9.5 Postage (economic tier + gateway funding)	42
9.6 Gateway-operator accountability	43
9.7 Vouch / introduction (optional)	43
9.8 Mixnet abuse	43
9.9 Group-address amplification	44
10. Versioning, Conformance, Governance	44
10.1 Protocol versioning	44
10.2 Capability negotiation (dual-stack)	44
10.3 Conformance levels	45
10.4 The spec is authoritative	45
10.5 Governance & licensing (intent)	45

10.6 Roadmap markers (non-normative)	45
11. Grounding & References	46
11.1 Verified building blocks	46
11.2 Corrections applied after verification	46
11.3 Consolidated honest limits	47
11.4 Implementation-stack notes (reference, Rust)	47
11.5 Selected bibliography	47
12. Operators, the Seam & Sustainability	48
12.1 Two deployment modes	48
12.2 The operator seam	48
12.3 The inviolable rule (normative)	49
12.4 Business & licensing model (informative)	49
12.5 Honest limits	49
13. DMTAP-Auth — Decentralized Identity & Login	49
13.1 Goals & non-goals	50
13.2 Substrate reuse	50
13.3 The native login ceremony	50
13.4 Sessions: key-bound, not bearer	52
13.5 Capabilities & delegation	52
13.6 Legacy bridge: OIDC / OAuth compatibility	53
13.7 Grounding & honest limits	54
14. Scaling & Deployment	55
14.1 Node classes	55
14.2 Horizontally-scalable gateways (normative patterns)	55
14.3 The mobile-only user (no always-on box)	56
14.4 The always-on-box user	56
14.5 Relay & buffer scaling	56
14.6 Hosted multi-tenant topology (the operator)	57
14.7 Grounding	57
15. Normative & Informative References	58
15.1 Normative — cryptography & encoding	58
15.2 Normative — transport, mesh & naming	58
15.3 Normative — legacy mail interop (the gateway & client edges)	58
15.4 Normative — identity, auth & anti-abuse	59
15.5 Informative — precedent & rationale	59
15.6 On reproduction	59
16. Numeric Parameters (v0)	60
16.1 Time, replay & clocks	60
16.2 Naming, KT & DHT	60
16.3 Mixnet & privacy	60
16.4 File tiers & transfer	60
16.5 Anti-abuse	61
16.6 Transport & relay	61
16.7 Crypto suites	61
16.8 Auth, sessions & group ordering	61

17. Feature-Parity Audit — Classic Mail/Calendar/Contacts vs DMTAP	62
17.0 Method	62
17.1 Mail	62
17.2 Contacts	70
17.3 Calendar	72
17.4 Cross-cutting	75
17.5 Summary table	76
17.6 Gaps to resolve	78
17.7 Net assessment	78
18. Appendix A: Wire Format (CDDL)	79
18.1 Conventions	79
18.2 Length & type governance by suite (normative)	81
18.3 Message-layer objects (§2)	82
18.4 Identity-layer objects (§1)	86
18.5 Transport-layer object (§4)	90
18.6 Group-layer objects (§5.8)	91
18.7 Auth-layer objects (§13)	92
18.9 Canonical signing & hashing preimages (normative)	93
18.10 Collected CDDL grammar (copy-paste block)	96
18.11 Source inconsistencies flagged (not silently diverged)	99
19. Appendix B: Protocol Operations	100
19.0 Conventions used in this appendix	100
19.1 Naming operations (§3)	101
19.2 Reachability operations (§4)	105
19.3 Delivery operations (§2.6, §2.7, §2.7a)	108
19.4 Async session initiation (§5.3)	112
19.5 Group operations (§5.1, §5.8)	116
19.6 DMTAP-Auth operations (§13)	123
19.7 Gateway operations (§7)	129
19.8 Files operations (§2.5, §5.5, §16.4)	131
19.9 JMAP method mapping (§8.1)	133
19.10 Gaps flagged (undefined behavior surfaced, not left implicit)	134
19.11 Operation count	135
20. Appendix C: State Machines	135
20.0 Conventions	135
20.1 Outbound MOTE delivery (sender side)	136
20.2 Inbound MOTE validation (recipient side)	138
20.3 Name resolution & pinning	140
20.4 Reachability establishment	142
20.5 MLS group / committer	143
20.6 DMTAP-Auth session	145
20.7 Node lifecycle / offline buffering	147
20.8 Composition summary (non-normative)	149
20.9 Honest gaps & normative fills (meta)	149
21. Appendix D: Error Codes & IANA Considerations	150

Part 1 — Error / Status Registry	150
21.1 Conventions and code format	150
21.2 Responder action vocabulary (normative)	151
21.3 Identity, Naming & Key Transparency errors (<code>0x01xx</code>)	151
21.4 Delivery & Validation — the MOTE object (<code>0x02xx</code>)	152
21.5 Transport & Reachability errors (<code>0x03xx</code>)	152
21.6 Messaging & Group errors — MLS (<code>0x04xx</code>)	152
21.7 Auth errors — DMTAP-Auth (<code>0x05xx</code>)	153
21.8 Gateway errors (<code>0x06xx</code>)	153
21.9 SMTP response-code mapping (inbound, informative reference to RFC 5321 / RFC 3463)	154
21.10 Anti-Abuse & Postage errors (<code>0x07xx</code>)	154
21.11 Files errors (<code>0x08xx</code>)	154
21.12 Traceability matrix	155
Part 2 — IANA Considerations	155
21.13 Overview and policy vocabulary	155
21.14 DMTAP Error/Status Code Registry	155
21.15 Algorithm Suites Registry (<code>suite</code> <code>u8</code>)	156
21.16 Message Kinds Registry (<code>kind</code> <code>u8</code>)	156
21.17 Challenge Types Registry	156
21.18 Name Backends Registry	156
21.19 KT Log Identification Registry	157
21.20 Envelope/Payload Extension Header Keys Registry (<code>Headers.ext</code>)	157
21.21 DNS Parameter Keys Registry (<code>_dmtap</code> / <code>_dmtap-gw</code>)	157
21.22 Capability Tokens Registry (negotiation, §10.2)	157
21.23 Extension & versioning procedure (normative)	157
21.24 Summary	158

0. Overview & Architecture

0.1 Goals

DMTAP is a protocol for authenticated, encrypted, metadata-private messaging between self-sovereign identities, with async (store-and-forward) delivery, no required central party, and an optional bridge to legacy email. One substrate carries **mail**, **chat**, **files**, and **decentralized identity/login** — the same keypair that receives your mail logs you in across the web (§13), with no central identity provider.

Concretely, DMTAP MUST provide:

1. **Sovereign identity** — a keypair you own; no account with any provider is required to be a DMTAP identity. The same identity serves mail, messaging, files, and web login (§13).
2. **Reachability without a static IP** — a node behind CGNAT, on a dynamic IP, is reachable by its key.
3. **Content, authenticity, and metadata privacy** — messages are end-to-end encrypted and signed, and the social graph (who talks to whom, when, how much) is hidden from a global passive observer.
4. **Continuity** — you never lose access to your identity (redundant, rotatable recovery) and can migrate your human name without losing existing contacts.
5. **Legacy interoperability** — you can exchange mail with the existing SMTP world, and existing OIDC apps can consume DMTAP login through a bridge (§13.6).
6. **Scales across device classes** — always-on nodes (Pi/NAS/VPS) hold the mailbox; intermittent devices (laptops, phones) participate as thin clients or push-woken nodes; gateways scale horizontally (§14).
7. **Future-proofing** — crypto-agility, transport independence, standards reuse; the system gets *simpler* as IPv6 spreads and legacy fades.

0.2 The two components

DMTAP is, deliberately, only two pieces of software plus DNS (which we do not build):

The Node (`node/`)

One binary, installed on any box that runs most of the time. It holds **all durable state** and does **all the real work**:

- Identity: the root keypair, device subkeys, recovery policy (§1).
- Store: the mailbox and file blobs (encrypted MOTEs + content-addressed chunks) (§2, §5).
- Mesh participation: peer discovery (DHT), relaying for others, delivery (§4).
- Mixnet client: onion-wrapping, cover traffic, sealed sender (§4, §6).
- Messaging: MLS groups for 1:1, chat, and file folders; MLS KeyPackages (§5.3).
- Client access: JMAP native, plus IMAP/POP/SMTP-submission compatibility (§8).
- The outbound **retry queue** — durability lives here, not in the middle.

A node MAY additionally run in **relay mode** (help NAT'd peers, if it has a public address) or **mix mode** (be a mixnet hop). These are capabilities of the same binary, not separate programs.

The Gateway (`gateway/`) — optional

The **only** component that speaks SMTP and the **only** one that is not content-blind (the legacy leg is unavoidably plaintext). It:

- receives inbound legacy mail (acts as MX), wraps it into a MOTE, attests it, and delivers into the mesh; returns SMTP 4xx if the recipient is offline so the *sending* server retries;
- sends outbound legacy mail, DKIM-signing as the user's domain via delegated selectors;
- carries the operational weight the system cannot avoid: **IP reputation**.

A node without legacy correspondents never invokes a gateway. At full adoption, the gateway is unnecessary. The gateway MAY be the node binary run in `--gateway` mode by an operator with a reputable IP and a domain.

DNS (not built here)

The naming substrate that maps a human name to a key. We publish and read records; we do not run DNS. See §3. DNS holds the **stable** binding (name → key); the mesh holds the **dynamic** binding (key → current location).

0.3 The three layers of indirection

The core trick that frees an address from any IP:

abc@def.com	—DNS/§3→	public key	—mesh/§4→	current location
NAME		IDENTITY		(IP/relay/mix path)
(human, stable)		(durable, portable)		(ephemeral, self-updated)

- **Name** → **key** is stable and lives in DNS (+ key transparency). It changes only when you migrate names.
- **Key** → **location** is dynamic and lives in the mesh (a signed, TTL'd DHT record the node republishes as its address changes).
- The **key is the identity**. Existing contacts route by key via the mesh and never need DNS again after first contact; a lost domain is a change of *name*, not of identity (§1.6).

0.4 Message-flow summary

DMTAP → DMTAP (the common path)

1. resolve abc@def.com → recipient key K (§3; cached/pinned after first contact)
2. fetch K's KeyPackages + current location (§4 DHT, §5.3 KeyPackages)
3. build a MOTE: sealed-sender, MLS/HPKE-encrypted to K, signed (§2, §5, §6)
4. send through the mixnet (private tier) or direct (fast tier) (§4, §6)
5. recipient node receives, verifies, decrypts, stores; acks (§2, §4)
(sender's node retries until ack – durability at the edge)

No gateway, no SMTP, no plaintext outside the endpoints.

Legacy → DMTAP (inbound)

gmail —SMTP→ Gateway —wrap+attest+encrypt to K→ mesh → recipient node
(if node offline: return SMTP 4xx; Gmail retries)

DMTAP → Legacy (outbound)

node —MOTE(legacy)→ Gateway —SMTP + delegated DKIM→ gmail
(node retries on failure)

0.5 Where state lives

State	Location	Notes
Keys, mailbox, files, retry queue	Node (the edge)	All durable state
Name → key	DNS + key-transparency log	Stable; small
Key → location	Mesh DHT	Dynamic; signed; TTL'd; self-republished
In-flight ciphertext	Mixnet / relay	Held only until delivered; content-blind
Legacy reputation	Gateway	The only non-trivial operational cost

The middle (mesh, mixnet, gateway) holds **no durable user data**. Durability is always punted to an edge: the sender's node retries; inbound legacy leans on the sending server's SMTP retry.

0.6 Privacy posture (summary; full model in §6)

- **Content & authenticity:** end-to-end encrypted (MLS/HPKE) and signed. Always.
- **Sender metadata:** hidden via **sealed sender** — intermediaries never learn the sender.
- **Social graph & timing:** hidden via a **mixnet** (onion routing + mixing delays) plus **cover traffic** and **size padding**. Email's asynchrony is what makes full-strength mixing affordable.
- **Recipient retrieval:** the **always-on node receives by push** through the mixnet, so there is *no store-and-poll step to hide* — the hardest metadata problem is dissolved by architecture, not by expensive PIR.
- **Discovery:** name→key lookups are routed *through* the mixnet, so the directory does not learn who is looking up whom.
- **Privacy tiers:** messages may choose **private** (full mixnet, minutes of latency) or **fast** (direct/low-hop, seconds, less metadata protection). Default is **private**; bulk file transfer uses **fast** for the payload and **private** for its control message.

Honest boundary: DMTAP targets a **global passive adversary**. Perfect resistance to a global *active* adversary with unlimited resources is not claimed; see §6.

0.7 Non-goals

- Real-time voice/video (separate WebRTC/SFU architecture).
- Blockchain/consensus (except optional self-sovereign naming in §3).
- Server-side search or server-side spam ML (search is on-device; anti-abuse is §9).

1. Identity Lifecycle

Identity is the foundation everything else hangs off. The governing principle:

Your key is your identity. Your domain, your IP, and your provider are all replaceable pointers to it.

This section defines keys, the key hierarchy, the recovery policy (and how recovery methods are themselves rotated), key rotation, and name/identity migration. All of these are **signed, versioned objects** published to the key-transparency log (§3), so every change is authenticated and every unauthorized change is *detectable*.

1.1 Algorithm suites & crypto-agility

Every signed/encrypted object carries a `suite` identifier. Implementations **MUST** reject unknown suites (fail closed), never guess.

suite	Sign	KEM / PKE	AEAD	Status
<code>0x01</code>	Ed25519	X25519 (HPKE)	ChaCha20-Poly1305	v0 REQUIRED
<code>0x02</code>	Ed25519 + ML-DSA-65	X25519 + ML-KEM-768 (hybrid)	ChaCha20-Poly1305	RESERVED (P

Suite `0x02` is the post-quantum migration target. A node **MAY** hold keys in multiple suites during migration; the transparency log records which suite is current. Downgrade is prevented by pinning (§3) and by the transparency log's monotonic history.

1.2 Keys and the identity hierarchy

An identity is rooted in a **root identity key** (`IK`), an offline-capable long-term signing keypair. `IK` signs everything authoritative but is used rarely; it **SHOULD** be held in cold form / recovery custody (§1.4).

```

IK (root identity key, Ed25519[+ML-DSA])      ← the identity; rarely used
├── signs ↓
├── DeviceKey_1 (per-device signing subkey)    ← day-to-day signing/auth
├── DeviceKey_2
├── KeyPackages (MLS: identity/signature + HPKE-init + PQ-KEM keys) ← async join (§5.3)
└── RecoveryPolicy vN (§1.4)                  ← how the identity is recovered

```

- **Device keys** authorize a specific device (phone, laptop, the always-on box). Each is a signing subkey signed by `IK` (a `DeviceCert`), with a `label`, `created`, and optional `expires`. Devices form the owner's **personal cluster** and share the mailbox via the encrypted CRDT sync of §5.
- **KeyPackages** (§5.3) let others start an encrypted MLS session with the identity while every device is offline. `KeyPackages` are signed by a device key; one-time `KeyPackages` are consumed per session.
- A DMTAP **address key** presented to correspondents is `IK`'s public half (the stable identity). Correspondents pin it on first use (§3).

DeviceCert (CBOR)

```

DeviceCert {
  suite:      u8,
  ik:         bytes,      // root identity public key
  device_key: bytes,      // device signing public key
  label:      tstr,       // "phone", "home-box", ...
  created:    u64,        // ms epoch
  expires:    ?u64,
  caps:       [+ tstr],   // "send","recv","relay","mix","gateway","admin"
  sig:        bytes,     // IK over the CBOR-encoded fields above
}

```

`caps` gates what a device *may participate in*. **No single device — including an admin device — may unilaterally change the recovery policy.** Changing `RecoveryPolicy` (§1.4) always requires either `IK` directly or satisfaction of the current `rotate_threshold` quorum; an

`admin` device counts only as *one factor* toward that quorum (it may, e.g., be a required member of the quorum, but never sufficient alone). This prevents a single stolen `admin` device from locking the owner out by rewriting recovery. See §1.4 for the authoritative signer rule.

1.3 The `Identity` object (published)

The current public identity is a signed, versioned object; its hash is the anchor everyone pins.

```
Identity {
  suites:  [+ u8],           // algorithm suites this identity supports, preference-ordered
  iks:     { u8 => bytes }, // identity public key per suite (Ed25519 for 0x01, +ML-DSA for 0x02)
  version: u64,             // monotonically increasing
  devices: [* DeviceCert],
  keypkgs: KeyPackageBundleRef, // location + hash of the current KeyPackage bundle (§5.3)
  recovery: RecoveryPolicyRef,  // hash of the current RecoveryPolicy (§1.4)
  names:    [* tstr],          // self-asserted name(s); trust only after forward name→ik veri
  prev:     ?bytes,           // hash of the previous Identity version (hash chain)
  ts:       u64,
  sig:      [+ bytes],        // one signature per suite in `suites`, over all of the above
}
```

Multi-suite support & PQ transition (normative). `suites` is a *set*, preference-ordered, so an identity can hold classical and PQ keys simultaneously during migration. Rules:

- A sender **MUST** use the **highest suite both parties support** (intersection of the sender's supported suites and the recipient's `Identity.suites`); if the intersection is empty, delivery fails closed (no silent downgrade).
- During transition the `Identity` **MUST** carry a signature under **every** suite in `suites` (`sig` is a list), so a verifier trusting either the classical or the PQ key can validate the object — this is what lets the network migrate without a flag day.
- Per-message suite is negotiated at **KeyPackage granularity** (§5.3): a `KeyPackage` advertises its suite, and the sender selects a recipient `KeyPackage` of the chosen suite. `Envelope.suite` records the suite actually used.
- A verifier **MUST** reject an `Identity` whose highest offered suite it cannot validate rather than fall back silently.

`prev` chains versions into a tamper-evident history mirrored in the transparency log (§3). A verifier accepts an `Identity` only if every `sig` validates under the corresponding key in `iks` and the chain is consistent with what it has previously pinned/seen.

1.4 Recovery policy (and rotating the recovery methods)

Recovery is **not** baked in at setup; it is a first-class, versioned, signed object so the recovery methods themselves can be rotated.

```
RecoveryPolicy {
  suite:    u8,
  ik:       bytes,
  version:  u64,
  methods:  [+ RecoveryMethod],
  recover_threshold: Threshold, // what regains access
  rotate_threshold: Threshold, // higher bar to change THIS policy
  prev:     ?bytes,            // hash chain
}
```

```

    ts:      u64,
    sig:      bytes,          // by IK, OR satisfied rotate_threshold quorum (reactive)
}

```

```
RecoveryMethod = PhraseMethod / DeviceMethod / SocialMethod
```

```
PhraseMethod { type:"phrase", recovery_key: bytes }           // key derived from a BIP39-style
```

```
DeviceMethod { type:"device", device_key: bytes, label: tstr }
```

```
SocialMethod { type:"social", guardians: [+ bytes], threshold: u8 } // Shamir shares
```

```
Threshold = { any_of: [+ MethodPredicate] } // e.g. 1 phrase OR 2 devices OR 2 guardians
```

A **MethodPredicate** names a satisfied factor: **Phrase** (the phrase-derived key), **Devices(n)** (any n device factors), **Guardians(n)** (any n social shares), or **Ik** (the root key itself). A **Threshold** 's **any_of** is met when **any** listed predicate is satisfied.

Rules:

1. **Authenticated.** Every version is signed by **IK** (proactive) or by satisfying the current **rotate_threshold** (reactive, mid-recovery). Unauthenticated policy changes are invalid.
2. **rotate_threshold** $\geq$ **recover_threshold**. A single compromised factor may (perhaps) recover access but MUST NOT be able to unilaterally rewrite the policy and lock the owner out.
3. **Weakening changes need the quorum, not IK alone (compromise defense).** A policy change that **removes or weakens any recovery factor** (drops a method, lowers a threshold, evicts a guardian/device) MUST satisfy **rotate_threshold** **even when signed by IK** — **IK** alone MUST NOT be sufficient to weaken recovery. This closes the *stolen- IK* takeover where an attacker proactively rewrites recovery to evict the owner and install their own factors. **Additive, non-weakening** changes (adding a redundant factor) MAY be signed by **IK** alone.
4. **Asymmetric veto window on weakening changes.** A recovery-weakening change MUST be published to the transparency log and take effect only **after a veto/delay window** (§16), during which the owner's monitoring devices (§3.5) can detect it and publish a **counter-signed veto/abort**. **The veto is deliberately asymmetric** to avoid a deadlock in which a compromised factor vetoes its *own* eviction:
 - A veto MUST itself satisfy the **rotate_threshold** quorum — a *single* prior factor (e.g. the very factor being removed) CANNOT veto. This ensures a stolen single factor cannot block its own removal.
 - A recovery change that itself satisfies **rotate_threshold** (a genuine quorum-backed recovery, §1.4 reactive path) **overrides any veto** and is NOT blockable — so legitimate recovery from compromise always wins over an attacker holding one not-yet-removed factor.

Because rule 3 already requires *every* weakening change to satisfy **rotate_threshold**, a rule-conforming weakening is inherently quorum-backed and thus veto-proof; the veto window is therefore **defense-in-depth** — it exists to detect and block a *non-conforming* (lesser-bar) weakening that slipped through a buggy/malicious implementation, not to gate normal recovery. A weakening change MUST NOT take effect instantly; non-weakening changes are not delayed.

5. **Real revocation.** Rotating a method out MUST re-key the underlying secret, not merely edit a list:

- rotate **phrase** → re-wrap the identity secret under the new phrase-derived key; old phrase becomes useless;
- change **guardians** / **threshold** → **redistribution** / **resharing** (a fresh access structure, Desmedt–Jajodia style), not merely proactive refresh; old shares fall below threshold and cannot reconstruct;
- remove **device** → rotate any identity/recovery material it held.

Recommended primitives (grounded):

- Use **Verifiable Secret Sharing** (Feldman or Pedersen VSS), **not** plain Shamir, so guardians can detect a corrupt share or a cheating dealer at reconstruction (hostile-guardian threat model). Plain Shamir has no cheating detection.
 - **Proactive Secret Sharing** refreshes shares under the *same* (M,N); **redistribution** changes the access structure (add/remove guardians, change threshold). Use the correct one per operation.
 - Prefer **SLIP-0039** for the mnemonic Shamir encoding (purpose-built: two-level groups, checksums, passphrase) over hand-rolling BIP39 + Shamir.
 - **Strongly consider FROST (RFC 9591)** threshold Ed25519 signatures so guardians *authorize* recovery **without ever reassembling the secret key in one place** — eliminating the single-point-of-compromise moment that Shamir reconstruction creates. This is the preferred design when the recovery secret is a signing key.
6. **Logged & detectable.** Every version is published to the transparency log; the owner's other devices monitor it and **MUST** alert on a change they did not initiate (intrusion detection, §3.5).

Recovery flows

- **Proactive rotation** (owner has access): publish `version+1` signed by `IK`. Routine hygiene; do it whenever a factor is suspected compromised.
- **Reactive recovery** (owner lost access): satisfy `recover_threshold` to reconstruct `IK` (or a fresh `IK` authorized by the old under `rotate_threshold`), then immediately publish a new policy that invalidates the compromised factor.

The bottom turtle

There is a foundational anchor (root phrase + social quorum). It is rotatable but requires the strongest `rotate_threshold`. Two worst cases bound it:

- **Loss.** Losing `IK` **and** enough recovery factors simultaneously is unrecoverable.
- **Compromise (the sharper case).** An attacker who obtains `IK` **plus** enough factors to satisfy `rotate_threshold` can rewrite recovery and effect an **owner-unrecoverable takeover** — *worse* than loss, because the owner is actively **evicted**, not merely locked out.

Both are mitigated by the same discipline: multiple independent, redundant, rotatable factors so no single loss is fatal; the weakening-quorum + veto-window rules above so a *partial* compromise cannot silently escalate to takeover; and KT monitoring (§3.5) so any takeover attempt is at least detectable and vetoable within the window.

Backup & restore (content continuity)

Recovery restores the **key**, not the **mailbox**: reconstructing `IK` onto a fresh device yields an authoritative identity over an **empty store**. Message/file history returns only from a surviving

cluster device (§5.6) or from a **portable encrypted backup**. DMTAP therefore defines a normative backup primitive: an owner's node **MUST** be able to export an **encrypted, integrity-protected backup** of the mailbox/CRDT state (§5.6) and file blobs (§5.5), sealed under a key derived from the recovery phrase (or a separate backup key held in `RecoveryPolicy`) and restorable onto an empty store after key recovery. Without a surviving device or such a backup, recovery preserves the **identity and all relationships** (§1.6) but **not prior content**. See §14.5 for how this differs from a relay/peer buffer (a buffer is not a backup).

1.5 Key rotation

To rotate `IK` (compromise, or scheduled PQ migration):

1. Generate `IK'`.
2. Publish an `Identity` version whose `sig` is by the **old** `IK` and whose body authorizes `IK'` (a cross-signed `KeyRotation` record: `old_ik` signs `new_ik` + `reason` + `ts`).
3. From then on `IK'` is authoritative; the transparency log's chain proves continuity.
4. Re-issue device certs and `KeyPackages` under `IK'`. Correspondents update the pin by following the signed chain (§3.4).

A verifier **MUST** accept `IK'` only via a valid chain from a previously-pinned `IK`, or via an explicit out-of-band re-verification.

1.6 Identity (name) migration — never losing your address

Because **key is identity**, losing a domain is a change of *name*, not of identity.

A `MoveRecord` rebinds the human name while preserving the key:

```
MoveRecord {
  suite:  u8,
  ik:     bytes,
  from:   tstr,           // "abc@old.com"
  to:     tstr,           // "abc@new.com" (or a self-sovereign name, §3.6)
  ts:     u64,
  prev:   ?bytes,
  sig:    bytes,          // by IK
}
```

Distribution (all three):

1. **Mesh/DHT** — the key→location record and `Identity.names` carry the new canonical name.
2. **Transparency log** — an auditable, ordered record of the move (and of the key discontinuity at the old name, defeating a squatter who later registers `old.com`).
3. **Push to contacts** — a signed MOTE announcing the move.

Because existing contacts **route by key via the mesh** and verify the `MoveRecord` against `IK`, they follow the owner automatically and cannot be redirected by a forged move. Only *new* contacts who know only the abandoned name are affected — the same, unavoidable tradeoff as giving up any domain, but the *identity and all existing relationships survive*.

Optional maximal durability: to make the literal address itself un-loseable and un-seizable, bind the name in a self-sovereign name backend instead of (or in addition to) DNS (§3.6). This is the only place DMTAP admits a name-chain, and it is confined to the name layer.

1.7 Object summary

Object	Signed by	Chained	In transparency log
Identity	IK	yes (prev)	yes
DeviceCert	IK	via Identity	via Identity
RecoveryPolicy	IK or rotate-quorum	yes	yes
KeyRotation	old IK $\rightarrow$ new IK	via Identity chain	yes
MoveRecord	IK	yes	yes

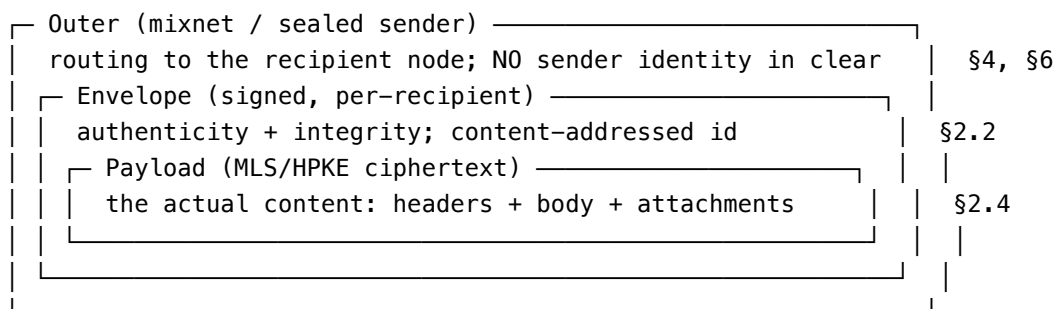
All identity-lifecycle operations share one machinery: **signed, versioned objects**, **threshold-gated changes**, and the **transparency log** as the audit trail (§3).

2. The MOTE Object

A **MOTE** (the atomic unit of DMTP) is a signed, encrypted, content-addressed message object. Mail, chat messages, file-share announcements, group events, and identity announcements are all MOTEs — one format, rendered differently by clients (§5, §8).

2.1 Layered structure

A MOTE has three nested layers, each serving a distinct purpose:



The **outer** layer is what mix nodes and relays see: an onion-wrapped, constant-length (padded) packet with no clear-text sender (sealed sender, §6). The **envelope** provides authenticity to the recipient. The **payload** is the end-to-end-encrypted content.

2.2 Envelope (CBOR)

```
Envelope {
  v:      u8,          // format version (0)
  suite:   u8,          // algorithm suite (§1.1)
  id:      bytes,       // content address of `ciphertext` (§2.2, hash-agile)
  to:      DeliveryTag, // routing target: recipient key, group id, or blinded tag (§2.2a)
  epoch:   ?bytes,      // MLS epoch / group context ref, if group (§5)
  ts:      u64,         // sender timestamp (ms epoch)
  kind:    u8,          // message kind (§2.3)
  keypkg:  ?KeyPackageRef, // present iff this initiates an MLS session (async join, §5.3)
  challenge: ?ChallengeResponse, // anti-abuse proof for cold senders (§2.2b, §9) –
                                   // verifiable WITHOUT decrypting `ciphertext`
  ciphertext: bytes,     // MLS or HPKE sealed Payload (§2.4)
  sender sig: bytes,     // detached sig by an EPHEMERAL per-message key over
```

```
// (id||to||ts||kind||challenge); gates abuse, reveals no identity
}
```

- **id** — content address = a 1-byte **hash-algorithm prefix** (multihash-style, for agility) followed by the digest. v0 default is **BLAKE3-256** (128-bit collision resistance, equal to SHA-256; fast Merkle/XOF structure ideal for chunking). BLAKE3 is cryptographically sound but **not FIPS/IETF-standardized** (the IETF draft expired); the agility prefix lets an implementation migrate to SHA-256/SHA-3 where compliance requires it without changing the address format. Pin the exact BLAKE3 mode + 256-bit output. Content addressing gives deduplication, integrity, and cacheability for free; identical ciphertext shares an **id**.
- **Signature placement.** For sealed-sender messages the *authenticating* signature and the sender's identity live **inside** the encrypted payload (`Payload.from` , `Payload.sig`), so intermediaries never see who signed. `sender_sig` in the envelope, when present, is a detached signature by an *ephemeral* per-message key (unlinkable) used only for spam/abuse gating at the recipient (§9); it does not reveal identity.
- **epoch** ties a message to an MLS group epoch so the recipient selects the right key.

2.2a Delivery tag (`to`) and recipient blinding

`to` is a **DeliveryTag** , one of:

- the recipient's **identity key** (default, simplest); or
- a **group id** (for MLS group messages, §5); or
- a **blinded delivery tag** — a per-contact value `BT = HKDF(shared_secret, epoch_day)` derived from a secret established at first contact, which the recipient's node recognizes but which is **unlinkable across time and across observers** to the recipient's persistent key.

Blinded tags are RECOMMENDED for the `private` tier. **Honest limit (reconciled with §6.4):** even a blinded tag does not hide *that a packet was delivered to a particular node* from the final mix / an observer of the recipient's link — an always-on node has a stable network presence (§6.4). Blinding removes the *persistent-key* linkage in the envelope; it does not remove last-hop delivery observability, which §6.4 addresses with node/identity decoupling and recipient-side cover traffic. Implementations **MUST NOT** present blinded tags as full recipient anonymity.

2.2b Anti-abuse challenge (`challenge`)

`challenge` is an optional `ChallengeResponse` carrying the sender's proof for cold contact (§9): an **ARC token**, a **proof-of-work solution**, a **postage stamp**, or a **vouch**. It is placed in the *envelope* (not the payload) so the recipient can evaluate abuse policy **without decrypting** — see the validation order in §2.7. Known contacts omit it (their MOTEs are accepted on the fast path). See §9 for the grammar, issuer-trust rules, and each proof type.

2.3 Message kinds

0x00	mail	long-form message (email semantics)
0x01	chat	short message (chat semantics)
0x02	reaction	emoji/ack on a referenced MOTE
0x03	edit	supersede a referenced MOTE
0x04	redact	request deletion of a referenced MOTE
0x05	file_offer	manifest + key for a content-addressed file (§5.5)
0x06	group_event	MLS handshake / membership change (§5.3)
0x07	receipt	delivery/read receipt (opt-in; §6)

0x08 presence ephemeral presence/typing (opt-in, off by default; §6)
 0x09 identity Identity/Move/RecoveryPolicy announcement (§1)
 0x0a system protocol control (capability negotiation, §10)
 0x40–0x7f reserved for extensions (§10)

Kinds `mail` and `chat` differ only in default client rendering and default privacy tier (§6): `mail` defaults to the `private` tier, `chat` MAY use the `fast` tier when both parties are online. Both are the same object over the same transport.

2.4 Payload (CBOR, encrypted)

The plaintext that is sealed into `Envelope.ciphertext`:

```
Payload {
  from:      bytes,           // sender identity key (IK) – revealed only to the recipient
  sig:       bytes,           // IK (or device key) over the canonical payload hash
  headers:   Headers,
  body:      Body,
  refs:      [* bytes],       // ids of MOTES this replies to / references (threading)
  attach:    [* Attachment],  // §2.5
  expires:   ?u64,            // requested expiry (client-enforced deletion)
  fs_ratchet: ?bytes,         // forward-secrecy ratchet material (§5.2)
}

Headers {
  thread:    ?bytes,          // stable thread/conversation id
  subject:   ?tstr,           // mail only
  mime:      ?tstr,           // content type of `body`
  cc:        [* bytes],       // additional recipient keys (fan-out is per-recipient MOTES)
  ext:       { * tstr => any }, // extension headers (§10)
}

Body = tstr / bytes           // text or opaque MIME
```

Only the recipient (or group members) can decrypt `Payload`, so **all sender identity, subject, recipients, threading, and content are hidden from the network** — this is what sealed sender + payload encryption buys.

2.5 Attachments and files

Small attachments MAY be inlined into `Payload.attach`. Larger files MUST be referenced by a content-addressed **manifest** and transferred out-of-band (direct/fast tier, §4.5), which is what makes DMTAP a file-share of arbitrary size (no protocol cap).

```
Attachment {
  name:      tstr,
  mime:      tstr,
  size:      u64,
  inline:    ?bytes,          // present iff small
  manifest:  ?ManifestRef,    // present iff large (§5.5)
  key:       bytes,           // per-file content key (recipient decrypts chunks)
}

ManifestRef { id: bytes, size: u64, chunks: u32 } // BLAKE3 Merkle-DAG root (§5.5)
```

The manifest lists chunk hashes (a BLAKE3 Merkle DAG). Chunks are fixed-size, individually encrypted and content-addressed, enabling **resumable, parallel, swarmed, deduplicated** transfer. Only the manifest + **key** travel in the (private) MOTE; the bulk chunks travel direct (§4.5). See §5.5 for the full file model.

Size tiers (normative threshold, reconciles §2.5 / §4.5 / §6.5). A file is handled by one of three paths, by size:

Tier	Size	Path
inline	≤ 1 padded packet (v0: ≤ 64 KiB after padding)	in <code>Attachment.inline</code> , inside the MOTE
normal	$>$ inline, ≤ 4 chunks (v0: ≤ 4 MiB)	manifest in MOTE; chunks also routed via t
large	$>$ normal	manifest in MOTE; chunks via the fast/only

The v0 numeric thresholds (64 KiB / 4 MiB) are parameters (§ numeric appendix) and MAY be tuned; the three-tier model is normative. This removes the earlier binary small/large ambiguity.

2.6 Delivery semantics

`deliver(outer_mote)` → recipient node receives, unwraps outer, verifies envelope, decrypts payload, stores, and returns:

`ack(id)` → recipient confirms receipt of MOTE `id`.

- **ack(id) transport.** An `ack` is a **small signed control MOTE** (kind `system`, §2.3) — or, on a live direct/ `fast` connection, a transport-level ack over the same channel — carrying `id` and signed by the recipient's device key, so the sender's retry queue (§4.7) can authenticate the confirmation. Acks are not themselves acked (no ack storm).
- **Durability = the sender's node retries** until `ack`, with exponential backoff and an `expires`-bounded deadline. The mixnet/relay holds nothing durably.
- **Deduplication.** A recipient that already holds `id` acks immediately without re-processing.
- **Ordering.** MOTES are not globally ordered; `ts` + `refs` + (for groups) `MLS epoch` provide causal/threading order. Clients **MUST** tolerate out-of-order and duplicate delivery.

2.7 Validation (recipient MUST)

Validation is ordered **cheapest-and-anonymous first**, so a flood of cold junk is rejected *before* any expensive asymmetric decryption (a decryption-DoS defense). On receipt a node **MUST**, in order:

1. Reject unknown `v` / `suite` (fail closed).
2. Verify `id` matches the content address of `ciphertext` (§2.2); drop on mismatch.
3. Verify `sender_sig` over (`id||to||ts||kind||challenge`) under its ephemeral key (cheap; no decryption). Drop on failure.
4. **Resolve** `to` to this node (or a group it belongs to). If `to` does not resolve, drop.
5. **Classify the sender** by `to` / pinning state: a **known contact** (fast path) vs an **unknown/cold sender**.
6. **For cold senders, enforce anti-abuse policy (§9) NOW, before decryption**, using the `challenge` field (ARC token / PoW / stamp / vouch) — all checkable without decrypting. Reject or defer per §9.2 / §2.7a if the challenge is absent or insufficient.
7. Decrypt `ciphertext` (MLS epoch key or HPKE to recipient key); drop on failure.
8. Verify `Payload.sig` under `Payload.from`; **on failure, discard silently and do not ack** (fail closed, matching steps 1–3). Otherwise verify `from` matches the pinned identity for a

known contact, or TOFU-pin on first contact (§3.4). For a cold sender whose `from` is now revealed, re-apply block/allow lists.

9. Apply `expires`, `refs`, `kind` semantics; store; `ack`.

Known contacts MAY skip step 6 (they are pre-authorized). Only known-contact MOTES reach decryption on the fast path; unknown senders must pass the anonymous abuse gate first.

2.7a Outcome of a failed/absent challenge (normative)

To reconcile §2.7 and §9.2, the disposition is by *degree*:

- **Invalid or forged** challenge, or failed `sender_sig` / `id` → **discard silently**, no user-visible effect, do not `ack` (except a duplicate `id`, which is `acked`).
- **Absent or below policy threshold** (a cold sender with no/weak proof) → **defer to a "requests" area** (not the inbox), rate-limited, never silently dropped and never surfaced as a normal message. The user MAY promote the sender (which pins them as a contact).

Implementations MUST NOT deliver an unproven cold MOTE to the inbox, and MUST NOT silently discard a well-formed-but-under-threshold one without the requests-area affordance.

2.8 Why this shape

- **Content-addressed** → dedup, integrity, caching, and file chunking all fall out of `id`.
- **Three layers** → clean separation of *routing privacy* (outer), *authenticity* (envelope), and *confidentiality* (payload).
- **Sealed sender** → the network sees ciphertext to an opaque destination and nothing else.
- **One object, many kinds** → mail, chat, files, groups, and identity share the format, so the transport, store, and crypto are built once (§5).

3. Naming & Directory (name → key)

Your **key is your identity and the only root of trust**; a `name@domain` is the everyday, human-facing pointer to it; the mesh finds you by key. This section covers the **stable** binding: how `abc@def.com` resolves to an identity key, how that binding is made tamper-evident, and how it degrades safely.

The layering (read this first). Three layers, never conflated:

- **The key is identity and proof.** Authenticity is *always* the key (`IK`, §1.2) — nothing else. A name is only a pointer to it.
- **DNS (and any name backend) is *discovery*, never proof.** It tells you which key *claims* a name; it does not attest it. A compromised registrar can lie about the pointer, so the pointer is never trusted on its own.
- **Key transparency (KT) makes the `name` → `key` binding tamper-evident**, so a silent key swap by DNS/registrar/directory is *detectable* (§3.5).
- **Pinning (TOFU) makes discovery a one-time event.** After first contact you route by the pinned key via the mesh (§4); DNS is not consulted again unless the signed identity chain says so. A later DNS/registrar compromise cannot redirect an existing relationship.

Stable anchor, rotatable keys (the real future-proofing). The human name binds to a **stable identity anchor** — the **root identity key `IK` (§1.2)** — *not* to a rotatable operational key. Day-to-day signing/device keys rotate *under* the identity without changing the name; even `IK` itself can rotate, and the suite can migrate to post-quantum, without the name changing — bridged by the signed hash chain (§1.5) and, for a change of the anchor's name, by

aliases + a signed `MoveRecord` (§1.6). So a `name@domain` **survives ordinary key rotation and PQ migration**: the `name → key` indirection is exactly what lets the key underneath change while the address people type stays the same. `IK` rotation is the rare migration event, not the common case.

3.1 Roles (who holds what)

- **The key (IK)** is the identity and the sole trust root. Everything below only *points* to it; nothing below can prove authenticity on its own.
- **DNS** (we do not run it — registrars/operators do) holds the stable `name → key` *pointer*. It is discovery: static and cacheable. It **MUST NOT** hold location (that is the mesh, §4), and it is **never** the proof — `KT` + pinning are.
- **Key transparency (KT)** makes the pointer *auditable* — since DNS is a trusted third party the owner does not control, `KT` lets a silent key swap be *detected* (§3.5).
- **The mesh DHT** holds the dynamic `key → location` record (§4).

3.2 DNS records

For `abc@def.com`, the resolver queries:

```
abc._dmtap.def.com.  IN  TXT  "v=dmtap1; suite=1; ik=<base64url IK>; id=<hash of Identity §1.3>;
                        kt=<KT log URL>; keypkgs=<KeyPackage bundle locator §5.3>"
_dmtap.def.com.      IN  SVCB 1 . ( ... )      ; optional service params, KT anchors
def.com.             IN  MX   ...             ; only if a legacy gateway serves the domain (§7)
```

- `ik` is the identity public key (or a hash of `Identity` that the mesh resolves to the full object). `id` pins the current `Identity` version (§1.3).
- Multiple `ik` / `suite` entries **MAY** appear during PQ migration (§1.1).
- DNSSEC **SHOULD** be enabled; it is not sufficient alone (hence `KT`).
- **did:web consistency (normative, for DMTAP-Auth §13.6)**. Where the same identity is also published as a `did:web` document (`did.json`), that document's key **MUST** be **byte-consistent with this DNS name → key binding and its KT entry** (same `IK`, same `Identity` hash). A verifier **MUST cross-check the two and pin** (§3.4); a `did.json` is the same discovery-only pointer as DNS and never proof on its own.

3.3 Resolution

`resolve(name)`:

1. DNS TXT/SVCB lookup for `name → { iks, id, kt, keypkgs }`
2. (first contact) verify against `KT` (§3.5): fetch a signed tree head + inclusion proof for this identity, and confirm no newer version supersedes it (rollback defense).
3. fetch full `Identity` (§1.3) from the mesh by ``id``; verify sig chain
4. PIN (`name → iks, id`) locally (TOFU); offer out-of-band verification to upgrade the pin
5. thereafter: route by key via the mesh (§4); DNS is not consulted again unless the pinned `Identity` chain says to (rotation/migration)

Fail-closed on unreachable KT (normative, §12 finding). If `KT` is unreachable, partitioned, or censored at **first contact**, the client **MUST NOT** silently TOFU-pin an unverified key — it **MUST** either refuse to pin (block) or hard-warn and require explicit user acceptance, and **MUST** prefer out-of-band verification. Silent downgrade to unverified TOFU is prohibited, because it enables an attacker to replay an old-but-validly-signed `Identity` (e.g. from before a legitimate rotation) precisely under the network conditions that make `KT` unreachable. Once

a key is pinned, later KT unavailability does not block routing (the relationship is already key-based).

DNS is the **front door, used once**. After first contact the relationship is key-based and DNS-independent — a later DNS/registrar compromise cannot silently redirect an existing contact.

3.4 Trust on first use (TOFU) + pinning

v0 trust model:

- On first resolution, **pin** (`name` → `ik`, `id`).
- Follow the signed `Identity` hash chain (§1.3–1.6) for rotations and migrations; accept a new key only via a valid chain from the pinned key.
- Offer **out-of-band verification** (safety-number / QR comparison of `ik`, §3.4.1) to upgrade a TOFU pin to a verified pin.
- A key that changes *without* a valid chain **MUST** raise a security warning, never silently update.

Honest limit: a MITM at the *very first* contact (before KT is consulted or before OOB verification) can substitute a key. KT (§3.5) closes this; OOB verification closes it immediately for high-value contacts.

3.4.1 Safety numbers (out-of-band key verification)

Out-of-band verification compares the **key**, not the name — it is the strongest trust upgrade and the one thing that closes a first-contact MITM immediately.

- A **safety number** (Signal-style) is a deterministic function of the parties' `IK`s (the fingerprint of the identity keys). Two contacts confirm they see the same value out-of-band — in person, over a trusted channel, or by scanning a **QR code**.
- **This is verification, not an address.** A safety number / fingerprint is *never* used to route or reach someone; it only confirms that the key you pinned is the key you meant. It does not appear in `Identity.names` and cannot be typed at to send mail.
- **Word rendering (optional).** Because digit strings are error-prone to compare aloud, a fingerprint **MAY** be rendered as a **word sequence** for easier human comparison: `words(fingerprint, wordlist)` over a curated **~1024-word, language-agnostic** list (short, pronounceable across major languages, no homophones/confusables/offensive collisions), 10 bits/word, with a folded **checksum** so a misheard word fails closed rather than comparing as a different key. **Proquints** (pronounceable 5-char syllables, 16 bits each) are an allowed language-neutral alternative encoding of the same bits.
- The word encoding exists **only** for this verification role — a comparison aid for confirming a key. It is deliberately **not** an address: DMTAP does not name people by their key digits. Because the safety number is taken over the full identity key, it carries the key's full strength; there is no separate truncated-name address to grind.

3.5 Key transparency (KT)

Status: designed-in, v0 ships a minimal form; full CONIKS/Key-Transparency-style logs are a v1 hardening.

- The owner's identity events (`Identity`, `RecoveryPolicy`, `KeyRotation`, `MoveRecord`, §1) are appended to an **append-only Merkle log**.

- Verifiers obtain **signed tree heads** and **inclusion/consistency proofs**; they gossip tree heads to detect **split-view/equivocation** (the log showing different histories to different observers).
- The owner's own devices **monitor** the log for the owner's entries and MUST alert on any change the owner did not initiate — turning KT into **identity intrusion detection**.
- Logs MAY be federated; a verifier trusts a *set* of logs and requires consistency across them. No single log is authoritative.

v0-minimal: a single append-only log with signed heads and inclusion proofs; monitoring and gossip-based equivocation detection land in v1.

3.6 Optional: self-sovereign naming (un-loseable addresses)

DNS names can expire or be seized. For a name that is **un-loseable** and **un-seizable** as long as the owner holds `IK`, DMTAP defines a pluggable **name backend** interface. A conforming backend maps `name` → `ik` such that only `IK` can update the binding and it cannot lapse without the owner's action. A name-chain (ENS-style) is one such backend.

This is the **only** place DMTAP admits a blockchain, it is **optional**, and it is confined to the name layer — nothing else in DMTAP depends on it. Resolution (§3.3) treats a self-sovereign name identically to a DNS name once `ik` is obtained.

3.7 Private lookups

To keep discovery metadata-private, name→key lookups **SHOULD** be routed **through the mixnet** (§6), so neither the DNS resolver nor a KT log learns *who* is asking. This replaces heavier private-contact-discovery schemes for v0; stronger schemes are a v1 option.

3.8 Onboarding tiers (day-one setup)

The identity is a **key**; DNS is a generated, auto-managed *projection* of it, never hand-edited. There are three onboarding tiers. A conformant client **SHOULD** default to Tier B (a provider-issued `name@domain`) and never require a user to author DNS.

Tier A — no domain (pure DMTAP)

Identity = key; the human name is a provider directory name or a self-sovereign name backend (§3.6) that resolves directly to the key. **No DNS, no domain.** DMTAP DMTAP only (the legacy world cannot resolve the name). Setup: generate key → claim a directory name → join mesh. Most users instead pick Tier B so their address also works for legacy email.

Tier B — gateway-owned domain (default; zero user DNS)

The user claims `alice@gw.example`; the **gateway operator owns `gw.example` and maintains all legacy records once, for all users:**

```
gw.example      MX      → gateway
gw.example      TXT      SPF (gateway sending IPs)
sel._domainkey  TXT      gateway DKIM key
_dmarc.gw.example TXT    DMARC policy
+ *@gw.example  name→key directory (service or programmatic/wildcard records)
```

The **user touches zero DNS**. DMTAP-native delivery is key-based; legacy interop works immediately because the shared domain is pre-configured and its IPs are warmed. Per-user legacy setup = none. This is Gmail-grade onboarding: you get an address, not a domain — a provider-issued `you@gw.example` that is a full, first-class `name@domain` (§3.9.1).

Tier C — vanity custom domain (auto-configured)

For `alice@yourbrand.com`, the domain's own DNS MUST carry MX/SPF/DKIM/DMARC (DMARC alignment is defined on the `From:` domain — unavoidable while legacy exists). But it MUST NOT be hand-edited: the provider auto-publishes the full record set via a **registrar API / the Domain Connect standard** or by hosting the domain's DNS. The user approves once; records self-update on key rotation.

DKIM delegation / DMARC alignment (all legacy-facing tiers)

The domain publishes the **gateway's DKIM public key** at `sel._domainkey.<domain>`; the gateway signs outbound with `d=<domain>`, so DKIM passes and **DMARC aligns** — Gmail/Outlook accept it as ordinary authenticated mail. The gateway holds only a delegated *DKIM* key, never the user's DMTAP identity key (§7.3). Inbound legacy uses the domain's MX → gateway (§7.2).

What cannot be skipped

Exactly one thing is irreducible: **some** operator must configure **one** legacy domain properly and **warm its sending IPs** (weeks). This is a one-time, per-domain operator task, amortized across all users on that domain — never a per-user task.

3.9 Names (one identity, many pointers)

The identity is always the **key**; every name is a memorable pointer to it, and one identity MAY hold several names at once — all resolving to the same key. DMTAP has **one recommended, headline form** and a few optional extras. This is deliberately *not* a ladder of equals.

Zooko's triangle, stated plainly. A name cannot be simultaneously *human-meaningful*, *global*, and *authority-free*. DMTAP's choice: names are human-meaningful and — through DNS federation — global, and it pays for that with an authority (DNS/registrar). But it **neutralizes** that authority by keeping the key the sole trust root, KT-auditing the binding (§3.5), and pinning on first use (§3.4): the authority can help you *discover* a key, it can never *forge* one undetected. The alternative corner (a flat authority-free name derived from the key) is not offered as an address — it is unwieldy, and it would name people by a rotatable artifact rather than the stable identity.

3.9.1 `name@domain` — the primary human address

`name@domain` is the everyday, recommended, headline address. It comes in two flavours, one format:

- **Provider-issued** — `you@envoiron.org` and the like (Bluesky-style): the majority who don't own a domain get a familiar, handle-shaped address for free from a provider, with zero DNS work (Tier B, §3.8).
- **Your own domain** — `you@yourbrand.com`, for full sovereignty (Tier C, §3.8).

Why this is the headline:

- **It survives key rotation and PQ migration.** The name binds to the stable anchor `IK`, not to a rotatable key; the key underneath can rotate or migrate to a PQ suite and the address is unchanged (`name` → `key` indirection, §1.5–1.6). This is the real future-proofing.
- **It's federated — no global-namespace squatting.** Uniqueness is *per-domain*, so there is no single global registry to squat or race: `alice@a.org` and `alice@b.org` are different people, and neither had to win a landrush.
- **One familiar format**, already understood by every human and every mail system.

- **It interoperates with legacy email for free** — the shared or owned domain carries MX/DKIM/DMARC (§3.8, §7), so `name@domain` is reachable from Gmail/Outlook on day one.

Authenticity is still the key: a `name@domain` is resolved (§3.3), KT-audited (§3.5), and pinned (§3.4) — the domain only points, it never proves. For a name that cannot expire or be seized, the same pointer can live in a self-sovereign name backend instead of DNS (§3.6), with resolution unchanged.

3.9.2 `@handle` — optional global registry (opt-in)

A flat, global `@handle` namespace is **optional, not the headline**. For handle-like UX, DMTAP prefers the **provider-federated** `you@provider` form above, which gives the same short-name feel *without* a single global namespace to fight over.

A conforming client MAY additionally offer an opt-in `@handle` directory. Because a *chosen, globally-unique* name requires arbitration (Zooko), the directory is a thin authority that:

- assigns each handle once (first-come-first-served) with an **anti-squat cost** (a small fee or proof-of-work, since assignment gives uniqueness but not scarcity);
- publishes `handle` → `key` to a **key-transparency log** (§3.5), so it is *auditable, not trusted* — it cannot silently repoint a handle without detection;
- **cannot hijack existing relationships** — contacts route by the pinned key (§1.6), so the handle is only an introduction.

Handles are normalized (NFC, lowercase, collapse consecutive dots; dots allowed as cosmetic separators, e.g. `@this.is.my.name`) and confusables/homoglyphs are reserved. The directory SHOULD be a **federated consortium running BFT consensus** (no single owner of the namespace, no chain) rather than a single operator; a name-chain (§3.6) is a further option.

Honest limit (why it is not the default): a single global namespace **reintroduces exactly the global squatting and impersonation that domain-federation avoids** — every desirable handle becomes a landrush and a phishing target (`@paypal` vs `@paypal`). That is the unavoidable price of a flat global name, and it is why `name@domain` is the recommended form and `@handle` is an explicit opt-in.

3.9.3 Petnames (local)

A user assigns **petnames** locally to contacts ("Mom" → a key). Petnames are human + zero-authority but *local-scope only* (not global), and never leave the device cluster (they are a social-graph artifact and MUST be stored encrypted at rest with the mailbox).

3.9.4 Aliases & subaddressing (one identity, many addresses)

For feature parity with legacy email, **one identity MAY hold multiple names** — all resolving to the same key — and support subaddressing:

- **Aliases.** An identity's `Identity.names` (§1.3) is a *list*: a user can hold several `name@domain` addresses (provider-issued and/or own-domain) at once, optionally an `@handle`, all pointing to the same key. Mail to any of them arrives in the same mailbox.
- **Retaining a legacy address.** Crucially, a **legacy email address you already own can be an alias**: point that domain's MX/DKIM at a gateway (§7) and add the address to `Identity.names`, so people who only know your old `you@oldprovider.com` still reach you (via the gateway → MOTE), while you migrate. This is the practical migration path — you don't lose your old address; it becomes one of your DMTAP aliases.

- **Subaddressing (plus-addressing).** `you+tag@domain` (and, for handles, `@you+tag`) is supported: the `+tag` is preserved for client-side filtering/labeling but resolves to the same key. Catch-all (`*@yourdomain`) is a domain-owner option (tier C).
- **Security — self-asserted names need forward verification (normative).** `Identity.names` is **self-asserted**: an identity can *list* any string, including a **victim's address**. A listed name therefore proves **nothing** on its own, and a client **MUST** trust or display a name in `Identity.names` **only after verifying the forward name → ik binding** (DNS + KT, §3.3–3.5) **also resolves to this same key** — i.e. the name points back. Before accepting a name (especially a **legacy alias**), the client **MUST** require **proof of control**: a DNS challenge under that name, or a **KT-anchored per-name record** binding `name → ik`. A name that does not verify back **MUST** be rendered **as unverified**, never shown as an authenticated address. Given that check, every *verified* alias resolves to the *same key*, so it cannot be used to impersonate — authenticity is always the key, not the name. Adding/removing an alias is a signed `Identity` version, audited via the same KT (§3.5). A legacy-alias's inbound mail is marked *legacy-origin* (not E2E before the gateway, §7.2), so the user sees which messages came the old way.

4. Transport: Mesh, Mixnet, Delivery

The mesh finds you and moves ciphertext to you; the mixnet hides who is talking to whom. The node **is** the mesh — relay and mix roles are capabilities of the node binary, not separate services.

4.1 Substrate: libp2p

DMTAP builds on **libp2p** rather than reinventing P2P:

- **Kademlia DHT** — peer routing and the `key → location` record store (see §4.2 for its security limits — it is the weakest link).
- **Circuit relay v2** — reachability for NAT'd nodes (the "relay" role).
- **AutoNAT v2 + DCUtR (hole punching)** — direct connections where possible.
- **Noise** (or TLS 1.3) — hop-by-hop channel security (in addition to end-to-end MOTE encryption; QUIC carries TLS 1.3 natively).
- **QUIC / TCP / WebSocket / WebRTC / WebTransport** transports.

A node dials **outbound** and holds connections, so CGNAT/dynamic-IP nodes are reachable.

Roaming — honest note: identity is a persistent keypair, not an IP. When a node's address changes, roaming is carried primarily by **re-publishing the location record (§4.2) and peers re-dialing**, not by QUIC connection migration. QUIC migration (RFC 9000) *can* preserve some live connections, but the Rust QUIC stack (`quinn`) has no multipath and address-change handling is imperfect; do not rely on it for seamless roaming.

4.2 The `key → location` record (DHT)

```
LocationRecord {
    ik:          bytes,          // identity key (DHT key = hash(ik))
    peer_id:     bytes,          // libp2p peer id (may be per-epoch / unlinkable, §6)
    addrs:       [* multiaddr], // current reachability hints (may be relay circuits, mix addrs)
    seq:         u64,            // monotonic sequence number; reject older-or-equal (rollback defense,
    ttl:         u64,
    ts:         u64,
    sig:         bytes,          // signed by a device key
}
```

The record follows the **IPNS pattern**: a self-certifying **value record** (DHT key = hash(ik)), signed, with a **monotonic sequence number** + **EOL/TTL** to defeat rollback/replay, stored at the **K closest peers**, and **aggressively republished** (DHT record lifetimes are short — hours — so staleness is a real failure mode). Use value records, not provider records.

CAUTION — signing does NOT stop eclipse attacks (the DHT is the weakest link)

Signing authenticates record *content*, not *routing*. Because a libp2p PeerId is `hash(pubkey)`, an attacker can cheaply generate IDs closest (XOR-distance) to a target key, fill honest routing tables, and control all lookups for that key — returning nothing or an *old, still-valid signed* record (censorship / rollback) **without forging anything**. This is a Sybil/eclipse attack at the routing layer, and it is the single most attackable part of DMTAP. Mitigations DMTAP REQUIRES/RECOMMENDS:

- **S/Kademlia disjoint-path lookups** (parallel, node-disjoint) and **IP-diversity caps** per k-bucket.
- **Aggressive republish** + accept only records with a newer sequence number (rollback defense).
- **The DHT is one discovery mechanism, not the root of trust.** Root of trust = the user's long-term keypair. Resolution order: (1) cached direct addresses, (2) **relay-reservation / rendezvous ("home relay") addresses**, (3) DHT as fallback. A fresh contact SHOULD have a non-DHT path (rendezvous/introduction) so it is not 100% dependent on a hostile public-DHT lookup.
- Closed/organizational deployments SHOULD use a **private DHT** (own protocol prefix) to shrink the Sybil surface.

4.3 Reachability ladder

Try in order; fall down only as needed:

1. **Direct** — IPv6, or IPv4 with port-forward/UPnP. No relay.
2. **Hole-punch** — DCUtR between two nodes (both online; always true for always-on boxes).
3. **Circuit relay** — a public-IP node relays; ciphertext-only, content-blind.

As IPv6 spreads, rungs 1–2 dominate and the relay role fades. Downtime is covered by sender-retry (§2.6) and optional **peer buffering** (buddy nodes hold ciphertext during an outage) — no central buffer required.

4.4 The mixnet (metadata privacy)

For the `private` tier, MOTEs are sent as **Sphinx-format**, constant-length, onion-wrapped packets through a sequence of **mix nodes** (Loopix/Nym-style):

- Each mix peels one layer; **no mix sees both sender and recipient**.
- Mixes add **randomized (Poisson) delays** and reorder, defeating timing correlation.
- Nodes emit **cover traffic** (loop + drop messages) at a steady rate so an observer cannot tell when real messages are sent/received. Cover rate is a **tunable knob** (§6).
- **Sealed sender** (§6.2): the sender identity is inside the payload, never in the outer packet.
- **Size padding**: MOTEs are padded to fixed buckets so size does not leak content.

Mix nodes are the same permissionless, content-blind contributor model as relays; incentive and Sybil-resistance are covered in §6.4 and §9.

Email's asynchrony is what makes full-strength mixing affordable: minutes of latency are acceptable for the `private` tier.

4.5 Bulk / file transfer

Large blobs (§5.5) **MUST NOT** traverse the mixnet (impractical bandwidth/latency). Instead:

- The **control MOTE** (`file_offer` : manifest + key) travels the **private** tier.
- The **chunks** transfer **direct** (fast tier), content-encrypted, **swarmed** from any holder (BitTorrent-style), resumable per chunk.

Honest tradeoff: the *fact and size* of a bulk transfer between two nodes is observable (direct), though the content is encrypted and the control message is private. See §6.

4.6 Privacy tiers

Tier	Path	Latency	Metadata privacy	Default for
private	mixnet + cover traffic	minutes	full (global passive)	mail, all control messages
fast	direct / low-hop mesh	sub-second	content only; graph observable	live chat (both online), bulk

Default is **private**. **fast** is opt-in per conversation/message. Choosing **fast** is itself metadata; clients **SHOULD** keep **private** as the standing default and cover it with cover traffic (§6).

4.7 Delivery state machine (sender)

QUEUED → SEALED (sealed sender + onion, §6) → IN_FLIGHT (mixnet/direct)
 → ACKED (done) | RETRY (backoff) | EXPIRED (drop, notify user)

Durability lives entirely in this sender-side queue. The middle is stateless.

4.8 Local, isolated, and delay-tolerant networks

DMTAP works in **remote environments with their own networks** — often *more easily than email*, because email needs an SMTP/IMAP server + MX + DNS, whereas two DMTAP nodes on the same network need no infrastructure at all.

- **Local discovery (mDNS).** Nodes on the same LAN discover each other via libp2p **mDNS** (`_p2p._udp.local`) with **zero configuration, no internet, and no central server**. A ship, a remote site, an air-gapped office, or a home LAN can run a fully functional local DMTAP mesh with nothing but the nodes. Private networks use a fingerprinted service name so they do not cross-discover.
- **Scope of the “easier than email” claim.** True specifically for the **local / same-network / isolated** case. Across the open internet, DMTAP still depends on the DHT/relays (§4.2–4.3), which have their own fragility.
- **Delay-tolerant store-and-forward is a DMTAP layer, not a libp2p feature.** libp2p is connection-oriented and provides no bundle/epidemic DTN routing. DMTAP **REQUIRES** its own store-and-forward: the sender-side retry queue (§4.7), **peer buffering**, and **sync-on-reconnect** of the device-cluster CRDT (§5.6). This is what lets an intermittently connected site queue locally and reconcile with the wider mesh when connectivity returns.
- **Radio transports (Bluetooth / Wi-Fi Direct / LoRa) are out of scope for v0.** libp2p ships no such transport (Briar's offline radio transports are its own Bramble code, not libp2p). Supporting them would mean writing a custom libp2p `Transport` — flagged as future work, not a v0 claim.

5. Messaging & Files (the unified substrate)

Mail, chat, and files are modes over one substrate: **MLS** (RFC 9420) for all session and group security, **MLS KeyPackages** for async initiation, and **content-addressed chunked blobs** for files. The same group primitive serves 1:1, group chat, email mailing-lists, multi-device, and shared file folders.

5.1 Why MLS everywhere

DMTAP standardizes on **MLS as the unifying crypto primitive**:

- **1:1** = a 2-member MLS group.
- **Group chat / mailing-list** = an MLS group.
- **Multi-device** = each of the owner's devices is a member (MLS handles this cleanly where pairwise ratchets get messy).
- **Shared file folder** = an MLS group over a set of manifests (§5.5).

MLS is **transport- and identity-agnostic** by design (RFC 9420, 2023; architecture in RFC 9750). Its abstract roles map onto DMTAP:

- **Delivery Service (DS)** → the DMTAP mesh/mixnet (§4).
- **Authentication Service (AS)** → DMTAP identity + KT (§1, §3). KT realizes the AS while *reducing* the trust placed in it — a good fit for a decentralized design.

MLS provides group forward secrecy and post-compromise security via the TreeKEM ratchet tree, with $O(\log n)$ member changes. v0 uses ciphersuites `0x0001` (MLS_128_DHKEMX25519_AES128GCM_SHA256_Ed25519) and `0x0003` (...CHACHA20POLY1305...); PQ migration uses MLS's own PQ ciphersuites (draft-ietf-mls-pq-ciphersuites, ML-KEM) and the MLS combiner (draft-ietf-mls-combiner), **not** a bolted-on external handshake.

CAUTION — epoch ordering is the hard part on a mesh

MLS trusts the DS for exactly one thing: **a total order on epochs** — Commits must be applied in an agreed order per group. A leaderless mesh has no natural serialization point, so **this ordering/consensus of Commits is the real difficulty of "MLS over P2P," not the crypto**. DMTAP **REQUIRES** that **MLS handshake** messages (Proposal / Commit / Welcome) travel over an **ordered, reliable channel** per group, while ordinary **application** messages (mail/chat/files) **MAY** travel over the reordering mixnet. Sending handshakes over the mixnet would stall or fork group state.

v0 ordered-channel: the group committer (normative). Each group has a **committer** — the node that serializes handshake messages into an append-only, hash-chained per-group log:

- **Selection**: the group creator is the initial committer. Committer identity is a signed field of the group state; every member knows it.
- **Rotation**: any member **MAY** be promoted to committer by a Commit (so committer role is not tied to one device's uptime). Committer **SHOULD** rotate on the current committer going offline past a timeout, or on member vote, via a Commit that all members apply.
- **Failover / liveness & deterministic succession**: if the committer is unreachable, members hold pending Proposals and initiate a **takeover that does not depend on the incumbent's cooperation**. The successor is **deterministic**: among live, non-faulted members the one with the **lowest member signing key** (canonical byte order) is the designated next committer (keys are unique; earliest join epoch is the fallback tie-break). The successor proposes a **takeover Commit** referencing the last agreed log head; it takes effect **only when it carries a threshold of member signatures** (a roster quorum, §16),

so a partitioned minority cannot install a rival committer and there is no split-brain over *who* takes over. (Edge case: a 2-member group cannot perform a takeover if one member is dead — the $> n/2$ quorum needs both — so a censoring committer in a 2-party group is resolved by leaving/recreating the group, not by takeover.) The hash-chained log makes a **fork detectable**: two Commits at the same log position with the same predecessor is proof of committer misbehavior; members **MUST** halt and alert (analogous to KT equivocation, §3.5) and recover per the fork-recovery path below.

- **Censorship is misbehavior — including *selective censorship***: a committer can *stall* but cannot *forge* (every handshake is member-signed). Crucially, a committer that stays live yet **withholds ordering of one specific pending, member-signed proposal** (e.g. never orders a member's own removal) past the **committer-liveness timeout (§16)** is treated **identically to a dead committer** and triggers the deterministic takeover above — total silence is *not* required to justify replacement. This defeats the selective-censor-and-block-your-own- replacement attack, since the successor rule and member-signature threshold need no consent from the incumbent.

Fork recovery (out of HALT). A detected fork halts the group (above); recovery is **out-of-band**. Members compare hash-chained log heads and identify the **last common epoch** both forks share; an **admin / owner** (per §5.8.2 authority) proposes a recovery Commit on top of that last common epoch, and — **for consistency with committer takeover (§16 roster quorum) and to deny a single admin unilateral fork-selection power** — the recovery Commit is canonical only when it carries the same $> n/2$ **member-signature quorum** as a takeover. This re-establishes a single canonical head that a strict majority endorsed, not one admin's choice. Members that applied the losing fork **roll back to the last common epoch** and re-apply from the recovery Commit; application messages committed only on the abandoned fork are re-submitted by their senders (sender retry, §2.6). This manual reconciliation is the v0 stopgap; **Decentralized MLS (draft-kohbrok-mls-dmls) is the eventual leaderless fix** that removes the single-orderer fork surface entirely.

Metadata exposure (honest, §6.6 item 7). The committer necessarily sees **all handshake traffic for its group** — an explicit exception to the "no single node sees both ends" framing. This is bounded by: the committer sees only *membership-change* metadata (not application message content or the social graph outside the group), rotating the role spreads exposure, and application traffic still uses the mixnet. Groups needing stronger metadata protection **SHOULD** rotate committer frequently.

Track `draft-kohbrok-mls-dmls` (Decentralized MLS) for a fully leaderless replacement; until it lands, the committer model above is v0.

5.2 Forward secrecy & (non-)deniability

- **Forward secrecy** comes from MLS epoch advancement. Healing (post-compromise security) is **per-epoch/per-Commit**, which is coarser than the Signal Double Ratchet's per-message healing — using MLS for 1:1 (§5.1) is a deliberate *simplicity-vs-optimality* trade, not a strict improvement over Double Ratchet.
- **Deniability — honest correction.** MLS is **signature-based and therefore non-repudiable**: every LeafNode and every FramedContent (handshake *and* application messages) is signed with the member's signature key. RFC 9750 states plainly that "*MLS does not make any claims with regard to deniability.*" DMTAP therefore **does NOT claim message deniability** at the MLS layer, and **MUST NOT** substitute shared-secret MACs for MLS's mandatory signatures (that would break RFC 9420 conformance). Deniability is an **explicit open design item**: if required, it must be engineered at another layer (e.g. exchanging the signature keys themselves over a deniable channel, per RFC 9750's note), or scoped to a pairwise/OTR-style sub-channel. Do not represent DMTAP messages as

deniable in v0.

5.3 Async session initiation (MLS-native)

To message an identity whose devices are all offline, DMTAP uses **MLS's own async-join mechanisms** rather than bolting on a separate PQXDH/X3DH handshake (which would add a second protocol and a second security proof for no benefit when everything is already MLS):

1. Each device pre-publishes signed **KeyPackages** (located via `Identity.keypkgs`, §1.3) to the mesh. A KeyPackage is MLS's prekey: identity/signature key, HPKE init key, and (for PQ) an ML-KEM init key via the MLS PQ ciphersuite.
2. The initiator **Adds** the offline member (a Commit) and sends a **Welcome** so the new member bootstraps group state on return; or the joiner uses an **External Commit** against the group's `GroupInfo` to self-join.
3. Post-quantum protection comes from the **MLS PQ ciphersuite / combiner** (§5.1), keeping a single key schedule.

KeyPackages are replenished by the owner's node; last-resort KeyPackages avoid exhaustion. (PQXDH-for-init would only be relevant if the 1:1 layer were Signal Double Ratchet rather than MLS; DMTAP chooses MLS everywhere, so it is not used.)

5.4 Message kinds in context

- `mail / chat` — the same MOTE; differ by default tier (§4.6) and client rendering.
- `reaction / edit / redact` — reference a prior MOTE by `id` via `refs`.
- `group_event` — MLS handshake messages (add/remove/update/commit).
- `receipt / presence` — opt-in, ephemeral, off by default (metadata-sensitive, §6).

5.5 Files: content-addressed, chunked, any size

A file is a **BLAKE3 Merkle-DAG manifest** over fixed-size encrypted chunks:

```
Manifest {
  id:      bytes,      // BLAKE3 root over chunk hashes (the content address)
  size:    u64,
  chunk_sz: u32,       // e.g. 1 MiB
  chunks:  [+ bytes],  // ordered chunk hashes
  key:     bytes,      // file content key (delivered inside the MOTE, §2.5)
  suite:   u8,
}
```

Properties:

- **Any size** — no protocol cap; the file is bounded only by the owner's storage.
- **Resumable / parallel / swarmed** — fetch chunks from any holder, restart per chunk, BitTorrent-style; a popular file becomes *more* available as more nodes hold it.
- **Deduplicated** — identical chunks (and identical files) stored once by content address.
- **Streamable** — consume in manifest order before full download.
- **Integrity** — every chunk self-verifies against its hash; the manifest self-verifies against `id`.

Availability tiers (files reintroduce storage economics):

Tier	Mechanism	Cost
Best-effort	origin node online + swarm from holders	\$0
Durable	encrypted peer-cache / replica	peer reciprocity
Always-available	paid replica / managed host	real storage cost

The always-on box gives best-effort/durable for free; "Dropbox-like always available" for large files is where a paid replica re-enters. Transfer uses the fast/direct tier (§4.5).

5.6 Multi-device (the personal cluster)

An owner's devices form an MLS group (`caps` per DeviceCert, §1.2). The mailbox, read/flag state, labels, and file index are replicated across devices as an **encrypted CRDT** synced over the mesh, converging under out-of-order delivery. Any device can send/receive; the always-on box is the anchor that guarantees receipt while other devices sleep.

5.7 Three products, shared components

Component	Mail	Chat	Files
Identity / keys (§1)			
MOTE object (§2)			
Mesh + mixnet (§4)			(control)
MLS group (§5.1)	(lists)	(channels)	(folders)
MLS KeyPackages (§5.3)			
Content-addressed blobs (§5.5)	(attach)	(attach)	
Fast tier (§4.6)	—	(live)	(bulk)
Device-cluster CRDT (§5.6)			

The one genuinely new component — the **MLS group** — unlocks all three. Real-time voice/video is **out of scope** (separate WebRTC/SFU architecture).

5.8 Groups as addressable identities

A group is an identity that has members. It has its **own keypair** and therefore its **own name** on the full naming ladder (§3.9): a group key-name, an `@handle` (e.g. `@team`), or a domain address (`team@company.com`). Sending to a group address delivers to all current members; membership is the group's MLS roster (§5.1). This single mechanism unifies mailing lists, chat channels, shared folders, and team inboxes — "a group with an address."

5.8.1 Two posting models

Model	Behavior
Broadcast / list	post to the address → every member receives a copy (distribution list, announce)
Collaborative / channel	shared, ordered conversation + shared state (chat channel, shared folder)

Both are MLS groups; they differ only in **posting policy** and **membership-visibility policy**, which are fields of the group state. A group MAY switch models by policy change (a Commit).

5.8.2 Roles & management

Group state carries **roles**: `owner` (≥ 1), `admin`, `member`, and optional `poster` (may send) vs `reader`. Management operations are **MLS Commits** ordered by the group committer (§5.1), authorized by role:

- **Add / remove member** — Add uses the invitee's KeyPackage (§5.3) + Welcome; Remove triggers file-key rotation for shared folders (§6.7). Requires `admin`.
- **Role change / transfer ownership** — requires `admin` / `owner`.
- **Policy change** (posting model, membership visibility, join policy) — requires `admin`.
- **Join policy**: `closed` (invite only), `request` (request $\rightarrow$ admin approval), `open` (anyone with the address may join, rate-limited + anti-abuse §9), or `vouch` (a member introduces).

All membership/role/policy changes are signed and appear in the group's hash-chained handshake log (§5.1), so a member can audit "who added/removed whom" — the group analog of identity key-transparency (§3.5). A malicious/coerced committer can stall but not forge (every change is member-signed); **selective non-ordering of a pending signed management proposal past the committer-liveness timeout is misbehavior that triggers the deterministic committer takeover (§5.1)**, and forks are detectable and recoverable (§5.1).

5.8.3 Membership privacy (subscriber-list)

Broadcast lists MUST support **hidden membership**: members receive via per-member sealed delivery (§6) and do not learn the other members' keys. MLS's tree exposes members to *each other* by default, so hidden-membership lists use a **relay/committer fan-out** where the list identity re-seals to each member individually rather than a shared member-visible tree — at the cost of the shared-group efficiency. Channels (member-visible) use the normal MLS tree. This resolves the earlier mailing-list-privacy gap: choose the model per group, and disclose it.

5.8.4 Delivery & scale

- Small groups: standard MLS group message, fan-out to members over the mesh/mixnet.
- Large lists: the committer's ordered log is the source of truth; delivery is **per-member** (sealed to each), pull-or-push, so a 10k-member list is 10k individual sealed deliveries, not a 10k-member MLS tree. This trades cryptographic group-sharing for scalability and hidden membership — the right trade for announce lists.
- **Anti-abuse through fan-out (normative, §9.9)**. Fan-out is an **amplification vector**: one post becomes N deliveries, so the recipient's per-sender policy (§9) MUST apply to the **original poster**, not to the list identity. The poster's **ARC token / postage / PoW** (§9) MUST be **carried through the fan-out** and presented on each per-member delivery, and each recipient evaluates it against the *poster* — no accountability laundering behind the group address. Fan-out MUST be **rate-limited per poster**, amplification for `open` (anyone-can-post) lists MUST be **capped**, and posting to a **large list** MUST require **postage or PoW** commensurate with the fan-out size.

5.8.5 Legacy interop

A group MAY have a **legacy address** (`team@company.com`) served by the gateway (§7): inbound legacy mail to the address is fanned out to members as MOTES; outbound from the list to legacy subscribers goes via the gateway. So a DMTAP group is reachable as an ordinary mailing-list address from the old world, while native members get the full encrypted/group experience. Legacy fan-out is bound by the **same per-poster anti-abuse carry-through and fan-out rate-limits** (§5.8.4, §9.9); the gateway attributes the origin (§9.6), so list fan-out cannot launder a spammer's accountability.

5.8.6 Group key custody, recovery & threshold (normative)

A group is an addressable identity (§5.8) with its **own keypair**, so it needs the same custody discipline as a personal identity (§1) — otherwise a single admin or committer holding the group key can unilaterally hijack `team@company.com`.

- **Threshold-held signing key.** The group's identity signing key **MUST** be **threshold-held by the group's owner / admin set** (FROST-style, reusing the §1.4 recovery machinery), so no single admin — and no committer — can sign as the group alone. Group-authoritative acts (below) require a threshold of admins, not one.
- **Group RecoveryPolicy + rotate_threshold.** A group has its own `RecoveryPolicy` (§1.4); changes to the group `Identity`, its recovery methods, or its key **MUST** satisfy the group's `rotate_threshold` (the weakening-quorum + veto-window rules of §1.4 apply), so a compromised admin device cannot rewrite group recovery or evict co-owners.
- **KT-logged group key events.** Group `Identity` / `KeyRotation` / `RecoveryPolicy` events **MUST** appear in key transparency (§3.5) exactly like personal ones, so members and the wider network detect an unauthorized group-key change. The committer (§5.1) orders *handshakes*; it is **not** authorized to change the group's identity key — that is a threshold act above the committer role.

6. Privacy & Threat Model

DMTAP protects **content, authenticity, and metadata**. This section states the threat model honestly, defines the guarantees, and marks the boundaries we do *not* claim.

6.1 Adversary model

Adversary	Capability	DMTAP
Network eavesdropper (local)	reads links near a node	defeat
Malicious relay / mix node	sees ciphertext it forwards	defeat
Curious directory / DNS / KT log	sees lookups	mitigation
Global passive adversary	observes all links, all timing	primary
Global <i>active</i> adversary (inject/drop/delay at will, unlimited resources)	shapes traffic to correlate	not feasible
Compromised endpoint (node seized/keylogged)	reads that node's plaintext	out of scope

DMTAP's headline guarantee is **strong metadata privacy against a global *passive* adversary**.

6.2 What is protected, and how

- **Content** — MLS/HPKE end-to-end encryption (§2, §5). Only recipients decrypt.
- **Authenticity** — signed payloads; identity bound to name via KT (§1, §3).
- **Sender identity** — **sealed sender**: the sender's identity and authenticating signature live *inside* the encrypted payload; intermediaries see only ciphertext to an opaque destination (§2.2). Honest scope: sealed sender hides the sender from *intermediaries*, but **does not hide the sender's IP** (the mixnet does that) and is a *metadata-reduction*, not elimination — receipt/timing side channels can statistically erode it (Martiny et al., NDSS 2021), which is why cover traffic (§6.3) and mixing are load-bearing, not optional.
- **Recipient identity** — the mixnet delivers by **push to an always-on node** whose network identity is decoupled from the human identity; there is **no store-and-poll step to hide**, which dissolves the private-retrieval (PIR) problem by architecture (§6.4).

- **Social graph & timing** — **mixnet** (onion routing + Poisson mixing delays) so no node sees both ends and timing correlation is defeated; plus **cover traffic** and **size padding** (§6.3).
- **Discovery** — name→key lookups routed through the mixnet, hiding *who* looks up *whom* (§3.7).

6.3 Mixnet, cover traffic, padding

- **Onion packets** (Sphinx-format, constant length) traverse a path of mix nodes; each peels one layer.
- **Mixing delay** — each mix holds packets a randomized (Poisson) time and reorders, so input and output streams cannot be timing-correlated.
- **Cover traffic** — nodes emit loop and drop cover messages at a steady rate; an observer cannot distinguish real activity from cover. Rate is a **tunable knob** (higher rate → more privacy, more bandwidth).
- **Size padding** — MOTE's are padded to fixed size buckets so length does not leak.

Email's asynchrony is the enabling property: minutes of latency are acceptable for the **private** tier, and higher latency yields stronger anonymity.

6.4 Why we avoid PIR (honest framing)

PIR (Pung, Talek) exists to hide **which record a client reads from an untrusted shared mailbox that holds everyone's mail** — a leak that only exists *because* those systems use a polled central store to support offline recipients. DMTAP makes a different **design choice**: the recipient runs an **always-on node that receives by push** through the mixnet, so there is no untrusted shared store being queried and thus no read-access-pattern to hide. We therefore avoid PIR's *problem* (and its large cost) — this is a genuine architectural difference, not a trick. (Note: Loopix itself *does* use provider store-and-poll to serve offline clients; DMTAP's always-on-push is a deliberate divergence, not an inherent property of mixnets.)

But push delivery is not "recipient anonymity, solved." Three residual exposures remain and **MUST** be handled:

1. **Last-hop / receipt visibility.** The final mix and a global observer of the recipient's link learn that *a packet was delivered to node X*. An always-on node has a **stable, targetable network presence** — itself a fingerprint. The node's network identity **MUST** be cryptographically and durably **decoupled** from the human identity (pseudonymous peer id, no re-identifying metadata).
2. **Receipt-timing.** Without cover on the delivery link, observing the node reveals *when* messages arrive. The recipient node **MUST** receive a steady **Poisson cover stream** so real receipts are indistinguishable from cover (as Loopix does with loop + link cover).
3. **Long-term intersection / statistical-disclosure attacks.** Persistent presence is exposed to correlation across many rounds; cover traffic bounds but does not eliminate this (cf. Vuvuzela's differential-privacy noise, which degrades over volume). This is inherited, not solved.

Offline buffering: if a node is down, a peer/relay holds sealed ciphertext, retrieved on return via an **unlinkable dead-drop token over the mixnet** (the buffer sees an anonymous pickup, not "user X"). Cheaper than PIR and sufficient for v0; full PIR remains an option for hostile-buffer scenarios.

6.5 Privacy tiers (and where each product sits)

Tier	Path	Latency	Graph privacy	Default for
private	full mixnet + cover	minutes	full (global passive)	mail, all control MOTEs, normal-size files
fast	direct / few-hop	sub-second	content only	live chat (both online), large-file bulk

- Default is `private`. `fast` is opt-in. Because *choosing fast* is itself a signal, clients keep `private` as the standing default and cover it with cover traffic.
- **Files:** the control MOTE is always `private`. **Normal-size files route through the mixnet like messages (fully metadata-private).** **Large-file bulk** uses `fast` — but **MUST** be **onion-routed (a few hops, Tor-style) + size-padded to buckets + swarmed from multiple holders**, accepting bandwidth cost. **This bulk tier is explicitly weaker:** like Tor, it provides relationship anonymity against *local/partial* adversaries but is **vulnerable to end-to-end traffic correlation** by an adversary observing both endpoints (Murdoch–Danezis, 2005). Swarming and padding raise cost; they do not guarantee anonymity. The strong guarantee is the *messaging* tier; moving large sensitive files inherits Tor's correlation exposure. See §5.5, §4.5.

6.6 Honest boundaries (what we do NOT claim)

1. **Global active adversary.** An adversary who can inject, drop, and delay traffic everywhere and correlate at scale can, given enough resources, degrade anonymity. Perfect resistance is bounded by the fundamental **anonymity (latency + bandwidth)** tradeoff; more cover traffic and delay approach it asymptotically but never reach a costless zero.
2. **Large-file bulk metadata.** Onion-routing + padding + swarming makes it *strong*, not *free* and not *perfect* — the fact and approximate volume of a large transfer may remain partially observable at high adversary capability.
3. **Endpoint compromise has cluster-wide blast radius.** A seized/keylogged device that is a synced cluster member (§5.6) exposes the **entire replicated mailbox and its history**, not just a device-local slice — phone theft is the common case. Mitigated by at-rest encryption, forward secrecy (§5.2), recovery/rotation (§1), and **OPTIONAL scoped sync** (e.g. recent-N- days on mobile) which implementations **SHOULD** offer; not eliminated.
4. **First-contact MITM.** Before KT/OOB verification, the *first* name→key resolution can be MITM'd; DMTAP fails closed if KT is unreachable at first contact (§3.3, §3.4).
5. **Metadata privacy is weakest-link.** One leaky component (a bad lookup, a client bug, a timing side-channel) can unravel graph privacy. It is harder to *achieve* than to *design*; implementations **MUST** be conservative and audited.
6. **v0 key transparency is not equivocation-proof.** A v0 single, non-gossiped KT log can show different histories to different observers (split-view) until v1 gossip/multi-log auditing exists (§3.5). In v0, KT is **tamper-evident-after-the-fact and self-monitorable, but not a trusted single source** — a network **SHOULD** run multiple independent logs even in v0, and clients **SHOULD** treat a single unaudited log as advisory, leaning on OOB verification for high-value contacts. This is a real tension with the sovereignty goal and is stated as such. For **DMTAP-Auth (§13)** specifically, a split view is a **silent per-RP account takeover**, so high-value login RPs **MUST** require **multi-log consistency or an OOB-verified pin even in v0** (§13.7).
7. **Group handshake ordering is a metadata concentration point.** The per-group committer/ ordering channel (§5.1) necessarily sees all of a group's handshake traffic; this is an explicit exception to the "no single node sees both ends" framing, bounded by rotating the committer and keeping application traffic on the mixnet.
8. **redact / expires are unenforceable** against a non-compliant recipient that already holds the plaintext — they are cooperative hints, not guarantees (relevant to any "right to erasure"

framing).

DMTAP states these boundaries in-product. Honest, disclosed limits beat a false "perfectly anonymous."

6.7 Data at rest

- The node encrypts the mailbox, file blobs, and keys **at rest** under a device/identity key.
- Messages get MLS **forward secrecy**; files use **per-file keys** (blast radius = one file) delivered over the forward-secret channel. Persistent, re-readable data cannot ratchet like ephemeral messages — this is a property of files, not a DMTAP flaw (§5.5).
- **Access revocation on group-member removal (normative)**. MLS removal blocks a removed member's *future* messages but does NOT revoke file keys they already hold. Therefore, when a member is removed from a shared file folder (§5.5), the node **MUST re-key and re-encrypt** every file the removed member had access to **by default**, and redistribute the new keys to remaining members. **Skipping the re-key is an explicit, surfaced opt-out** (e.g. low-sensitivity shared media where the cost outweighs the benefit), never the silent default; folders marked **confidential** MUST always re-key (opt-out forbidden). Without a re-key, a removed member retains indefinite read access to already-shared files. Clients MUST surface that pre-removal copies a member already downloaded cannot be recalled (the un-share limit, § file-safety).
- Recovery custody (§1.4) governs how at-rest keys survive device loss.

7. The Legacy Gateway (optional)

The gateway is the **only** component that speaks SMTP and the **only** one not content-blind (the legacy leg is unavoidably plaintext). It is **optional** — a node with no legacy correspondents never uses one, and at full DMTAP adoption it is unnecessary. It MAY be the node binary run in `--gateway` mode by an operator with a reputable IP and a domain.

7.1 Responsibilities

- **Inbound** (legacy → DMTAP): act as MX for a domain, receive SMTP, translate to a MOTE, attest it, and deliver into the mesh.
- **Outbound** (DMTAP → legacy): accept a `mail` MOTE marked for a legacy address, translate to RFC 5322, DKIM-sign as the sender's domain via **delegated selectors**, and send via SMTP.
- Carry the one irreducible operational cost: **IP reputation** (warmup, feedback loops, block-list remediation, abuse handling).

The gateway is **stateless**: durability is punted to the edges (§7.4).

7.2 Inbound

1. Gmail connects to `def.com` MX = gateway; SMTP transaction.
2. Reject spam early, before DATA where possible (RBL/DNSBL, SPF/DMARC, greylisting, per-IP rate limits) — never accept the bulk of spam (§9).
3. Look up recipient key K for RCPT TO (via DNS/directory §3).
4. Wrap the RFC 5322 message into a MOTE (`kind=mail`), encrypt to K, and set an ATTESTATION: gateway signs "received via gateway G at T from <SMTP envelope>" **with a domain-anchored attestation key** (§7.2a), so the recipient can verify it arrived through a gateway genuinely authorized for the recipient's domain.
5. Deliver into the mesh to K (§4). If K's node is unreachable, return SMTP 4xx so the

SENDING server retries (durability punted to the legacy sender). Store nothing.

7.2a Attestation key binding (normative)

An attestation is worthless unless its signing key is provably bound to a gateway the domain actually authorized — otherwise any operator could forge "legitimate legacy origin" for a domain it does not serve. Therefore the domain **MUST** publish the gateway's **attestation public key**, analogous to DKIM's selector, in DNS (and MAY anchor it in KT):

```
<sel>._dmtap-gw.def.com. IN TXT "v=dmtapgwl; k=<attestation public key>"
```

Recipient nodes **MUST** verify an inbound MOTE's attestation signature against a key published under the recipient's own domain (or an explicitly trusted gateway set), **MUST** reject attestations that do not verify, and **MUST** mark accepted ones as *legacy-origin* (not end-to-end encrypted before the gateway). This upgrades the former "MAY verify" to a default-on check with a cryptographic anchor.

7.3 Outbound & DKIM delegation

1. node → gateway: a mail MOTE for bob@gmail.com (over the mesh, authenticated).
2. gateway translates MOTE → RFC 5322.
3. gateway DKIM-signs as the sender's domain using a DELEGATED selector: the domain owner publishes the gateway's DKIM public key at <selector>._domainkey.def.com – so the gateway signs AS def.com WITHOUT ever holding the user's DMTAP identity key.
4. gateway SMTPs to the destination MX, enforcing TLS via MTA-STS/DANE.
5. On failure, report to the node; the NODE retries (owns the queue). Store nothing.

DKIM delegation cleanly separates *deliverability reputation* (the gateway's) from *identity* (the user's key, never shared).

7.4 Statelessness & durability

- Inbound: unreachable recipient → SMTP 4xx → the **legacy sender** retries.
- Outbound: send failure → the **user's node** retries.
- The gateway holds no queue and no mailbox. Restart it and nodes/senders re-drive; nothing to recover or leak.

7.5 Decentralization & economics (summary; anti-abuse in §9)

- Many independent gateway operators MAY register in a directory with {pubkey, reputation, region, price, stake}; nodes select by reputation-to-destination.
- Per-identity accountability + operator stake keep a shared reputation pool safe to decentralize (§9). DKIM delegation makes operators swappable (change the delegated selector).
- Postage (§9) MAY fund outbound legacy sending (the stamp the sender attaches is redeemed by the delivering gateway), making it revenue-neutral and doubling as spam pricing.

7.6 Dual-stack addressing

A single abc@def.com is reachable both ways (§3.2): DMTAP senders resolve name→key and go native (mesh, no gateway); legacy senders use MX→gateway. Capability is discovered per-sender; no coordination is required, and native traffic grows while gateway traffic fades.

7.7 Fairness, self-host backstop & non-lock-in

DMTAP does not — and cannot — mandate that every gateway serve everyone. It instead makes **no gateway load-bearing**, so fair access is a structural property, not a rule requiring anyone's compliance.

You never need a specific gateway

- **DMTAP-native needs no gateway.** DMTAP delivery is key-based over the mesh (§4); a user with only DMTAP correspondents never invokes one.
- **Self-host is always available.** Any user with a reputable IP and a domain MAY run the gateway (§7) themselves and depend on no third party. This **self-host backstop** makes gateway access a *right* (you can always serve yourself), not a grant.

Why a universal-service mandate is infeasible

A protocol rule "every gateway MUST accept all traffic" is:

1. **Unenforceable** — no authority can compel a sovereign operator in a decentralized system; refusal or silent degradation cannot be prevented.
2. **Economically self-defeating** — a gateway forced to accept all traffic inherits the abuse that destroys its IP reputation, degrading the service for everyone. This is precisely why open SMTP relays died.

So DMTAP does not attempt it. Fairness is achieved by the four mechanisms below instead.

How fairness is achieved

1. **The self-host backstop** (above) — the guaranteed right to serve yourself.
2. **A competitive, swappable market.** DKIM delegation (§7.3) + key-identity (§1) mean **zero lock-in**: a user switches operators with a DNS/DKIM change and no data migration (the box is the authority). If one operator refuses, another serves; switching is free.
3. **The accountability layer makes open service viable.** Open relays failed because abuse was unattributable. DMTAP attributes every send to an anonymous-but-accountable ARC token + optional postage + operator stake (§9), so a gateway *can afford* to serve strangers openly — abuse is priced and per-token-bannable, not a reputation-destroying free-for-all. Openness is no longer suicidal.
4. **An optional commons gateway.** A non-profit or protocol-funded operator MAY commit to universal, non-discriminatory service (a "public option"), funded by postage/donations, as **one operator among many** — not a mandate on all. Reputation ratings (§7.5) reward open operators and down-rank discriminatory ones, applying soft, market-driven fairness pressure.

The result is stronger than a mandate could enforce: because you can always route around, self-serve, or switch — and because accountability makes genuinely-open gateways survivable — no operator can act as a gatekeeper, without any unenforceable obligation being imposed.

8. Client Access

The node is a multi-protocol front-end over one MOTE store. Every protocol is a *view* of the same mailbox. Client-facing protocols terminate **on the node**; the node is reached through the mesh (SNI/stream routing over the relay/mesh) so no static IP is needed.

8.1 JMAP (native)

- The node exposes **JMAP** (RFC 8620 / RFC 8621) over a local HTTP endpoint (and, for remote devices, over an authenticated mesh stream).
- JMAP is the modern sync surface: `query`, `get`, `set`, `changes`, `push`. It maps directly onto the MOTE store and the device-cluster CRDT (§5.6).
- New DMTAP-native clients SHOULD prefer JMAP + the native MOTE/MLS APIs, which expose DMTAP-only features (identity verification, postage, privacy tier, file manifests).

8.2 IMAP / POP / SMTP-submission (compatibility)

To let existing mail clients (Apple Mail, Outlook, Thunderbird, mutt) work unchanged:

- The node runs **IMAP**, **POP3**, and **SMTP-submission** servers locally, projecting the MOTE store as folders/flags and accepting outbound submissions.
- Reached through the mesh (SNI-passthrough / stream routing); the node terminates TLS and speaks the legacy protocol; the relay/mesh never decrypts.
- **Auth = app-passwords**: the node issues app-specific passwords mapped to the identity, so legacy clients authenticate without touching the keypair.
- **Encryption is transparent to the owner's own client**: the node decrypts MOTES and presents normal RFC 5322/MIME to the authenticated client. The E2E boundary is between *parties*, not between the owner and their own device.
- Outbound submission → the node converts to a MOTE (native) or routes to a gateway (legacy destination).

Compatibility support MAY be deprecated over time (like SMTP) as native clients mature; it is not required for conformance, but is RECOMMENDED for adoption.

8.3 Multi-device

Devices form the owner's personal MLS cluster (§5.6); each runs its own client surface and syncs the mailbox/flags/labels/file-index via encrypted CRDT over the mesh. The always-on node anchors receipt while other devices sleep.

8.4 Calendar & contacts (first-class, decentralized)

Calendar and contacts are **not** separate central services — they are additional **MOTE kinds** stored in the same node, end-to-end encrypted, synced across the device cluster (§5.6), and shared/invited via the same MLS groups (§5) as everything else. They inherit the full decentralized model: your node holds them, no provider can read them, and there is no central CalDAV/CardDAV server.

- **Native**: calendar events and contacts are represented as **JSCalendar (RFC 8984)** and **JSContact (RFC 9553)** MOTES, synced via **JMAP** (Calendars/Contacts) alongside mail (§8.1). Calendar invitations and scheduling (iTIP-style) ride as MOTES between participants — no central scheduling server; free/busy and RSVP are messages, not a server query.
- **Compatibility**: the node exposes **CalDAV (RFC 4791)** and **CardDAV (RFC 6352)** servers on `localhost` /over the mesh, projecting the calendar/contact MOTE store as iCalendar (RFC 5545) and vCard (RFC 6350) so existing clients (Apple Calendar/Contacts, Thunderbird, DAVx) work unchanged — reached through the mesh with app-passwords, exactly like IMAP (§8.2).

Compatibility DAV surfaces MAY be deprecated over time like the mail ones; native JMAP + MOTE is the forward path.

8.5 The decentralization invariant (all data classes)

Every data class DMTAP carries — **mail, chat, files, calendar, contacts, identity, and login (§13)** — obeys the same rule: it lives on the **user's node**, is **end-to-end encrypted**, syncs across the user's **device cluster**, shares via the same **MLS groups**, and routes over the same **mesh/mixnet** — with **no central server** for any of it. Legacy protocols (IMAP/POP/SMTP/CalDAV/CardDAV) and the OIDC bridge (§13.6) are **edge-compatible surfaces only**; they never become a central store or a required intermediary. The node is the authority for *everything*, uniformly. There is no data class that quietly depends on a central service.

Silent grants that would **redirect or delegate** that data — **auto-forward rules**, new **capability delegations** (§13.5), and new **RP-session authorizations** (§13.4) — **MUST** be surfaced to the owner's device cluster and logged to KT self-monitoring (§3.5), so a business-email-compromise-style silent redirect or delegation is owner-visible, not covert.

9. Anti-Abuse & Postage

DMTAP must stop spam/abuse **without** a central filter and **without** breaking sealed-sender anonymity. The tension: sealed sender hides who sent a message, but the recipient still needs to rate-limit and block abusers. The resolution is **anonymous but accountable** tokens.

9.1 Principles

1. **Authenticated-by-default.** Every native MOTE is authenticated to the recipient (payload signature, §2.4); there is no anonymous unauthenticated injection like open SMTP.
2. **Recipient policy is local.** The recipient node decides what to accept — allow known contacts, challenge strangers, block per-identity — *before* surfacing to the user (§2.7).
3. **Cost for cold contact.** Reaching a stranger costs something (a token, proof-of-work, or a paid stamp), so bulk unsolicited sending is uneconomic. Known contacts are free.
4. **Anonymity preserved.** Abuse control **MUST NOT** deanonymize the sender or link a sender across recipients.

9.2 Recipient policy

```
Policy {
  allow:      [* ContactRef],    // known keys / group members → free, always accept
  challenge:  ChallengeSpec,    // unknown senders must satisfy this
  block:      [* Token/KeyRef],  // per-identity or per-token blocks
  rate:       RateLimit,        // per-sender-token limits
}
ChallengeSpec = pow(bits) / token(issuer) / stamp(amount) / vouch(guardianSet)
```

A stranger's first MOTE **MUST** carry a valid challenge response or it is dropped/deferred to a "requests" area, never delivered to the inbox.

9.3 Anonymous rate-limit tokens (the core mechanism)

DMTAP uses **Privacy-Pass anonymous tokens** (RFC 9576 architecture, RFC 9577 HTTP scheme, RFC 9578 issuance) so a recipient can rate-limit/block a sender **without learning who they are** and **without the sender being linkable across recipients**.

CAUTION — **pick the right variant.** Vanilla Privacy Pass tokens are issuance-unlinkable but do **not**, alone, give the exact combination DMTAP needs: *per-recipient rate-limiting* AND *cross-recipient unlinkability*. That combination requires per-origin-scoped is-

suance — **Anonymous Rate-limited Credentials (ARC)**, the Privacy Pass WG work (`draft-yun-privacypass-arc`). DMTAP tokens **MUST** use ARC-style per-origin binding, not plain tokens, for the anonymity + accountability guarantee to hold simultaneously.

- A sender obtains blinded tokens from an **issuer** bound to a *rate budget*.
- The sender attaches a token to a cold MOTE; the recipient verifies it is valid and unspent, enforces per-token rate limits, and can **block a token/issuer** if it misbehaves — all without identity.
- Tokens are **unlinkable across recipients** (blind signatures), so token use does not build a cross-recipient graph.

9.3.1 Issuer trust is what gives a token value (normative)

A token is only as trustworthy as its **issuer**, and issuance **MUST** have a cost or scarcity or the whole "cost for cold contact" model collapses (a spammer would self-issue unlimited free tokens). Therefore:

- The recipient's policy scores issuers (parallel to gateway reputation, §7.5). A token's granted rate budget is a function of **issuer trust at the recipient**, not of the token alone.
- A token from an **unknown/unvetted issuer (including the sender's own node)** **carries a default rate budget of ZERO** — i.e. it counts as *no token*, forcing fallback to PoW (§9.4) or postage (§9.5). Self-issuance thus buys anonymity but **no cost relief**.
- Trusted issuers (a recipient's own provider, a staked gateway, a vouched community issuer) make issuance costly/rate-limited on their side and stake reputation on not minting for abusers; a recipient extends budget to their tokens.

This is the load-bearing rule that makes ARC anti-abuse real rather than bypassable.

9.3.2 The unlinkability collective-defense tension (honest)

ARC's cross-recipient unlinkability is in **direct tension** with identifying a repeat spammer who hits many recipients: the very property that protects a legitimate sender's social graph also prevents recipients from cross-linking an abuser. DMTAP resolves this at the **issuer layer, not the recipient layer**: repeat abuse is bounded by the issuer throttling/revoking its own issuance to a misbehaving client (who is *not* anonymous to its own issuer), and by recipients down-scoring issuers that mint for abusers. Recipient-side cross-recipient linking is deliberately unavailable; DMTAP does not claim it. (Signal solves the analogous problem with delivery tokens; DMTAP generalizes it to open, issuer-scored anonymous tokens.)

9.4 Proof-of-work (fallback)

Where no token issuer is available, a stranger **MAY** attach a **proof-of-work** (a hashcash-style puzzle over the MOTE `id` + recipient + date) whose difficulty the recipient policy sets. PoW imposes cost on bulk senders while remaining trivial for a single legitimate message.

CAUTION. PoW is a **last-resort** tier, not primary: there is no live standard, and plain hashcash asymmetry favors organized spammers (botnets/GPUs) over low-power legitimate senders (phones). DMTAP PoW **MUST** use a **memory-hard** function (Argon2/scrypt-style) to flatten that asymmetry, and difficulty **SHOULD** be adaptive. Prefer the token/reputation path (§9.3) whenever available.

9.5 Postage (economic tier + gateway funding)

Postage is a signed, prepaid, real-money credit voucher (NOT a cryptocurrency):

- A sender **MAY** attach postage to a cold MOTE; heavy senders pay, contacts are free.

- **Postage doubles as the gateway fee:** when a MOTE is bridged to legacy (§7), the delivering gateway **redeems the stamp**, so outbound legacy sending is paid-for and revenue-neutral.
- Postage is enforced by the recipient/gateway; it is orthogonal to token/PoW anti-spam and MAY be combined.

9.5.1 Issuance, redemption & double-spend (normative)

Postage is a **bearer instrument for real money**, so it needs a settlement model, not just a signature. DMTAP specifies the minimum:

- **Issuance:** a postage **issuer** (a provider/gateway with a real-money float) mints a stamp = `sign_issuer(serial, amount, expiry, [audience])`. The issuer debits the buyer's account at mint time. `audience` MAY scope a stamp to a specific recipient/gateway.
- **Redemption + double-spend prevention:** a stamp is single-use. The **redeeming party** (recipient node or gateway) **MUST check the stamp's `serial` against the issuer's redemption endpoint** (online check) or accept the issuer's signed short-lived spent-list; the issuer marks the serial spent atomically. A stamp presented twice is rejected on the second redeem.
- **Cross-operator settlement:** the redeemer claims the amount from the issuer against the serial; issuers reconcile out-of-band (standard interchange). "Trivial" was an overstatement — it requires an issuer-side ledger, specified here.
- **Failure mode:** if the issuer is unreachable at redemption, the stamp is treated as *unverified* (falls back to token/PoW policy), never accepted on faith. No offline bearer acceptance of real money.

Postage issuers are subject to the same reputation/trust scoring as token issuers (§9.3.1); an untrusted issuer's stamps carry no weight.

9.6 Gateway-operator accountability

For decentralized legacy gateways (§7):

- **Per-identity accountability:** every outbound handoff carries the sender's signature + postage, so a gateway attributes abuse to a *token/identity*, not an IP — making a shared reputation pool safe to decentralize.
- **Operator stake/bond:** operators post collateral; poisoning the pool slashes it (Sybil resistance + incentive alignment).
- **Reputation routing:** nodes route outbound to operators by measured deliverability; bad operators lose traffic.

9.7 Vouch / introduction (optional)

A cold sender MAY present a **vouch** — an introduction token signed by a contact the recipient trusts (a mutual connection) — to bypass PoW/stamp. This bootstraps first contact through the social graph without a central authority, and MUST itself be rate-limited to prevent vouch farming.

9.8 Mixnet abuse

Mix nodes are content-blind and cannot filter content. Abuse of the mixnet itself (flooding) is bounded by: token/PoW admission at the *entry* (a MOTE without valid postage/token is not accepted into the mixnet by the sender's own entry policy), operator rate limits, and stake for mix operators (§6.4, §7.5). Cover traffic is rate-bounded per node.

9.9 Group-address amplification

A **group address** (§5.8) is a first-class abuse-amplification vector: a single post fans out to N recipients, and a naïve design evaluates the anti-abuse challenge **once at list ingress** and then attributes delivery to the **group**, bypassing each recipient's per-sender policy (§9.2) and **laundering the poster's accountability**. DMTAP forbids this:

- **Origin accountability carries through fan-out — with the right proof per list type.** The **poster's** challenge proof **MUST** be carried on each fanned-out per-member delivery; each recipient applies its per-sender policy to the **original poster**, not the group identity. *Which* proof depends on the list's membership model, because ARC tokens are per-origin (per-recipient) scoped and cross-recipient-unlinkable (§9.3) and therefore **do not compose** with fan-out to many recipients:
 - **Member-visible channels** (§5.8.3): the poster knows the members, so it **MAY** mint a per-member ARC token (one per recipient origin). ARC carry-through applies here.
 - **Hidden-membership lists** (§5.8.3): the poster does *not* know the members (that's the point), so per-member ARC is impossible without breaking membership privacy. These lists **MUST** use **postage or PoW scoped to the list address** as the poster's proof (a single list-scoped proof the committer/relay verifies at ingress and vouches per-delivery), not per-member ARC. The committer attests "this fan-out carried a valid list-scoped proof from poster P" so recipients still get origin accountability without learning a cross-recipient ARC graph. **Honest limit:** this hands the hidden-list committer a trust power recipients must accept — it could false-vouch (launder a spammer), misattribute (frame a poster or evade a recipient's block on P), or under-size the "commensurate" proof. It is bounded (the hidden list's committer *is* the list operator, which already re-seals the fan-out and knows the full membership — roughly the mailing-list trust you already accept), and abuse is detectable (recipients down-score a committer/list that vouches for spam, §7.5). Member-visible channels avoid this power entirely (each recipient verifies ARC independently), so hidden-membership trades a committer-trust assumption for its privacy — disclose it, don't hide it.
- **Per-poster fan-out rate-limits.** A list **MUST** rate-limit fan-out **per poster** and **cap amplification** for **open**-join lists (anyone-can-post), whose amplification is otherwise unbounded.
- **Cost to post to large lists.** Posting to a large list **MUST** require **postage or PoW** commensurate with the fan-out size, so mass amplification is not free.
- **Legacy fan-out** (§5.8.5) is bound by the same rules; the gateway attributes the origin (§9.6).

10. Versioning, Conformance, Governance

10.1 Protocol versioning

- Every wire object carries a format version (`v`) and algorithm suite (`suite`).
- Unknown `v` / `suite` **MUST** be rejected (fail closed), never guessed.
- New message kinds use the reserved `0x40–0x7f` range (§2.3); a node ignores unknown kinds it is not required to process, but **MUST NOT** ack a kind it cannot validate.

10.2 Capability negotiation (dual-stack)

- A sender discovers a recipient's capabilities via the recipient's **Identity** /DNS record (native DMTAP vs legacy-only) and picks the path per-recipient (§3, §7.6).

- **system** MOTES (kind `0x0a`) carry capability announcements between nodes (supported suites, privacy tiers, MLS ciphersuites, extensions).
- **Opportunistic upgrade**: a legacy correspondent is reached via the gateway; if they later publish a DMTAP key, native delivery is used automatically. No flag day.

10.3 Conformance levels

An implementation is **DMTAP-conformant** at a level if it passes the corresponding suite:

Level	Requires
Core	Identity (§1), MOTE (§2), naming resolution + TOFU + v0-minimal KT publish/verify
Private	Core + mixnet + sealed sender + cover traffic + privacy tiers (§4, §6)
Groups & Files	Core + MLS groups + content-addressed file transfer (§5)
Legacy	Core + gateway inbound/outbound + DKIM delegation (§7)
Clients	Core + JMAP; IMAP/POP/SMTP-submission compat RECOMMENDED (§8)
Auth	Core + DMTAP-Auth login ceremony with origin binding + key-bound sessions (§13); O

The **conformance test suite** (in `conformance/`, TBD) is the *operational definition* of compatibility. "DMTAP-compatible" means "passes the suite," not "resembles the reference." This is the primary defense against fragmentation.

10.4 The spec is authoritative

Independent implementations **MUST** be buildable from this specification alone. The Rust reference in `node/` and `gateway/` is a proof and a set of libraries, **not** normative. Where reference and spec disagree, the spec governs (or the discrepancy is filed as a bug).

10.5 Governance & licensing (intent)

- **Standards track**: pursue an **IETF Internet-Draft** for the wire protocol and object formats, aiming for RFC status (as JMAP and MLS did). Neutral governance is what lets competitors adopt without fearing capture.
- **Licensing**: the **specification** and the **reference implementation** (node, gateway, client, libraries) ship under the **MIT license** (Apache-2.0 dual-licensing under consideration for its explicit patent grant). Everything a user touches and everything trust depends on is open; permissive licensing maximizes adoption by any party, competitors included.
- **Open software + paid operations**: commercial sustainability comes from a thin, **private control-plane** (a hosted operator) that implements the **operator seam** (`crates/dmtap-seam`) to bill *operations* — never gating any protocol or privacy feature. See §12 for the full model and the inviolable rule (privacy/crypto are never behind the seam).
- **Reuse over reinvention**: DMTAP deliberately composes existing standards — MLS (RFC 9420), HPKE (RFC 9180), JMAP (RFC 8620/8621), libp2p, DNS/DNSSEC/SVCB, key transparency, Privacy Pass. The novelty is the *composition and transport*, not new cryptography.

10.6 Roadmap markers (non-normative)

- **v0**: Core + Private (minimal KT via TOFU+pinning) + Groups & Files + Legacy gateway.
- **v1 hardening**: full CONIKS/KT with monitoring + gossip; onion-routed bulk; anonymous tokens at scale; PQ suite `0x02` migration; optional self-sovereign naming backend.
- **Later research**: stronger private contact discovery; scalable private retrieval for hostile-buffer scenarios; deniable-group properties; metadata-privacy for very large files.

11. Grounding & References

This section records the standards each design choice rests on, the corrections applied after verification, and a consolidated statement of honest limits. DMTAP deliberately **composes existing standards**; the novelty is the composition and transport, not new cryptography.

11.1 Verified building blocks

Layer	Choice	Standard / source
PKE	HPKE	RFC 9180 (DHK)
Signatures	Ed25519 (PureEdDSA)	RFC 8032
Key agreement	X25519	RFC 7748
PQ KEM	ML-KEM-768 (cat-3)	FIPS 203
PQ signatures	ML-DSA-65 (cat-3)	FIPS 204 (SLH-1)
Hybrid HPKE KEM	X-Wing (ML-KEM-768 X25519)	draft-connolly-c
Hashing	BLAKE3-256 + agility prefix	(BLAKE3 not F)
Serialization	Deterministic CBOR (byte-wise/core key order)	RFC 8949 §4.2 (
Group/session	MLS	RFC 9420; archi
PQ-MLS	ML-KEM ciphersuites + combiner	draft-ietf-mls-p
Async join	MLS KeyPackages + external commits	RFC 9420
Sealed sender	payload-embedded sender	Signal (2018)
Mixnet packet	Sphinx	Danezis & Goldb
Mixnet design	Loopix (Poisson mix, loop/drop/mix cover)	Piotrowska et al
Client sync	JMAP	RFC 8620 / RFC
Mesh	libp2p (Kad, Relay v2, DCUtR, AutoNAT v2, Noise, QUIC)	libp2p specs
Location record	IPNS-style signed value record	IPFS/IPNS
Naming	DNS TXT (key) + SVCB (service)	RFC 9460; cf. D
Key transparency	Merkle log + STH + inclusion/consistency/absence	CT RFC 6962; C
Anti-abuse tokens	Privacy Pass + ARC scoping	RFC 9576/9577/
PoW fallback	memory-hard hashcash	Back 1997; Argo
Recovery	VSS + SLIP-0039 + FROST	Feldman/Peders
Self-sovereign naming (opt)	name-chain (ENS-style)	Zooko's Triangle

11.2 Corrections applied after verification

1. **MLS handshake ordering** — Commit/Proposal/Welcome **MUST** use an ordered channel, **not** the reordering mixnet (§5.1). This is the hardest part of MLS-over-mesh; track [draft-kohbrok-mls-dmls](#).
2. **Async init** — use MLS-native **KeyPackages** + **external commits**, not a bolted-on PQXDH (§5.3).
3. **Deniability** — MLS is **non-repudiable**; DMTAP does **not** claim message deniability and does not swap MLS signatures for MACs (§5.2). Open design item.
4. **No-PIR claim** — reframed honestly: push delivery avoids PIR's problem but leaves last-hop visibility, receipt-timing, and intersection exposure — mitigated by node/identity decoupling + recipient-side cover traffic (§6.4).
5. **DHT security** — record signing eclipse resistance; added S/Kademlia disjoint paths, IP-diversity buckets, seq#/EOL rollback defense, and a **non-DHT rendezvous fallback**; the DHT is one discovery mechanism, not the root of trust (§4.2).
6. **QUIC roaming** — carried by location re-publish + re-dial, not seamless QUIC migration ([quinn](#) has no multipath) (§4.1).

7. **Sealed sender scope** — hides sender from intermediaries, not the IP (mixnet does that); metadata-*reduction*, not elimination (NDSS 2021 side-channel) (§6.2).
8. **Bulk-file tier** — explicitly weaker; Tor-style onion routing is subject to end-to-end correlation; swarming/padding raise cost, not guarantees (§6.5).
9. **Recovery** — VSS over plain Shamir; FROST to avoid key reassembly; SLIP-0039 encoding; "redistribution" vs "proactive refresh" distinguished (§1.4).
10. **Anti-abuse tokens** — need **ARC** (per-origin rate-limited + cross-recipient unlinkable), not vanilla Privacy Pass; PoW must be **memory-hard**, last-resort (§9).
11. **Hash agility** — content-address digests carry a multihash-style prefix (§2.2).
12. **DNS is discovery, not trust** — a DNS binding is only trustworthy via KT; SVCB is for service params, a dedicated TXT for the key (§3).
13. **DTN / radio** — store-and-forward is a DMTAP-built layer (not libp2p); Bluetooth/Wi-Fi Direct/LoRa are future work, not v0 (§4.8).

11.3 Consolidated honest limits

- **Global active adversary** with unlimited resources is not fully defeated — the **Anonymity Trilemma** (Das et al., IEEE S&P 2018) makes strong anonymity cost latency and/or bandwidth. DMTAP targets a **global passive adversary** and *bounds* (not eliminates) partial-active exposure.
- **Intersection / statistical-disclosure attacks** against always-on nodes are inherited; cover traffic bounds, does not eliminate, them.
- **First-contact MITM** is possible until KT gossip or out-of-band verification (§3.4).
- **KT without gossip** catches later tampering and enables self-monitoring, but a single equivocating log is not caught until independent auditors/gossip exist (§3.5).
- **Large-file bulk** metadata is weaker than message metadata (Tor-class), by necessity.
- **Persistent-file forward secrecy** cannot match ephemeral-message FS (files must stay readable); mitigated by per-file keys + FS-delivered keys + at-rest encryption (§6.7).
- **Key loss** of **IK** plus all recovery factors is unrecoverable (the bottom turtle, §1.4).
- **PQ onion (Sphinx)** as published is not PQ-safe; PQ mix packet formats are open research.

11.4 Implementation-stack notes (reference, Rust)

- **Prefer formally-verified PQ crates** (`libcrux-ml-kem`, `libcrux-ml-dsa`) over the unaudited RustCrypto `ml-kem` / `ml-dsa`; pin past the `ml-dsa` advisory GHSA-5x2r-hc65-25f9.
- **openmls** — independently audited (SRLabs, 2025) — is the MLS choice.
- **hpke** (rozbb) or **hpke-rs** (Cryspen) — not `ed25519-dalek-hpke` (experimental).
- **ed25519-dalek** $\geq 2.x$, **x25519-dalek**, **blake3**, **ciborium** — mature.
- **Mixnet**: `sphinx-packet` / `nym-sphinx-*` are low-cadence standalone; track Nym directly.
- **rust-libp2p** is production-usable but churns its API (pin versions); `go-libp2p` is the more mature interop reference — a deliberate memory-safety-vs-maturity trade.

11.5 Selected bibliography

- Danezis & Goldberg, *Sphinx: A Compact and Provably Secure Mix Format*, IEEE S&P 2009.
- Piotrowska, Hayes, Elahi, Meiser, Danezis, *The Loopix Anonymity System*, USENIX Sec 2017.
- Das, Meiser, Mohammadi, Kate, *Anonymity Trilemma*, IEEE S&P 2018.
- van den Hooff et al., *Vuvuzela*, SOSP 2015; Angel & Setty, *Pung*, OSDI 2016; Cheng et al., *Talek*, ACSAC 2020.

- Dingledine, Mathewson, Syverson, *Tor*, USENIX Sec 2004; Murdoch & Danezis, *Low-Cost Traffic Analysis of Tor*, IEEE S&P 2005.
- Melara et al., *CONIKS*, USENIX Sec 2015; RFC 6962 (Certificate Transparency).
- Martiny et al., *Improving Signal's Sealed Sender*, NDSS 2021.
- RFCs 9180, 8032, 7748, 8949, 9420, 9750, 8620, 8621, 9460, 6376, 7929, 9576/9577/9578, 9591.
- FIPS 203, 204, 205; `draft-connolly-cfrg-xwing-kem`; `draft-ietf-mls-pq-ciphersuites`; `draft-ietf-mls-combiner`; `draft-kohbrok-mls-dmls`; `draft-yun-privacypass-arc`.

12. Operators, the Seam & Sustainability

DMTAP is deployed in one of two modes, and the boundary between them is a small, explicit **operator seam**. This section is partly *informative* (business/licensing intent) and partly *normative* (the inviolable rule and the seam's guarantees).

12.1 Two deployment modes

Mode	Who runs it	Cost	Limits
Self-host	The user, on their own box	\$0	none — fully functional, unrestricted
Hosted operator	A provider (e.g. a commercial control-plane)	paid	plan quotas on <i>operations</i> only

The OSS is a **complete product on its own**. Self-host is not a crippled tier: it has every protocol, client, and privacy feature. A hosted operator adds *convenience* (someone else runs the box, warms the IPs, manages the domain), not *capability*.

12.2 The operator seam

The seam is the clean boundary the OSS exposes so an operator can add billing and multi-tenant management **without forking or patching the protocol**. It is four capabilities (reference crate: `crates/dmtap-seam`, contract: its `CONTRACT.md`):

- **Metering** — the OSS emits usage events at the real cost centers (gateway egress, storage, relay bytes, message counts, vanity domains).
- **Provisioning** — create/suspend accounts and @-addresses across onboarding tiers A/B/C (§3.8).
- **Policy** — per-account quotas/entitlements on operations (storage caps, send caps, domain counts, rate limits).
- **GatewayAuthz** — authorize legacy egress with per-identity-token accountability (§9), preserving sealed sender.

Each capability has a **self-host default that is unlimited/no-op**, so with no operator the OSS runs unrestricted. A hosted operator implements the same capabilities — in-process (Rust traits) or out-of-process (HTTP/events) — to add billing and quotas.

Fail-open to function, fail-safe on security (MUST): if the operator is unreachable, the OSS MUST NOT break user-facing mail/chat/files. Metering queues and retries (usage may under-count during an outage), and quota **Policy** falls back to allow (a billing concern, not a security one). **GatewayAuthz is different and MUST NOT fail open to "allow."** GatewayAuthz is a security control (§9): the accountability it enforces is exactly what prevents unattributable legacy egress — the open-relay failure mode §7.7 exists to avoid. On operator-unreachable, GatewayAuthz MUST fall back to a **safe default**: permit legacy egress only to **already-established contacts** (and to senders carrying valid self-contained proof —

postage/PoW verifiable without the operator), and **deny cold/unproven legacy egress** for the outage window. A generic "fail-open to allow" for this capability is prohibited.

12.3 The inviolable rule (normative)

Privacy, cryptography, metadata privacy, and recovery **MUST NEVER** be behind the seam or a paywall. There **MUST** be no seam hook, quota, or plan gate capable of disabling encryption, weakening the mixnet, reducing metadata privacy, or denying a user access to their own keys or mailbox. The seam meters and limits **operations and organizational concerns only**. Premium is for *running it*, never for *protection*.

A conformant operator implementation **MUST NOT** expose any control that violates this rule; a conformant OSS build **MUST NOT** consult the seam for any privacy/crypto decision.

12.4 Business & licensing model (informative)

DMTAP follows an **open-software + paid-operations** model — the GitLab-style split done cleanly, without a crippled free tier:

- **Everything a user touches, and everything trust depends on, is open** — protocol, spec, node, gateway, client, crypto. Shipped under the **MIT license** (Apache-2.0 dual-licensing is under consideration for its explicit patent grant, relevant to novel mechanisms such as anti-abuse postage/tokens). Libraries other implementations embed are permissively licensed to maximize adoption.
- **The paid layer is a thin, private control-plane** (a *hosted operator*) that implements the seam contract and bills **operations**: hosted nodes, storage, legacy egress / IP reputation, vanity domains, org/SLA. It withholds no protocol, client, or privacy feature.
- **The moat is reputation + network + being the best operator**, not the code. Because openness is what lets the network grow large enough to be worth hosting, full openness costs little and is itself a trust requirement for a privacy product (a closed client cannot credibly claim "we can't read your mail"). Competitors hosting DMTAP grow the network the operator is central to.

12.5 Honest limits

- **Free-rider economics**: paying (convenience) customers subsidize free self-hosters. Intended and viable (the convenience majority pays; the sovereign minority self-hosts), provided the paid tier genuinely covers cost + margin.
- **Commoditization**: open software means thin margins on the software itself; margin comes from operations and scale, not licenses.
- **No protocol control**: required for adoption, but it means an operator can be out-executed — the defense is execution and trust, not a license.
- **The open-core temptation recurs**: pressure to move "just one" feature behind the paywall will appear. The inviolable rule (§12.3) is the bright line; drift there is fatal to the brand and is prohibited for privacy/crypto features.

13. DMTAP-Auth — Decentralized Identity & Login

Your DMTAP identity is a keypair (§1) with a human name resolvable to that key (§3). The same identity that receives your mail can log you in **everywhere**, with **no central identity provider**. This section specifies DMTAP-Auth: sovereign, decentralized web login built on the identity and naming layers, plus a bridge so existing "Sign in with OIDC" apps work unchanged.

The governing principle mirrors the rest of DMTAP:

Your key is your identity — for mail, for messaging, and for login. No provider can revoke it, surveil it, or lock you out. Your node is your identity provider.

13.1 Goals & non-goals

DMTAP-Auth MUST provide:

1. **Decentralized login** — a relying party (RP) authenticates you as `alice@yourdomain` by verifying a signature from your key; there is no Google/Okta in the middle.
2. **Phishing resistance** — a signature obtained by a look-alike site MUST NOT authenticate to the real site.
3. **No bearer tokens after login (native path)** — native sessions are bound to your key (proof-of-possession), so a stolen token is useless without the key. (The legacy OIDC *bridge* path yields a classical bearer token and forfeits this — see §13.4, §13.6, §13.7.)
4. **Legacy interoperability** — existing OIDC/OAuth relying parties can consume DMTAP-Auth through a compatibility bridge.
5. **Recoverable & revocable** — losing a device does not lose your identity (§1.4), and a single app/device session can be revoked without rotating your whole identity.

Non-goals: replacing WebAuthn/passkeys (DMTAP-Auth *uses* them); acting as an authorization server for arbitrary third-party API scopes beyond login + basic profile + delegated capabilities (§13.5).

13.2 Substrate reuse

DMTAP-Auth introduces almost no new cryptography — it reuses:

- **Identity** (§1): root key `IK`, device subkeys, the signed `Identity` object.
- **Naming** (§3): `name` → `key` via DNS + key transparency — this *is* issuer discovery.
- **Recovery** (§1.4): because your key now guards *all* your logins, recovery is even more load-bearing; DMTAP-Auth inherits it directly.
- **Transparency log** (§3.5): device/session authorizations and revocations are logged, so unauthorized login grants are detectable (§13.4).

13.3 The native login ceremony

1. RP shows "Sign in with DMTAP"; user enters `alice@yourdomain`
2. RP resolves `name` → `key` + auth endpoint (§3 lookup; DID/OIDC discovery, §13.6)
3. RP creates a Challenge:


```
{ rp_origin, nonce, issued_at, exp, aud, [scope] }
```
4. The challenge is presented to a TRUSTED CLIENT on the user's side (browser/OS/app), which binds and displays the VERIFIED `rp_origin`, and runs a WebAuthn/passkey user-verification ceremony (§13.3.1). BEFORE signing, the client generates a fresh per-RP, per-device SESSION keypair (§13.4) and computes `cnf = H(session_pubkey)`.
5. The user's key signs `H(rp_origin || nonce || issued_at || exp || aud || cnf)` (canonical, §2)
6. RP verifies the signature against `alice`'s pinned key (§3.4), that `rp_origin ==` its own origin, nonce unused, not expired → authenticated, and binds the session **ONLY** to the key named by `cnf` (proof-of-possession, §13.4)

The signed statement is a structured, origin-scoped, nonce-bound challenge (the SIWE/CAIP-122 pattern, hardened — see §13.7). The `aud` field binds the assertion to the intended RP. The `cnf` field (confirmation key, RFC 7800 style) binds the assertion to the session key the client generated *before* signing, so a captured assertion — including one seen by the

bridge (§13.6) — **cannot be replayed with an attacker-chosen session key** (session-hijack defense). The RP MUST bind the session to `cnf` and to nothing else.

13.3.1 Origin binding is the load-bearing property (normative)

Phishing resistance comes **entirely** from binding the assertion to the *true* RP origin, and that binding MUST be injected and enforced by a **trusted client component on the user's side**, never by the signer trusting a value handed to it by the RP.

- In a browser, this is **WebAuthn** (W3C Rec L2 / CR L3): the browser writes the *observed* origin into `clientDataJSON`, the authenticator signs over its hash, and credentials are scoped by `rpId`. An assertion produced at `alice-yourdomain.evil.com` cannot validate for `yourdomain`. DMTAP-Auth's browser ceremony MUST use WebAuthn for exactly this.
- **The remote-node hazard (CRITICAL)**. If the user's *always-on node* signs challenges that arbitrary parties relay to it, origin binding **evaporates** — a phisher relays the real challenge, gets the node to sign, and replays it. FIDO2 cross-device auth (CTAP 2.2 "hybrid") mitigates the analogous remote-approval risk with **BLE proximity**, which a remote node cannot use. Therefore DMTAP-Auth REQUIRES, for any node-signed login:
 1. the challenge carries `rp_origin` and `aud`, and the node signs *over them*;
 2. a trusted approval surface (the node's own authenticated client/app, or a paired passkey) **displays the verified `rp_origin`** to the human and requires explicit approval per login;
 3. the node MUST bind the challenge to a **pending, user-initiated login intent** — a nonce the node *itself* minted at the moment the user actively **started** a login on the node's own authenticated client. This is what "authenticated request channel" means: the node only signs a challenge that **matches an intent it originated a request for**. An unsolicited challenge **relayed** by a third party has no matching pending intent and MUST be refused — there is nothing for it to match. The node MUST rate-limit and log approvals (consent-farming defense).

Preferred — and, for a remote node, the **only permitted** — design: the **passkey/WebAuthn ceremony happens in the user's client**, and the node's key is only invoked *after* a successful local user-verification bound to the origin — i.e. the passkey gates the node's signature. Concretely, use the **WebAuthn PRF extension** (over CTAP2 `hmac-secret`) so the passkey deterministically derives the key that unlocks the node's signing key — the identity key never leaves the node and never touches the RP (the deployed `wwWallet` pattern). A **bare node-signed mode** — the node signing on its own authority, without a local user-verification bound to the origin — is **FORBIDDEN** for any remote node. Nodes MUST NOT offer "approve any challenge" modes.

Users without a passkey (availability, normative). The load-bearing requirement is a *trusted client that enforces origin-binding + intent-matching + user-verification* — a FIDO passkey is the strongest such client but not the only permitted one. A user with no passkey/PRF authenticator MAY instead use an **authenticated paired companion client** (their own signed-in Enviro app on a device paired to the node, §5.6) as the trusted client: it originates the login intent, performs user verification (biometric/PIN), and only then authorizes the node's signature.

Honest calibration (normative — do not overclaim). The companion path preserves **intent-matching** (a relayed unsolicited challenge has no matching pending intent and is refused), so it keeps the H1 relay hole closed. But it is **NOT equivalent to a passkey**: a WebAuthn passkey binds the *machine-observed* origin (the browser writes the actual navigated origin into `clientDataJSON` and enforces `rpId` scoping), whereas the companion receives

`rp_origin` as *data* and can only *display* it — degrading origin verification to the **user-verified mode that §13.7 limit 1 calls weaker**, leaving residual look-alike/homograph-origin exposure. To restore machine-enforced comparison, a companion client **MUST TOFU-pin the RP's origin on first login and fail closed on any origin mismatch thereafter**, so an established RP cannot later be confused with a look-alike. Passkey (machine origin binding) remains the RECOMMENDED mode; the companion path is the acceptable fallback with this stated, weaker-against-look-alikes property. What is forbidden is *any* mode with no trusted client enforcing intent — never the absence of a FIDO passkey specifically.

13.4 Sessions: key-bound, not bearer

After login, the RP and client establish a session that is **sender-constrained to the user's key**, so a leaked token cannot be replayed by a thief:

- Sessions **SHOULD** use **DPoP (RFC 9449)** — each request carries a fresh proof-of-possession JWT signed by a session key bound at login — or **GNAP (RFC 9635)** continuation, which is key-based end to end.
- Session keys are **per-RP, per-device** ephemeral keys authorized by a device key (§1.2), not `IK` itself. This limits blast radius and enables granular revocation.
- **Revocation**: revoking one app/device session publishes a revocation to the transparency log and/or a short-lived status endpoint; it **MUST NOT** require rotating `IK`. Rotating a device key (§1.5) revokes all its sessions at once. Losing `IK` and recovering (§1.4) **MUST** invalidate all prior session authorizations.
- **Bounded re-validation (normative)**. Because an RP validates the login-time delegation **once**, `IK` rotation (§1.5), device revocation, or recovery (§1.4) would not otherwise reach a live RP session. Therefore session authorizations **MUST** be **short-lived** (DMTAP-Auth session TTL + idle-timeout, §16), and an RP **MUST NOT** treat a login-time delegation as valid indefinitely: it **MUST re-validate the delegation** against the user's status endpoint or KT head at a **bounded interval** (RP re-validation interval, §16), or bind the session to a **revocation-list epoch** it **MUST** refresh at that interval. A session whose delegation no longer validates **MUST** be terminated at the next check.
- **On unreachable status/KT (normative, sensible default)**. If the status endpoint / KT head is unreachable at a re-validation check, the RP **MUST NOT** honor the session indefinitely (fail-open would let an attacker who partitions the endpoint keep a revoked session alive) and **SHOULD NOT** hard-fail instantly (that would log everyone out on a transient outage). Instead the RP **MUST** honor the last successfully-validated delegation only until a bounded **grace window** (= 2× the re-validation interval, §16), then fail closed and require re-authentication. This bounds an attacker's post-revocation persistence to the grace window while tolerating brief outages — mirroring §3.3's fail-closed-on-unreachable-KT stance.
- **Bridge path caveat (honest)**. These key-bound / proof-of-possession guarantees hold on the **native** path. A **bridged** login (§13.6) mints a classical OIDC ID Token — a *bearer* token — so the “stolen token is useless without the key” property (goal 3) is **forfeited on the bridge path**: it inherits classical OIDC bearer-token risk. Only the native path is key-bound end to end. Implementers **MUST NOT** assume proof-of-possession on bridged sessions.

13.5 Capabilities & delegation

For “let this app act on my behalf” (beyond login), DMTAP-Auth uses **capability delegation** rather than opaque scopes: a signed, offline-verifiable, attenuable capability token (**UCAN-style**, informative) delegates a *specific, least-privilege* right (e.g. “read calendar MOTEs for

24h”) from `IK` /device key to an app or another of the user's nodes. Delegations are attenuable (a device can only sub-delegate a subset), time-bound, and revocable. This also carries authorization **across the user's own device cluster** (§5.6).

Owner-visible grants (BEC defense, normative). A new capability delegation, a new RP-session authorization (§13.4), and any **auto-forward** / **redirection rule** change **MUST** be routed through the owner's **device-cluster notification + KT self-monitoring path** (§3.5), exactly like identity events (§1.4) — so a **silent grant** an attacker installs (the classic business-email- compromise move: quietly delegating access or auto-forwarding mail) is **visible to the owner's other devices** and alertable. Silent, unlogged authorization is prohibited.

13.6 Legacy bridge: OIDC / OAuth compatibility

Existing apps speak “Sign in with Google/OIDC,” not DMTAP-Auth. The bridge makes DMTAP identity consumable by them — the same native-path/legacy-bridge duality as mail (§7):

- **Per-user issuer (native-ish).** OpenID Connect Discovery 1.0 (Final) permits WebFinger-based **issuer discovery** from a user identifier, with the issuer at any host. So `alice@yourdomain` can advertise her **own node as her OIDC issuer**, self-issuing ID Tokens (the **SIOP v2** shape — a draft, not final). Deployed precedent: **Solid-OIDC** (user-controlled OPs; note it trusts a WebID-named *issuer*, it does not embed keys) and **IndieAuth** (identity = a URL you own; a W3C Note + IndieWeb living standard, not a Recommendation — cite the living standard). DMTAP-Auth expresses the binding as **did:web rooted at the user's mail domain** (`did:web:yourdomain:users:alice` → `did.json`), the DID method that matches §3's DNS name→key. The `did.json` **MUST** be **byte-consistent with the §3 DNS name → key binding and its KT entry** (same `IK`, same current Identity hash); an RP **MUST cross-check the did.json key against DNS + KT and pin it** (§3.4), never trusting the DID document alone — it is the same discovery-not-proof pointer as DNS (§3.2).
- **Hosted bridge (compat).** Because mainstream RP libraries assume a *fixed* issuer allowlist and rarely implement WebFinger discovery or dynamic client registration, DMTAP-Auth defines a **bridge OIDC Provider**: a standard, well-known OP whose backend is DMTAP-Auth. Legacy RPs add it like any OIDC provider; the bridge performs the §13.3 ceremony and mints a standard ID Token. **The bridge signs that ID Token with its OWN key, so a compromised bridge could forge logins for its RPs exactly as any classical IdP could.** DMTAP-Auth bounds this two ways: (a) the minted ID Token **MUST embed the user's own §13.3 signed assertion** (and its `cnf`) as a verifiable claim, so an RP **MAY additionally verify the user's key directly** and stop trusting the bridge's signature alone; (b) every token the bridge mints **MUST** be appended to a **bridge-transparency log** (KT-style, §3.5) that the user's node **monitors**, so a bridge minting a login the user never performed is **detectable** (the same intrusion-detection posture as identity KT). The bridge is a *convenience operator*, swappable and self-hostable — it sees login events but never the user's key (it verifies signatures), and it is content-blind to mail. **Honest framing:** a bridge-mediated login trusts the bridge **to the same degree as any classical IdP**; only the **native path** (RPs verifying the user's signature directly, §13.3) removes the trusted third party entirely (§13.7).
- **DID interop.** `alice@yourdomain`'s `name→key` binding is expressible as **did:web** (DNS/HTTPS-rooted, matching §3) so DID-aware verifiers and Verifiable-Credential ecosystems (OID4VP, Final) can consume DMTAP identities. `did:key` expresses a raw-key identity (tier A, §3.6).

Adoption grows exactly like mail: native RPs verify signatures directly; legacy RPs use the bridge; the bridge's importance fades as native support spreads.

13.7 Grounding & honest limits

Grounded standards (verified):

Layer	Standard	Status
Origin-bound ceremony	WebAuthn (W3C Rec L2, CR L3) + CTAP2 (FIDO2)	Rec / final
Challenge pattern	SIWE ERC-4361 (Final) + CAIP-122	final
Keypair web-auth precedent	Nostr NIP-07 / NIP-98	deployed
Key-bound sessions	DPoP RFC 9449 / GNAP RFC 9635	RFC
Capability delegation	UCAN v1.0	community spec (inf)
Issuer discovery / self-issued	OIDC Discovery 1.0 (Final) + SIOP v2 (draft)	final / draft
User-controlled OP precedent	Solid-OIDC , IndieAuth (living std)	deployed
Identifier interop	DID Core (W3C Rec), did:web / did:key	Rec / community
Modern OAuth security	OAuth 2.1 (draft) + RFC 9700 BCP	draft / RFC

Honest limits (stated in-product):

1. **Origin binding depends on a trusted client.** The anti-phishing guarantee holds only where a trusted browser/OS/app enforces `rp_origin` (WebAuthn). Raw keypair signing where the *user visually verifies* the origin (SIWE/NIP-98 style) is **weaker** and MUST NOT be the only mode; DMTAP-Auth improves on it via §13.3.1 but a compromised or absent trusted client degrades to user-verified security.
2. **Remote-node consent farming.** A remote always-on node approving relayed challenges cannot use proximity; the mitigations in §13.3.1 (pending-intent binding, origin display, per-login approval, rate-limit, log) bound but do not eliminate the risk. Passkey-gated signing is therefore the **only permitted** remote-node mode (bare node-signed is forbidden, §13.3.1).
3. **Key loss = login loss.** Your key now guards every login, so recovery UX (§1.4) is critical; this raises the stakes on getting recovery right, not a new vulnerability.
4. **RP adoption is chicken-and-egg.** The OIDC bridge (§13.6) is what makes DMTAP-Auth useful before native RP support exists — the same bootstrap strategy as the mail gateway.
5. **SIOP v2 is a draft** and WebFinger issuer discovery is the least-implemented corner of OIDC; the hosted bridge is required precisely for this reason, not optional polish.
6. **No PKI beyond DNS/CA.** `did:web` (and DNS-rooted naming, §3) bottoms out at DNS + TLS/CA — there is no independent proof binding domain to key, so a DNS/registrar/CA compromise is an identity compromise. Key transparency (§3.5) makes such a substitution *detectable*, not impossible; high-value use SHOULD add out-of-band verification. **For AUTH specifically**, a v0 single KT log that can present a **split view** (§6.6 item 6) is a **silent per-RP account takeover**, so DMTAP-Auth **REQUIRES** — even in v0 — that high-value login RPs verify the `name → key` binding against **multiple independent KT logs** (consistency across them) or an **out-of-band-verified pin**, never a single unaudited log.
7. **Login is a deliberate identity disclosure.** Authenticating to a relying party intentionally reveals your identity *to that RP*; this is opt-in and per-RP, and MUST NOT be conflated with or allowed to weaken the mail/messaging metadata-privacy guarantees (§6). Session keys are per-RP (§13.4) so RPs cannot correlate you across sites via the auth layer. This cross-site unlinkability holds **against the RPs**, not against the **bridge** (§13.6), which sees *all* of a user's bridged logins, nor against the user's own **issuing node/host**, which learns *which RP* a login resolves to. To bound the bridge's view, the bridge SHOULD issue **pairwise/blinded subject identifiers** (a distinct `sub` per RP), never one global subject.
8. **The hosted bridge is a trusted third party.** A bridge-mediated login (§13.6) trusts the

bridge's signing key **to the same degree as any classical IdP** — a compromised bridge can forge logins for its RPs. The embedded user assertion + `cnf` (so an RP MAY verify the user's key directly) and the bridge-transparency log the user's node monitors (§13.6) **bound and make detectable** this risk, but do not remove it. Only the **native path** (§13.3, RPs verifying the user's signature) eliminates the trusted third party.

14. Scaling & Deployment

DMTAP must work for a user with only a phone, a user with an always-on box, and an operator hosting millions of mailboxes — and the gateway fleet must scale horizontally. This section specifies the node classes, the reachability/buffer layers, and the scaling patterns, grounded in how production mail and P2P systems actually operate.

14.1 Node classes

Class	Examples	Role
Always-on node	Pi, NAS, home server, VPS	The authority — holds the mailbox, does
Intermittent node / thin client	Laptop, phone	A client that syncs to an always-on node
Relay	any public-IP node	Reachability hop for NAT'd nodes; conten
Mix	staked operator node	A mixnet hop (§6); content-blind
Gateway	accountable operator	Legacy SMTP bridge (§7); the only non-c

A phone is **never** a durable endpoint (§14.3). An always-on node is the durable authority (§14.4). Relay/mix/gateway are stateless middle roles that scale horizontally (§14.2, §14.5).

14.2 Horizontally-scalable gateways (normative patterns)

The gateway (§7) is stateless and **MUST** scale as a **shared-nothing worker fleet**:

1. **Identical, interchangeable instances.** Every gateway instance runs the same config; there is no whole-cluster coordination on the hot path (coordination is a scaling bottleneck — the KumoMTA design principle). Add instances to add capacity.
2. **IPs owned by an egress layer, not the instances.** Outbound source IPs are selected by an egress-proxy layer (weighted round-robin over **egress pools** of warmed IPs), so worker instances stay stateless and IPs are managed independently of instance count.
3. **DKIM keys distributed read-only** to all instances (never generated per-instance), so signing is stateless-safe.
4. **Per-ISP warmup state.** An IP is "warm for Gmail, cold for Outlook" independently; pool sizing and warmup track **per receiving ISP** (the SES managed-pool model), spilling overflow to a shared pool during warmup.
5. **One coordinating tier for the irreducible shared state.** IP reputation and rate/throttle counters are the *one* thing a shared-nothing fleet cannot make node-local; hold them in a small shared store (Redis-style) feeding automated **health-check pool demotion** (Good/Poor/New by bounce & complaint rates), not in the workers.
6. **Inbound MX at scale:** DNS MX priority + equal-preference randomization (RFC 5321 §5.1) for coarse load spreading, Anycast receiving IPs, and accept-then-forward MTAs behind a source-IP-preserving LB (XCLIENT/PROXY protocol) so downstream filtering still sees the true sender.

Honest limits (MUST disclose): IP reputation is irreducibly shared mutable state — the one non-local part of an otherwise shared-nothing fleet. **Warm-IP inventory is a hard**

scaling floor: you cannot grow outbound faster than you can warm IPs (weeks), so burst capacity is bounded by the pre-warmed pool, not by instance count. DNS MX round-robin is sender-controlled (you cannot force even distribution). Anycast inbound must be paired with stable routing so a route flap does not break a live SMTP session.

14.3 The mobile-only user (no always-on box)

A phone cannot hold a socket open or be reachable, and platform push is **best-effort, not guaranteed** on both APNs and FCM. Therefore a phone is a **push-woken thin client that drains a queue it does not host** — never a durable node. When the user has no always-on box, the queue is a **hosted, content-blind relay-mailbox** (the Chatmail model):

sender → relay-mailbox (E2EE ciphertext, short TTL) → wake-push (content-free, ≤4KB) via APNs/FCM through a notification proxy → phone wakes, opens its own authenticated connection, **DRAINS** the queue, decrypts locally, then reconciles on foreground

Requirements:

- **Push carries no content** — only a wake signal (the device token is itself encrypted where possible). Apple/Google never see plaintext. The design is **wake-and-fetch**, never deliver-in-push (APNs payload ≤4KB; silent push is throttled if excessive).
- **Push is a latency optimization, not delivery.** The client **MUST** still poll/reconcile on foreground; DMTAP **MUST NOT** treat a push as delivery confirmation. Wakes **SHOULD** be coalesced/batched to avoid iOS silent-push throttling.
- **The relay-mailbox is a buffer, not an archive** (§14.5): short TTL (~weeks), content-blind, delete-after-inactivity. The durable copy lands on the device once fetched.

This is the *only* architecture that works for a user with no home box, and it matches deployed practice (Delta Chat/Chatmail, Signal, Matrix/Sygnal — all wake-and-fetch with content-free push).

14.4 The always-on-box user

The Pi/NAS/VPS is the **durable authority**: it holds the mailbox, terminates client protocols (JMAP/IMAP, §8), and participates in the mesh. The owner's phone/laptop are thin clients that sync to it (§5.6 device cluster). Reachability (the box is usually NAT'd) comes from the relay layer (§14.5). Brief box downtime is covered by the same relay-mailbox/gateway short-queue buffer as §14.3; once the box returns and fetches, the durable copy lives on the box.

14.5 Relay & buffer scaling

Relays (reachability). Run **many small, independent, stateless relay nodes** — libp2p circuit-relay v2 needs zero coordination between relays, so the fleet scales by adding cheap nodes. Relays are a **reachability hop only**, with tight caps (go-libp2p defaults: ~128 reservations, 16 circuits/peer, 2 min / 128 KB per circuit). Discovery **SHOULD** use an **operator-run static/rendezvous list**, not sole reliance on the public DHT.

Honest limit (MUST disclose): the public libp2p/IPFS DHT + relay path is designed for *brief hole-punch assistance*, not sustained mailbox sync — undialable-peer dial timeouts and republish load make it unsuitable as the sole production discovery/relay path. DMTAP deployments **SHOULD** run their own relay fleet with tuned caps and rendezvous discovery, using the DHT (if at all) only for opportunistic discovery. Direct connectivity (IPv6, hole-punch) is always preferred; relays carry the residual hard-NAT minority (§4.3).

Buffers (offline holding). Ciphertext for an offline node is held in a **content-blind relay-mailbox with a short TTL and delete-after-inactivity** (Chatmail model: ~20-day message TTL, ~90-day inactive-account purge as reference defaults). This decouples availability from any single peer's uptime while keeping per-account cost near-zero (no long-term archive). Peer buffering (§4.3) is an alternative when no hosted buffer is desired, but its durability is tied to the buffering peer's uptime — weak for mobile-only. The gateway's short queue (§7.4) is only the legacy-translation hop and **MUST NOT** become a store.

Honest limit: a relay-mailbox is a **buffer, not an archive** — a node offline past the TTL loses undelivered mail. Durability **MUST** land at the recipient's edge once fetched. Senders retry (§2.6) within their own deadline regardless.

Buffer is not backup (§1.4). Neither a relay-mailbox nor a peer buffer (§4.3, peer-buffer TTL §16.6) is a **content backup**: each holds only *undelivered* ciphertext within its TTL, and key recovery (§1.4) restores onto an **empty store**. Durable content continuity requires a surviving cluster device (§5.6) or the **portable encrypted backup** of §1.4 — not the buffer.

14.6 Hosted multi-tenant topology (the operator)

For an operator hosting many mailboxes (Enviro Cloud, §12), the sensible horizontally-scalable shape is **stateless routed front + per-tenant sandboxed compute + object-storage persistence + scale-to-zero**:

- **Stateless front:** hostname-routed proxy on an **Anycast** network, TLS-terminating, back-hauling to the tenant's cell.
- **Per-tenant isolation:** one logical app / sandbox (microVM, e.g. Firecracker) **per customer**, so a compromised tenant cannot read another's secrets or content; equivalently a namespace-per-tenant + sandboxed runtime on Kubernetes.
- **Storage:** per-account **encrypted object-storage bucket** (§ Enviro Cloud uses R2/Tigris, free egress); isolation enforced at the app/bucket boundary, not the platform sandbox alone.
- **Scale-to-zero + push-wake:** idle mailboxes cost only storage; a request/push cold-boots the cell (~sub-second). This fits mailboxes (idle most of the time) and the §14.3 push model.
- **Multi-region:** place each tenant's cell in its home region behind the Anycast front; control-plane metadata (tiny) in a shared control-plane DB (Neon), content in per-tenant buckets.

Because there is no long-term archive on the hot path and mailboxes are mostly idle, the per-mailbox cost floor is near-zero (a documented ~1 GB RAM / 1 CPU serves thousands of light mailboxes), which is what makes the operator economics in §12 work.

14.7 Grounding

Patterns grounded in: KumoMTA (shared-nothing MTA, egress pools), AWS SES (per-ISP managed warmup), SendGrid/Mailgun (per-stream pools, health-check demotion), RFC 5321 §5.1 (MX priority), Cloudflare (Anycast inbound), Delta Chat/Chatmail + Signal + Matrix/Sygnal (wake-and-fetch push, relay-mailbox), APNs/FCM (best-effort, ≤4 KB, throttling), libp2p circuit-relay v2 + Protocol Labs DHT findings (relay caps, discovery), Fly Machines "one app per customer" + Kubernetes multi-tenancy (per-tenant sandbox, scale-to-zero). See §11 for the full bibliography.

15. Normative & Informative References

DMTAP is a **profile that composes existing standards**. This section is the authoritative reference list. **Normative** references are required to implement DMTAP correctly; **Informative** references are precedent, rationale, or optional interop. Full text is not reproduced here — RFCs are permanently available at <https://www.rfc-editor.org/rfc/rfcNNNN> and W3C/OpenID/other specs at the URLs given. DMTAP cites them by identifier and profiles them; where DMTAP narrows or extends a referenced spec, the DMTAP text is normative for DMTAP.

15.1 Normative — cryptography & encoding

Ref	Title	DMTAP use
RFC 9180	Hybrid Public Key Encryption (HPKE)	seal-to-key encryption; DHKEM(X25519)/HKEM
RFC 8032	EdDSA (Ed25519)	signatures (§1, §2)
RFC 7748	Elliptic Curves for Security (X25519)	key agreement (§1)
RFC 9420	The Messaging Layer Security (MLS) Protocol	group/session security for 1:1, groups, files, m...
RFC 9750	The MLS Architecture	Delivery/Authentication Service roles → mes...
RFC 8949	Concise Binary Object Representation (CBOR)	wire serialization; deterministic (core) enc...
FIPS 203	ML-KEM (Module-Lattice KEM)	PQ KEM (suite 0x02, §1.1)
FIPS 204	ML-DSA (Module-Lattice signatures)	PQ signatures (suite 0x02, §1.1)
BLAKE3	BLAKE3 cryptographic hash	content addressing (§2.2), hash-agile prefix; m...

15.2 Normative — transport, mesh & naming

Ref	Title	DMTAP use
libp2p specs	Kademlia DHT, Circuit Relay v2, DCUtr, AutoNAT v2, Noise	mesh, reachability, N...
RFC 9000	QUIC	primary transport; co...
RFC 1035 / 9460	DNS / SVCB & HTTPS RRs	name→key records; u...
RFC 6376	DomainKeys Identified Mail (DKIM)	gateway signs-as-dom...
RFC 8461 / 7672	MTA-STS / DANE (SMTP TLS)	enforce TLS on the le...

15.3 Normative — legacy mail interop (the gateway & client edges)

Ref	Title	DMTAP use
RFC 5321	Simple Mail Transfer Protocol (SMTP)	gateway legacy world; MX priority/...
RFC 5322	Internet Message Format	RFC822 MOTE translation at the g...
RFC 2045–2049	MIME	attachment/body representation acro...
RFC 9051	IMAP4rev2	client-compat surface projected from
RFC 3501	IMAP4rev1	broader legacy-client compatibility (§...
RFC 6409	Message Submission (port 587)	client outbound submission (§8)
RFC 1939	POP3	optional legacy-client compatibility (§...
RFC 8620 / 8621	JMAP (Core / Mail)	native modern client sync surface (§8...
RFC 4791	Calendaring Extensions to WebDAV (CalDAV)	legacy calendar-client compat, project...
RFC 6352	vCard Extensions to WebDAV (CardDAV)	legacy contacts-client compat, project...
RFC 5545	iCalendar	calendar object body format (§8.4)
RFC 6350	vCard 4.0	contact object body format (§8.4)
RFC 8984 / 9553	JSCalendar / JSContact	native JSON calendar/contact repres...

Ref	Title	DMTAP use
-----	-------	-----------

15.4 Normative — identity, auth & anti-abuse

Ref	Title	DMTAP use
RFC 6749	OAuth 2.0 Authorization Framework	base framework for th
RFC 7636	PKCE	REQUIRED on any D
RFC 9449	OAuth 2.0 Demonstrating Proof of Possession (DPoP)	key-bound, sender-con
RFC 9068	JWT Profile for OAuth 2.0 Access Tokens	token format for the O
RFC 7009 / 7662	OAuth Token Revocation / Introspection	session revocation wit
RFC 8414	OAuth 2.0 Authorization Server Metadata	.well-known discovery
RFC 9700	Best Current Practice for OAuth 2.0 Security	security baseline for §
RFC 9576 / 9577 / 9578	Privacy Pass (Architecture / HTTP / Issuance)	anonymous anti-abuse
WebAuthn L2	Web Authentication (W3C Rec, L2; L3 CR)	origin-bound login cer
CTAP2	Client to Authenticator Protocol (FIDO2)	passkey/authenticator

15.5 Informative — precedent & rationale

Ref	Title
RFC 9635	Grant Negotiation and Authorization Protocol (GNA)
draft-ietf-oauth-v2-1	OAuth 2.1
ERC-4361 / CAIP-122	Sign-In with Ethereum / Sign-In with X
W3C DID Core 1.0	Decentralized Identifiers
IndieAuth (living std)	Identity = a URL/domain you own
SIOP v2 (draft)	Self-Issued OpenID Provider
Solid-OIDC (CG report)	WebID + DPoP over OIDC
Nostr NIP-07 / NIP-98	keypair browser signing / HTTP auth
UCAN v1.0	User-Controlled Authorization Networks
Sphinx (Danezis & Goldberg, 2009)	mix packet format
Loopix (USENIX Sec 2017)	mixnet design
Anonymity Trilemma (IEEE S&P 2018)	anonymity latency/bandwidth
CONIKS (USENIX Sec 2015) / RFC 6962	key transparency / Certificate Transparency
SLIP-0039 / RFC 9591 (FROST)	Shamir mnemonic / threshold signatures
Chatmail / Delta Chat, Matrix/Sygnal, Signal	minimal mail server, push gateway, sealed sender
draft-connolly-cfrg-xwing-kem	X-Wing hybrid KEM
draft-ietf-mls-pq-ciphersuites / -combiner	PQ-MLS
draft-kohbrok-mls-dmls	Decentralized MLS
draft-yun-privacypass-arc	Anonymous Rate-Limited Credentials

15.6 On reproduction

DMTAP does not vendor the full text of referenced standards. RFCs are reproducible under BCP 78 / the IETF Trust Legal Provisions with their notices intact, but referencing by identifier is the norm and avoids staleness. Implementers should read the referenced specs directly at the URLs above; this document defines only the **DMTAP profile** over them.

16. Numeric Parameters (v0)

Interoperability requires agreed numbers, not qualitative "hours"/"minutes." This appendix fixes the v0 defaults. Each is a **parameter** an implementation MAY tune within stated bounds, but the defaults here are what a conformance run (§10) checks. All are versioned with the protocol.

16.1 Time, replay & clocks

Parameter	Default	Notes
Clock-skew tolerance	± 120 s	max accepted difference between <code>ts</code> and receive
Challenge/nonce validity window	120 s	server-issued single-use nonce for auth (§13) & a
Replay cache retention	≥ 300 s	seen- <code>id</code> / seen-nonce cache MUST cover the ske
MOTE <code>expires</code> max	1 year	cap on requested client-side expiry
Sender retry deadline	72 h	after which an undelivered MOTE fails to the us
Sender retry backoff	exp, base 30 s, cap 1 h	exponential with jitter

Nodes MUST NOT rely on synchronized clocks for correctness — only for the skew-bounded windows above and for ordering hints.

16.2 Naming, KT & DHT

Parameter	Default	Notes
Key-name length	8 words (80 bits)	§3.9.1; 12 words (128 bits) for adversary-proof mode
Key-name wordlist size	1024	language-agnostic; +1 checksum word
KT signed-tree-head poll	≤ 6 h	client re-checks its own entry (self-monitoring)
KT gossip interval (v1)	≤ 1 h	tree-head gossip for equivocation detection
DHT location-record TTL	2 h	signed; republish before expiry (§4.2)
DHT republish interval	45 min	< TTL, with jitter
DHT lookup redundancy (K)	20	store at K closest; S/Kademlia disjoint paths ≥ 3
Location seq-number	monotonic u64	rollback defense; reject older-or-equal

16.3 Mixnet & privacy

Parameter	Default	Notes
Mix path length	3 hops	Sphinx onion (§4.4)
Per-hop mix delay	exp, mean 5 s	Poisson mixing; <code>private</code> tier
Cover-traffic rate	Poisson, mean 30 s/msg	tunable per node; higher = more privacy, more b
Padded packet size (bucket)	2 KiB	Sphinx constant-length cell after padding
<code>private</code> -tier end-to-end latency	seconds–minutes	consequence of mixing (§6)

16.4 File tiers & transfer

Parameter	Default	Notes
Inline attachment threshold	≤ 64 KiB	rides the message (§2.5)

Parameter	Default	Notes
Normal-file threshold	≤ 4 MiB (≤ 4 chunks)	chunks via mixnet — full privacy (§6.5)
Chunk size	1 MiB	fixed; content-addressed (§5.5)
Large-file bulk	> 4 MiB	fast/onion bulk path — weaker privacy (§6.5)
Swarm parallelism	≤ 8 sources	per-file concurrent fetch

16.5 Anti-abuse

Parameter	Default	Notes
Cold-sender PoW	memory-hard (Argon2id), adaptive	last-resort tier (§9.4)
PoW puzzle scope	<code>id recipient nonce(epoch)</code>	fresh epoch nonce to prevent precomputation
Unknown-issuer token budget	0	self-issued/unvetted → no rate budget (S)
Requests-area retention	30 days	unproven cold MOTE held here, not in S
Postage redemption check	online (issuer)	no offline bearer acceptance (§9.5.1)

16.6 Transport & relay

Parameter	Default	Notes
Relay reservation TTL	1 h	libp2p circuit-relay v2 (§14.5)
Relay per-circuit cap	2 min / 128 KiB	brief hole-punch assist only — not sustained sync
Reachability ladder	IPv6 → hole-punch → relay	prefer direct (§4.3)
Offline-buffer TTL	20 days	relay-mailbox message hold (§14.5)
Peer-buffer TTL	20 days	buddy-node ciphertext hold (§4.3, §14.5); not an arch
Inactive-account purge	90 days	relay-mailbox (§14.5)
Push payload	≤ 4 KiB, content-free	wake-and-fetch only (§14.3)

16.7 Crypto suites

suite	Sign	KEM	AEAD	Hash	S
<code>0x01</code>	Ed25519	X25519 (HPKE)	ChaCha20-Poly1305	BLAKE3-256	v
<code>0x02</code>	Ed25519+ML-DSA-65	X-Wing (X25519+ML-KEM-768)	ChaCha20-Poly1305	BLAKE3-256	P

16.8 Auth, sessions & group ordering

Parameter	Default	Notes
DMTAP-Auth session TTL	24 h	key-bound session lifetime before
DMTAP-Auth session idle-timeout	30 min	idle expiry of a key-bound session
RP delegation re-validation interval	≤ 15 min	RP re-checks status endpoint
Recovery-weakening veto window	72 h	delay before a factor-weakening
Recovery veto quorum	<code>rotate_threshold</code>	a veto MUST satisfy this (as
Committer-liveness timeout	5 min	pending signed proposal upon
Committer-takeover hysteresis	2 consecutive misses	takeover only after the timeout
Committer roster quorum	$> n/2$ ($\lceil (n+1)/2 \rceil$ of current members)	a takeover Commit is valid o

Parameter	Default	Notes
Group join-request expiry	30 days	a <code>request -mode</code> join with no

All numeric values here are v0 defaults; a future protocol version MAY revise them, and capability negotiation (§10.2) carries any non-default profile.

17. Feature-Parity Audit — Classic Mail/Calendar/Contacts vs DMTAP

This section is a **sense-check**, not a pitch. For every feature a working email/calendar/ contacts user expects, it states how DMTAP provides it (or doesn't), whether the mapping is sound in a decentralized/E2E system, and how the security properties compare to legacy. A "doesn't map / genuinely harder" verdict is treated as a valid, useful outcome — this document exists to surface exactly those, not to hide them.

17.0 Method

Each feature gets four lines:

- **How** — the DMTAP mechanism that provides it, with a § back-reference.
- **Sense-check** — does this make sense architecturally in a sealed-sender, E2E, no-central-server system, or is it in tension with that model?
- **Security** — better / same / worse than legacy IMAP/SMTP-era email, and why.
- **Open issue** — anything unresolved. "None." if the mapping is complete.

Verdicts used in the summary (§17.5): **Clean** (works cleanly, same or better), **Different** (provided, but the mechanism or UX genuinely changes), **Harder** (a real capability loss or added difficulty, disclosed rather than hidden), **N/A** (the legacy feature's premise doesn't survive translation, and that is correct, not a bug).

52 features are covered: 29 mail, 7 contacts, 11 calendar, 5 cross-cutting.

17.1 Mail

1. Folders — Clean

- **How:** JMAP `Mailbox` semantics over the MOTE store (§8.1); mailbox assignment is part of the replicated per-account state kept in sync across the device cluster as an encrypted CRDT (§5.6: "the mailbox, read/flag state, labels, and file index are replicated across devices").
- **Sense-check:** Folders are pure client/account-local organization over an already-decrypted store; nothing about decentralization touches this.
- **Security:** Same as legacy, but the folder structure itself is encrypted at rest (§6.7) and never visible to any intermediary — legacy IMAP folder names/counts are visible to the provider by construction; here they are not.
- **Open issue:** None.

2. Labels/tags — Clean

- **How:** Same CRDT-replicated state as folders (§5.6); DMTAP's model treats labels and folders as the same mechanism (JMAP keywords/mailboxes), not two systems.
- **Sense-check:** Client-local metadata; no tension with the protocol.

- **Security:** Same as legacy in function; better in exposure (never leaves the encrypted store/CRDT sync channel).
- **Open issue:** None.

3. Filters/rules (server-side Sieve vs client-side) — Different

- **How:** DMTAP has no third-party server that ever sees plaintext, so classic "runs on the provider's Sieve server while you're offline" doesn't exist as such. But the **always-on node is the user's own authority**, not a third party (§8.2: "the E2E boundary is between *parties*, not between the owner and their own device") — it already decrypts every MOTE for the owner. Filter rules therefore run **on the owner's own node**, which is functionally "server-side" (always-on, runs while the user is offline) without being a *third party's* server. This is arguably the correct re-siting of the feature, not a downgrade.
- **Sense-check:** Sound. The property legacy Sieve actually gives users ("rules apply even when my client is closed") is fully preserved because the node, not the client, is always-on. What's lost is a *provider-hosted* Sieve service independent of any of the user's own hardware — that specific shape doesn't survive, correctly, since it would require a third party to hold plaintext.
- **Security:** Better: no third party ever evaluates rules against your plaintext. Same blast-radius caveat as everything else on the node: a compromised node sees what Sieve would have seen anyway.
- **Open issue:** No rule language is specified in v0 (Sieve itself, RFC 5228, MAY simply be reused verbatim by an implementation running on the node — nothing prevents it). Left to implementations; not a protocol gap since filtering is entirely local to the node.

4. Search (on-device, no server-side index) — Different

- **How:** Explicitly a non-goal to centralize (§0.7: "no server-side search"); each node indexes its own plaintext mailbox locally. A thin client (phone) either queries its home node live or searches its local cache (§14.1: thin clients hold "cache only").
- **Sense-check:** Correct outcome for an E2E system — a searchable *shared* server-side index is precisely the kind of central plaintext store DMTAP is designed to not need. The interesting nuance: because the "server" (the node) is *yours*, on-device search on the always-on node is not actually weaker than a provider's server-side search in capability — only a phone's *local* cache-only search is meaningfully limited, and that's a phone-storage limit, not a protocol one.
- **Security:** Better — no provider ever builds a searchable index of your mail (a historically real target: subpoenas, breaches, ad-targeting scandals all rooted in provider-side search indices).
- **Open issue:** No shared/cross-user search corpus exists (see spam classification, item 20) — a consequence of the same design choice, not a separate gap.

5. Threading/conversations — Clean

- **How:** `Headers.thread` (stable thread id) + `Payload.refs` (reply/reference chain, §2.4); clients render either by explicit thread id or reference-graph reconstruction.
- **Sense-check:** Directly maps; MOTEs are inherently unordered on the wire (§2.6) so explicit thread/ref fields are the *correct* mechanism (vs. legacy's fragile `In-Reply-To` / `References` header heuristics, which routinely break across clients).
- **Security:** Same or better — thread linkage is inside the encrypted payload, invisible to the network (legacy threading headers are plaintext SMTP headers, visible to every relay).
- **Open issue:** None.

6. Drafts — Clean

- **How:** A draft is a MOTE that has not yet been dispatched — it exists only as local/ device-cluster CRDT state (§5.6) before a `kind` (§2.3) is ever sent. No protocol object is needed because "sent" is the only state transition the wire format cares about.
- **Sense-check:** Sound; drafts are inherently pre-wire and so are naturally client/cluster state, syncing across the owner's own devices exactly like read/flag state.
- **Security:** Better — a draft never touches the network, the mixnet, or any intermediary until dispatched (legacy webmail drafts are stored server-side in plaintext by the provider).
- **Open issue:** None.

7. Attachments (any size) — Clean

- **How:** Inline for small, content-addressed chunked manifest for large, with three explicit size tiers and their privacy cost (§2.5, §5.5, §6.5, §16.4): inline ≤ 64 KiB, normal ≤ 4 MiB (full mixnet privacy), large > 4 MiB (fast/onion bulk path, weaker metadata privacy).
- **Sense-check:** Clean and in fact a genuine upgrade — no protocol-imposed size cap at all (legacy mail is capped ~ 25 MB by providers); the tradeoff is disclosed rather than hidden (privacy weakens as size grows, §6.5).
- **Security:** Content and integrity are strictly better (per-chunk hashes, dedup, resumable swarm transfer, §5.5). Metadata privacy is weaker for large files specifically — the spec is explicit about this (§6.6.2) rather than pretending otherwise.
- **Open issue:** None beyond the disclosed large-file metadata tradeoff (§6.5), which is a known, stated limit, not an unresolved gap.

8. Rich text/HTML — Clean

- **How:** `Headers.mime` + `Body` (tstr/bytes, §2.4) carries the content type and body exactly as MIME does today; nothing about encryption or decentralization constrains body format.
- **Sense-check:** Non-issue; purely a client-rendering concern.
- **Security:** Same as legacy for content (the usual HTML-mail render-time risks — remote image beacons, tracking pixels — are a **client** hygiene concern, unchanged by DMTAP). One improvement: since delivery is E2E, a tracking pixel can no longer also be a *provider-side* read-confirmation mechanism the way it can when a webmail provider proxies remote images; it's still a client-side leak if the client fetches remote content, same as always.
- **Open issue:** None (client responsibility to proxy/strip remote content, as today).

9. Signatures (email sig block) — Clean

- **How:** Plain client-local text/HTML prepended to `Body` at compose time. No protocol object.
- **Sense-check:** Non-issue.
- **Security:** Unchanged; note this is unrelated to the *cryptographic* `sig` field (§2.4) — naming collision with legacy terminology only, worth a footnote in client UX copy so users don't conflate "email signature" with "message signing."
- **Open issue:** None.

10. Aliases + plus-addressing — Clean

- **How:** `Identity.names` is a list (§1.3); one identity holds many names (key-name, handles, `name@domain`) all resolving to the same key, and legacy addresses you already own can be folded in as aliases (§3.9.4). Plus-addressing (`you+tag@domain`) is explicitly preserved for client-side filtering and resolves to the same key (§3.9.4).

- **Sense-check:** Clean, and stronger than legacy: every alias is cryptographically the same identity, so — unlike legacy plus-addressing, which is an unauthenticated convention any sender can spoof around — a DMTAP alias cannot be used to impersonate (§3.9.4: "authenticity is always the key, not the name").
- **Security:** Better — alias additions/removals are signed `Identity` versions, audited via KT (§3.5); a legacy provider can silently disable/redirect an alias, a DMTAP alias change is tamper-evident.
- **Open issue:** None.

11. Forwarding (manual) — Different

- **How:** Forwarding as *composing a new message that quotes/attaches the original* works unchanged — the forwarder decrypts, re-encrypts to the new recipient, done. What does **not** carry over is legacy "Redirect"/`Resent-From` semantics — literally re-transmitting the original message *as though the original sender sent it*, preserving `From:`. That requires forging the original sender's signature, which a DMTAP node cannot do (`Payload.sig` is bound to the original sender's key, §2.4).
- **Sense-check:** The impossible half (bit-identical redirect impersonating the original sender) is precisely the thing that *should* be impossible in an authenticated system — legacy `Resent-From` is a well-known, still-live spoofing vector. Losing it is a feature, not a bug.
- **Security:** Better: DMTAP forwarding is always honestly attributed to the forwarder; legacy redirect-forwarding is a documented BEC (business email compromise) and phishing amplifier.
- **Open issue:** None — flagged here so implementers don't try to "fix" this by reintroducing forgeable redirect.

12. Auto-forward (rule-based, all mail to another address) — Different

- **How:** A standing rule on the owner's own node (§8.2 model, same siting as filters, item 3): decrypt, re-encrypt-and-send-as-new-message to the configured address, for every matching incoming MOTE.
- **Sense-check:** Sound mechanically. Worth flagging because silent auto-forward rules are a real-world account-takeover technique (attacker adds a hidden Gmail/Exchange forwarding rule to exfiltrate mail after a credential compromise) — the same attack is *possible* here if an attacker gains node/admin access, since forwarding-rule changes are ordinary local configuration, not a signed identity-lifecycle object.
- **Security:** Same residual risk as legacy for this specific attack (it's a node-compromise consequence, not a protocol flaw), but DMTAP has the machinery to do *better*: identity and recovery-policy changes are logged and self-monitored for owner alerting (§3.5); auto-forward rule changes currently are not analogous first-class signed/logged objects.
- **Open issue: Gap.** Recommend implementations surface auto-forward rule changes through the same device-cluster-notification path used for read/flag sync (§5.6), so a newly added silent forwarding rule is visible to the owner's other devices, mirroring the KT self-monitoring pattern (§3.5) that already protects identity/recovery changes. Not currently specified.

13. Vacation / auto-responder / out-of-office — Different (improved)

- **How:** A client/node-local automation: reply-once-per-sender-per-window with a template. Because DMTAP validates the anti-abuse challenge for cold senders *before* decryption (§2.7 step 6), an auto-responder can be scoped to fire only for senders that already cleared that gate — i.e. real, accountable senders, not arbitrary inbound traffic.
- **Sense-check:** Fits cleanly, and interacts well with §9's anti-abuse design rather than fighting it.

- **Security:** Genuinely better than legacy on one specific, real problem: **auto-responder backscatter**. Legacy auto-responders regularly bounce vacation replies to forged `From:` addresses used by spammers, spamming innocent third parties. Because DMTAP senders are cryptographically authenticated (`Payload.from`, §2.4) rather than a spoofable SMTP `MAIL FROM`, an auto-responder always replies to a real party — the backscatter failure mode structurally cannot occur.
- **Open issue:** None.

14. Read receipts — Clean

- **How:** `kind=0x07 receipt`, explicitly **opt-in**, ephemeral, off by default (§2.3, §5.4, §6.2).
- **Sense-check:** Correctly modeled as metadata-sensitive rather than a free default — read receipts leak exactly the kind of fine-grained timing/behavior signal the rest of the spec works hard to hide (§6.6.3's intersection-attack concern applies at the small scale of "did they read it and when" too), so opt-in-only is the right call, not an oversight.
- **Security:** Better — legacy read receipts (or worse, silent tracking pixels used as a read-receipt substitute) are usually either on-by-default or invisible/non-consensual; here the feature only exists when the recipient affirmatively enables it, and it rides inside the encrypted payload (not observable by any intermediary either way).
- **Open issue:** None.

15. Scheduled send — Clean

- **How:** The originating node holds the MOTE undispatched until the scheduled time, then sends normally. Best done from the owner's always-on node (§14.4) so the schedule survives a sleeping phone/laptop.
- **Sense-check:** Trivial client/node feature; no protocol change needed.
- **Security:** Same as legacy for the "will it actually fire" guarantee (both depend on *something* being up at the scheduled time); better for confidentiality (the pending message sits encrypted on the user's own node rather than in a provider's plaintext outbox).
- **Open issue:** A scheduled send fired from a thin client with no always-on node (§14.3's mobile-only user) needs the hosted relay-mailbox or the phone itself to be awake at send time — same durability caveat as any mobile-only delivery in this model, not specific to scheduling.

16. Snooze — Clean

- **How:** Pure local/CRDT UI state (hide until time X, then resurface) — no wire format involvement at all, same as legacy.
- **Sense-check:** Non-issue.
- **Security:** Unchanged.
- **Open issue:** None.

17. Undo send — Different

- **How:** Only meaningful during the **pre-dispatch window** the client itself imposes (hold the MOTE N seconds before actually handing it to the outbound queue) — identical in kind to how Gmail's "undo send" actually works (a client-side delay, not a true retraction). Once the MOTE has left the node, "undo" degrades to the same mechanism as message recall (item 28): a cooperative `redact` /edit request.
- **Sense-check:** Correct and honestly scoped — true after-the-fact undo (retracting bytes already delivered to and possibly displayed by the recipient) is not achievable in *any* system, legacy or DMTAP; the useful part of "undo send" (a grace-period buffer) transfers perfectly.

- **Security:** Same as legacy in the pre-dispatch window; same *honest limit* as legacy post-dispatch (neither system can un-deliver bytes already in a recipient's hands) — DMTAP is more upfront about this limit (§6.6.8) than most legacy clients' UI copy is.
- **Open issue:** None — the limit is disclosed, not hidden.

18. Mark read/unread, flag/star — Clean

- **How:** Explicitly named in §5.6: "the mailbox, read/flag state, labels, and file index are replicated across devices as an encrypted CRDT."
- **Sense-check:** Direct match; this is exactly the multi-device-state problem MLS-based device clustering is designed to solve cleanly (no per-pair ratchet mess, §5.1).
- **Security:** Better — flag/star state syncs end-to-end encrypted across your own devices rather than living as plaintext provider-side account metadata.
- **Open issue:** None.

19. Archive — Clean

- **How:** Same CRDT-replicated mailbox-state mechanism as folders/labels (§5.6); "archive" is a mailbox-membership change like any other.
- **Sense-check:** Non-issue.
- **Security:** Same as folders (item 1).
- **Open issue:** None.

20. Spam/junk + blocklists/allowlists — Different

- **How:** Recipient-local `Policy{allow, challenge, block, rate}` (§9.2), enforced *before* decryption for unknown senders (§2.7 step 6), with a graded outcome (silent-discard vs. requests-area defer, §2.7a) rather than a binary accept/reject.
- **Sense-check:** This is the correct re-architecture for a sealed-sender, no-central-scanner system: since no third party ever sees plaintext to run ML classification against, the defense has to move to a place that doesn't need content — cryptographic cost-to-reach (challenge/PoW/postage/vouch, §9.3–9.7) substitutes for content filtering at the point where content filtering is architecturally unavailable.
- **Security:** Mixed, disclosed honestly. **Better:** spam that never gets a valid challenge is rejected before any provider (or anyone) reads a single byte of it — a strictly stronger privacy property than "our ML model read your mail to decide it's spam." **Harder:** there is no shared cross-user corpus or network effect the way a single large provider's spam model benefits from billions of samples — each recipient's local policy starts cold and reputation is scored per-issuer (§9.3.1) rather than per-message-content. A node MAY still run local content-based classification on whatever lands in its own requests-area (it holds the plaintext of its own mail), but there is no cross-user training signal.
- **Open issue:** No shared abuse-signal exchange between independent nodes/operators is specified beyond issuer/gateway reputation scoring (§7.5, §9.3.1) — an acceptable, disclosed gap given the metadata-privacy goals, not an oversight.

21. Quotas — Clean

- **How:** The protocol itself imposes **no** storage cap (§5.5: "bounded only by the owner's storage"). Quotas are a policy/billing layer, entirely outside the protocol, imposed only by a *hosted* operator on operations (storage caps, send caps, §12).
- **Sense-check:** Correctly kept out of the wire protocol; quotas are a business-model concern, not a messaging concern, and self-hosters have none by construction.

- **Security:** Explicitly protected: §12 states there **MUST** be no quota/plan gate capable of disabling encryption — i.e. a hosted operator cannot use quota enforcement as a backdoor into weakening the security model.
- **Open issue:** None.

22. Import/export (mbox/EML) — Clean

- **How:** The node's IMAP compatibility surface (§8.2) lets any existing mbox/EML tool work against it unchanged, exactly as against a legacy IMAP server; native export is a MOTE-store dump via JMAP.
- **Sense-check:** Direct hit — this is precisely what the compatibility surfaces (§8.1, §8.2) exist for.
- **Security:** Same as legacy for the exported artifact itself (an mbox/EML file is plaintext once exported, same as it always was); better in that the export is pulled by an app-password-authenticated client from your own node, not handed to a third party to generate on your behalf.
- **Open issue:** None.

23. Delegation / send-on-behalf — Different

- **How:** Two real mechanisms, neither identical to legacy "delegate access": (a) `DeviceCert.caps` (§1.2) includes a `send` capability — an assistant's device can be issued a narrowly-scoped device certificate signed by `IK`, letting it send *as the identity* without holding the root key; (b) capability delegation (§13.5, UCAN-style) generalizes further — "let this app/person act on my behalf" for a specific, time-bound, attenuable right, which extends naturally from login-delegation to mail-send-delegation.
- **Sense-check:** Sound mechanically, but note a real semantic gap from legacy Exchange-style delegation: a `send`-capable device signs as the *same identity* indistinguishable from the principal — legacy delegate-send shows a distinguishing `Sender: /"on behalf of"` header so the recipient knows an assistant actually hit send. DMTAP has no equivalent marker by default.
- **Security:** The delegated device is narrowly scoped (only `send`, not `admin` /recovery, §1.2) and revocable without rotating the whole identity — tighter blast-radius control than legacy delegate access, which often just grants full mailbox access. The one regression: the recipient cannot tell a delegate sent it, where legacy could show that distinction.
- **Open issue: Gap.** No standard `Headers.ext` field is defined for "sent by delegate device X of identity Y" — straightforward to add as an extension header (§10) but not yet specified.

24. Shared mailboxes — Clean

- **How:** A "team inbox" is explicitly one of the named group models (§5.8.1): a group identity with its own address; every member of the MLS group receives posts to the shared address.
- **Sense-check:** Direct hit — "a group with an address" (§5.8) is exactly the shared-mailbox primitive, generalized.
- **Security:** Better — membership, roles (owner/admin/member/poster/reader, §5.8.2), and every add/remove/role change are signed and appear in an auditable hash-chained log (§5.8.2), unlike a legacy shared mailbox whose access-grant history is usually just an admin-console audit log controlled by the provider.
- **Open issue:** None.

25. Distribution lists — Clean

- **How:** The broadcast/list posting model of groups-as-address (§5.8.1): post to the address, every member gets a sealed per-member copy; membership is typically hidden (§5.8.3).
- **Sense-check:** This *is* the feature §5.8 was designed to unify — direct, first-class mapping, not a stretch.
- **Security:** Better on the specific point legacy mailing lists get wrong by default: hidden membership via per-member sealed delivery (§5.8.3) means subscribers don't learn each other's identities, whereas legacy list software (majordomo/mailman-era) frequently exposes the subscriber list or at least each poster's address to all recipients.
- **Open issue:** Large lists trade cryptographic group-sharing for per-member sealed fan-out (§5.8.4) — a stated, deliberate scalability/privacy tradeoff, not an unresolved question.

26. Catch-all — Clean

- **How:** Explicitly named as a tier-C domain-owner option (§3.9.4).
- **Sense-check:** Direct hit, no tension — it's just another alias-resolution rule at the naming layer.
- **Security:** Same considerations as legacy catch-all (a broader attack surface for misdirected/spam mail landing in one inbox); DMTAP's per-alias `Identity` signing (§3.9.4) at least makes the set of valid catch-all-eligible names auditable via KT, which legacy catch-all configuration is not.
- **Open issue:** None.

27. Priority / importance flag — N/A (minor)

- **How:** No dedicated field in v0. Could be carried in `Headers.ext` (§2.4's extension-header mechanism, formalized in §10) exactly like any other MIME-era header that isn't security-relevant.
- **Sense-check:** Not a decentralization-relevant feature at all — it's cosmetic metadata that legacy clients themselves rarely enforce meaningfully (X-Priority is famously ignored/abused by spammers to fake urgency).
- **Security:** No change either way; note that letting a *sender* self-declare "urgent" is already a weak signal in legacy mail and remains exactly as weak here — not something DMTAP needs to strengthen, since the real anti-abuse gate (§9) is what actually matters for triage.
- **Open issue:** Minor — a client convention for `ext.priority` is undefined but trivial to add; low value, not worth a normative field in v0.

28. Message recall (and why it's only cooperative) — Harder (honestly, same as legacy)

- **How:** `kind=0x03 edit` / `kind=0x04 redact` reference a prior MOTE by `id` (§2.3) and ask the recipient's client to supersede/delete it.
- **Sense-check:** The honest answer is that recall was **never really possible** in legacy email either (Exchange's "recall" famously fails silently once the recipient has opened the message) — DMTAP just refuses to pretend otherwise. §6.6 item 8 states it plainly: "`redact` / `expires` are unenforceable against a non-compliant recipient that already holds the plaintext — they are cooperative hints, not guarantees."
- **Security:** Arguably *more* honest than legacy, which is worse in one specific way: because DMTAP is E2E, there is no provider-side copy a "recall" could plausibly reach into even in principle (legacy recall's occasional partial success stems from the message often still sitting in a shared Exchange store the sender's organization controls) — so DMTAP recall is unambiguously cooperative-only, with no false hope of provider-side deletion.
- **Open issue:** None — this is a disclosed, irreducible limit (§6.6.8), not a gap to close.

29. DKIM / SPF / DMARC — Clean

- **How:** Entirely the gateway's job (§7): DKIM via delegated selectors so the gateway signs `d=<domain>` without ever holding the user's DMTAP key (§7.3); SPF/DMARC records maintained once per domain by the gateway operator (Tier B, §3.8) or auto-published by the provider for vanity domains (Tier C, §3.8); DMARC alignment follows directly from DKIM delegation.
 - **Sense-check:** Correctly scoped as a **legacy-interop** concern only — DMTAP-native delivery needs none of this (authenticity comes from the payload signature, §2.4, not domain-level email authentication). DKIM/SPF/DMARC only matter on the leg that still speaks SMTP.
 - **Security:** Same guarantees as legacy for that leg (this is literally legacy DKIM/SPF/DMARC, unmodified), with one structural improvement: the gateway never holds the user's actual identity key (§7.3) — a compromised or malicious gateway operator can forge out-bound DKIM-valid legacy mail (same blast radius as any legacy DKIM-holding provider) but can never impersonate the user's DMTAP-native identity, which is a strictly separate keyspace.
 - **Open issue:** None (this is deliberately unmodified legacy machinery, by design).
-

17.2 Contacts

30. Contact cards (vCard/JSContact) — Clean

- **How:** Native representation is **JSContact (RFC 9553)** MOTEs, synced via JMAP Contacts (§8.1, §8.4); CardDAV (RFC 6352) projects the same store as vCard (RFC 6350) for legacy clients (§8.4).
- **Sense-check:** Direct hit; contacts are just another MOTE kind sharing the same node, encryption, and sync machinery as mail (§8.4: "not separate central services").
- **Security:** Better — your address book is end-to-end encrypted and synced across your own devices; no provider (Google Contacts-style) ever holds a plaintext copy or mines it.
- **Open issue:** None.

31. Contact groups — Different

- **How:** Two distinct things get conflated under one legacy name, and DMTAP correctly separates them: (a) a *reachable* group with its own address and MLS roster is the groups-as-address primitive (§5.8) — full distribution-list semantics; (b) a plain *organizational* tag ("Family", "Work") with no address of its own is just local metadata on contact MOTEs, needing no group entity at all.
- **Sense-check:** This separation is actually clearer than legacy, which uses "contact group" for both an address-able mailing list and a purely-local organizational label under one confusing UI concept.
- **Security:** Local-tag groups: no change, purely local. Addressable groups: same improvements as distribution lists (item 25) — hidden membership option, auditable add/remove.
- **Open issue:** None; flagged only so implementers don't collapse the two into one object.

32. Shared address books — Different (extrapolated, not verbatim spec'd)

- **How:** The spec explicitly generalizes the "MLS group over a set of manifests" pattern to shared file folders (§5.1, §5.7); a shared contacts collection is the same pattern applied to contact MOTEs instead of file manifests — consistent with §8.4's framing that contacts share "the same MLS groups (§5) as everything else."

- **Sense-check:** Sound by direct analogy to the spec's own stated pattern, but the spec does not literally spell out "shared address book" as its own named construct the way it does for shared file folders and team inboxes.
- **Security:** Same benefits as any MLS-group-shared data: encrypted, member-auditable, re-keyed on member removal (§6.7's removal/re-key rule applies here too, since a departed team member's read access to a shared contact list should not persist any more than to a shared file folder).
- **Open issue: Minor gap.** Worth an explicit line in a future revision of §8.4 naming "shared address book = MLS group over contact MOTE" the same way §5.5/§5.7 do for files, so implementers don't have to infer it.

33. Auto-complete / directory (org-wide GAL equivalent) — Harder

- **How:** No default provider-hosted org directory exists in DMTAP's model. What's available: the global handle directory (§3.9.2, not org-scoped, thin/KT-audited), and — if an organization wants a GAL-equivalent — self-assembling one as a shared-contacts group (item 32) that all org members are added to.
- **Sense-check:** This is a genuine, disclosed difference, not a design flaw: a legacy GAL is a provider-hosted, always-populated directory *because* the provider already holds everyone's plaintext account data centrally. DMTAP has no equivalent central store to build a directory from — an org has to deliberately construct one.
- **Security:** Arguably better once built (no provider-side directory to scrape/breach for an org chart), but the automatic, zero-setup nature of a legacy GAL is genuinely lost until an org sets up its own shared-contacts group.
- **Open issue: Gap.** No first-class "organizational directory" object/onboarding flow is specified; left as an application built from the shared-address-book primitive (item 32). Worth naming this as a needed piece of on-boarding tooling for org/business deployments.

34. Contact photos — Clean

- **How:** Rides the vCard `PHOTO` field / JSContact media property, or a small inline `Attachment` (§2.5) for larger images.
- **Sense-check:** Non-issue.
- **Security:** Same as any other attachment/inline content — encrypted at rest and in transit, unlike legacy contact photos which are often fetched live from a provider's CDN (Gravatar-style), leaking a network request per contact-card view.
- **Open issue:** None.

35. Import/export — Clean

- **How:** Same CardDAV compatibility surface as item 30 (§8.4) — existing tools (Apple Contacts, DAVx) work unchanged.
- **Sense-check:** Direct hit.
- **Security:** Same as mail import/export (item 22): pulled from your own node under app-password auth, not generated by a third party.
- **Open issue:** None.

36. Contact key / verification (the DMTAP upgrade) — Clean, and the headline improvement

- **How:** Every contact is bound to a real identity key, pinned on first contact (TOFU, §3.4) and upgradeable to an out-of-band-verified pin (safety-number/QR comparison, §3.4); the key is tracked through key transparency (§3.5) so a silent key swap is detectable, and any

identity-migration (§1.6) is followed automatically and safely because contacts route by key, not name.

- **Sense-check:** This is not something legacy contacts have *any* analog for — a legacy contact card is just a display-name/email string with zero cryptographic binding, which is precisely why display-name spoofing and lookalike-domain phishing work at all. Adding real key identity to the contact model is the single most consequential upgrade in this whole audit.
- **Security:** Strictly better, not just "different." A "verified contact" (OOB-checked safety number) makes impersonation via display-name or lookalike-address cryptographically impossible for that relationship — legacy email has no equivalent ceiling on how good contact trust can get.
- **Open issue:** Adoption of OOB verification is, as always with this pattern (Signal/WhatsApp safety numbers), a UX problem more than a protocol one — most users will stay at TOFU-pinned rather than fully OOB-verified. This is a known, general limit of TOFU (§3.4's honest-limit note), not specific to contacts.

17.3 Calendar

37. Events — Clean

- **How:** Native **JSCalendar (RFC 8984)** MOTEs, synced via JMAP Calendars (§8.4); CalDAV (RFC 4791) projects the store as iCalendar (RFC 5545) for legacy clients.
- **Sense-check:** Direct hit, same "just another MOTE kind" pattern as mail/contacts (§8.4).
- **Security:** Better — no central CalDAV server ever holds your calendar in plaintext; end-to-end encrypted and device-cluster-synced like everything else.
- **Open issue:** None.

38. Recurring events (RRULE) — Clean

- **How:** JSCalendar (RFC 8984) has first-class recurrence-rule support carried natively; CalDAV/iCalendar projection (§8.4) exposes the legacy RRULE form unchanged for compatible clients.
- **Sense-check:** Direct hit; recurrence expansion/exceptions are an ordinary client responsibility, unaffected by decentralization.
- **Security:** No change; note RFC 8984 was itself designed specifically to fix long-standing `VTIMEZONE` /RRULE ambiguity bugs in iCalendar — an incidental improvement DMTAP inherits by choosing JSCalendar as the native form rather than legacy iCalendar.
- **Open issue:** None.

39. Invitations + RSVP + iTIP scheduling (peer-to-peer, no central server) — Clean (the cleanest mapping in this document)

- **How:** Explicitly specified (§8.4): "Calendar invitations and scheduling (iTIP-style) ride as MOTEs between participants — no central scheduling server; free/busy and RSVP are messages, not a server query."
- **Sense-check:** This is arguably not even a stretch — legacy iTIP (RFC 5546, `METHOD: REQUEST / REPLY / CANCEL`) was *already* conceptually a message-passing protocol riding over email; DMTAP simply carries the same request/reply pattern over MOTEs instead of MIME email. The architecture and the legacy feature were already aligned before DMTAP existed.
- **Security:** Better — invitation/RSVP content is end-to-end encrypted and sender-authenticated (payload signature, §2.4), whereas legacy iTIP-over-SMTP invitations are exactly as

spoofable as any other unauthenticated SMTP mail (a well-known calendar-phishing vector: forged meeting invites).

- **Open issue:** None.

40. Free/busy — Different

- **How:** Also explicit in §8.4: free/busy is "a message, not a server query" — a live request MOTE to the participant, answered by a reply MOTE, gated by the same recipient policy (§9) as any other cold contact.
- **Sense-check:** Correct given the no-central-server model, but genuinely changes the *availability characteristic* of the feature: legacy free/busy is typically a continuously-queryable published resource (a `.ifb` URL, or a CalDAV/EWS free-busy report) answerable at any time regardless of whether the person is "present." A message-based free/busy needs the recipient's node reachable (or queued for wake, §14.3) to answer, so it is not instantaneously queryable during their downtime the way a static published resource is.
- **Security:** Better in the common case — a stranger cannot silently scrape your calendar availability the way a public `.ifb` URL can be scraped; a free/busy request is gated by the same anti-abuse policy (§9) as a message from that sender. Harder for the legitimate "public booking page" use case (Calendly-style): there is no first-class, always-answerable public-availability resource defined.
- **Open issue: Gap.** No spec'd analog for a *publicly queryable* availability resource (vs. person-to-person messaged free/busy). Achievable as a broadcast-group address (§5.8) with an automated responder (same pattern as resource booking, item 47) but not written up as its own construct.

41. Reminders/alarms — Clean

- **How:** JSCalendar's alarm component, fired locally by whichever device is awake, using the same CRDT sync (§5.6) that already carries read/flag/label state so any device can pick up the reminder.
- **Sense-check:** Direct hit; identical mechanism to mail's read-state sync.
- **Security:** Same as items 1/18 — encrypted, device-cluster-synced, never provider-visible.
- **Open issue:** None.

42. Shared calendars — Different (same extrapolation as item 32)

- **How:** MLS group over calendar-event MOTES, the same shared-folder pattern §5.1/§5.7 establish for files; §8.4 confirms calendar shares "via the same MLS groups (§5) as everything else."
- **Sense-check:** Sound by direct analogy; not verbatim named as its own construct any more than shared address books are.
- **Security:** Same member-auditable, re-keyed-on-removal properties as any shared-folder MLS group (§6.7).
- **Open issue:** Same minor documentation gap as item 32 — worth an explicit naming pass.

43. Calendar delegation (assistant manages a calendar) — Different

- **How:** Same two mechanisms as mail delegation (item 23): a narrowly-scoped `DeviceCert` capability, or a UCAN-style capability delegation (§13.5) for "create/modify calendar MOTES for identity X, time-bound."
- **Sense-check:** Sound; reuses the general delegation primitive rather than inventing a calendar-specific one — consistent with the spec's stated preference for one mechanism over many.

- **Security:** Same tradeoff as item 23: tighter scoping than legacy (a calendar-only capability, not full mailbox access) but no visible "modified by delegate" marker by default.
- **Open issue:** Same as item 23 — no `Headers.ext` /event-property convention yet defined for attributing a delegate's edit.

44. Availability (working-hours preference, distinct from free/busy) — N/A

- **How:** Purely a local client preference (what hours you're "available" for booking purposes); never needs to leave the device.
- **Sense-check:** Not a protocol-relevant feature at all.
- **Security:** No change.
- **Open issue:** None.

45. Time zones — Clean (mild improvement)

- **How:** JSCalendar (RFC 8984) carries IANA-timezone-aware date/time fields natively; Cal-DAV projection maps to legacy `VTIMEZONE` /iCalendar for compatible clients (§8.4).
- **Sense-check:** Non-issue architecturally.
- **Security:** N/A (not a security property); functionally an improvement, since RFC 8984 was specifically designed to avoid the recurring `VTIMEZONE` -vs- `RRULE` interaction bugs that plague legacy iCalendar across DST transitions and floating times.
- **Open issue:** None.

46. Attachments on events — Clean

- **How:** Same `Attachment` /manifest mechanism as mail (§2.5), reused unchanged.
- **Sense-check:** Direct hit; no calendar-specific handling needed.
- **Security:** Better than legacy in the same way mail attachments are (item 7): no provider-imposed size ceiling, content-addressed integrity.
- **Open issue:** None.

47. Resource booking (rooms, equipment) — Clean

- **How:** A bookable resource is simply **another DMTAP identity** — a keypair with its own address (the same "group/identity with an address" pattern as §5.8) whose automation policy auto-accepts or conflict-checks incoming invitation MOTEs (item 39) on its own. No special protocol primitive is needed: it's an ordinary identity running automation, exactly like a "bot" node.
- **Sense-check:** This is a clean, arguably elegant emergent property: legacy resource booking requires a special mailbox type and server-side logic maintained by Exchange/Google Workspace; here it falls out for free from "identities can run automation over ordinary messages," needing zero new machinery.
- **Security:** Better in one respect — the resource's booking policy is enforced by its own node under its own key, not by a shared central admin console; auditable the same way any identity's activity is (signed messages, KT).
- **Open issue: Minor gap.** No reference automation-policy language for such "bot identities" (auto-accept rules, conflict resolution) is specified anywhere in the document — purely an implementation/client concern, but worth flagging as unwritten territory for a future client-behavior appendix.

17.4 Cross-cutting

48. Multi-device sync — Clean

- **How:** The owner's devices form a personal MLS group/cluster (§5.6); mailbox, flags, labels, and file index replicate as an encrypted CRDT, converging under out-of-order delivery. This is the one genuinely new component (§5.7) that unlocks mail, chat, and files together.
- **Sense-check:** This is DMTAP's strongest area — MLS's group-membership model is a better fit for "N devices, one identity" than legacy IMAP's per-device independent connections (no shared crypto identity across devices) or than pairwise Double-Ratchet fan-out (§5.2 notes this explicitly as the reason MLS was chosen).
- **Security:** Better — sync is end-to-end encrypted between the owner's own devices, not just between the owner and a central server; legacy IMAP sync trusts the provider to relay state between devices in whatever form the provider stores it.
- **Open issue:** Endpoint compromise has cluster-wide blast radius (§6.6.3) — a stolen synced device exposes the whole replicated mailbox history, not just a device-local slice. Mitigated by optional scoped sync (recent-N-days on mobile) but not eliminated; disclosed explicitly in §6.6, not a hidden gap.

49. Offline access — Clean

- **How:** The always-on node holds the full plaintext store and is the offline-first authority (§14.4); thin clients (§14.1) cache what they've synced, same as any modern IMAP client with local cache.
- **Sense-check:** Sound; offline access on the always-on node is unconditional (it's the source of truth), and thin-client offline access is bounded by whatever's been cached — the same shape modern mail apps already have (a phone's Gmail app also only shows what it's cached).
- **Security:** Same or better — the cache on a thin client is itself part of the encrypted-at-rest model (§6.7), not a plaintext local SQLite file the way many legacy mail apps cache today.
- **Open issue:** None beyond the disclosed cluster-blast-radius point already covered in item 48.

50. Push/notifications — Clean, and a genuine privacy improvement

- **How:** Wake-and-fetch (§14.3): push carries no content, only a wake signal (≤ 4 KiB, content-free, §16.6), through APNs/FCM via a notification proxy; the device wakes, opens its own authenticated connection, and drains/decrypts locally. "Push is a latency optimization, not delivery" — the client MUST still reconcile on foreground.
- **Sense-check:** Correctly modeled as the *only* architecture that works for a no-always-on- box user (§14.3), matching deployed precedent (Delta Chat/Chatmail, Signal, Matrix/Sygnal).
- **Security:** A real, stated improvement: Apple/Google's push infrastructure never sees plaintext or even a content preview — unlike many legacy mail providers whose push payloads have historically included message preview text delivered through Apple/Google's own push infrastructure (a real, historically-exploited leak surface). Content-free wake push structurally cannot leak content through the push channel.
- **Open issue:** iOS silent-push throttling requires wake coalescing/batching (§14.3) — a known platform constraint, not a DMTAP-specific gap.

51. Account migration — Clean, and better than legacy

- **How:** MoveRecord (§1.6) rebinds the human name while preserving the key, distributed via mesh/DHT, the transparency log, and a signed push-to-contacts MOTE; because contacts route by key (not name) after first contact, they follow the move automatically without being redirectable by a forged move.

- **Sense-check:** Correctly frames losing a domain as a change of *name*, not of *identity* — the reverse of legacy, where losing your provider or domain *is* losing your identity (new address, contacts must be manually re-notified, history often orphaned).
- **Security:** Strictly better: the move is signed and KT-audited (a squatter later registering the abandoned domain cannot hijack existing relationships, §1.6), and existing contacts survive the migration with zero action — legacy migration has no equivalent continuity guarantee at all.
- **Open issue:** Only *new* contacts who know solely the abandoned name are unreachable post-move — an unavoidable, disclosed tradeoff (§1.6), not a design flaw.

52. Backup/restore — Different

- **How:** Two separate things, and the spec is strong on one and silent on the other. (a) **Identity backup** — the recovery policy (§1.4: phrase/device/social-guardian methods, threshold-gated, VSS/FROST-hardened) is genuinely best-in-class: the key itself is effectively un-loseable given redundant recovery factors. (b) **Mailbox-content backup/DR** — i.e., “restore my actual mail/calendar/contacts data after data loss,” not just “recover my key” — is not separately specified as a DMTAP mechanism. It's implicitly provided by (i) multi-device replication (§5.6, each device holds/caches a copy, an incidental N-way backup) and (ii) whatever the node operator does at the infra layer — self-hosters own this entirely; a hosted hosted-operator (§14.6) presumably backs the per-tenant object-storage bucket, but this is an operator/infra choice, not a protocol guarantee.
- **Sense-check:** The identity half is exactly the right thing for a decentralized protocol to specify rigorously (§1.4 does). The *content*-backup half is legitimately a node/infra concern more than a protocol concern in a self-sovereign model (there is no central server whose backup policy the protocol could mandate) — but that also means there is currently no documented “export a full encrypted backup archive of everything” primitive the way, e.g., Signal ships a backup file, which a security-conscious self-hoster would reasonably expect.
- **Security:** Identity recovery: strictly better than legacy (“forgot password” flows, provider-controlled account recovery). Content backup/DR: no worse than legacy in the self-hosted case (you were always responsible for backing up your own server), but legacy *hosted* webmail has an implicit “the provider backs up their datacenters” guarantee that has no explicit DMTAP-protocol equivalent — it's an operator SLA question, not specified here.
- **Open issue: Gap.** Recommend a future revision define a client-level “export full encrypted backup archive” primitive (mailbox + calendar + contacts + keys-if-desired) at the client-access layer (§8), independent of any specific operator's infra practices, so self-hosters have a documented, portable disaster-recovery path that doesn't depend on reverse-engineering the IMAP/CalDAV/CardDAV export surfaces for a full-fidelity backup.

17.5 Summary table

#	Feature	Verdict	One-line note
1	Folders	Clean	JMAP mailboxes over CRDT-synced state (§5.6)
2	Labels/tags	Clean	Same CRDT state as folders (§5.6)
3	Filters/rules	Different	Run on the owner's own always-on node, not a third party (
4	Search	Different	On-device by design (§0.7); no cross-user index
5	Threading	Clean	<code>Headers.thread</code> + <code>refs</code> (§2.4)
6	Drafts	Clean	Pre-dispatch CRDT state (§5.6)

#	Feature	Verdict	One-line note
7	Attachments (any size)	Clean	Content-addressed manifests, 3 size tiers (§2.5, §5.5, §6.5)
8	Rich text/HTML	Clean	<code>mime / Body</code> fields (§2.4)
9	Signatures (text)	Clean	Client-local, unrelated to crypto <code>sig</code>
10	Aliases + plus-addressing	Clean	<code>Identity.names</code> list (§3.9.4); non-spoofable
11	Forwarding (manual)	Different	Works; forged- <code>From:</code> redirect correctly impossible
12	Auto-forward	Different	Works; rule-change auditing not yet specified (gap)
13	Vacation/auto-responder	Different	Improved: no backscatter (authenticated senders)
14	Read receipts	Clean	<code>kind=0x07</code> , opt-in, off by default (§2.3, §6)
15	Scheduled send	Clean	Node holds MOTE until send time
16	Snooze	Clean	Local UI state only
17	Undo send	Different	Only within pre-dispatch window, same as legacy in truth
18	Mark read/unread, flag/star	Clean	Explicitly named in §5.6
19	Archive	Clean	Mailbox-membership change (§5.6)
20	Spam/junk + block/allow lists	Different	Cost-based (§9) replaces central ML; no shared corpus
21	Quotas	Clean	Not a protocol concept; operator policy only (§12)
22	Import/export (mbox/EML)	Clean	Via IMAP compat surface (§8.2)
23	Delegation/send-on-behalf	Different	<code>DeviceCert.caps /UCAN</code> (§1.2, §13.5); no "on behalf of" ma
24	Shared mailboxes	Clean	Team-inbox group model (§5.8.1)
25	Distribution lists	Clean	Groups-as-address, hidden membership (§5.8)
26	Catch-all	Clean	Named tier-C option (§3.9.4)
27	Priority/importance	N/A	Cosmetic; undefined ext header, low value
28	Message recall	Harder	Cooperative-only, disclosed (§6.6.8); legacy recall never reall
29	DKIM/SPF/DMARC	Clean	Gateway-only, delegated selectors (§7, §3.8)
30	Contact cards	Clean	JSCalendar/CardDAV (§8.4)
31	Contact groups	Different	Split cleanly into addressable groups vs. local tags
32	Shared address books	Different	Extrapolated from shared-folder pattern (§5.1/§5.7); not ver
33	Auto-complete/directory	Harder	No default org directory; must be built as a shared-contacts
34	Contact photos	Clean	vCard field / inline attachment
35	Import/export	Clean	CardDAV compat surface (§8.4)
36	Contact key/verification	Clean	The headline upgrade — real key binding + KT + OOB
37	Events	Clean	JSCalendar/CalDAV (§8.4)
38	Recurring events (RRULE)	Clean	Native in JSCalendar; fixes legacy TZ/RRULE bugs
39	Invitations+RSVP+iTIP	Clean	Explicit, cleanest mapping in the audit (§8.4)
40	Free/busy	Different	Message/reply, not an always-queryable resource; no public-a
41	Reminders/alarms	Clean	Same CRDT sync as read/flag state
42	Shared calendars	Different	Extrapolated shared-folder pattern, same as item 32
43	Calendar delegation	Different	Same mechanism/gap as mail delegation (item 23)
44	Availability (working hours)	N/A	Purely local preference
45	Time zones	Clean	JSCalendar fixes legacy VTIMEZONE issues
46	Attachments on events	Clean	Same mechanism as mail attachments
47	Resource booking	Clean	A resource is just an automated identity; no new primitive r
48	Multi-device sync	Clean	Core strength of the design (§5.6, §5.7)
49	Offline access	Clean	Always-on node is offline-first authority (§14.4)
50	Push/notifications	Clean	Content-free wake-and-fetch (§14.3); real privacy win
51	Account migration	Clean	<code>MoveRecord</code> , contacts follow by key (§1.6)
52	Backup/restore	Different	Identity recovery is excellent (§1.4); content backup/DR unc

Tally: **34 Clean, 16 Different, 2 N/A, 1 Harder** (of the 2 features graded strictly "Harder," auto-complete/directory is the one with no fallback beyond manual setup; message

recall's "Harder" grade is really "same honest limit as legacy," included as Harder only because it is a capability legacy nominally advertises and DMTAP explicitly refuses to overclaim).

17.6 Gaps to resolve

Ranked by how load-bearing the gap is:

1. **Full-fidelity backup/restore export** (item 52) — no client-level "export everything, encrypted, portable" primitive is specified. Identity recovery (§1.4) is solved; mailbox/calendar/contacts disaster-recovery for self-hosters is not. Recommend a §8 addition.
2. **Auto-complete/organizational directory** (item 33) — no zero-setup GAL equivalent; needs either a documented "org directory = shared-contacts group" pattern and onboarding flow, or an explicit acceptance that this is permanently a build-it-yourself feature for organizations.
3. **Public, always-queryable availability/booking page** (item 40) — free/busy-as-message covers person-to-person scheduling well but has no analog for a public booking page (Calendly-style); worth a short note on composing it from a broadcast group + automated responder (same pattern as resource booking, item 47).
4. **Delegate-send/edit attribution marker** (items 23, 43) — delegated `send`-capable devices are indistinguishable from the principal; a `Headers.ext` convention for "sent/edited by delegate device X" would close this small but real gap versus legacy's `Sender: /`"on behalf of" distinction.
5. **Auto-forward rule-change auditing** (item 12) — silent forwarding-rule injection is a live real-world account-takeover technique; recommend surfacing rule changes through the same device-cluster-notification path already used for identity/KT self-monitoring (§3.5).
6. **Explicit naming of "shared address book" / "shared calendar" as named constructs** (items 32, 42) — the mechanism (MLS group over a data-class's MOTE's) is sound and consistent with how §5.5/§5.7 name shared file folders, but §8.4 doesn't spell it out the same way for contacts/calendar; a documentation gap, not a mechanism gap.
7. **No shared cross-node abuse-signal exchange** (item 20) — a disclosed, accepted tradeoff of the privacy model rather than an oversight, but worth tracking as adoption grows and spam patterns emerge that per-recipient-only policy handles less well than a shared corpus would.

None of these gaps require new cryptography or a new core primitive — every one is either a documentation gap (naming an already-consistent pattern), an extension-header convention (§10), or a client/onboarding feature to be built on existing primitives (groups, delegation, CRDT sync).

17.7 Net assessment

DMTAP achieves **practical feature parity** with classic email/calendar/contacts, and does so by mapping the overwhelming majority of legacy features onto existing primitives rather than inventing per-feature machinery: the MOTE kind system (§2.3), the device-cluster CRDT (§5.6), groups-as-address (§5.8), and the JMAP/IMAP/CalDAV/CardDAV compatibility surfaces (§8) between them account for 40+ of the 52 features audited here with no new mechanism required.

Of the remainder: a handful of features are **correctly** not preserved in their legacy form because that form depended on a central plaintext store or a forgeable identity that DMTAP deliberately removes (server-side spam ML, forged-`From:` redirect, a central searchable index, message recall as a real guarantee) — these are disclosed, intentional, and in most cases a net security improvement, not a parity failure. A smaller number are **genuine, resolvable gaps** (§17.6) — none load-bearing enough to block adoption, all closeable with documentation, an extension header, or an onboarding pattern rather than new protocol design.

And a few features are not just preserved but **substantively upgraded** by the decentralized model: verified contact identity (item 36) eliminates a whole class of impersonation/spoofing attacks legacy contacts have no defense against; account migration (item 51) gives continuity of address and relationships that legacy providers cannot; content-free push (item 50) closes a real historical leak surface; auto-responders (item 13) structurally cannot backscatter; and aliases (item 10) cannot be used to impersonate, unlike legacy plus-addressing.

Bottom line: nothing a classic email/calendar/contacts user relies on daily is missing without an honest, documented reason — either a clean or different-but-equivalent mapping exists, or the feature's legacy form depended on a central-plaintext-store property that DMTAP correctly declines to reintroduce. The open items in §17.6 are real but narrow, and closing them is client/documentation work on top of the existing primitives, not a redesign.

18. Appendix A: Wire Format (CDDL)

This appendix is the **machine-checkable, normative** rendering of every DMTAP wire object defined in prose in §1–§13. Where prose and this appendix appear to differ, the intent of the prose section governs and the divergence is a spec bug to reconcile (§10.4); the known divergences are catalogued in §18.11. Independent implementations **MUST** be able to encode and decode every object below, byte-for-byte identically, from this text alone.

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY**, and **OPTIONAL** are to be interpreted as in RFC 2119 / RFC 8174.

18.1 Conventions

18.1.1 Serialization — deterministic CBOR

All wire objects are encoded as **CBOR** (RFC 8949) using **Core Deterministic Encoding** (RFC 8949 §4.2). An encoder **MUST**, and a verifier of a signed object **MUST** re-check that:

1. Integers, string lengths, array/map counts, and tags use the **shortest possible** encoding of the argument (RFC 8949 §4.2.1, "preferred serialization"); no indefinite-length items are permitted.
2. **Map keys are sorted by their deterministic CBOR encoding, compared bitwise, ascending** (RFC 8949 §4.2.1). Because every object below is keyed by small unsigned integers, this reduces to ascending numeric key order (`0`, `1`, `2`, ...) with `0x00–0x17` encoded in one byte, `0x18–0xff` in two, and so on — but implementations **MUST** compare the *encoded key bytes*, not the abstract integer, so the two agree for all keys used here.
3. No duplicate map keys appear. A decoder **MUST** reject a map containing a duplicate key.
4. Floating-point values do not appear anywhere in DMTAP wire objects. Booleans appear only where a rule below admits `bool`.
5. NaN/Infinity, CBOR tags other than those explicitly defined here, and the CBOR `undefined` (major-type 7, value 23) **MUST NOT** appear and **MUST** be rejected on decode.

CBOR `null` (major-type 7, value 22, the byte `0xf6`) is the canonical representation of an **absent optional field** *only inside a signing preimage* (§18.9); on the wire an absent optional field is simply **omitted from the map**, never present with a `null` value. A decoder **MUST** reject a wire map that carries an optional key whose value is `null`.

18.1.2 Integer-key convention (COSE/CWT-style)

Every DMTAP object that is a CBOR **map** is keyed by **small unsigned integers**, exactly as COSE (RFC 9052) and CWT (RFC 8392) key their maps, for compactness and to make the deterministic key order trivial. Keys are assigned **per object type**, starting at `1`, and are

never reused with a different meaning across versions of the same object. Key `0` is reserved as a **variant discriminator** in the tagged-choice objects (`DeliveryTag` , `ChallengeResponse` , `RecoveryMethod`); no non-choice object uses key `0` .

Integer keys ≥ 64 (`0x40`) are RESERVED for future/extension fields, mirroring the reserved message-kind range (§2.3). A decoder processing a **signed** object MUST reject any key it does not recognize (fail closed) so that the signing preimage is unambiguous; a decoder processing an **unsigned** object MAY ignore unknown keys ≥ 64 . Text-keyed extensibility is confined to `Headers.ext` (§18.3.6).

18.1.3 Endianness

CBOR encodes all integers big-endian by construction (RFC 8949 §3), so map/field integers carry no separate endianness rule. Where a **signing/hashing preimage** concatenates raw fixed-width integers *outside* CBOR (only `Envelope.sender_sig` , §18.9.1), each such integer is encoded **big-endian, fixed width**: a `u8` as 1 byte, a `u64` as 8 bytes. All other preimages are deterministic CBOR and inherit CBOR's encoding.

18.1.4 Algorithm-suite tagging

Every top-level signed or encrypted object carries a `suite` field (a `u8` , §16.7). The suite selects, as a **set**, the signature algorithm, the KEM/PKE, the AEAD, and the content-hash for that object:

suite	Sign	KEM/PKE	AEAD	Hash
<code>0x01</code>	Ed25519	X25519 (HPKE, RFC 9180)	ChaCha20-Poly1305	BLAKE3-256
<code>0x02</code>	Ed25519 + ML-DSA-65	X-Wing (X25519 + ML-KEM-768)	ChaCha20-Poly1305	BLAKE3-256

A decoder MUST reject an object whose `suite` it does not implement (fail closed, §1.1); it MUST NOT guess. The `suite` value governs the **length and structure** of every `ik-pub` , `sig-val` , and encapsulated-key byte string in the same object (§18.2). The DMTAP `suite` (`u8`) is **distinct** from the MLS ciphersuite (a `u16` , e.g. `0x0001` , negotiated *inside* `Envelope.ciphertext` per RFC 9420, §5.1); the two never share a field.

18.1.5 Hash-agility prefix

Every content-address and every inter-object hash reference (`hash` in the grammar) is a **multihash-style, single-byte algorithm prefix followed by the raw digest**. The prefix values are drawn from the multicodec registry, truncated to one byte for v0 (all values below fit in one byte):

Prefix byte	Algorithm	Digest length	Status
<code>0x1e</code>	BLAKE3-256	32 B	v0 REQUIRED (default)
<code>0x12</code>	SHA2-256	32 B	RESERVED (compliance migration)
<code>0x16</code>	SHA3-256	32 B	RESERVED

Thus a v0 `hash` is exactly **33 bytes**: `0x1e || BLAKE3-256(preimage)` . The prefix lets an implementation migrate the digest algorithm (e.g. to SHA-256 where FIPS compliance requires it, §2.2) **without changing the address format**. A verifier MUST reject a `hash` whose prefix byte it does not implement, and MUST reject a digest whose length does not match the prefix's fixed length.

BLAKE3 is used in its **default (unkeyed) hashing mode with 256-bit (32-byte) output**. The `derive_key` and `keyed_hash` modes are NOT used for content addresses. Where BLAKE3 is used as a Merkle construction over chunk hashes, the domain-separated tree of §18.9.5 applies, not BLAKE3's internal tree.

18.1.6 Domain separation & signing (general rule)

Every signature in DMTAP is computed over a **domain-separated preimage**:

```
preimage = DS-tag || body
signature = Sign(sk_suite, preimage)
```

where `DS-tag` is the object-specific ASCII string from §18.9 terminated by a single `0x00` byte, and `body` is defined per object (usually the deterministic CBOR of the object **with its signature field(s) omitted**). Domain separation guarantees that a signature valid for one object type can never be replayed as a signature for another. A verifier **MUST** reconstruct the exact `DS-tag || body` and **MUST** reject any signature that does not verify against it under the key selected by `suite`.

For `suite = 0x02` (hybrid PQ), a `sig-val` is the **concatenation** `Ed25519_sig || ML-DSA-65_sig` and BOTH component signatures **MUST** verify; `Identity.sig` additionally carries one `sig-val` per suite in `suites` (§18.4.1).

18.1.7 CDDL prelude (shared productions)

The following CDDL rules (RFC 8610) are referenced by every object subsection and are collected once here. `.size` constraints marked "v0" hold for `suite = 0x01`; under `suite = 0x02` the suite-governed lengths change and the constraint is relaxed to the ranges noted in comments.

```
; — scalar aliases —————
u8      = uint .size 1           ; 0 .. 255
u16     = uint .size 2           ; 0 .. 65535
u32     = uint .size 4           ; 0 .. 4294967295
u64     = uint                   ; 0 .. 2^64-1 (CBOR shortest form)
ts      = u64                    ; ms since Unix epoch (§16.1)
suite   = 0x01..0xff            ; DMTAP algorithm-suite id (§18.1.4)

; — crypto byte strings (lengths suite-governed, §18.1.4) —————
hash    = bytes .size (33..129) ; 1-byte alg prefix || digest; v0 = 33
ik-pub  = bytes                  ; identity/device public key; v0 Ed25519 = 32 B
sig-val = bytes                  ; detached signature; v0 Ed25519 = 64 B
enc-key = bytes                  ; HPKE/AEAD content key; v0 = 32 B
peer-id = bytes                  ; libp2p PeerId = multihash(pubkey)
maddr   = tstr                   ; multiaddr, textual form (e.g. "/ip6/.../quic-v1")
```

18.2 Length & type governance by suite (normative)

For every byte-string field whose length is "suite-governed," the exact length under each suite is fixed by the table below. An implementation **MUST** reject a field whose length does not match the object's `suite`.

Field class	suite = 0x01	suite = 0x02
ik-pub (signing)	32 B (Ed25519)	32 B 1952 B (Ed25519 ML-DSA-65)
sig-val	64 B (Ed25519)	64 B 3309 B (Ed25519 ML-DSA-65)
enc-key (AEAD content key)	32 B (ChaCha20)	32 B (ChaCha20)
HPKE encapsulated key (inside ciphertext)	32 B (X25519)	32 B 1088 B (X-Wing)
hash digest	32 B (BLAKE3-256)	32 B (BLAKE3-256)

The PQ (0x02) lengths are RESERVED and given for forward planning; a v0 implementation implements only 0x01 and MUST reject 0x02 fail-closed until it supports it.

18.3 Message-layer objects (§2)

18.3.1 Envelope (§2.2)

```
Envelope = {
  1 => u8,           ; v      format version, = 0 in v0
  2 => suite,         ; suite  algorithm suite (§18.1.4)
  3 => hash,          ; id     content address of field 10 (ciphertext)
  4 => DeliveryTag,   ; to     routing target (§18.3.2)
  ? 5 => bytes,       ; epoch  MLS epoch / group-context ref (present iff group)
  6 => ts,            ; ts     sender timestamp (ms epoch)
  7 => u8,            ; kind    message kind (§2.3)
  ? 8 => KeyPackageRef, ; keypkg present iff this initiates an MLS session (§18.3.4)
  ? 9 => ChallengeResponse, ; challenge anti-abuse proof (§18.3.3)
  10 => bytes,        ; ciphertext MLS/HPKE sealed Payload (§18.3.5)
  11 => sig-val,      ; sender_sig detached sig by an EPHEMERAL per-message key
}
```

Field	Key	Type	Presence	Meaning & constraints
v	1	u8	MUST	Format version. MUST equal 0 in v0; a decoder MUST reject 0x02
suite	2	suite	MUST	Algorithm suite actually used for this MOTE (§18.1.4)
id	3	hash	MUST	Content address of the exact bytes of field 10 (ciphertext)
to	4	DeliveryTag	MUST	Routing target: recipient key, group id, or blinded tag
epoch	5	bytes	OPTIONAL	MLS epoch / group-context reference; present iff the group is an MLS group
ts	6	ts	MUST	Sender wall-clock timestamp, ms since Unix epoch. S
kind	7	u8	MUST	Message kind (§2.3): 0x00 mail ... 0x0a system; 0x4
keypkg	8	KeyPackageRef	OPTIONAL	Present iff this MOTE initiates an MLS session again
challenge	9	ChallengeResponse	OPTIONAL	Anti-abuse proof for a cold sender (§9), evaluated <i>w</i>
ciphertext	10	bytes	MUST	The MLS PrivateMessage or HPKE-sealed Payload
sender_sig	11	sig-val	MUST	Detached signature by an ephemeral, per-message key

18.3.2 DeliveryTag (§2.2a)

A tagged choice; key 0 is the variant discriminator.

DeliveryTag = KeyTag / GroupTag / BlindedTag

KeyTag = { 0 => 1, 1 => ik-pub } ; deliver to a recipient identity key (default)

GroupTag = { 0 => 2, 1 => bytes } ; deliver to an MLS group id (§5)
 BlindedTag = { 0 => 3, 1 => bytes } ; blinded per-contact tag BT = HKDF(ss, epoch_day)

Variant	Disc. (key 0)	Value (key 1)	Meaning & constraints
KeyTag	1	ik-pub	The recipient's persistent identity public key. Simplest; links
GroupTag	2	bytes (group id)	The stable group identifier (§5.8). Resolves to a group the r
BlindedTag	3	bytes (BT)	A per-contact blinded tag BT = HKDF(shared_secret, epoch_d

18.3.3 ChallengeResponse (§2.2b, §9)

A tagged choice; key 0 selects the proof type. All four are verifiable **without decrypting** the payload (§2.7).

ChallengeResponse = ArcToken / PowSolution / PostageStamp / Vouch

```

ArcToken = {
  0 => 1,           ; discriminator: ARC anonymous rate-limited credential (§9.3)
  1 => bytes,       ; issuer      issuer identifier / public key ref
  2 => bytes,       ; token       ARC presentation (per-origin-scoped)
  3 => bytes,       ; origin      recipient-origin scope the token is bound to
  ? 4 => bytes,     ; nonce       optional freshness nonce
}

PowSolution = {
  0 => 2,           ; discriminator: memory-hard proof-of-work (§9.4)
  1 => tstr,        ; algo        MUST be "argon2id" in v0
  2 => [ u32, u32, u32 ], ; params   Argon2id (m_KiB, t_iters, p_planes)
  3 => bytes,       ; epoch_nonce recipient-issued epoch nonce (anti-precompute, §16.5)
  4 => bytes,       ; solution   value satisfying the difficulty target
  5 => u8,          ; difficulty leading-zero-bit target set by recipient policy
}

PostageStamp = {
  0 => 3,           ; discriminator: prepaid real-money stamp (§9.5)
  1 => bytes,       ; issuer      postage issuer public key ref
  2 => bytes,       ; serial      unique serial (double-spend key, §9.5.1)
  3 => u64,         ; amount      value in minor currency units
  4 => tstr,        ; currency    ISO-4217 code, e.g. "USD"
  5 => ts,          ; expiry      stamp expiry (ms epoch)
  ? 6 => bytes,     ; audience    OPTIONAL recipient/gateway scope
  7 => sig-val,     ; sig         issuer signature (§18.9.7)
}

Vouch = {
  0 => 4,           ; discriminator: social introduction (§9.7)
  1 => ik-pub,      ; voucher     IK of the vouching contact (trusted by recipient)
  2 => ik-pub,      ; subject     key of the cold sender being vouched for
  3 => ik-pub,      ; recipient   the recipient this vouch is scoped to
  4 => ts,          ; exp         vouch expiry (ms epoch); MUST be rate-limited
  5 => sig-val,     ; sig         signature by `voucher` (§18.9.7)
}

```

Object	Field	Key	Type	Presence	Meaning & constraints
ArcToken	disc	0	1	MUST	Selects ARC.
	issuer	1	bytes	MUST	Issuer identity/key ref. An unknown/v
	token	2	bytes	MUST	The ARC presentation, per-origin-sco
	origin	3	bytes	MUST	The recipient-origin the credential is sco
	nonce	4	bytes	OPTIONAL	Freshness nonce within the 120 s validit
PowSolution	disc	0	2	MUST	Selects PoW (last-resort tier, §9.4).
	algo	1	tstr	MUST	MUST be "argon2id" (memory-hard) i
	params	2	[u32,u32,u32]	MUST	Argon2id parameters (memory KiB, iter
	epoch_nonce	3	bytes	MUST	Recipient-issued epoch nonce; scope is
	solution	4	bytes	MUST	The value whose Argon2id digest meets
	difficulty	5	u8	MUST	Leading-zero-bit target; SHOULD be ad
PostageStamp	disc	0	3	MUST	Selects postage.
	issuer	1	bytes	MUST	Issuer key ref; subject to reputation sco
	serial	2	bytes	MUST	Unique serial; the double-spend key. Th
	amount	3	u64	MUST	Face value in minor units of currency .
	currency	4	tstr	MUST	ISO-4217 currency code.
	expiry	5	ts	MUST	Expiry; an expired stamp MUST be rej
	audience	6	bytes	OPTIONAL	Scopes the stamp to a specific recipient,
	sig	7	sig-val	MUST	Issuer signature over (serial, amount, c
Vouch	disc	0	4	MUST	Selects vouch.
	voucher	1	ik-pub	MUST	IK of the introducing contact; the recipi
	subject	2	ik-pub	MUST	The cold sender's key being introduced.
	recipient	3	ik-pub	MUST	The recipient the vouch is scoped to; M
	exp	4	ts	MUST	Expiry; vouches MUST be rate-limited
	sig	5	sig-val	MUST	Signature by voucher over (subject, r

18.3.4 KeyPackageRef (§2.2, §5.3)

Reference to a **single** recipient KeyPackage consumed to initiate an MLS session. Distinct from KeyPackageBundleRef (§18.4.3), which locates the recipient's *whole* published bundle.

```

KeyPackageRef = {
  1 => hash,           ; ref  content address of the consumed KeyPackage
  2 => suite,           ; suite suite advertised by that KeyPackage
  ? 3 => tstr,         ; loc  OPTIONAL locator hint where it was fetched
}

```

Field	Key	Type	Presence	Meaning & constraints
ref	1	hash	MUST	Content address of the specific KeyPackage consumed. One-time KeyP
suite	2	suite	MUST	Suite the KeyPackage advertises; MUST equal Envelope.suite . Per-m
loc	3	tstr	OPTIONAL	Locator hint (mesh/relay address) where the KeyPackage bundle was

18.3.5 Payload (§2.4)

The plaintext sealed into Envelope.ciphertext . Only the recipient (or group members) can decrypt it; all sender identity, subject, recipients, threading, and content live here.

```

Payload = {
  1 => ik-pub,           ; from      sender identity key (IK); revealed only to recipient
  2 => sig-val,           ; sig       IK (or device key) over the canonical payload hash
  3 => Headers,           ; headers  (§18.3.6)
  4 => Body,              ; body    (§18.3.6)
  5 => [* hash],          ; refs    ids of MOTEs replied-to/referenced (threading)
  6 => [* Attachment],    ; attach  (§18.3.7)
  ? 7 => ts,              ; expires requested client-enforced expiry
  ? 8 => bytes,           ; fs_ratchet forward-secrecy ratchet material (§5.2)
}

```

Field	Key	Type	Presence	Meaning & constraints
from	1	ik-pub	MUST	Sender identity key (IK). For a known conta
sig	2	sig-val	MUST	Signature by from (IK) or an IK-authorized
headers	3	Headers	MUST	Message headers (§18.3.6). MAY be an emp
body	4	Body	MUST	Message body: text (tstr) or opaque MIME
refs	5	[* hash]	MUST (MAY be empty)	Content addresses of MOTEs this one replie
attach	6	[* Attachment]	MUST (MAY be empty)	Attachments (§18.3.7). Small ones inline; la
expires	7	ts	OPTIONAL	Requested expiry; client-enforced deletion. I
fs_ratchet	8	bytes	OPTIONAL	Forward-secrecy ratchet material (§5.2). Op

18.3.6 Headers and Body (§2.4)

```

Headers = {
  ? 1 => bytes,           ; thread  stable thread/conversation id
  ? 2 => tstr,            ; subject mail only
  ? 3 => tstr,            ; mime    content type of Body
  4 => [* ik-pub],        ; cc      additional recipient keys (fan-out is per-recipient)
  ? 5 => { * tstr => any }, ; ext     extension headers (§10)
}

```

Body = tstr / bytes ; UTF-8 text, or opaque MIME bytes

Field	Key	Type	Presence	Meaning & constraints
thread	1	bytes	OPTIONAL	Stable thread/conversation id for grouping; in
subject	2	tstr	OPTIONAL	Human subject line; meaningful for mail kind
mime	3	tstr	OPTIONAL	Media type of Body (e.g. "text/plain; charse
cc	4	[* ik-pub]	MUST (MAY be empty)	Additional recipient identity keys. Delivery fa
ext	5	{* tstr => any}	OPTIONAL	Text-keyed extension headers (§10). The only
Body	—	tstr / bytes	(as Payload.body)	tstr UTF-8 text; bytes opaque MIME p

18.3.7 Attachment and ManifestRef (§2.5)

```

Attachment = {
  1 => tstr,              ; name    display file name
  2 => tstr,              ; mime    media type
  3 => u64,               ; size    plaintext size in bytes
  ? 4 => bytes,           ; inline  present iff small (≤ inline threshold)
}

```

```

    ? 5 => ManifestRef,    ; manifest present iff large
    6 => enc-key,          ; key      per-file content key
}

```

```

ManifestRef = {
    1 => hash,              ; id      BLAKE3 Merkle-DAG root of the Manifest (§18.9.5)
    2 => u64,               ; size   total plaintext size
    3 => u32,               ; chunks NUMBER of chunks (a count)
}

```

Object	Field	Key	Type	Presence	Meaning & constraints
Attachment	name	1	tstr	MUST	Display name; MUST be treated as untrusted
	mime	2	tstr	MUST	Declared media type.
	size	3	u64	MUST	Plaintext size in bytes.
	inline	4	bytes	OPTIONAL	The encrypted-then-inlined content, present iff the
	manifest	5	ManifestRef	OPTIONAL	Reference to the file's manifest, present iff the
	key	6	enc-key	MUST	Per-file content key; the recipient decrypts chu
ManifestRef	id	1	hash	MUST	Content address = Merkle-DAG root of the M
	size	2	u64	MUST	Total plaintext size, equal to Manifest.size .
	chunks	3	u32	MUST	Count of chunks (note: in ManifestRef , chu

18.3.8 Manifest (§5.5)

```

Manifest = {
    1 => hash,              ; id      BLAKE3 Merkle root over chunk hashes (the content address)
    2 => u64,               ; size   total plaintext size
    3 => u32,               ; chunk_sz fixed chunk size (e.g. 1 MiB, §16.4)
    4 => [+ hash],          ; chunks ordered chunk hashes (list, ≥ 1)
    5 => enc-key,           ; key    file content key (delivered inside the MOTE, §2.5)
    6 => suite,             ; suite  suite for chunk encryption + hashing
}

```

Field	Key	Type	Presence	Meaning & constraints
id	1	hash	MUST	Content address = domain-separated BLAKE3 Merkle root over the
size	2	u64	MUST	Total plaintext file size in bytes.
chunk_sz	3	u32	MUST	Fixed chunk size; every chunk except possibly the last is exactly thi
chunks	4	[+ hash]	MUST	Ordered list of per-chunk content addresses <code>h_i = prefix BLAKE</code>
key	5	enc-key	MUST	File content key; used to encrypt/decrypt each chunk. Delivered on
suite	6	suite	MUST	Suite governing chunk AEAD and the digest algorithm of each <code>h_i</code>

18.4 Identity-layer objects (§1)

18.4.1 Identity (§1.3)

The published, signed, versioned identity; its `id` (a `hash` of this object, §18.9.4) is the anchor everyone pins.

```

Identity = {
  1 => [+ u8],           ; suites    supported suites, preference-ordered (a SET)
  2 => { + u8 => ik-pub }, ; iks      identity public key per suite
  3 => u64,               ; version  monotonically increasing
  4 => [* DeviceCert],    ; devices
  5 => KeyPackageBundleRef, ; keypkgs  location+hash of current KeyPackage bundle (§18.4.3)
  6 => hash,              ; recovery hash of the current RecoveryPolicy (§18.4.4)
  7 => [* tstr],          ; names    canonical human name(s) (§3)
  ? 8 => hash,            ; prev     hash of previous Identity version (hash chain)
  9 => ts,                ; ts
  10 => [+ sig-val],      ; sig      ONE signature per suite in `suites`, over the body
}

```

Field	Key	Type	Presence	Meaning & constraints
<code>suites</code>	1	<code>[+ u8]</code>	MUST	The suites this identity supports, a
<code>iks</code>	2	<code>{+ u8 => ik-pub}</code>	MUST	Map from each suite in <code>suites</code> to
<code>version</code>	3	<code>u64</code>	MUST	Monotonic version. A verifier MUS
<code>devices</code>	4	<code>[* DeviceCert]</code>	MUST (MAY be empty)	The identity's device certificates (§
<code>keypkgs</code>	5	<code>KeyPackageBundleRef</code>	MUST	Location + hash of the current Ke
<code>recovery</code>	6	<code>hash (RecoveryPolicyRef)</code>	MUST	Hash of the current <code>RecoveryPolicy</code>
<code>names</code>	7	<code>[* tstr]</code>	MUST (MAY be empty)	Canonical human name(s), e.g. "a
<code>prev</code>	8	<code>hash</code>	OPTIONAL	Hash of the previous <code>Identity</code> ver
<code>ts</code>	9	<code>ts</code>	MUST	Publication timestamp.
<code>sig</code>	10	<code>[+ sig-val]</code>	MUST	One signature per suite in <code>sui</code>

18.4.2 DeviceCert (§1.2)

```

DeviceCert = {
  1 => suite,           ; suite
  2 => ik-pub,          ; ik      root identity public key
  3 => ik-pub,          ; device_key device signing public key
  4 => tstr,            ; label   "phone", "home-box", ...
  5 => ts,              ; created
  ? 6 => ts,            ; expires
  7 => [+ tstr],        ; caps    capability strings
  8 => sig-val,         ; sig      IK over the body (§18.9)
}

```

Field	Key	Type	Presence	Meaning & constraints
<code>suite</code>	1	<code>suite</code>	MUST	Suite of <code>device_key</code> and <code>sig</code> .
<code>ik</code>	2	<code>ik-pub</code>	MUST	The root identity public key that issued this cert; MUST mat
<code>device_key</code>	3	<code>ik-pub</code>	MUST	The device's signing public key (day-to-day signing/auth).
<code>label</code>	4	<code>tstr</code>	MUST	Human device label. UTF-8.
<code>created</code>	5	<code>ts</code>	MUST	Issuance time.
<code>expires</code>	6	<code>ts</code>	OPTIONAL	Expiry; after it, verifiers MUST NOT accept <code>device_key</code> . Abs
<code>caps</code>	7	<code>[+ tstr]</code>	MUST	Capability set gating participation: "send", "recv", "relay"
<code>sig</code>	8	<code>sig-val</code>	MUST	IK signature over the body (§18.9.3).

18.4.3 `KeyPackageBundleRef` (§1.3, §5.3)

Locates and pins the identity's **whole** published `KeyPackage` bundle (distinct from `KeyPackageRef`, §18.3.4, which names one consumed package).

```
KeyPackageBundleRef = {
  1 => tstr,           ; loc      mesh/relay locator for the bundle
  2 => hash,           ; id       content address of the bundle
  ? 3 => [+ u8],       ; suites   suites advertised by the bundle
}
```

Field	Key	Type	Presence	Meaning & constraints
<code>loc</code>	1	<code>tstr</code>	MUST	Locator (mesh key, relay URL, or DNS <code>keypkgs=</code> value, §3.2) where
<code>id</code>	2	<code>hash</code>	MUST	Content address pinning the exact bundle bytes; a fetched bundle M
<code>suites</code>	3	<code>[+ u8]</code>	OPTIONAL	Suites the bundle offers; if present, MUST be a subset of <code>Identity.s</code>

18.4.4 `RecoveryPolicy`, `RecoveryMethod`, `Threshold` (§1.4)

`RecoveryPolicyRef` = `hash` ; hash of a `RecoveryPolicy` (`Identity.recovery`)

```
RecoveryPolicy = {
  1 => suite,
  2 => ik-pub,           ; ik
  3 => u64,             ; version
  4 => [+ RecoveryMethod], ; methods
  5 => Threshold,        ; recover_threshold what regains access
  6 => Threshold,        ; rotate_threshold  higher bar to change THIS policy
  ? 7 => hash,           ; prev hash chain
  8 => ts,
  9 => sig-val,          ; sig by IK, OR by a satisfied rotate_threshold quorum
}
```

`RecoveryMethod` = `PhraseMethod` / `DeviceMethod` / `SocialMethod`

`PhraseMethod` = { 0 => 1, 1 => bytes } ; recovery_key from a SLIP-0039/BIP39 phrase

`DeviceMethod` = { 0 => 2, 1 => ik-pub, 2 => tstr } ; device_key, label

`SocialMethod` = { 0 => 3, 1 => [+ ik-pub], 2 => u8 } ; guardians, threshold (VSS/FR0ST shares)

`Threshold` = { 1 => [+ MethodPredicate] } ; any_of –
satisfied if ANY predicate holds

`MethodPredicate` = { 1 => method-type, 2 => uint } ; method + required count

`method-type` = "phrase" / "device" / "social"

Object	Field	Key	Type	Presence	Meaning & constraints
<code>RecoveryPolicy</code>	<code>suite</code>	1	<code>suite</code>	MUST	Suite of <code>ik</code> / <code>sig</code> .
	<code>ik</code>	2	<code>ik-pub</code>	MUST	The identity this policy
	<code>version</code>	3	<code>u64</code>	MUST	Monotonic; reject $\leq$ last
	<code>methods</code>	4	<code>[+ RecoveryMethod]</code>	MUST	The recovery factors (ph
	<code>recover_threshold</code>	5	<code>Threshold</code>	MUST	Predicate that regains a

Object	Field	Key	Type	Presence	Meaning & constraints
PhraseMethod	rotate_threshold	6	Threshold	MUST	Predicate to change this
	prev	7	hash	OPTIONAL	Hash of the previous pol
	ts	8	ts	MUST	Timestamp.
	sig	9	sig-val	MUST	Signed by IK (proactive)
	disc	0	1	MUST	Phrase factor.
DeviceMethod	recovery_key	1	bytes	MUST	Key derived from a mne
	disc	0	2	MUST	Device factor.
	device_key	1	ik-pub	MUST	The device signing key a
SocialMethod	label	2	tstr	MUST	Human label.
	disc	0	3	MUST	Social/guardian factor.
	guardians	1	[+ ik-pub]	MUST	Guardian keys holding V
Threshold	threshold	2	u8	MUST	Number of guardian sha
	any_of	1	[+ MethodPredicate]	MUST	Disjunction: satisfied if a
	method	1	method-type	MUST	One of "phrase" , "devi
MethodPredicate	count	2	uint	MUST	Number of factors of me

18.4.5 KeyRotation (§1.5)

Cross-signed record authorizing a new root key. Distributed via the **Identity** chain and the KT log.

```

KeyRotation = {
  1 => suite,
  2 => ik-pub,           ; old_ik  the retiring root key
  3 => ik-pub,           ; new_ik  the incoming root key
  4 => tstr,             ; reason  free-text reason (e.g. "compromise", "pq-migration")
  5 => ts,
  ? 6 => hash,           ; prev
  7 => sig-val,          ; sig    by OLD_IK over (old_ik, new_ik, reason, ts)
}

```

Field	Key	Type	Presence	Meaning & constraints
suite	1	suite	MUST	Suite of the signature (old_ik 's suite). During PQ migration this r
old_ik	2	ik-pub	MUST	The retiring root key; MUST be the currently-pinned IK.
new_ik	3	ik-pub	MUST	The incoming root key. A verifier accepts it only via this valid cha
reason	4	tstr	MUST	Human/audit reason string.
ts	5	ts	MUST	Timestamp.
prev	6	hash	OPTIONAL	Hash chaining into identity history.
sig	7	sig-val	MUST	Signature by old_ik over the body (§18.9.3), proving continuity.

18.4.6 MoveRecord (§1.6)

Rebinds the human name while preserving the key.

```

MoveRecord = {
  1 => suite,
  2 => ik-pub,           ; ik
  3 => tstr,             ; from  "abc@old.com"
}

```

```

4 => tstr,           ; to      "abc@new.com" or a self-sovereign name (§3.6)
5 => ts,
? 6 => hash,         ; prev
7 => sig-val,        ; sig     by IK
}

```

Field	Key	Type	Presence	Meaning & constraints
suite	1	suite	MUST	Suite of <code>ik</code> / <code>sig</code> .
ik	2	ik-pub	MUST	The identity performing the move; the key is unchanged by a move.
from	3	tstr	MUST	The old canonical name.
to	4	tstr	MUST	The new canonical name (or self-sovereign name).
ts	5	ts	MUST	Timestamp.
prev	6	hash	OPTIONAL	Hash chain.
sig	7	sig-val	MUST	Signature by IK (§18.9.3). Contacts route by key and verify against

18.5 Transport-layer object (§4)

18.5.1 LocationRecord (§4.2)

The self-certifying `key` → `location` value record stored in the DHT (IPNS pattern).

```

LocationRecord = {
  1 => ik-pub,           ; ik      identity key (DHT key = multihash(ik))
  2 => peer-id,          ; peer_id  libp2p PeerId (MAY be per-epoch/unlinkable, §6)
  3 => [* maddr],        ; addrs   current reachability hints
  4 => u64,               ; seq     monotonic sequence number (rollback defense, §16.2)
  5 => u64,               ; ttl     record lifetime in seconds
  6 => ts,                ; ts      Publication time.
  7 => sig-val,          ; sig     signed by a device key
}

```

Field	Key	Type	Presence	Meaning & constraints
ik	1	ik-pub	MUST	Identity key; the DHT key is <code>multihash(ik)</code> .
peer_id	2	peer-id	MUST	libp2p PeerId; MAY be a per-epoch, unlinkable id t
addrs	3	[* maddr]	MUST (MAY be empty)	Reachability hints (multiaddrs): direct, relay-circuit
seq	4	u64	MUST	Monotonic sequence number; a resolver MUST reject
ttl	5	u64	MUST	Record lifetime in seconds (v0 default 2 h, §16.2); re
ts	6	ts	MUST	Publication time.
sig	7	sig-val	MUST	Signed by a device key (not necessarily IK) author

Divergence flagged (§18.11): §4.2's inline CBOR omits an explicit `seq` field, yet §16.2 normatively requires a "Location seq-number | monotonic u64" for rollback defense. This appendix adds `seq` (key 4) as REQUIRED, per §16.2, rather than overloading `ts`.

18.6 Group-layer objects (§5.8)

18.6.1 GroupState

The signed, versioned snapshot of a group's roster and policy, derived from the group's hash-chained MLS handshake log (§5.1). The **authoritative** ordering is the log; `GroupState` is the committer-signed projection members pin.

```
GroupState = {
  1 => bytes,           ; group_id
  2 => suite,
  3 => bytes,           ; epoch      MLS epoch / group-context ref
  ? 4 => tstr,          ; name       group's human name (§5.8), if any
  5 => ik-pub,          ; committer  the node serializing handshakes (§5.1)
  6 => posting-model,   ; posting_model
  7 => visibility,      ; membership_visibility
  8 => join-policy,     ; join_policy
  9 => [+ RosterEntry],; roster
  10 => hash,           ; log_head   hash-chained handshake-log head
  11 => u64,            ; version    monotonic
  12 => ts,             ; ts
  13 => sig-val,        ; committer_sig by `committer` over the body (§18.9.6)
}
```

```
RosterEntry = {
  1 => ik-pub,          ; member
  2 => [+ role],        ; roles
  3 => ts,              ; joined
}
```

```
posting-model = "broadcast" / "collaborative"
visibility     = "hidden" / "visible"
join-policy    = "closed" / "request" / "open" / "vouch"
role           = "owner" / "admin" / "member" / "poster" / "reader"
```

Object	Field	Key	Type	Presence	Meaning & constraints
<code>GroupState</code>	<code>group_id</code>	1	bytes	MUST	Stable group identifier; the g
	<code>suite</code>	2	suite	MUST	Suite of <code>committer_sig</code> and
	<code>epoch</code>	3	bytes	MUST	Current MLS epoch reference
	<code>name</code>	4	tstr	OPTIONAL	Group human name (<code>@team</code> ,
	<code>committer</code>	5	ik-pub	MUST	The member serializing hand
	<code>posting_model</code>	6	posting-model	MUST	"broadcast" (list: post→all;
	<code>membership_visibility</code>	7	visibility	MUST	"hidden" (per-member seale
	<code>join_policy</code>	8	join-policy	MUST	"closed" (invite only), "req
	<code>roster</code>	9	[+ RosterEntry]	MUST	Current members with roles;
	<code>log_head</code>	10	hash	MUST	Hash-chained head of the gro
	<code>version</code>	11	u64	MUST	Monotonic; reject $\leq$ last pin
	<code>ts</code>	12	ts	MUST	Snapshot time.
	<code>committer_sig</code>	13	sig-val	MUST	Signature by <code>committer</code> over
<code>RosterEntry</code>	<code>member</code>	1	ik-pub	MUST	Member identity key.

Object	Field	Key	Type	Presence	Meaning & constraints
	roles	2	[+ role]	MUST	≥ 1 role from {owner, admin
	joined	3	ts	MUST	Join timestamp.

18.6.2 GroupEvent (kind 0x06)

A thin wrapper carrying an opaque MLS handshake PDU with the committer's ordering. The MLS `MLSMMessage` (Proposal/Commit/Welcome) is defined by RFC 9420 and is **opaque** to DMTAP framing.

```
GroupEvent = {
  1 => bytes,           ; group_id
  2 => bytes,           ; epoch
  3 => bytes,           ; mls      opaque MLSMessage (RFC 9420 TLS-encoded)
  4 => u64,             ; log_seq  position in the hash-chained handshake log
  5 => hash,            ; prev     previous log entry's hash (chain)
  6 => sig-val,         ; committer_sig committer's ordering signature (§18.9.6)
}
```

Field	Key	Type	Presence	Meaning & constraints
group_id	1	bytes	MUST	Target group.
epoch	2	bytes	MUST	Epoch the handshake applies to.
mls	3	bytes	MUST	The opaque, TLS-encoded MLS <code>MLSMMessage</code> (RFC 9420). DMT
log_seq	4	u64	MUST	Strictly increasing position in the per-group log. A gap or dupli
prev	5	hash	MUST	Hash of the previous log entry (<code>GroupEvent</code>), chaining the log. C
committer_sig	6	sig-val	MUST	Committer signature over (group_id, epoch, mls, log_seq, prev

18.7 Auth-layer objects (§13)

18.7.1 Challenge (§13.3)

Created by the relying party (RP); presented to a trusted client, which binds and displays the verified origin.

```
Challenge = {
  1 => tstr,           ; rp_origin  the RP's true web origin (scheme://host[:port])
  2 => bytes,          ; nonce      single-use server nonce
  3 => ts,             ; issued_at
  4 => ts,             ; exp
  5 => tstr,           ; aud        intended RP audience identifier
  ? 6 => [* tstr],     ; scope     OPTIONAL requested scopes / capabilities
}
```

Field	Key	Type	Presence	Meaning & constraints
rp_origin	1	tstr	MUST	The RP's true origin. Phishing resistance depends entirely on
nonce	2	bytes	MUST	Single-use nonce; valid ≤ 120 s (§16.1). Reuse MUST be reject

Field	Key	Type	Presence	Meaning & constraints
issued_at	3	ts	MUST	Issue time.
exp	4	ts	MUST	Expiry; an assertion after <code>exp</code> MUST be rejected.
aud	5	tstr	MUST	Audience binding the assertion to the intended RP.
scope	6	[* tstr]	OPTIONAL	Requested login scopes / delegated capabilities (§13.5).

18.7.2 Assertion (§13.3)

The user's signed response. The signature is over the **canonical hash of the origin-bound fields** (§13.3 step 5), so a look-alike origin cannot produce a valid assertion for the real RP.

```
Assertion = {
  1 => tstr,           ; rp_origin  echoed; MUST equal the Challenge value
  2 => bytes,          ; nonce      echoed
  3 => ts,              ; issued_at  echoed
  4 => ts,              ; exp        echoed
  5 => tstr,            ; aud        echoed
  6 => ik-pub,          ; from      signer: session key or device key (per-RP, per-device)
  7 => sig-val,         ; sig        over the origin-bound preimage (§18.9.8)
}
```

Field	Key	Type	Presence	Meaning & constraints
rp_origin	1	tstr	MUST	Echo of <code>Challenge.rp_origin</code> ; the RP MUST verify it equals its own
nonce	2	bytes	MUST	Echo of <code>Challenge.nonce</code> ; MUST be unused and within its validity
issued_at	3	ts	MUST	Echo.
exp	4	ts	MUST	Echo; RP MUST reject if now > <code>exp</code> .
aud	5	tstr	MUST	Echo; MUST match the RP.
from	6	ik-pub	MUST	The signing key: a per-RP, per-device session key authorized by
sig	7	sig-val	MUST	Signature by <code>from</code> over the origin-bound preimage (§18.9.8). Session

18.9 Canonical signing & hashing preimages (normative)

Two independent implementations MUST produce **identical** signatures and content addresses. This requires an exact, shared definition of the bytes fed to `Sign` / `Hash`. Except where noted, every signature is over `DS-tag || det_cbor(object[] sig)`, where:

- `DS-tag` is the object's ASCII domain-separation string from the table below, **terminated by one `0x00` byte**;
- `det_cbor(object[] sig)` is the RFC 8949 §4.2 deterministic encoding of the object **with its signature field(s) removed from the map entirely** (not set to `null`) and all other fields present exactly as they appear on the wire.

Object	Signed field(s)	DS-tag (ASCII, + <code>0x00</code>)	Preimage body
Envelope	sender_sig (k11)	DMTAP-v0/envelope-sender	§18.9.1 (concatenation, not whole-object)
Payload	sig (k2)	DMTAP-v0/payload	§18.9.2 (over payload hash)
Identity	sig (k10, per suite)	DMTAP-v0/identity	<code>det_cbor(Identity [] {10})</code>

Object	Signed field(s)	DS-tag (ASCII, + 0x00)	Preimage body
DeviceCert	sig (k8)	DMTAP-v0/device-cert	det_cbor(DeviceCert [] {8})
RecoveryPolicy	sig (k9)	DMTAP-v0/recovery-policy	det_cbor(RecoveryPolicy [] {9})
KeyRotation	sig (k7)	DMTAP-v0/key-rotation	det_cbor(KeyRotation [] {7})
MoveRecord	sig (k7)	DMTAP-v0/move-record	det_cbor(MoveRecord [] {7})
LocationRecord	sig (k7)	DMTAP-v0/location-record	det_cbor(LocationRecord [] {7})
GroupState	committer_sig (k13)	DMTAP-v0/group-state	det_cbor(GroupState [] {13})
GroupEvent	committer_sig (k6)	DMTAP-v0/group-event	det_cbor(GroupEvent [] {6})
PostageStamp	sig (k7)	DMTAP-v0/postage-stamp	det_cbor(PostageStamp [] {7})
Vouch	sig (k5)	DMTAP-v0/vouch	det_cbor(Vouch [] {5})
Assertion	sig (k7)	DMTAP-v0/auth-assertion	§18.9.8

For `Identity` (`sig` is a **list**, one entry per suite): the **same** preimage `DS-tag || det_cbor(Identity [] {10})` is signed once per suite in `suites`, and the results are placed in `sig` in `suites` order. Each entry is a `sig-val` per its suite (§18.2).

18.9.1 Envelope.sender_sig

Per §2.2/§2.7 the ephemeral sender signature is over the concatenation (`id || to || ts || kind || challenge`), **not** the whole envelope. Exactly:

```
preimage = "DMTAP-v0/envelope-sender" || 0x00
           || id_bytes                ; field 3, the full hash byte string (prefix || digest)
           || det_cbor(to)             ; field 4, deterministic CBOR of the DeliveryTag map
           || u64be(ts)                ; field 6, 8 bytes big-endian
           || u8(kind)                 ; field 7, 1 byte
           || challenge_enc             ; field 9: det_cbor(ChallengeResponse) if present,
                                     ; else the single byte 0xf6 (CBOR null)

sender_sig = Sign(sk_ephemeral, preimage)
```

`id_bytes` is the raw byte string of `Envelope.id` (its CBOR head is *not* included). `to` and `challenge` are included as their deterministic CBOR encodings so structured tags are covered unambiguously. When `challenge` is absent, exactly one byte `0xf6` is appended (this is the only place a `null` appears in a preimage, §18.1.1). A verifier **MUST** reconstruct this preimage and check `sender_sig` under the ephemeral key **before** any decryption (§2.7 step 3).

18.9.2 Payload.sig

```
payload_hash = BLAKE3-256( det_cbor(Payload [] {2}) ) ; 32 bytes, no prefix
preimage     = "DMTAP-v0/payload" || 0x00 || payload_hash
Payload.sig  = Sign(sk_from, preimage)                ; sk_from = IK or device key
```

`from` (key 1) is included in the hashed body, binding the signature to the claimed sender. A device-key signature is valid only if that device key is authorized by a current `DeviceCert` under `from`'s `Identity` (§1.2). Verified at §2.7 step 8.

18.9.3 Identity-family objects

`Identity`, `DeviceCert`, `RecoveryPolicy`, `KeyRotation`, `MoveRecord`, `LocationRecord` all use the general rule: `Sign(sk, DS-tag || 0x00 || det_cbor(object [] {sig-key}))` with the DS-tags in

the table above. The signing key is: `IK` for `Identity / DeviceCert / MoveRecord`; `IK` or a satisfied `rotate_threshold` quorum for `RecoveryPolicy`; `old_ik` for `KeyRotation`; an authorized `device key` for `LocationRecord`.

18.9.4 Object content addresses (`Envelope.id` , `Identity anchor`)

`Envelope.id` is the content address of the **exact bytes** of `Envelope.ciphertext` , with **no** domain separation (so identical ciphertext dedups regardless of context, §2.8):

```
Envelope.id = 0x1e || BLAKE3-256( ciphertext_bytes ) ; v0 prefix 0x1e = BLAKE3-256
```

The `Identity anchor` everyone pins (the `id` referenced from DNS/KT, §3.2) is the content address of the fully-signed `Identity` object:

```
Identity_id = 0x1e || BLAKE3-256( det_cbor(Identity) ) ; the complete, signed object
```

Other `hash` references to a whole object (`recovery` , `keypkgs.id` , `prev` , `KeyPackageRef.ref` , `GroupState.log_head`) are computed the same way — `prefix || BLAKE3-256(det_cbor(referenced object))` — with no domain separation, since a content address is a pure function of the bytes.

18.9.5 Merkle-DAG manifest root (`Manifest.id` , `ManifestRef.id`)

The manifest root is an **RFC 6962-style binary Merkle tree** over the ordered chunk hashes, using the object's suite hash (v0 BLAKE3-256) with **domain-separated leaf/node prefixes** so a leaf can never be reinterpreted as an internal node (second-preimage defense):

; 1. Chunking & per-chunk hash (chunks are encrypted, then hashed)

```
enc_i = AEAD_encrypt(Manifest.key, plaintext_chunk_i) ; suite AEAD, per §2.5
```

```
h_i = 0x1e || BLAKE3-256( enc_i ) ; the value stored in Manifest.ch
```

; 2. Merkle tree over the ORDERED chunk hashes `h_0 ... h_{n-1}`

```
leaf(h_i) = BLAKE3-256( 0x00 || h_i ) ; 32-byte digest, no prefix
```

```
node(left, right) = BLAKE3-256( 0x01 || left || right ) ; 32-byte digest, no prefix
```

```
MTH([h_0]) = leaf(h_0)
```

```
MTH(h_0 ... h_{n-1}), n > 1 = node( MTH(h_0 ... h_{k-1}), ; k = largest power of 2 < n
                                MTH(h_k ... h_{n-1}) ) ; (RFC 6962 split rule)
```

; 3. Manifest content address

```
Manifest.id = 0x1e || MTH(h_0 ... h_{n-1})
```

`ManifestRef.id` (in an `Attachment`) **MUST** equal the referenced `Manifest.id`. The tree uses the RFC 6962 non-power-of-two split (no padding; unpaired subtrees carried up), so the root is deterministic for any chunk count $n \geq 1$. Each `h_i` self-verifies a fetched chunk; the tree self-verifies the manifest against `id` (§5.5).

18.9.6 Group objects

`GroupState.committer_sig` and `GroupEvent.committer_sig` use the general rule with tags `DMTAP-v0/group-state` / `DMTAP-v0/group-event`. These signatures assert **ordering by the committer only**; they do NOT substitute for the member signatures inside each MLS handshake (`GroupEvent.mls`), which every member **MUST** verify independently (§5.1, §5.8.2).

18.9.7 Anti-abuse objects

`PostageStamp.sig` and `Vouch.sig` use the general rule (`Sign(sk_issuer|voucher, DS-tag || 0x00 || det_cbor(obj` with tags `DMTAP-v0/postage-stamp` / `DMTAP-v0/vouch` . `ArcToken` and `PowSolution` carry no DMTAP signature of their own — an ARC presentation is verified by the ARC protocol (`draft-yun-privacypass-arc`) and a PoW by recomputing the Argon2id digest over `id || recipient || epoch_nonce` (§16.5).

18.9.8 Assertion.sig (auth)

Per §13.3 step 5 the signature is over the hash of the origin-bound fields:

```
auth_hash    = BLAKE3-256( det_cbor([ rp_origin, nonce, issued_at, exp, aud ] )
preimage     = "DMTAP-v0/auth-assertion" || 0x00 || auth_hash
Assertion.sig = Sign(sk_session, preimage)
```

The hashed array is a fixed 5-element CBOR array in exactly that order (echoing the `Challenge` fields), so the RP reconstructs it from its own issued `Challenge` and MUST reject any mismatch of `rp_origin` / `aud` . The signing key is the per-RP session key (§13.4), not `IK` .

18.10 Collected CDDL grammar (copy-paste block)

The following is the complete, self-contained CDDL for every DMTAP wire object. An implementer MAY copy this single block into a CDDL tool (RFC 8610) as the normative schema. It is internally consistent and consistent with §1, §2, §5, §13.

```
; =====
; DMTAP v0 – Wire Format (Appendix A, §18)
; Deterministic CBOR (RFC 8949 §4.2); integer-keyed maps (COSE/CWT).
; =====

; — scalar aliases —————
u8      = uint .size 1
u16     = uint .size 2
u32     = uint .size 4
u64     = uint
ts      = u64                ; ms since Unix epoch
suite   = 0x01..0xff        ; DMTAP algorithm suite

; — crypto byte strings (lengths suite-governed, §18.2) —————
hash    = bytes .size (33..129)    ; 1-byte alg prefix || digest; v0 = 33
ik-pub  = bytes                    ; v0 Ed25519 = 32 B
sig-val = bytes                    ; v0 Ed25519 = 64 B
enc-key = bytes                    ; v0 = 32 B
peer-id = bytes
maddr   = tstr

; — message layer (§2) —————
Envelope = {
  1 => u8, 2 => suite, 3 => hash, 4 => DeliveryTag,
```

```

    ? 5 => bytes, 6 => ts, 7 => u8,
    ? 8 => KeyPackageRef, ? 9 => ChallengeResponse,
    10 => bytes, 11 => sig-val,
}

```

```

DeliveryTag = KeyTag / GroupTag / BlindedTag

```

```

KeyTag      = { 0 => 1, 1 => ik-pub }

```

```

GroupTag    = { 0 => 2, 1 => bytes }

```

```

BlindedTag  = { 0 => 3, 1 => bytes }

```

```

ChallengeResponse = ArcToken / PowSolution / PostageStamp / Vouch

```

```

ArcToken     = { 0 => 1, 1 => bytes, 2 => bytes, 3 => bytes, ? 4 => bytes }

```

```

PowSolution  = { 0 => 2, 1 => tstr, 2 => [u32, u32, u32],
                  3 => bytes, 4 => bytes, 5 => u8 }

```

```

PostageStamp = { 0 => 3, 1 => bytes, 2 => bytes, 3 => u64, 4 => tstr,
                  5 => ts, ? 6 => bytes, 7 => sig-val }

```

```

Vouch        = { 0 => 4, 1 => ik-pub, 2 => ik-pub, 3 => ik-pub,
                  4 => ts, 5 => sig-val }

```

```

KeyPackageRef = { 1 => hash, 2 => suite, ? 3 => tstr }

```

```

Payload = {

```

```

    1 => ik-pub, 2 => sig-val, 3 => Headers, 4 => Body,
    5 => [* hash], 6 => [* Attachment], ? 7 => ts, ? 8 => bytes,

```

```

}

```

```

Headers = {

```

```

    ? 1 => bytes, ? 2 => tstr, ? 3 => tstr,
    4 => [* ik-pub], ? 5 => { * tstr => any },

```

```

}

```

```

Body = tstr / bytes

```

```

Attachment = {

```

```

    1 => tstr, 2 => tstr, 3 => u64,
    ? 4 => bytes, ? 5 => ManifestRef, 6 => enc-key,

```

```

}

```

```

ManifestRef = { 1 => hash, 2 => u64, 3 => u32 }

```

```

Manifest = {

```

```

    1 => hash, 2 => u64, 3 => u32, 4 => [+ hash], 5 => enc-key, 6 => suite,

```

```

}

```

```

; — identity layer (§1) —————

```

```

Identity = {

```

```

    1 => [+ u8], 2 => { + u8 => ik-pub }, 3 => u64,
    4 => [* DeviceCert], 5 => KeyPackageBundleRef, 6 => hash,
    7 => [* tstr], ? 8 => hash, 9 => ts, 10 => [+ sig-val],

```

```

}

```

```

DeviceCert = {

```

```

    1 => suite, 2 => ik-pub, 3 => ik-pub, 4 => tstr,

```

```

    5 => ts, ? 6 => ts, 7 => [+ tstr], 8 => sig-val,
}

KeyPackageBundleRef = { 1 => tstr, 2 => hash, ? 3 => [+ u8] }

RecoveryPolicyRef = hash
RecoveryPolicy = {
    1 => suite, 2 => ik-pub, 3 => u64, 4 => [+ RecoveryMethod],
    5 => Threshold, 6 => Threshold, ? 7 => hash, 8 => ts, 9 => sig-val,
}
RecoveryMethod = PhraseMethod / DeviceMethod / SocialMethod
PhraseMethod    = { 0 => 1, 1 => bytes }
DeviceMethod    = { 0 => 2, 1 => ik-pub, 2 => tstr }
SocialMethod    = { 0 => 3, 1 => [+ ik-pub], 2 => u8 }
Threshold       = { 1 => [+ MethodPredicate] }
MethodPredicate = { 1 => method-type, 2 => uint }
method-type     = "phrase" / "device" / "social"

KeyRotation = {
    1 => suite, 2 => ik-pub, 3 => ik-pub, 4 => tstr,
    5 => ts, ? 6 => hash, 7 => sig-val,
}

MoveRecord = {
    1 => suite, 2 => ik-pub, 3 => tstr, 4 => tstr,
    5 => ts, ? 6 => hash, 7 => sig-val,
}

; — transport layer (§4) —————
LocationRecord = {
    1 => ik-pub, 2 => peer-id, 3 => [* maddr],
    4 => u64, 5 => u64, 6 => ts, 7 => sig-val,
}

; — group layer (§5.8) —————
GroupState = {
    1 => bytes, 2 => suite, 3 => bytes, ? 4 => tstr, 5 => ik-pub,
    6 => posting-model, 7 => visibility, 8 => join-policy,
    9 => [+ RosterEntry], 10 => hash, 11 => u64, 12 => ts, 13 => sig-val,
}
RosterEntry    = { 1 => ik-pub, 2 => [+ role], 3 => ts }
posting-model  = "broadcast" / "collaborative"
visibility     = "hidden" / "visible"
join-policy    = "closed" / "request" / "open" / "vouch"
role           = "owner" / "admin" / "member" / "poster" / "reader"

GroupEvent = {
    1 => bytes, 2 => bytes, 3 => bytes, 4 => u64, 5 => hash, 6 => sig-val,
}

; — auth layer (§13) —————

```

```

Challenge = {
  1 => tstr, 2 => bytes, 3 => ts, 4 => ts, 5 => tstr, ? 6 => [* tstr],
}
Assertion = {
  1 => tstr, 2 => bytes, 3 => ts, 4 => ts, 5 => tstr,
  6 => ik-pub, 7 => sig-val,
}

```

18.11 Source inconsistencies flagged (not silently diverged)

While formalizing, the following discrepancies among §1/§2/§4/§5/§16 were found. This appendix resolves each explicitly rather than picking one silently; each SHOULD be reconciled in the prose (§10.4):

1. **Identity.keypkgs vs Identity.prekeys**. §1.3's object names the field `keypkgs` (type `KeyPackageBundleRef`); §5.3 refers to the same field as `Identity.prekeys`. This appendix uses **keypkgs** (§1.3, the authoritative object definition) — key 5 of `Identity`. Recommend fixing §5.3's prose to say `keypkgs`.
2. **Two distinct KeyPackage reference types**. §2.2's `Envelope.keypkg` is a `KeyPackageRef` (a single consumed package); §1.3's `Identity.keypkgs` is a `KeyPackageBundleRef` (the whole published bundle). The names are close enough to conflate. This appendix keeps them as two separate objects (§18.3.4, §18.4.3) and documents the distinction.
3. **LocationRecord missing seq**. §4.2's inline CBOR has no explicit sequence number, but §16.2 normatively requires a monotonic `Location seq-number (u64)` for rollback defense. This appendix adds **seq** as a **REQUIRED** field (key 4, §18.5.1). Recommend adding it to §4.2's CBOR.
4. **chunks means two different things**. In `ManifestRef` (§2.5) `chunks` is a `u32 count`; in `Manifest` (§5.5) `chunks` is a `[+ bytes]` **list of hashes**. Both are retained with their source meanings and the collision is called out in §18.3.7/§18.3.8. Recommend renaming `ManifestRef.chunks` → `chunk_count` in a future revision to remove the overload.
5. **Threshold predicate type undefined in prose**. §1.4 gives `Threshold = { any_of: [+ MethodPredicate] }` but never defines `MethodPredicate`. This appendix defines it as `{ method: ("phrase"/"device"/"social"), count: uint }` (§18.4.4), matching the prose example "1 phrase OR 2 devices OR 2 guardians". ("guardian" in that example maps to method `"social"`.) Recommend adding this definition to §1.4.
6. **RecoveryMethod discriminator representation**. §1.4 tags variants by a string field `type:"phrase"|"device"|"social"`. For a uniform integer-keyed choice, this appendix encodes the discriminator at **key 0** with integer values 1/2/3 (§18.3.3 convention), preserving the variant meaning while keeping deterministic small-integer keys throughout. This is a *format* normalization, not a semantic change; noted for consistency review.
7. **DMTAP suite (u8) vs MLS ciphersuite (u16)**. §1.1/§16.7 define `suite` as a one-byte DMTAP id (`0x01 / 0x02`); §5.1 also cites MLS ciphersuites `0x0001 / 0x0003`. These are different registries in different layers and **MUST NOT** share a field; §18.1.4 states this explicitly to prevent an implementer from conflating them.
8. **Group-state object was prose-only**. §5.8 describes roster/roles/join-policy/posting-model/ membership-visibility/committer but gives no CBOR. This appendix constructs

`GroupState` and `RosterEntry` (§18.6.1) faithfully to that prose; being newly formalized, they SHOULD get a review pass against any reference implementation.

Object count: this appendix gives a normative CDDL rule and a per-field semantics table for **24 wire objects** — `Envelope`, `DeliveryTag`, `ChallengeResponse`, `KeyPackageRef`, `Payload`, `Headers`, `Body`, `Attachment`, `ManifestRef`, `Manifest`, `Identity`, `DeviceCert`, `KeyPackageBundleRef`, `RecoveryPolicy`, `RecoveryMethod`, `Threshold`, `KeyRotation`, `MoveRecord`, `LocationRecord`, `GroupState`, `RosterEntry`, `GroupEvent`, `Challenge`, `Assertion` — plus their tagged sub-variants (`KeyTag` / `GroupTag` / `BlindedTag`; `ArcToken` / `PowSolution` / `PostageStamp` / `Vouch`; `PhraseMethod` / `DeviceMethod` / `SocialMethod`; `MethodPredicate`) and the shared scalar prelude (§18.1.7). Counting the four choice-variant families and `MethodPredicate` as distinct encodable structures brings the total to **36 CDDL-defined structures**, all collected in §18.10.

19. Appendix B: Protocol Operations

This appendix is the **operational cross-reference** of the specification: every operation any DMTAP role performs, specified to the depth needed to implement it without re-deriving it from the narrative sections. Where a narrative section (§§0–16) already states a rule, this appendix does not restate the rationale — it restates the rule as a normative, testable step and gives it a worked trace. Where this appendix and a narrative section conflict, the narrative section governs (§10.4) and the conflict is a bug in this appendix.

19.0 Conventions used in this appendix

Spec block. Every operation is specified with the same seven fields, in this order:

- **Purpose** — one paragraph, what the operation is for.
- **Initiator / Responder** — which role(s) perform which side. DMTAP is peer-to-peer, so these are not fixed "client"/"server" roles; they are named per operation (e.g. *sender* / *recipient node*, *RP* / *trusted client*).
- **Parameters** — each typed, marked **MUST** (required) or **OPTIONAL**, with its source section.
- **Preconditions** — what **MUST** already hold before the procedure may begin.
- **Procedure** — numbered, normative steps. A step that duplicates an ordering constraint from §2.7 or elsewhere keeps that section's step numbering visible in parentheses.
- **Success result** — the observable outcome and what state changes.
- **Failure modes** — every defined error condition and its response. No condition is left without an outcome (§2.7a's rule — "no undefined behavior" — is applied to every operation in this appendix, not only delivery).
- **Idempotency / retry** — whether re-invoking with the same parameters is safe, and what a retrying caller should expect.

Each operation also gets a **worked trace**: an annotated request/response exchange over the actual DMTAP objects (CBOR maps shown as pseudo-JSON for readability; field names match §1–§16 verbatim). Traces use role tags in place of IMAP's fixed `C:` / `S:`, since DMTAP has no client/server asymmetry at the protocol level:

A: the initiating party (sender, resolver, RP, gateway – named per operation)
 B: the responding party (recipient node, resolver target, trusted client)
 DHT: the mesh DHT (§4.2), not a single peer
 KT: the key-transparency log (§3.5)

A trace line is `TAG → OBJECT{fields}` (send) or `TAG ← OBJECT{fields}` (receive), with a trailing `# comment`. Fields irrelevant to the point being illustrated are elided with `...`.

Error taxonomy. Three outcome classes recur across every operation in this appendix, matching §2.7a's precedent:

Class	Meaning
Reject (fail-closed)	Malformed, unauthenticated, or policy-violating input. The operation MUST NOT proceed.
Defer	Input is well-formed but cannot be authorized/decided yet (offline peer, unreachable peer, etc.).
Silent drop	Reserved for the narrow, explicitly-listed cases where surfacing the failure to the sender is not appropriate.

An operation's Failure-modes table classifies every listed condition into one of these three.

19.1 Naming operations (§3)

19.1.1 `resolve(name) → identity`

Purpose. Turn a human-facing `name@domain` (or self-sovereign name, §3.6) into a KT-verified, pinned identity key, per the resolution algorithm of §3.3.

Initiator / Responder. Initiator: any node needing to address `name` for the first time (resolving node, "A"). Responder: the DNS/name-backend resolver chain and the KT log ("KT"); there is no single peer responder.

Parameters.

- `name` (`tstr`, MUST) — `local@domain` form (§3.9.1) or a self-sovereign name (§3.6).
- `expected_suite` (`u8`, OPTIONAL) — if the caller already holds a prior pin for `name` (re-resolution after a claimed rotation), the suite it was last pinned under.
- `require_oob` (`bool`, OPTIONAL, default `false`) — caller requests that the operation refuse to complete without out-of-band verification (§3.4.1), regardless of KT availability.

Preconditions. None (this is the first-contact entry point). If `name` is already pinned locally, callers SHOULD use the local pin directly and only invoke `resolve` again when the pinned `Identity` chain (§1.3) itself signals a rotation/migration (§3.3 step 5).

Procedure (normative; mirrors §3.3, numbered identically).

1. DNS/name-backend lookup: query `<local>._dmtap.<domain>.TXT` (§3.2) → `{iks, id, kt, keypkgs}`. If no record resolves: **fail** `NAME_NOT_FOUND`.
2. **(first contact only)** Fetch a signed KT tree head + inclusion proof for `id` from `kt` (§3.5). If KT is unreachable, partitioned, or censored: apply the §3.3 fail-closed rule — refuse to pin, or (if the caller's policy allows) hard-warn and require explicit user acceptance. Silent TOFU is prohibited. If `require_oob` is set, this step is mandatory and a KT outage is a hard failure, not a warn-and-continue.
3. Fetch the full `Identity` object (§1.3) from the mesh by `id`; verify every `sig` in `Identity.sig` validates under the corresponding `iks[suite]`, and that the chain (`prev`) is consistent with anything previously pinned for this `name`. Reject on any signature failure or chain inconsistency (`CHAIN_INVALID`).
4. Pin (`name → iks, id`) locally (TOFU), recording the pin as **unverified** unless step 5 ran.
5. **(optional, if `require_oob` or user-initiated)** Perform out-of-band safety-number comparison (§3.4.1); on match, upgrade the pin to **verified**.
6. Return the pinned identity. Thereafter, routing for this contact uses the mesh (§4) by key; `resolve` is not invoked again for this contact unless the pinned `Identity` chain itself carries a new version.

Success result. A locally pinned (`name`, `iks`, `id`, `suites`, `keypkgs`, `pin_state` ∈ {`unverified`, `verified`}) tuple, usable by `lookup-location` (§19.2.2) and `deliver` (§19.3.1).

Failure modes.

Condition

No DNS/name-backend record for `name`

KT unreachable at first contact, `require_oob` unset

KT unreachable, `require_oob` set

`Identity` signature fails to validate under any advertised suite

`Identity.prev` chain inconsistent with a previously-seen version for this `name` (rollback/fork)

Only suites the resolving node does not implement are offered

Resolution succeeds but returns a key that contradicts an **already-pinned** key for `name`, with no valid `KeyR`

Idempotency / retry. Idempotent: re-invoking with the same `name` after a prior success returns the same pin (or a newer one only via a valid chain). Safe to retry on `NAME_NOT_FOUND` or `KT-unreachable` failures with backoff; retrying does not change any durable state until a success path completes.

Example trace.

A: `resolve("alice@example.org")`

A → DNS: `TXT? alice._dmtap.example.org`

DNS ← A: `"v=dmtap1; suite=1; ik=b64:MFk...; id=blake3:9f2a...; kt=https://kt.example.org/log; key"`

A → KT: `signed-tree-head + inclusion-proof? id=blake3:9f2a...`

KT ← A: `{ tree_head, proof }` # verifies; no newer version exists

A → mesh: `fetch Identity by id=blake3:9f2a...`

mesh ← A: `Identity{ suites:[1], iks:{1: MFk...}, version:3, prev:blake3:7c0d..., names:["alice@e"`

A: `verify sig under iks[1]` # OK

A: `verify prev chain against nothing-prior (first contact)` # OK, nothing to contradict

A: `PIN (alice@example.org → ik=MFk..., id=9f2a..., pin_state=unverified)`

A: `resolve() → { ik: MFk..., suites:[1], keypkgs: ..., pin_state: unverified }`

19.1.2 `publish-identity(identity)`

Purpose. Publish a new or updated `Identity` object (§1.3) — initial identity creation, device addition/removal, suite migration, or any field change — to the transparency log and the mesh, making it discoverable and auditable.

Initiator / Responder. Initiator: the identity owner's node holding `IK` (or a device authorized per §1.2/§1.4). Responder: the KT log (append) and the mesh DHT/content store (publish by content address).

Parameters.

- `identity` (`Identity`, **MUST**) — the fully-formed object (§1.3): `suites`, `iks`, `version`, `devices`, `keypkgs`, `recovery`, `names`, `prev`, `ts`, `sig`.
- `announce` (`bool`, **OPTIONAL**, default `true`) — whether to also push a signed `identity` kind MOTE (§2.3 kind `0x09`) to existing contacts announcing the change.

Preconditions.

1. `identity.version` = (previous published version) + 1, or `1` for initial publication.
2. `identity.prev` = hash of the previous published `Identity` (or absent, only for `version=1`).
3. `identity.sig` contains a valid signature under **every** suite listed in `identity.suites` (§1.3's multi-suite rule) — a partial signature set is invalid, not partially accepted.

4. For a `RecoveryPolicy` change carried transitively via `identity.recovery`: the *authorizing* principal is `IK` (proactive) or a satisfied `rotate_threshold` quorum (reactive) — never a single `admin`-capable device alone (§1.2, §1.4 rule 1–2).

Procedure (normative).

1. Validate preconditions locally before publishing (a node MUST NOT publish an object it cannot itself verify).
2. Append `identity` to the owner's KT log entry stream: submit to the log operator(s); receive a signed tree head + inclusion proof covering this entry.
3. Publish `identity` to the mesh DHT/content store, addressed by its content hash (so `resolve`'s step 3, §19.1.1, can fetch it).
4. Update the DNS/name-backend `id=` pointer (§3.2) to the new object's hash (out of protocol scope for *how* the domain record is edited — see §3.8 onboarding tiers — but the new `id` MUST be published before or atomically with the DNS update, never after, so there is no window where DNS points to a since-superseded object).
5. If `announce`, construct and send a `kind=0x09 identity` MOTE (§2.3) to every contact in the owner's local address book, carrying the new `Identity` (or a reference to it) so the owner's own devices (self-monitoring, §3.5) and contacts learn of the change without waiting for their own next `resolve`.
6. The owner's **other devices** monitor the KT log for entries under this identity and MUST alert the owner if a new version appears that none of the owner's devices initiated (intrusion detection, §3.5) — this is a standing background procedure, not a step of this operation, but `publish-identity` is precisely the event it watches for.

Success result. New `Identity` version durably in the KT log (with inclusion proof) and discoverable on the mesh by content hash; existing contacts notified if `announce` was set.

Failure modes.

Condition	Class	Response
<code>version</code> is not exactly <code>previous+1</code>	Reject	<code>VERSION_CONFLICT</code> ; caller must re-
<code>prev</code> does not match the hash of the currently-published object	Reject	<code>CHAIN_MISMATCH</code> (concurrent-pub
Signature set does not cover every suite in <code>suites</code>	Reject	<code>INCOMPLETE_SIGNATURE</code> ; publicatio
<code>RecoveryPolicy</code> change authorized by less than <code>rotate_threshold</code>	Reject	<code>INSUFFICIENT_QUORUM</code> ; §1.4 rule 1
KT log operator unreachable	Defer	Publication is queued/retried; th
DHT publish fails (no reachable peers to store at)	Defer	Retry with backoff; KT entry alr

Idempotency / retry. Not naturally idempotent (each call advances `version`), but concurrent-publish races are made safe by the `prev`-chain precondition: a second publish attempt built against a stale `prev` is rejected (`CHAIN_MISMATCH`) rather than silently forking the identity history, forcing the caller to rebase on the accepted head and retry.

Example trace.

```
A: publish-identity(Identity{ version:4, prev:blake3:9f2a..., devices:[...,new_phone_cert],
                               suites:[1], iks:{1:MfK...}, sig:[Ed25519(...)] })
A: local check: version 4 == prev-version(3)+1                # OK
A: local check: sig covers suite 1 (the only suite in `suites`) # OK
A → KT:  append(Identity{v4})
KT ← A:  { tree_head_42, inclusion_proof }
A → DHT:  put(hash(Identity{v4}) → Identity{v4})
DHT ← A:  stored at 20 closest peers
A: (announce=true) → contacts: MOTE{ kind:0x09, payload: ref(Identity{v4}) } # to each pinned c
A: publish-identity() → { version:4, tree_head:42 }
```

19.1.3 publish-move(move_record)

Purpose. Rebind a human name (`from` $\rightarrow$ `to`) while preserving the identity key, per §1.6, distributing the rebinding via all three required channels.

Initiator / Responder. Initiator: identity owner's node ($\mathbf{IK}$). Responders: KT log, mesh, and every currently-known contact.

Parameters.

- `move_record` (MoveRecord, MUST) — {suite, ik, from, to, ts, prev, sig} (§1.6), sig by IK.

Preconditions.

1. `from` is a name currently in the owner's published `Identity.names`.
2. `to` is either a name the owner already controls resolution for (its own domain/self- sovereign backend, §3.6) or a newly claimed provider-issued name (§3.8 Tier B/C onboarding already completed for `to` before this call).
3. `move_record.sig` validates under the current pinned `IK`.

Procedure (normative).

1. Validate preconditions locally.
2. Publish `move_record` to the KT log (auditable, ordered — defeats a later squatter of `from` claiming continuity, §1.6).
3. Publish an updated `Identity` version (§19.1.2) whose `names` list adds `to` (and MAY retain `from` as a legacy alias per §3.9.4, or drop it if the domain is truly lost).
4. Update the mesh `LocationRecord / Identity.names` so future `resolve("to")` calls succeed immediately and existing key-based routing (mesh) is unaffected (contacts already routing by key never needed `to` at all).
5. Push a signed `kind=0x09` `identity` MOTE carrying `move_record` to every existing contact (push to contacts, §1.6 item 3). Contacts verify `move_record.sig` against their **pinned** `IK` for the owner (not against a fresh `resolve` of `from`, which could be squatted after abandonment) and update their local display name; routing is unaffected because it was already key-based.

Success result. `to` resolves to the owner's key; `move_record` is durably logged; existing contacts' clients display the new name and continue routing by the unchanged key.

Failure modes.

Condition
<code>from</code> not in the owner's current <code>Identity.names</code>
<code>move_record.sig</code> does not validate under pinned <code>IK</code>
<code>to</code> is not yet resolvable (onboarding for <code>to</code> incomplete)
KT log unreachable
A contact's client cannot verify <code>move_record.sig</code> against its pinned <code>IK</code> (e.g. the contact never actually had <code>to</code>)

Idempotency / retry. Re-publishing the same `move_record` is safe (KT append is content-addressed dedup, §2.2's "identical ciphertext shares an id" principle applies analogously); re-sending the contact-announcement MOTE to a contact that already acked it is a duplicate under §2.6 and is acked-without-reprocessing by that contact.

Example trace.

```
A: publish-move(MoveRecord{ ik:MFk..., from:"alice@oldhost.com", to:"alice@example.org",
                             ts:1752600000000, prev:blake3:aa11..., sig:Ed25519(...) })
```

```

A → KT:  append(MoveRecord)
KT ← A:  { tree_head_43, inclusion_proof }
A → DHT: update Identity.names += "alice@example.org"
A → bob (existing contact, routed by key, not by name):
    MOTE{ kind:0x09, to: bob_key, payload: { move_record } }
bob ← A: verifies move_record.sig against PINNED ik=MFk...      # matches, trusted
bob: updates local display name "alice@oldhost.com" → "alice@example.org"
bob: routing unaffected – still addressing ik=MFk... directly
bob → A: ack(id)

```

19.2 Reachability operations (§4)

19.2.1 publish-location(location_record)

Purpose. Publish/refresh the signed `key → location` `LocationRecord` (§4.2) that lets peers find the node's current reachability hints (addresses, relay circuits) without a static IP.

Initiator / Responder. Initiator: the node itself (every node publishes its own record; no third party publishes on a node's behalf). Responder: the DHT (the K closest peers to `hash(ik)`, §4.2).

Parameters.

- `ik` (bytes , MUST) — identity key; the DHT key is `hash(ik)` .
- `peer_id` (bytes , MUST) — current libp2p peer id (MAY be per-epoch/unlinkable, §6).
- `addrs` ([* multiaddr] , MUST) — current reachability hints: direct addresses, relay circuits, mix addresses (§4.3).
- `ttl` (u64 , MUST) — record lifetime; v0 default 2 h (§16.2).
- `seq` (u64 , MUST) — monotonically increasing sequence number (rollback defense).
- `sig` (bytes , MUST) — signature by a device key over the above.

Preconditions.

1. `seq` strictly greater than the last `seq` this node has published (monotonic, never reused even across restarts — a node MUST persist its last-used `seq` or derive it from a monotonic clock to avoid accidental rollback after a crash).
2. The signing device key is a current, non-expired, non-revoked `DeviceCert` (§1.2).

Procedure (normative; the IPNS-pattern value-record publish, §4.2).

1. Assemble `LocationRecord{ik, peer_id, addrs, ttl, ts, sig}` with `seq` per the precondition.
2. Sign with a device key (not necessarily `IK` — location changes constantly and MUST NOT require the offline-capable root key).
3. Store the record at the **K closest peers** to `hash(ik)` (§16.2: $K=20$), using **S/Kademlia disjoint-path** lookups (≥ 3 node-disjoint paths) to reduce single-eclipse exposure (§4.2 caution).
4. Schedule **aggressive republish**: re-invoke this operation at the republish interval (§16.2: 45 min, jittered), strictly before `ttl` elapses, incrementing `seq` each time even if `addrs` is unchanged (a stale-but-still-valid record is itself a failure mode the mesh must avoid, §4.2).

Success result. The record is stored (with best-effort replication) at the K closest peers; a `lookup-location` for `ik` by any peer within `ttl` returns this record.

Failure modes.

Condition

seq not strictly greater than the node's last-published value
 Signing device key expired/revoked
 Fewer than a usable quorum of the K closest peers accept the store (partial write, e.g. under active eclipse)
 No DHT peers reachable at all (total network partition)

Idempotency / retry. Each invocation MUST use a fresh, strictly-increasing seq, so exact re-invocation with identical parameters is not meaningful — but re-publishing after a failed attempt with a bumped seq is always safe and is exactly what the republish schedule does.

Example trace.

```
A: publish-location(ik=MfK..., peer_id=Qm7f..., addrs=[/ip6/.../udp/443/quic, /relay-circuit/...]
    ttl=7200000, seq=118)
A: local check: 118 > last_published_seq(117)                # OK
A: sign with device_key "home-box"
A → DHT: put(hash(MfK...), LocationRecord{seq:118, ...}) via 3 disjoint paths
DHT ← A: stored at 18/20 of the K closest peers (2 timed out –
acceptable, quorum met)
A: schedule republish at now+45min (jitter ±5min), seq→119
A: publish-location() → { stored_at: 18, ttl_expires: now+2h }
```

19.2.2 lookup-location(ik) → LocationRecord

Purpose. Retrieve the current key → location record for a peer, so a sender can attempt delivery over the reachability ladder (§19.2.3).

Initiator / Responder. Initiator: any node wanting to reach ik. Responder: the DHT (the K closest peers to hash(ik)), or a non-DHT path (cached address, rendezvous) tried first per §4.2's resolution order.

Parameters.

- ik (bytes, MUST) — the target identity key.
- allow_dht (bool, OPTIONAL, default true) — whether to fall back to the public DHT if non-DHT paths fail; a closed/organizational deployment MAY set this false and use only its private DHT (§4.2).

Preconditions. None (this is itself the discovery step); however per §4.2, a node SHOULD attempt non-DHT paths first regardless of whether it has ever contacted ik before.

Procedure (normative; §4.2's resolution order).

1. Try **cached direct addresses** last successfully used for ik (if any). If a cached address answers a liveness probe, return it immediately — skip DHT entirely.
2. Try **relay-reservation / rendezvous ("home relay") addresses** if the caller has an out-of-band or previously-learned rendezvous hint for ik. This step exists precisely so a fresh contact is not 100% dependent on a hostile public-DHT lookup (§4.2).
3. **DHT fallback** (only if allow_dht): query the K closest peers to hash(ik) via S/Kademlia disjoint-path lookups; among returned candidate records, accept only the one with the **highest seq** (rollback defense) whose sig validates under a current device key of ik's pinned Identity. Discard any record with seq ≤ a previously-seen seq for this ik (reply/rollback).
4. If multiple returned records disagree (different peers return different "highest" records) — a signature that the eclipse/censorship caution (§4.2) warns can happen — prefer the record

with the highest `seq` that a majority of the disjoint-path queries agree on; if no majority exists, treat as `LOCATION_UNRESOLVED` (do not guess).

Success result. A verified `LocationRecord` for `ik`, usable to attempt the reachability ladder (§19.2.3).

Failure modes.

Condition	Class	Response
No cached, rendezvous, or DHT record found	Defer	<code>LOCATION_UNRESOLVED</code>
A found record's <code>sig</code> does not validate under <code>ik</code> 's current pinned devices	Reject	Discard
A found record's <code>seq</code> is $\leq$ the highest <code>seq</code> already seen for <code>ik</code> (stale/replayed/rollback)	Reject	Discard
Disjoint-path queries disagree with no majority (possible eclipse, §4.2 caution)	Defer	<code>LOCATION_UNRESOLVED</code>
<code>allow_dht=false</code> and no non-DHT path exists	Reject	<code>LOCATION_UNRESOLVED</code>

Idempotency / retry. Fully idempotent (a pure read); safe to retry at will. Because DHT record lifetimes are short (§16.2 TTL 2 h), a caller SHOULD NOT cache a `lookup-location` result past the record's own `ttd`.

Example trace.

```
A: lookup-location(ik=MFk...)
A: cache check: no prior direct address for MFk...
A: rendezvous check: no known home-relay hint for MFk...
A → DHT: get(hash(MFk...)) via 3 disjoint paths, allow_dht=true
DHT ← A: path1 → LocationRecord{seq:118,...}; path2 → LocationRecord{seq:118,...};
          path3 → LocationRecord{seq:104,...}          # stale, from an eclipse-adjacent peer
A: majority (2/3) agree on seq:118 → accept it; discard seq:104
A: verify sig under MFk...'s current device key          # OK
A: lookup-location() → LocationRecord{seq:118, addrs:[...], ttd_expires: now+~1h50m}
```

19.2.3 The reachability ladder (attempt sequence)

Purpose. Given a resolved `LocationRecord`, attempt an actual connection to the peer in the order that prefers direct connectivity and falls back only as needed (§4.3), because rungs 1–2 are cheaper, faster, and expose no third-party relay.

Initiator / Responder. Initiator: the sending node. Responder: the target node (and, for rung 3, a relay operator's node).

Parameters.

- `location` (`LocationRecord`, MUST) — from `lookup-location` (§19.2.2).
- `deadline` (`u64`, OPTIONAL) — caller's own timeout for exhausting the ladder before falling back to store-and-forward (mixnet buffering, sender retry, §4.7); if absent, use the operation's own per-rung timeouts summed.

Preconditions. A valid, non-expired `LocationRecord` for the target (§19.2.2's success result).

Procedure (normative; try in order, fall down only as needed — §4.3).

- Direct.** Attempt a direct connection using `location.addrs` entries that are non-relay multiaddrs (IPv6 preferred; IPv4 with port-forward/UPnP acceptable). No relay involved. If a direct connection completes its transport handshake (Noise/TLS 1.3, §4.1), STOP — success.
- Hole-punch.** If step 1 fails (both sides behind NAT), attempt **AutoNAT v2 + DCUtr** coordinated hole-punching. This requires both nodes reachably online simultaneously —

true by construction for two always-on boxes. If the punched connection completes, STOP — success.

3. **Circuit relay.** If steps 1–2 fail, use a **circuit relay v2** hop from `location.addrs` 'relay-circuit entries (or a discovered relay per §14.5). The relay sees ciphertext only (content-blind); it is a reachability hop, not a store. If a relay circuit connects, STOP — success (with the caveat that this rung carries the weakest metadata privacy of the three, §6.6 item 2 as it applies to bulk; for ordinary MOTE delivery over relay, content remains E2E-encrypted regardless of rung).
4. **All rungs exhausted.** Fall back to store-and-forward: the sender's retry queue (§4.7) holds the MOTE and retries per the backoff schedule (§16.1); if a peer-buffering buddy node or a relay-mailbox is configured for the target, offer it there (§4.3, §14.5) rather than failing immediately.

Success result. An established transport connection at the lowest-numbered rung that worked, over which `deliver` (§19.3.1) proceeds.

Failure modes.

Condition	Class
Rung 1 fails (no direct path)	Defer (internal to the ladder)
Rung 2 fails (hole-punch coordination fails or one side is not simultaneously online)	Defer
Rung 3 fails (no relay reachable, or all relay reservations exhausted, §16.6 caps)	Defer
All rungs fail and <code>deadline</code> elapses	Reject (to the caller of <code>attempt-reachability</code>)

Idempotency / retry. The ladder itself is stateless per attempt; re-running it is always safe and is exactly what the sender-retry state machine (§19.3.3, §4.7) does on each backoff tick — it does not remember which rung last succeeded, since NAT/network conditions change.

Example trace.

```
A: attempt-reachability(location=LocationRecord{addrs:[/ip6/2001:db8:....udp/443/quic,
                                                    /p2p-circuit/.../relay1]})
A → B: QUIC direct dial /ip6/2001:db8:....udp/443/quic
B ↹ A: (timeout – B is behind CGNAT with no IPv6, direct fails)
A: rung 1 FAILED → try rung 2
A ↹ AutoNAT/DCUtr ↹ B: coordinated hole-punch attempt
A ↹ B: hole-punch SUCCEEDS                                     # both online, punch completes
A: reachability established via rung 2 (hole-punch)
A: proceed to deliver() over this connection
```

19.3 Delivery operations (§2.6, §2.7, §2.7a)

19.3.1 `deliver(outer_mote)`

Purpose. The recipient-side procedure that receives a MOTE off the wire, validates it in the cheapest-first order (§2.7), and either stores it for the user, defers it to the requests area (§2.7a), or silently drops it. This is the single most security-critical operation in the protocol: every failure path is defined, matching §2.7's decryption-DoS defense.

Initiator / Responder. Initiator: the sending node (over whatever transport rung §19.2.3 established, or via the mixnet, §4.4). Responder: the recipient's node — the party that runs this entire procedure.

Parameters.

- `outer_mote` (`Envelope`, `MUST`) — the full envelope as defined in §2.2: `v, suite, id, to, epoch, ts, kind, keypkg, challenge, ciphertext, sender_sig`. (The

"outer" mixnet wrapper, if present, has already been peeled by the time this operation's input is the `Envelope`; onion unwrapping is a transport-layer concern of §4.4, not restated here.)

Preconditions. None — this operation is the entry point for unauthenticated network input by definition, which is exactly why its ordering is normatively fixed (§2.7).

Procedure (normative — reproduces §2.7 verbatim as the responder's procedure, with the §2.7a disposition rule folded into step 6/9).

1. Reject unknown `v / suite` (fail closed). → **Silent drop** if this is the outer-most check with no `sender_sig` yet verifiable; treated as malformed input.
2. Verify `id` matches the content address of `ciphertext` (BLAKE3-256 by default, §2.2). Drop on mismatch.
3. Verify `sender_sig` over `(id || to || ts || kind || challenge)` under the envelope's ephemeral key — cheap, no decryption. Drop on failure.
4. **Resolve `to`** to this node's own key, or a group this node belongs to (`DeliveryTag` resolution, §2.2a: identity key, group id, or blinded tag recognized via the node's own per-contact secret). If `to` does not resolve to anything this node holds, drop (this node is not a valid recipient — do not guess or forward).
5. **Classify the sender:** known contact (pinned `to` /blinded-tag state matches an existing contact) vs. unknown/cold sender.
6. **Cold-sender gate (§9, before decryption).** If cold, evaluate `challenge` against local `Policy` (§9.2) — ARC token validity + issuer trust (§9.3.1), PoW solution (§9.4), postage validity (§9.5.1), or vouch (§9.7) — entirely without decrypting `ciphertext`. Known contacts skip this step. The outcome is exactly the §2.7a table:
 - Invalid/forged `challenge` (fails cryptographically, e.g. bad ARC signature, invalid PoW, double-spent postage serial) → **silent drop**, do not `ack` (unless `id` is a duplicate, which is acked per step 9's dedup rule).
 - Absent, or present but below the recipient's policy threshold → **defer to the requests area** (§2.7a): stored undecrypted-state-pending or decrypted-but-quarantined per implementation choice (implementations MAY decrypt to render a preview in the requests area, since decryption itself is not the resource being protected once the sender has *some* accountable proof — but MUST NOT surface it as an inbox message). Rate-limited per §9.2's `RateLimit`. Retained for the requests-area retention window (§16.5: 30 days), then purged if never promoted.
 - Valid and at/above threshold → proceed to step 7 exactly as a known contact would.
7. Decrypt `ciphertext` (MLS epoch key for group `epoch`, or HPKE to the recipient's key for 1:1 §5.3 async-init). Drop on decryption failure (wrong epoch, corrupt ciphertext, key not held).
8. Verify `Payload.sig` under `Payload.from`. For a known contact, `from` MUST match the pinned identity (§3.4); a mismatch is treated as a forged/relayed message — drop, do not `ack`. For a cold sender whose `from` is only now revealed (post-decryption), re-apply the recipient's block/allow lists against the now-known identity — a sender that passed the anonymous challenge gate (step 6) but is on an explicit per-identity block list (§9.2 `block`) is still rejected here.
9. Apply `expires / refs / kind` semantics (§2.4, §2.3); **store** the MOTE (inbox, if step 6/8 cleared it fully; requests area, if step 6 deferred it); **ack** (§19.3.2). A MOTE whose `id` this node **already holds** (dedup, §2.6) is acked immediately at whichever step the duplicate is detected, without re-running the remaining steps.

Success result. The MOTE is stored (inbox or requests area) and an `ack(id)` is sent to

the sender. Exactly one of three terminal states is reached for every input: **stored+acked**, **deferred+acked** (requests area), or **dropped+unacked** (silent, only for cryptographically invalid input per §2.7a) — there is no fourth, undefined outcome.

Failure modes.

Condition	Class
Unknown <code>v / suite</code>	Silent drop
<code>id</code> content-address of <code>ciphertext</code>	Silent drop
<code>sender_sig</code> fails to verify	Silent drop
<code>to</code> does not resolve at this node	Silent drop
Cold sender, <code>challenge</code> invalid/forged	Silent drop
Cold sender, <code>challenge</code> absent/below threshold	Defer to requests area
Decryption fails (wrong epoch/key/corrupt)	Silent drop
<code>Payload.sig</code> fails under <code>Payload.from</code> , OR <code>from</code> mismatches a known contact's pin	Silent drop
<code>from</code> (revealed post-decrypt) is on the recipient's explicit block list	Silent drop
Duplicate <code>id</code> (already held)	N/A — not a failure
<code>expires</code> in the past relative to receipt (already-expired MOTE)	Reject (soft)

Idempotency / retry. Fully idempotent on `id`: re-delivering an already-acked `id` is a no-op that still results in an `ack` (dedup, step 9). This is what makes sender-side retry (§19.3.3) safe to run blindly on any timeout without a separate "did they already get it" check.

Example trace (known contact, fast path).

```
A → B: Envelope{ v:0, suite:1, id:blake3:7ab2..., to: bob_ik, epoch:null, ts:..., kind:0x01,
               challenge:null, ciphertext:..., sender_sig:Ed25519(...) }
B: step1 v/suite known                                # OK
B: step2 recompute blake3(ciphertext) == id            # OK
B: step3 verify sender_sig under envelope's ephemeral key # OK
B: step4 resolve to=bob_ik → this node                 # OK, matches
B: step5 classify: alice_ik is a pinned known contact   # KNOWN
B: step6 SKIPPED (known contact, §2.7 "known contacts MAY skip step 6")
B: step7 decrypt ciphertext (HPKE to bob's key)        # OK
B: step8 verify Payload.sig under Payload.from == alice_ik (pinned) # matches
B: step9 store to inbox; kind=0x01 chat rendered in Talk thread
B → A: ack(id=blake3:7ab2...)
```

Example trace (cold sender, deferred to requests area).

```
C → B: Envelope{ ..., to: bob_ik, kind:0x00, challenge: ARC{issuer:"unknown-node-xyz", ...},
               ciphertext:..., sender_sig:... }
B: steps 1–4 pass as above
B: step5 classify: sender's blinded tag/key unrecognized → COLD
B: step6 evaluate challenge: ARC token, but issuer "unknown-node-xyz" is unvetted
    → per §9.3.1 unvetted issuer default rate budget = 0 → treated as "no token"
    → falls back to policy: no PoW/postage attached either → BELOW THRESHOLD
B: DEFER → store in requests area (undecrypted or preview-decrypted per implementation),
    rate-limit counter for this challenge-class incremented, retention clock started (30d)
B → C: ack(id)                                         # deferred MOTES ARE acked –

# only invalid/forged ones are
```

19.3.2 `ack(id)`

Purpose. Confirm receipt of a specific MOTE `id` to the sender, terminating that MOTE's sender-side retry loop (§4.7).

Initiator / Responder. Initiator: the recipient node (sent as a consequence of `deliver`, §19.3.1). Responder: the sender's retry-queue state machine (§19.3.3), which consumes the ack.

Parameters.

- `id` (bytes , MUST) — the content address being acknowledged.
- `ack_sig` (bytes , OPTIONAL) — implementations MAY sign the ack (e.g. under the same ephemeral mechanism as `sender_sig`) so a relay cannot forge acks on the recipient's behalf; this is an implementation hardening, not specified further at the object-format level in v0.

Preconditions. The recipient has completed `deliver`'s procedure for `id` to a terminal state that the §2.7a table marks as ack-eligible: **stored** (inbox) or **deferred** (requests area). A **silently dropped** MOTE MUST NOT be acked (§2.7a).

Procedure (normative).

1. Construct a minimal `ack{id}` message.
2. Send it back over the same channel/rung the `Envelope` arrived on if still open, or via a fresh `deliver`-style send addressed to the original sender's key if the channel has since closed (an ack is itself routed like any other small MOTE-adjacent message — it does not require a dedicated wire object beyond `id`).

Success result. The sender's retry-queue entry for `id` transitions to **ACKED** (§4.7) and is removed from the retry schedule.

Failure modes.

Condition	Class
Ack is sent but never reaches the sender (network loss)	Defer (at the sender)
Recipient tries to ack an <code>id</code> it silently dropped (implementation bug)	Reject (protocol-level should-never-happen)

Idempotency / retry. Sending the same `ack(id)` multiple times is harmless — the sender's retry-queue transition to **ACKED** is itself idempotent (an already-acked entry receiving another ack is a no-op).

Example trace. See `deliver`'s traces above — both end in `B → A: ack(id=...)`.

19.3.3 Sender retry state machine (dedup and durability)

Purpose. Own durability from the sender's side, since the mixnet/relay middle holds nothing (§0.5, §4.7): retry an unacked MOTE with backoff until `ack` or the retry deadline expires.

Initiator / Responder. Initiator/owner: the sending node. There is no "responder" — this is a purely local state machine driven by the outcomes of `attempt-reachability` (§19.2.3), `deliver`'s remote effect (whether an `ack` comes back), and wall-clock time.

Parameters.

- `mote_id` (bytes , MUST) — the MOTE being tracked.
- `expires` (u64 , OPTIONAL) — the MOTE's own requested expiry (§2.4); bounds the retry deadline from above if smaller than the default.
- `retry_deadline` (u64 , OPTIONAL, default per §16.1: 72 h) — the hard ceiling after which the MOTE fails to the user regardless of `expires`.

Preconditions. A MOTE has been constructed and handed to the outbound queue (i.e. it has left the **QUEUED** /draft state, §4.7).

Procedure (normative state machine, §4.7).

1. **QUEUED** → the MOTE is sealed (sender-blinded, onion-wrapped if **private** tier, §6.2) → **SEALED**.
2. **SEALED** → an in-flight send attempt is made (mixnet path for **private**, direct/reachability-ladder for **fast**) → **IN_FLIGHT**.
3. On **ack(id)** received → **ACKED**. Terminal; remove from the retry queue.
4. On send failure or no **ack** within the current backoff window → **RETRY**: re-attempt with **exponential backoff** (§16.1: base 30 s, cap 1 h, with jitter), returning to step 2 each time (a fresh reachability-ladder attempt, since network conditions may have changed).
5. If **retry_deadline** (or the smaller of **expires**, if set) elapses while still un-acked → **EXPIRED**. Terminal; the sender's client notifies the user the message could not be delivered (§4.7).

Success result. Either **ACKED** (delivered) or **EXPIRED** (failed, user-notified) — no MOTE is left in indefinite limbo; every entry reaches one of exactly two terminal states.

Failure modes.

Condition	Class	Response
Recipient permanently unreachable (deleted identity, permanently offline)	Reject (eventual)	EXPIRED at r
expires requested shorter than the default retry deadline	Defer, bounded	The smaller o
Sender's own node restarts mid-retry	Defer	The retry que
A duplicate ack arrives for an already- ACKED /removed entry	N/A	Ignored; no st

Idempotency / retry. The entire point of this operation is retry-safety: every re-attempt at step 2 is a fresh, idempotent send of the same immutable **Envelope** (same **id**), relying on the recipient's dedup (§2.6) to make repeated delivery harmless.

Example trace.

```

A: QUEUED mote_id=blake3:7ab2...
A: SEALED (onion-wrapped, private tier)
A: IN_FLIGHT attempt 1 via mixnet
  ... 90s pass, no ack ...
A: RETRY backoff=30s → attempt 2
  ... no ack ...
A: RETRY backoff=60s → attempt 3
A ← B: ack(id=blake3:7ab2...)
A: ACKED - removed from retry queue

```

19.4 Async session initiation (§5.3)

19.4.1 **fetch-keypackage(ik)** → **KeyPackage**

Purpose. Retrieve a signed MLS KeyPackage (MLS's prekey) for an identity so a sender can initiate an encrypted session with a peer whose devices are all currently offline, without a separate PQXDH/X3DH protocol (§5.3).

Initiator / Responder. Initiator: the node wishing to start a session (sender). Responder: the mesh (KeyPackages are located via **Identity.prekeys** / **keypkgs**, §1.3, and fetched from wherever the owner's node last published them — typically the mesh content store, republished by the owner's own node).

Parameters.

- **ik** (**bytes**, **MUST**) — target identity key (already resolved, §19.1.1).
- **suite** (**u8**, **OPTIONAL**) — preferred suite; if absent, use the highest suite common to sender and **ik**'s advertised **Identity.suites** (§1.3's negotiation rule).

Preconditions. `ik` has been resolved and its `Identity.keypkgs` (a `KeyPackageBundleRef`) is known.

Procedure (normative).

1. Fetch the `KeyPackageBundleRef`-addressed bundle from the mesh.
2. Select one `KeyPackage` matching the negotiated `suite` (§1.3: "per-message suite is negotiated at `KeyPackage` granularity"). Prefer a **one-time** `KeyPackage` if available (MLS's consume-once prekey); fall back to a **last-resort** `KeyPackage` (§5.3) only if no one-time package remains, so an identity is never left un-initiable purely from prekey exhaustion.
3. Verify the `KeyPackage`'s own signature under a current device key of `ik`'s `Identity` (a `KeyPackage` not traceable to a currently-valid device is untrustworthy — reject it and try the next available package, if any).
4. Mark the selected one-time `KeyPackage`, if used, as **consumed** locally to avoid re-offering it (the owner's node performs the authoritative consumption/replenishment; the fetching sender's local marking is advisory only, to avoid retrying an already-used package against the same responder in a race).

Success result. A verified `KeyPackage` usable as the `Add` target in `add-member` (§19.4.2).

Failure modes.

Condition	Class	Response
No <code>KeyPackage</code> bundle published (or bundle empty — full exhaustion)	Defer	Retry
Fetches <code>KeyPackage</code> 's signature does not validate under any current device key	Reject	<code>KEYPAK_INVALID</code>
No common suite between sender and <code>ik</code> 's advertised suites	Reject	<code>NO_COMMON_SUITE</code>
A one-time <code>KeyPackage</code> is fetched by two concurrent initiators (race)	Defer (for the loser)	The other initiator wins

Idempotency / retry. Fetching is idempotent as a read; consuming a one-time package is explicitly **not** idempotent (that is its purpose) — a retry after a consumption race **MUST** fetch a fresh package, not resubmit the same one.

Example trace.

```
A: fetch-keypackage(ik=bob_ik, suite=1)
A → mesh: get(Identity(bob_ik).keypkgs)
mesh ← A: KeyPackageBundle{ packages: [KP_onetime_17, KP_onetime_18, KP_lastresort] }
A: select KP_onetime_17 (suite 1, one-time)
A: verify KP_onetime_17.sig under bob's device_key "home-box"      # OK
A: mark KP_onetime_17 consumed (local, advisory)
A: fetch-keypackage() → KP_onetime_17
```

19.4.2 `add-member(group_or_1to1, keypackage) → {Commit, Welcome}`

Purpose. Bring an offline party into an encrypted session (a new 1:1, or an existing group) via MLS's native async-join mechanism: an `Add` `Commit` plus a `Welcome` message the joining party uses on next contact (§5.3 step 2).

Initiator / Responder. Initiator: an existing member (for a group) or the party starting a 1:1 (a 1:1 is a 2-member group from creation, §5.1). Responder: the group's committer (§5.1) for ordering, and the invitee (eventual, asynchronous responder via `Welcome` consumption).

Parameters.

- `group_state` (opaque MLS group context, **MUST**) — the current `GroupInfo` /ratchet-tree state (or, for a fresh 1:1, the freshly-created 2-member group state).
- `keypackage` (`KeyPackage`, **MUST**) — from `fetch-keypackage` (§19.4.1).

- **role** (**tstr** , OPTIONAL) — for group adds beyond a bare 1:1, the invitee's initial role (**member** / **poster** / **reader** , §5.8.2); requires **admin** capability on the initiator (see §19.5.2's precondition, which this operation shares when used for an existing group).

Preconditions.

1. The initiator holds current, valid group state (is a current member with the necessary capability — **admin** for a pre-existing group per §5.8.2; any member for a fresh 1:1).
2. **keypackage** has been verified per **fetch-keypackage** 's step 3.
3. The initiator can reach the group's **committer** (§5.1) to submit the **Add** Commit for ordering — see §19.5.5 if the committer is unreachable.

Procedure (normative).

1. Construct an MLS **Add** proposal referencing **keypackage** .
2. Submit the proposal (bundled into a **Commit**) to the group's ordered handshake channel — the committer (§5.1). The committer appends it to the hash-chained per-group log at the next position.
3. On the Commit being accepted into the log, all current members apply it, advancing the group's **epoch** .
4. Construct a **Welcome** message (MLS-standard: encrypted group secrets + the ratchet tree, addressed to the new member's **keypackage** 's init key) and send it as a **kind=0x06 group_event MOTE** (§2.3) to the invitee's key, **to = keypackage** 's owning identity — this MOTE travels like any other (subject to the invitee's own cold-sender gate, §2.7, if the inviter is not already a known contact of the invitee — see the failure-mode note below).
5. The invitee, on eventually coming online and receiving the **Welcome** MOTE (via ordinary **deliver** , §19.3.1), uses it to bootstrap local group state at the current **epoch** , without having needed to be online for steps 1–3.

Success result. The group (or new 1:1) advances to a new **epoch** including the new member; the new member, once it processes the **Welcome** , holds full current group state.

Failure modes.

Condition

Initiator lacks **admin** capability for a pre-existing group

Committer unreachable to accept the Commit

keypackage already consumed (race with another initiator, §19.4.1)

Welcome MOTE is deferred to the invitee's requests area because the inviter is not yet a known contact of the

Two concurrent **Add** s for the same group produce conflicting Commits at the same log position

Idempotency / retry. Not idempotent at the MLS layer (each successful **Add** advances **epoch** once); a failed submission (e.g. committer timeout) is safe to retry with a fresh proposal once the committer situation resolves, since an un-applied proposal has no group-state effect yet.

Example trace.

```
A (group admin): add-member(group_state=G@epoch5, keypackage=KP_onetime_17, role="member")
A → committer: Proposal{Add, KP_onetime_17}
committer: appends Commit_6{Add(KP_onetime_17)} to hash-chained log at position 6
committer → all current members (incl. A): Commit_6
members: apply Commit_6 → G@epoch6, new member's leaf present (pending Welcome)
A → bob (invitee, via bob_ik resolved from KP_onetime_17's owner):
```

```

    MOTE{ kind:0x06, to: bob_ik, payload: Welcome{ epoch:6, secrets:..., tree:... } }
    ... bob offline; MOTE retried per §19.3.3 ...
bob (later, online): deliver() processes the Welcome MOTE → bootstraps G@epoch6 locally
bob → A: ack(id)

```

19.4.3 external-commit(group_info) → Commit

Purpose. Let a party **self-join** a group using its published `GroupInfo`, without waiting for an existing member to **Add** them — MLS's other async-join primitive (§5.3 step 2, "or the joiner uses an External Commit against the group's `GroupInfo` to self-join"). This is also the mechanism behind **open** / **request** join policies (§5.8.2, §19.5.4).

Initiator / Responder. Initiator: the joining party. Responder: the group's committer (must still order the resulting Commit into the hash-chained log, §5.1 — external commit does not bypass ordering, only bypasses needing an existing member to sponsor it).

Parameters.

- `group_info` (`GroupInfo`, **MUST**) — the group's current public join information (ratchet-tree public state + `epoch`), obtained via the group's address (§5.8: resolving a group address works like resolving any identity, §19.1.1) or an out-of-band invite link.
- `join_policy_proof` (**OPTIONAL**) — for **request** / **vouch** join policies (§5.8.2), whatever satisfies the policy (an admin-approval token, or a vouch from an existing member); absent for **open** groups.

Preconditions.

1. The group's **join** policy (§5.8.2) is **open**, or the joiner holds a valid `join_policy_proof` for **request** / **vouch**.
2. `group_info` is current (its `epoch` is not so stale that the resulting Commit would be rejected as building on an obsolete tree — see failure modes).

Procedure (normative).

1. Construct an MLS `ExternalCommit` against `group_info`'s ratchet tree, adding the joiner as a new leaf, signed by the joiner's own key.
2. Submit the `ExternalCommit` to the committer for ordering into the hash-chained log, exactly as any other Commit (§5.1) — an external commit is still subject to the same total-order requirement; it is a *different kind* of Commit, not a different ordering channel.
3. On acceptance, all current members (including the joiner) apply it; `epoch` advances.
4. For **open** groups this is rate-limited and anti-abuse-gated (§5.8.2: "anyone with the address may join, rate-limited + anti-abuse §9") exactly like a cold-sender MOTE, since an `ExternalCommit` submission is, from the committer's perspective, an unsolicited request from a potentially-unknown party.

Success result. The joiner is a full group member at the new `epoch`, having bootstrapped state directly from `group_info` without needing any existing member online to sponsor it.

Failure modes.

Condition

Join policy is **closed**

request / **vouch** policy and no valid proof supplied

`group_info.epoch` is stale (group has since advanced past what the joiner's `ExternalCommit` was built against)

open group, rate limit exceeded for this source (anti-abuse, §9.2)

Committer unreachable

Condition

Two external commits race at the same `epoch`

Idempotency / retry. Not idempotent (each success advances `epoch` once); retries after `EPOCH_STALE` MUST rebuild the `ExternalCommit` against the freshly-fetched `group_info`, not resubmit the stale one.

Example trace (open group).

```
A: external-commit(group_info=GroupInfo{group="team@company.com", epoch:12, tree:...})
A: join_policy = "open" (no proof needed)
A → committer: ExternalCommit{ new_leaf: A_leaf, based_on_epoch:12 }
committer: rate-limit check for A's source – OK, under threshold
committer: appends Commit_13{ExternalCommit(A)} to log
committer → all members: Commit_13
members (incl. A): apply → G@epoch13, A now a full member with role="member" (default for open)
```

19.5 Group operations (§5.1, §5.8)

19.5.1 `create-group(params)` → `GroupState`

Purpose. Instantiate a new MLS group as an addressable identity (§5.8: "a group is an identity that has members... its own keypair and therefore its own name"), fixing its initial committer, roles, and posting/visibility/join policies.

Initiator / Responder. Initiator: the creating party. No responder beyond the mesh/KT publication steps (a brand-new group has no other members yet, though it MAY be created with an initial member set via `add-member`, §19.4.2, applied after creation).

Parameters.

- `group_keypair` (generated, MUST) — the group's own identity keypair (§5.8); the group is published as an `Identity` (§1.3) exactly like a person's, with `names` optionally carrying a `@handle` or `name@domain` (§3.9).
- `posting_model` ("broadcast" | "channel", MUST) — §5.8.1.
- `visibility` ("hidden" | "member-visible", MUST) — membership-visibility policy (§5.8.1); MUST be "hidden" if `posting_model="broadcast"` and the deployment wants subscriber-list privacy (§5.8.3) — see the precondition below.
- `join_policy` ("closed" | "request" | "open" | "vouch", MUST) — §5.8.2.
- `creator_role` (implicit `owner`, not a parameter) — the creator is always the initial `owner` (§5.8.2).
- `legacy_address` (tstr, OPTIONAL) — a `team@company.com` address served by a gateway for legacy interop (§5.8.5).

Preconditions.

- If `visibility="hidden"` (subscriber-list privacy, §5.8.3), the implementation MUST use the relay/committer fan-out delivery model (§5.8.3), not the shared MLS tree — this is a structural choice made at creation time, not switchable without effectively recreating the group's delivery mechanism (switching `posting_model` /policy via a Commit, §5.8.1, does not retroactively change how *already-delivered* membership information was exposed).
- `group_keypair` has not been previously published (a fresh identity).

Procedure (normative).

- Generate `group_keypair`.

2. Initialize MLS group state with the creator as the sole member, role `owner`, and the requested `posting_model` / `visibility` / `join_policy` as signed fields of the group state (§5.8.2: "all membership/role/policy changes are signed and appear in the group's hash-chained handshake log").
3. Set the creator as the **initial committer** (§5.1: "the group creator is the initial committer. Committer identity is a signed field of the group state; every member knows it").
4. Publish the group's `Identity` (§19.1.2, reused verbatim — a group identity publishes exactly like a personal one) so `resolve("team@company.com")` or `resolve("@team")` finds it.
5. If `legacy_address` is set, configure the gateway (§7, §5.8.5) to fan out inbound legacy mail to the address as MOTEs to current members.
6. Optionally, invite an initial member set via `add-member` (§19.4.2), applied as Commits against `epoch=0` / `1` immediately after creation.

Success result. A published group `Identity`, resolvable by name, with the creator as sole member/owner/committer, ready to accept `join`s (§19.5.4) or `add-member` invitations.

Failure modes.

Condition	Class
Requested name (<code>@handle</code> or <code>name@domain</code>) already claimed by another identity	Reject
<code>visibility="hidden"</code> requested with <code>posting_model="channel"</code>	Reject (policy, not protocol)
<code>legacy_address</code> requested but no gateway configured/available for that domain	Defer

Idempotency / retry. Not idempotent (each call mints a fresh `group_keypair`); a failed `publish` (`publish-identity` 's own failure modes, §19.1.2) is safe to retry with the same already-generated keypair, since the keypair generation step has no side effects until publication.

Example trace.

```
A: create-group(posting_model="channel", visibility="member-visible", join_policy="request",
               names=["@design-team"])
A: generate group_keypair GK
A: init MLS group, sole member A, role=owner, committer=A
A: publish-identity(Identity{ iks:{1:GK_pub}, names:["@design-team"], version:1, ... })
A → KT: append; A → DHT: put
A: create-group() → GroupState{ group_ik: GK_pub, epoch:0, members:[A(owner)],
                               posting_model:"channel", join_policy:"request" }
```

19.5.2 `group-add` / `group-remove` / `group-role-change` / `group-policy-change`

Purpose. The four Commit-based group-management operations (§5.8.2), ordered by the committer like any other handshake message (§5.1), each requiring the role §5.8.2 specifies.

Initiator / Responder. Initiator: a member holding the required role. Responder: the group's committer (ordering) and all current members (applying).

Parameters (per sub-operation).

- `group-add(keypackage, role)` — `keypackage` (MUST, from `fetch-keypackage`, §19.4.1), `role` (MUST, one of `owner|admin|member|poster|reader`, §5.8.2). Mechanically this is `add-member` (§19.4.2) invoked against an existing group — restated here as its own named entry point because §5.8.2 names it separately and gives it a distinct role requirement.
- `group-remove(member_ik)` — `member_ik` (bytes, MUST) — the member to remove.
- `group-role-change(member_ik, new_role)` — both MUST.

- `group-policy-change(field, new_value)` — `field` (MUST, one of `posting_model`, `visibility`, `join_policy`), `new_value` (MUST, matching that field's type in §19.5.1).

Preconditions (role gating, normative per §5.8.2).

Operation	Required role
<code>group-add</code>	<code>admin</code> (or <code>owner</code>)
<code>group-remove</code>	<code>admin</code> (or <code>owner</code>)
<code>group-role-change</code> / transfer ownership	<code>admin</code> / <code>owner</code> (transferring ownership itself typically requires <code>owner</code>)
<code>group-policy-change</code>	<code>admin</code> (or <code>owner</code>)

Procedure (normative).

1. Initiator constructs the appropriate MLS Proposal: `Add` (via `keypackage`), `Remove` (via `member_ik`'s leaf), `Update`-with-role-attribute change, or a policy-field `Update` — whichever the group's MLS extension mechanism uses to carry non-cryptographic group-state fields (roles/policy are signed fields of the group state riding alongside the ratchet tree, §5.8.2).
2. Submit to the committer for ordering (§5.1); the committer appends it to the hash-chained log at the next position.
3. On acceptance, all members apply it; `epoch` advances.
4. **`group-remove` additionally triggers file-key rotation** for shared folders (§5.8.2: "Remove triggers file-key rotation for shared folders (§6.7)") — the removing operation MUST, for any folder marked confidential (§6.7), re-key and redistribute new file keys to the remaining members, since MLS removal alone blocks only *future* messages, not previously-held file keys.
5. Every accepted change appears in the group's hash-chained handshake log (§5.8.2), auditable by any member ("who added/removed whom").

Success result. The group's roster/role/policy state advances to a new `epoch` reflecting the change, logged and auditable.

Failure modes.

Condition
Initiator lacks the required role
<code>group-remove</code> targets a member not currently in the roster
<code>group-role-change</code> attempts to remove the last <code>owner</code> without designating a successor
<code>group-policy-change</code> sets <code>visibility="hidden"</code> on a group whose <code>posting_model="channel"</code> and members have
Committer unreachable
File-key rotation (step 4) fails to reach a remaining member (offline)

Idempotency / retry. Not idempotent (each success advances `epoch`); safe to retry against the current `epoch` after a committer-unreachable defer resolves.

Example trace (`group-remove` with file-key rotation).

```
A (admin): group-remove(member_ik=carol_ik)
A → committer: Proposal{Remove, carol_ik}
committer: appends Commit_9{Remove(carol_ik)} to log
committer → members: Commit_9
members: apply → G@epoch9, carol no longer in roster
A: for each confidential shared folder F in this group:
    generate new file key K'_F
```

re-encrypt F's manifest-referenced content under K'_F (§6.7)
 distribute K'_F to remaining members via a `group\_event` MOTE
 carol: retains any already-downloaded plaintext (un-share limit, §6.7) but cannot decrypt
 future content or future manifest updates under K'_F

19.5.3 `post-to-group(group_ik, mote)`

Purpose. Send a message to a group's address, fanned out per the group's `posting_model` (§5.8.1) and delivery scale regime (§5.8.4).

Initiator / Responder. Initiator: any member with `poster` capability (or `member`, if the group does not distinguish poster/reader, §5.8.2). Responder: every current member (small groups, standard MLS fan-out) or the committer acting as fan-out relay (large lists, §5.8.4).

Parameters.

- `group_ik` (bytes, MUST) — the group's identity key (`to` in the resulting `Envelope s`, §2.2a).
- `mote` (application `Payload`, MUST) — the content, exactly as any 1:1 MOTE.

Preconditions.

1. The initiator holds `poster` (or equivalent, per the group's role scheme) capability.
2. The group's current `epoch` state is held by the initiator (it must encrypt to the current epoch's tree/keys).

Procedure (normative, branches on scale/model per §5.8.4).

1. **Small groups (standard MLS fan-out).** Encrypt one MLS application message under the current `epoch`; the mesh/mixnet delivers it once to the group's routing structure; each member's node decrypts locally with its own leaf key. `to = group_ik` or the MLS group's `DeliveryTag` (§2.2a).
2. **Large lists (per-member sealed fan-out).** The committer's ordered log is authoritative for membership; delivery is **per-member**, sealed individually to each subscriber (§5.8.4) — mechanically, this is `deliver` (§19.3.1) invoked once per member with a per-member-sealed envelope, not one shared-tree MOTE. This is what gives hidden-membership lists (§5.8.3) their privacy: no shared tree exposes the roster.
3. Either way, each recipient's own `deliver` procedure (§19.3.1) applies unchanged — group membership itself is the recipient's "known contact" classification (step 5 of §2.7), so fellow members are never treated as cold senders by each other.

Success result. Every current member (or subscriber, for hidden lists) receives the message, each acking independently per §19.3.2.

Failure modes.

Condition	Class
Initiator lacks <code>poster</code> capability (<code>reader</code> -only role attempting to post)	Reject
Initiator's local <code>epoch</code> is behind the group's current <code>epoch</code> (missed a recent Commit)	Reject (local)
One member (of many, large-list fan-out) is offline	Defer (per-member)
A member has been removed but the sender's cached roster is stale	Reject (at the removal)

Idempotency / retry. Each post is a distinct MOTE (`id` = content address of that specific ciphertext); re-sending identical content produces a distinguishable *new* MOTE per recipient (sealed per-member framing differs even for identical plaintext, in the large-list case) — not naturally deduplicated across a wholesale re-post; deduplication (§2.6) applies per-envelope `id` as usual if the exact same send is retried by the sender-retry machine.

Example trace (hidden-membership broadcast list).

```
A (poster on @design-team): post-to-group(group_ik=GK_pub, mote={ kind:0x01, body:"standup at 10"
A: local epoch check – current # OK
A: posting_model = "broadcast", visibility = "hidden"
A → committer(=A's own node, or the current committer): fan-out request
committer: for each of 40 subscribers, seal an individual Envelope{to: blinded_tag_i, ...}
committer → subscriber_1..40: 40 individual deliver(s), each subject to that subscriber's own
known-contact/cold-sender classification (§2.7) against the list's
posting identity, not against each other
subscriber_17 (offline): enters that subscriber's own sender-retry loop for this one Envelope
subscriber_1..16,18..40: ack independently
```

19.5.4 join(group_id, mode)

Purpose. Request or perform membership entry into a group under its declared `join_policy` (§5.8.2): `closed`, `request`, `open`, or `vouch`.

Initiator / Responder. Initiator: the prospective member. Responder: the group's committer (ordering), and — for **request / vouch** — an admin (approval) or an existing member (vouching).

Parameters.

- `group_ik` (bytes , MUST) — resolved group identity (§19.1.1 works identically for groups).
- `mode` (implicit from the group's own state, not caller-chosen) — the operation behaves per whichever `join_policy` the group currently has; the caller does not select a mode, it discovers one from `group_info` / `Identity` .
- `vouch_token` (OPTIONAL) — required if `join_policy="vouch"`: a signed introduction from an existing member (§5.8.2, and the general vouch mechanism of §9.7).

Preconditions. `group_ik` resolves and its current `join_policy` is known (via its published `GroupInfo / Identity`).

Procedure (normative, branches by policy).

1. `join_policy="closed"` → **not performable** by this operation; the prospective member must be invited via `group-add` (§19.5.2) by an existing `admin`. `join` immediately fails.
2. `join_policy="open"` → equivalent to `external-commit` (§19.4.3) with no proof required; subject to that operation's own rate-limiting/anti-abuse gate.
3. `join_policy="request"` → submit a join request (itself deliverable as a small signed message to the group's admin set, gated by the *admins'* own cold-sender policy if the requester is unknown to them, §2.7); on admin approval, the admin performs `group-add` (§19.5.2) on the requester's behalf, OR issues an approval token the requester then presents to `external-commit` (§19.4.3) as `join_policy_proof` — either implementation is conformant, but the requester **MUST NOT** be able to self-admit without *some* admin-signed approval artifact.
4. `join_policy="vouch"` → present `vouch_token` (signed by an existing member) to `external-commit` (§19.4.3) as `join_policy_proof`; the committer/group verifies the voucher is a current member and that the vouch itself is rate-limited (§9.7: vouches "MUST itself be rate-limited to prevent vouch farming").

Success result. The requester becomes a group member at a new epoch (immediately for open / vouch once approved; after admin action for request).

Failure modes.

Condition

```
join_policy="closed"
```

Condition

```

join_policy="request" , admin never responds
join_policy="vouch" , voucher is not a current member (removed since issuing the vouch, or never was one)
join_policy="vouch" , voucher exceeds their own vouch rate limit
join_policy="open" , requester exceeds the group's open-join rate limit

```

Idempotency / retry. A repeated `join` request while a `request`-mode approval is pending is harmless but SHOULD be deduplicated client-side (no protocol-level harm in an admin seeing two identical pending requests, but implementations SHOULD collapse them for UX).

Example trace (`vouch`).

```

A: join(group_ik=GK_pub, vouch_token=Vouch{ voucher: carol_ik, group: GK_pub,
                                          sig: Ed25519(carol_device_key, ...) })
A → committer: ExternalCommit{ new_leaf: A_leaf, join_policy_proof: vouch_token }
committer: verify carol_ik is a current member                # OK
committer: check carol_ik's vouch-rate counter                # under limit
committer: appends Commit{ExternalCommit(A)} to log
committer → members: Commit
members (incl. A): apply → A is now a member

```

19.5.5 `committer-elect` / `committer-rotate` (failover)

Purpose. Establish or change which member serializes the group's handshake log (§5.1), and recover liveness when the current committer is unreachable.

Initiator / Responder. Initiator: the group creator (initial election, folded into `create-group`, §19.5.1) or any member proposing rotation. Responder: all members, who must apply the same Commit to agree on the new committer.

Parameters.

- `new_committer_ik` (bytes , MUST) — the member being promoted.
- `reason` (tstr , OPTIONAL) — "scheduled", "timeout", "vote" — informative only.

Preconditions.

1. `new_committer_ik` is a current group member.
2. For `reason="timeout"`: the current committer has been unreachable past the group's configured liveness timeout (implementation-defined; not fixed numerically in v0, see §19.10).

Procedure (normative, §5.1).

1. **Committer identity is a signed field of the group state**; rotation is itself a Commit (`committer-rotate`), submitted like any other group-management Commit (§19.5.2's mechanism) — but critically, *who orders this particular Commit* is exactly the problem being solved, so see step 2.
2. **Normal rotation** (current committer reachable, e.g. `reason="scheduled"` or "vote"): the current committer itself orders and appends the rotation Commit (it is, after all, still live) naming `new_committer_ik`; all members apply it, and `new_committer_ik` becomes authoritative for future ordering.
3. **Failover rotation** (`reason="timeout"`, current committer unreachable): members hold pending Proposals and either (a) wait for the unreachable committer to return, or (b) elect a new committer via a Commit that **references the last agreed log head** — since no ordering authority is currently live, this election Commit is itself accepted by member **quorum agreement** on the referenced log head (an implementation-defined quorum rule;

the spec fixes the *log-head-reference* requirement, not the quorum size — see §19.10) rather than by a single committer's say-so.

4. Once `new_committer_ik` is agreed, it resumes ordering exactly as any committer would.

Success result. The group has a single, agreed, live committer, with the transition itself recorded in the hash-chained log (auditable, §5.8.2).

Failure modes.

Condition	Class
<code>new_committer_ik</code> is not a current member	Reject
Members disagree on the referenced log head during failover election (partition)	Reject → escalates to fork-
Old committer returns after a failover rotation already completed	Defer, reconciled
No live majority reachable at all (severe partition)	Defer

Idempotency / retry. Rotation is not idempotent (each success is one Commit); retried election attempts after a resolved partition should reference the now-current log head, not a stale one.

Example trace (failover).

```
[committer carol_ik has been unreachable for 3x the group's configured liveness timeout]
A: committer-rotate(new_committer_ik=A_ik, reason="timeout")
A → other members (bob, dave): propose election, referencing last agreed log head = Commit_9
bob, dave: agree Commit_9 is indeed the last one they've applied too
A, bob, dave: quorum (3/4 members, meets this group's configured threshold) → accept
          Commit_10{committer-rotate(A_ik)} at position 10 (referencing Commit_9)
A: becomes committer; resumes ordering
[carol later returns, offline the whole time]
carol: receives Commit_10 on reconnect; sees it correctly chains from Commit_9 (her own last
      known state); accepts it; rejoins as an ordinary member, does NOT contest
```

19.5.6 Fork-detection halt

Purpose. Detect and respond to committer misbehavior (or a failed failover producing disjoint quorums): two Commits at the same log position with the same predecessor (§5.1: "The hash-chained log makes a fork detectable... members MUST halt and alert").

Initiator / Responder. Detected by any member on receipt of conflicting log entries; there is no single "initiator" — this is a passive-detection, active-response procedure every member runs continuously.

Parameters. N/A (triggered by observed state, not called with arguments) — but conceptually takes `commit_a`, `commit_b` (both referencing the same `prev` at the same position) as its detected input.

Preconditions. A member has received two distinct, validly-signed Commits both claiming the same log position with the same `prev` hash (a structural impossibility in honest operation of a single-writer hash-chained log — proof of committer misbehavior, or of an unresolved failover race, §19.5.5).

Procedure (normative, §5.1, §6.6 item 7 cross-reference).

1. On detecting `commit_a` and `commit_b` both claiming position N with identical `prev`: **MUST halt** — stop applying further Commits from either branch, and stop treating either as authoritative.
2. **MUST alert** the member (surface a security warning, analogous to KT equivocation detection, §3.5) — this is member-facing, not silent, since a stalled/forking committer is a detectable-but-not-forgeable event (§5.1: "a committer can stall but not forge").

3. Group membership/application-message processing for this group **SHOULD** pause for handshake-dependent operations (adds/removes/role/policy changes) until resolved; ordinary application messages already encrypted under the last-agreed **epoch** **MAY** continue to be processed (the fork concerns *future* handshake state, not already-established message keys).
4. Resolution is **out of the committer's hands** — members must manually (or via a higher-level, out-of-band group-recovery convention, not fixed in v0) agree which branch to keep, or re-create the group from the last pre-fork agreed state. This is a deliberate consequence of "cannot forge" (every Commit is member-signed) not implying "cannot stall or fork the log" — those failure modes remain, bounded by detectability + committer rotation, not eliminated.

Success result. There is no "success" result for a fork — this operation's success is **correctly refusing to silently pick a branch**. The observable outcome is a halted group requiring member/administrator intervention.

Failure modes.

Condition	Class
A fork is detected but a member's implementation does not halt (bug)	Reject (implementation N/A)
Fork is actually a stale duplicate delivery of the <i>same</i> Commit (not a true fork)	N/A
No resolution convention is agreed by the group after halting	Defer, indefinitely

Idempotency / retry. N/A — this is a detection-and-halt procedure, not an invokable operation with retry semantics; it re-triggers identically every time a member re-observes the same conflicting pair.

Example trace.

```

bob (group member): receives Commit_10a{committer-rotate(A_ik)} referencing prev=Commit_9
                    ALSO receives Commit_10b{committer-rotate(eve_ik)} referencing prev=Commit_9
bob: both signatures verify individually – but SAME position (10), SAME prev (9), DIFFERENT content
bob: FORK DETECTED
bob: HALT further Commit application for this group
bob: ALERT – surfaces "group state conflict, admin action required" to the user
bob: continues rendering already-decrypted application messages under epoch 9 (last agreed)
    but does not apply either Commit_10a or Commit_10b

```

19.6 DMTAP-Auth operations (§13)

Conformance note (normative). These operation sketches are **subordinate to §13's hardened requirements**; where an op block below is less specific, §13 governs. In particular: (a) the client **MUST** generate the session keypair *before* signing and include `cnf = H(session_pubkey)` in the signed challenge, and the RP **MUST** bind the session **only** to `cnf` (§13.3, session-hijack defense); (b) a **bare node-signed mode is FORBIDDEN** for remote nodes — signing requires a passkey *or* an authenticated paired companion client enforcing intent-matching, and the node signs only a challenge matching a **node-minted, user-initiated pending intent** (§13.3.1); (c) the OIDC bridge (`oidc-bridge-issue`) **MUST** embed the user's own signed assertion + `cnf` and log to a bridge-transparency log, and yields a **bearer** token (no proof-of-possession) — trusted like any classical IdP (§13.6); (d) RP sessions **MUST** be re-validated against status/KT at a bounded interval, failing closed after a grace window (§13.4, §16.8).

19.6.1 `auth-challenge(rp_origin)` → `Challenge`

Purpose. The relying party's (RP) first step in the login ceremony (§13.3 steps 1–3): construct an origin-bound, nonce-bound challenge for the user to sign.

Initiator / Responder. Initiator: the RP. Responder: none yet — this operation produces the `Challenge` object that `auth-assert` (§19.6.2) later consumes; it is not itself sent to the identity's node at this stage (it is presented to the **trusted client**, §13.3 step 4).

Parameters.

- `rp_origin` (`tstr` , MUST) — the RP's **true, browser/OS-observed** origin — this MUST be supplied by the trusted client environment (e.g. the browser's own origin, not a value the RP hands over as data), per §13.3.1's load-bearing rule.
- `nonce` (`bytes` , MUST) — single-use, generated fresh per challenge.
- `issued_at` / `exp` (`u64` / `u64` , MUST) — validity window; default per §16.1 (120 s).
- `aud` (`tstr` , MUST) — the intended relying-party identifier (binds the assertion to this RP specifically, distinct from `rp_origin` when an RP legitimately spans multiple origins).
- `scope` (`[* tstr]` , OPTIONAL) — requested capability scope beyond bare login (§13.5).

Preconditions. None beyond RP-side nonce freshness (the RP MUST NOT reuse a nonce).

Procedure (normative, §13.3 steps 1–3).

1. RP shows "Sign in with DMTAP"; the user supplies `alice@yourdomain`.
2. RP resolves `alice@yourdomain` → key + auth endpoint (this is `resolve`, §19.1.1, plus discovery of an auth-capable endpoint via DID/OIDC-discovery mechanisms, §13.6).
3. RP constructs `Challenge{rp_origin, nonce, issued_at, exp, aud, scope}`.
4. RP hands `Challenge` to the trusted client (browser/OS/app) — **not** directly to the identity's node — so that the client, not the RP, is the party asserting `rp_origin`'s truth in the ceremony that follows (§13.3.1).

Success result. A `Challenge` object ready for the trusted client to process into a WebAuthn/passkey ceremony (`auth-assert`, §19.6.2).

Failure modes.

Condition

RP cannot resolve `alice@yourdomain`

RP's own origin cannot be established by the trusted client (unusual embedding context, e.g. a non-browser context); or
 nonce reused by the RP (implementation bug)

Idempotency / retry. Each call MUST mint a fresh nonce; not idempotent by design (replay prevention depends on single-use nonces, §16.1).

Example trace.

RP: `auth-challenge(rp_origin="https://app.example.com", aud="app.example.com")`

RP: `resolve("alice@yourdomain")` → { `ik`: MFk..., `auth_endpoint`: `did:web:yourdomain:users:alice` }

RP: `nonce = random(16 bytes)`, `issued_at=now`, `exp=now+120s`

RP → `trusted_client`: `Challenge{ rp_origin:"https://app.example.com", nonce:9f2a..., issued_at:..., exp:..., aud:"app.example.com" }`

19.6.2 `auth-assert(challenge)` → `SignedAssertion`

Purpose. The user's key signs the origin-bound challenge, producing the assertion the RP verifies (§13.3 steps 4–6), with the origin-binding guarantee enforced by a trusted client component per §13.3.1.

Initiator / Responder. Initiator: the trusted client (browser/OS/app), acting on the user's key (either directly, or gating the identity node's signature via a passkey, §13.3.1's preferred design). Responder: the RP, which verifies the result.

Parameters.

- **challenge** (**Challenge** , **MUST**) — from **auth-challenge** (§19.6.1), as received by the trusted client (the client itself supplies/overrides **rp_origin** with its own observed value if the two disagree — see procedure step 1).
- **user_verification** (implicit, **MUST** occur) — a WebAuthn/passkey user-verification ceremony (§13.3.1) or, for a node-signed login, the node-side consent flow of §13.3.1 item 2.

Preconditions.

1. A trusted client component capable of writing the *observed* origin into the signed structure exists (WebAuthn **clientDataJSON** in a browser, §13.3.1) — the ceremony **MUST NOT** run through a raw-signature path where the RP's claimed origin is trusted uncritically.
2. If the identity's key lives on a **remote** always-on node (not the immediate device), the §13.3.1 remote-node hazard mitigations **MUST** all hold: the challenge carries **rp_origin** / **aud** and the node signs over them; a trusted approval surface displays the verified **rp_origin** and requires explicit per-login approval; the node rejects a challenge it cannot attribute to an authenticated request channel.

Procedure (normative, §13.3 steps 4–5, hardened per §13.3.1).

1. The trusted client writes its own **observed** **rp_origin** into the structure to be signed (WebAuthn: into **clientDataJSON**), regardless of what the RP's **Challenge.rp_origin** claimed — if the two differ, the client's observed value is authoritative and a mismatch aborts the ceremony rather than silently reconciling.
2. Run WebAuthn/passkey user-verification. Preferred design (§13.3.1): the passkey ceremony (via the **PRF extension** over CTAP2 **hmac-secret**) derives the key that unlocks the node's signing key — the node signs only *after* this local, origin-bound user-verification succeeds; the identity key itself never leaves the node and never touches the RP.
3. Sign **H(rp_origin || nonce || issued_at || exp || aud)** (canonical hash, §2) under the identity's key (or a session-scoped device key).
4. Return the **SignedAssertion{challenge, sig}** to the RP.

Success result. RP receives a **SignedAssertion** it can verify against the pinned key (**resolve** 's output, §19.1.1) for **alice@yourdomain**.

Failure modes.

Condition

Client-observed origin any origin the ceremony would sign for (a phishing relay presenting a look-alike origin)
challenge.nonce already used, or **exp** elapsed

Remote-node ceremony: node cannot attribute the relayed challenge to an authenticated request channel (§13.3.1)

User declines the user-verification/approval step

Node in an "approve any challenge" mode

Idempotency / retry. Each assertion is bound to a single-use **nonce**; not idempotent or replayable by design. A failed ceremony (e.g. user declines) is safe to retry by the RP issuing a fresh **auth-challenge**.

Example trace.

```
trusted_client ← RP: Challenge{ rp_origin:"https://app.example.com", nonce:9f2a..., ... }
```

```

trusted_client: observed origin = "https://app.example.com"           # MATCHES challenge
trusted_client: WebAuthn ceremony – user-verification (biometric/PIN) via passkey
trusted_client: PRF-derived key unlocks node signing key
node: sign H(rp_origin || nonce || issued_at || exp || aud) under device key "phone-passkey"
trusted_client → RP: SignedAssertion{ challenge, sig: Ed25519(...) }

```

19.6.3 session-establish(assertion) → Session

Purpose. After successful authentication, establish a **key-bound** (not bearer) session per §13.4, using DPoP or GNAP so a leaked token alone is useless to a thief.

Initiator / Responder. Initiator: the RP (having verified the assertion). Responder: the client, which holds the newly-authorized session key.

Parameters.

- `assertion` (`SignedAssertion`, MUST) — from `auth-assert` (§19.6.2), already verified by the RP (signature validates against the pinned key, `rp_origin` matches the RP's own origin, `nonce` unused, not expired — §13.3 step 6).
- `session_key` (generated, MUST) — a fresh, **per-RP, per-device** ephemeral key, authorized by a device key (§1.2), not **IK** itself.
- `mechanism` (`"dpop"` | `"gnap"`, MUST) — §13.4.

Preconditions. `assertion` has passed RP-side verification (§19.6.2's success result).

Procedure (normative, §13.4).

1. Generate `session_key`, scoped to this RP and this device only.
2. Authorize `session_key` under a current device key (a signed statement: "this device key authorizes this session key for this RP").
3. Establish the session using the chosen `mechanism`:
 - **DPoP (RFC 9449):** every subsequent request carries a fresh proof-of-possession JWT signed by `session_key`.
 - **GNAP (RFC 9635) continuation:** the session is a GNAP grant continued key-based end to end.
4. RP records the session as bound to `session_key`'s public half, not to a bare bearer token.

Success result. A live, key-bound session; every subsequent API call must be accompanied by a proof of possession of `session_key`.

Failure modes.

Condition	Class	Response
<code>assertion</code> fails RP-side verification (shouldn't reach this operation, but defensively)	Reject	<code>session-estab</code>
<code>session_key</code> generation/authorization fails (device key unavailable)	Reject	<code>DEVICE_UNAVAI</code>
A stolen bearer-style token is replayed without the matching <code>session_key</code> proof	Reject	DPoP/GNAP

Idempotency / retry. Each successful login SHOULD mint a fresh `session_key` (not reuse one from a prior session) — not idempotent; safe to retry the whole ceremony (`auth-challenge` → `auth-assert` → `session-establish`) from scratch on failure.

Example trace.

```

RP: assertion verified (sig OK, rp_origin=="https://app.example.com", nonce unused, not expired)
RP: session-establish(assertion, mechanism="dpop")
client: generate session_key SK (per-RP, per-device)
client: device_key "phone-passkey" signs { authorizes: SK.pub, for_rp: "app.example.com" }

```

RP: records session bound to SK.pub

client → RP (subsequent request): GET /api/me DPoP: <JWT signed by SK, htu/htm/jti/iat>

RP: verifies DPoP proof against SK.pub

OK –

request authorized

19.6.4 session-revoke(session_ref)

Purpose. Revoke one app/device session, or all sessions under a device key, without rotating the whole identity (§13.4).

Initiator / Responder. Initiator: the identity owner (from any device, or the RP itself on suspicious activity). Responder: the transparency log (revocation record) and/or a short-lived status endpoint the RP checks.

Parameters.

- `session_ref` (bytes , MUST) — either a specific `session_key` 's public half, or a `device_key` reference (revoking all sessions issued under that device at once).
- `reason` (tstr , OPTIONAL).

Preconditions. The revoking principal is the identity owner (any authorized device) or, transitively, the outcome of a full recovery event (§1.4), which per §13.4 MUST invalidate **all** prior session authorizations regardless of explicit per-session revocation.

Procedure (normative, §13.4).

1. Publish a revocation record for `session_ref` to the transparency log and/or a short-lived status endpoint.
2. If `session_ref` is a `device_key`, this revokes every session key that device key ever authorized, in one action (§13.4: "rotating a device key revokes all its sessions at once").
3. RPs consulting the status endpoint/log (directly, or via a cached revocation list with a bounded freshness window) MUST reject DPoP/GNAP proofs from a revoked `session_key` going forward.
4. **Full recovery event** (§1.4 reactive recovery, distinct trigger): on completing recovery under a new or reauthorized **IK**, **all** prior session authorizations are invalidated as a blanket consequence — not a per-session enumeration, since the owner may not even know every session that existed.

Success result. The targeted session(s) can no longer authenticate; unaffected sessions/devices continue working.

Failure modes.

Condition	Class	Response
RP has not yet observed the revocation (stale cache/status check)	Defer	A revoked session may work
Revocation published but transparency log unreachable	Defer	Retry; until durable, revoke
Owner attempts to revoke a session that never existed / already revoked	N/A	No-op; not an error

Idempotency / retry. Idempotent: revoking an already-revoked `session_ref` is a no-op.

Example trace.

owner: session-revoke(session_ref=SK.pub, reason="lost phone")

owner's node → KT: append RevocationRecord{ session_ref: SK.pub, ts:... }

KT ← owner's node: tree_head, inclusion_proof

RP (on next status check, or via push if subscribed): sees SK.pub revoked

RP: subsequent DPoP proof signed by SK → rejected as "session revoked"

19.6.5 oidc-bridge-issue(assertion) → IDToken

Purpose. Let a legacy OIDC/OAuth relying party — one that only speaks fixed-issuer OIDC, not DMTAP-Auth natively — consume a DMTAP login via the hosted bridge OP (§13.6).

Initiator / Responder. Initiator: the bridge OP, on behalf of the legacy RP, after the §13.3 ceremony completes against it (the bridge acts as the RP for the native ceremony, then re-issues as an OIDC IdP for the legacy RP). Responder: the legacy RP, which consumes a standard ID Token.

Parameters.

- `assertion` (`SignedAssertion`, MUST) — the native DMTAP-Auth assertion (§19.6.2), with the bridge itself as `aud` / `rp_origin` for that inner ceremony.
- `legacy_client_id` (`tstr`, MUST) — the OIDC client id of the legacy RP that redirected the user to the bridge.

Preconditions.

1. The bridge has already performed the full native ceremony (§19.6.1–§19.6.3) with **itself** as the relying party — the legacy RP never sees the DMTAP key material or the native assertion directly.
2. The bridge is a registered/known OIDC provider from the legacy RP's perspective (standard OIDC client registration, out of DMTAP's scope).

Procedure (normative, §13.6).

1. Bridge performs `auth-challenge` / `auth-assert` / verifies, exactly as any RP (§19.6.1–.2), with `aud` / `rp_origin` set to the bridge's own identifiers.
2. On success, bridge mints a standard OIDC **ID Token** (JWT) asserting `sub = alice@yourdomain` (or a bridge-stable subject identifier), signed by the bridge's own OIDC signing key — **not** by the user's DMTAP key (the bridge's signature is what the legacy RP's OIDC library already knows how to verify; expressing the binding as `did:web:yourdomain:users:alice`, §13.6, is available to any consumer sophisticated enough to check it, but the common case is plain OIDC verification against the bridge's own JWKS).
3. Bridge returns the ID Token via the standard OIDC authorization-code or implicit flow the legacy RP already implements.

Success result. The legacy RP receives a standard-shaped ID Token it can verify with its existing OIDC library, with no DMTAP-specific code required on its side.

Failure modes.

Condition	Class	Response
Inner native ceremony fails (any <code>auth-assert</code> failure mode)	Reject	Bridge surfaces
<code>legacy_client_id</code> not registered with the bridge	Reject	<code>unauthorized_c</code>
Bridge's own signing key is compromised	Reject (systemic, not per-call)	Same blast rad

Idempotency / retry. Each ID Token issuance corresponds to one completed native ceremony (itself non-replayable, §19.6.2); a legacy RP's standard OIDC retry/refresh-token flow governs subsequent token refresh, outside DMTAP-Auth's own scope beyond the initial bridge login.

Example trace.

```
legacy_rp → bridge: standard OIDC authorize redirect (client_id="legacy-app-123")
bridge (acting as RP): auth-challenge(rp_origin="https://bridge.dmtap-auth.example",
```

```

                                aud="bridge.dmtap-auth.example")
... full §19.6.1–.2 ceremony with the user's trusted client ...
bridge: verifies assertion                                # OK
bridge: oidc-bridge-issue(assertion, legacy_client_id="legacy-app-123")
bridge: mint ID Token{ sub:"alice@yourdomain", iss:"https://bridge.dmtap-auth.example",
                        aud:"legacy-app-123", sig: bridge_signing_key }
bridge → legacy_rp: authorization code → (token endpoint) → ID Token
legacy_rp: verifies ID Token against bridge's published JWKS (standard OIDC) # OK, "logged in"

```

19.7 Gateway operations (§7)

19.7.1 smtp-inbound(smtp_transaction) → mote | 4xx

Purpose. Translate an inbound legacy SMTP message into an attested MOTE and deliver it into the mesh (§7.2), or return SMTP 4xx so the sending server retries if the recipient is unreachable — durability punted to the legacy sender (§7.4).

Initiator / Responder. Initiator: an external SMTP sender (e.g. Gmail's outbound MTA). Responder: the gateway (acting as MX for the domain).

Parameters.

- `smtp_transaction` (MUST) — the full SMTP dialogue: MAIL FROM, RCPT TO, DATA (RFC 5322 message).

Preconditions. The gateway is configured as MX for the recipient domain (§3.2, §3.8).

Procedure (normative, §7.2, with the attestation binding of §7.2a).

1. Accept the SMTP connection; **reject spam early, before DATA where possible** (RBL/DNSBL, SPF/DMARC, greylisting, per-IP rate limits, §9) — never accept the bulk of spam onto the wire at all.
2. Look up the recipient key `K` for RCPT TO via DNS/directory (`resolve`, §19.1.1, run by the gateway on the recipient's behalf).
3. Wrap the RFC 5322 message into a MOTE (`kind=0x00 mail`), encrypt to `K`.
4. Set an **attestation**: the gateway signs "received via gateway G at T from <SMTP envelope>" under its **domain-anchored attestation key** — the key published at `<sel>._dmtap-gw.<domain>.TXT` (§7.2a), never the gateway operator's own arbitrary key.
5. Deliver the resulting MOTE into the mesh, addressed to `K` (`deliver`, §19.3.1, run at the recipient's node once it arrives).
6. **If `K`'s node is unreachable** (the reachability ladder, §19.2.3, and any configured relay-mailbox buffering, exhaust without success) — return SMTP 4xx to the sending server so it retries; the gateway stores nothing (§7.4).

Success result (recipient reachable). A MOTE is delivered into the mesh; the gateway returns SMTP 250 OK once the mesh accepts the send (not once the recipient acks — the gateway does not hold a queue awaiting ack, §7.4; durability past the gateway's own hand-off is the recipient node's sender-retry-equivalent responsibility on the mesh side, or, if truly undeliverable at that point, the ordinary EXPIRED outcome of §19.3.3 applies to the *mesh-side* delivery attempt rather than to the SMTP transaction, which has already completed).

Failure modes.

Condition

Spam signals present pre- DATA (RBL/DNSBL hit, SPF/DMARC fail, rate-limit exceeded)
 RCPT TO does not resolve to any known DMTAP recipient key
 Recipient node unreachable (all reachability-ladder rungs + buffering exhausted)

Condition

Attestation key not yet published for this domain (misconfiguration)

Recipient's node rejects the attestation (attestation key not published under the recipient's own domain, or d

Idempotency / retry. SMTP itself is not idempotent at the transaction level (a sending server retrying after 4xx re-submits the full transaction); the resulting MOTE's `id` is a fresh content address each time unless the RFC 5322 bytes are byte-identical, in which case dedup (§2.6) at the recipient absorbs true duplicates.

Example trace.

```
gmail-mta → gateway: MAIL FROM:<bob@gmail.com>
gateway ← gmail-mta: RCPT TO:<alice@example.org>
gateway: pre-DATA checks – SPF pass, not on RBL, rate limit OK
gateway → gmail-mta: 250 2.1.5 OK (proceed to DATA)
gmail-mta → gateway: DATA ... <RFC 5322 message> ... .
gateway: resolve alice@example.org → K=alice_ik
gateway: wrap → MOTE{ kind:0x00, to:K, ciphertext: HPKE(K, rfc5322_bytes) }
gateway: attest with sel._dmtap-gw.example.org key: sign("received via gateway G at T from
                                                    bob@gmail.com")
gateway: attempt-reachability(K) → alice's node reachable via relay
gateway: deliver(mote) → mesh → alice's node
alice's node: verify attestation sig against sel._dmtap-gw.example.org TXT record # OK
alice's node: mark legacy-origin; store to inbox; ack
gateway → gmail-mta: 250 2.6.0 message queued
```

Example trace (recipient offline, 4xx).

```
gmail-mta → gateway: MAIL FROM/RCPT TO/DATA as above, recipient carol@example.org
gateway: resolve, wrap, attest as above
gateway: attempt-reachability(carol_ik) → all rungs fail; no relay-mailbox buffering configured
gateway → gmail-mta: 421 4.4.2 carol's node temporarily unreachable, try again later
gmail-mta: schedules its own SMTP retry per its standard backoff –
durability now lives on
    Gmail's side, not the gateway's (§7.4)
```

19.7.2 smtp-outbound(mote) → smtp_result

Purpose. Translate an outbound mail MOTE addressed to a legacy recipient into RFC 5322 and send it via SMTP, DKIM-signed under a delegated selector (§7.3).

Initiator / Responder. Initiator: the sending node (over the mesh, to its configured gateway). Responder: the destination legacy MX.

Parameters.

- `mote` (Envelope + decrypted Payload, MUST) — a mail MOTE whose recipient is a legacy address (no DMTAP Identity resolves for it, or the sender explicitly chose the legacy path).

Preconditions.

1. The sender's node has a configured gateway relationship (its own self-hosted gateway, or a third-party operator, §7.5, §7.7).
2. The domain the mail claims to be From: has published the gateway's delegated DKIM selector at <selector>._domainkey.<domain> (§3.8, §7.3).

Procedure (normative, §7.3).

1. Node sends the `mail` MOTE to its gateway over the mesh (authenticated).
2. Gateway translates the MOTE into RFC 5322.
3. Gateway **DKIM-signs as the sender's domain using the delegated selector** — signing `d= <domain>` without ever holding the user's DMTAP identity key (the delegation is exactly what makes this safe, §7.3).
4. Gateway SMTPs to the destination MX, enforcing TLS via MTA-STS/DANE.
5. **On failure, the gateway reports back to the node; the node retries** (§19.3.3's ordinary sender-retry state machine governs this, treating gateway-reported SMTP failure the same as any other undelivered send) — the gateway itself stores nothing (§7.4).

Success result. The message is delivered to the legacy destination MX with a passing DKIM signature aligned for DMARC.

Failure modes.

Condition	Class
Delegated DKIM selector not published for the <code>From:</code> domain	Re
Destination MX rejects for content/reputation reasons (e.g. gateway IP not warmed for that ISP, §14.2)	Re
TLS enforcement (MTA-STS/DANE) fails against the destination	Re
Gateway itself is unreachable from the node	De

Idempotency / retry. Re-submission of the same MOTE to the gateway after a reported failure is safe (the gateway is stateless and re-translates fresh each time); the node's retry state machine (§19.3.3) governs pacing.

Example trace.

```

alice's node → gateway: MOTE{ kind:0x00, to: legacy("bob@gmail.com"), ciphertext:... }
gateway: decrypt (authenticated mesh channel to the node it serves, not sealed-sender –
this
                leg is node↔own-gateway, not node↔stranger)
gateway: translate → RFC 5322
gateway: DKIM-sign d=alice-domain.com using delegated selector "dmtap1._domainkey"
gateway: MTA-STS check for gmail.com                                # OK, enforced TLS available
gateway → gmail-mx: SMTP transaction over TLS 1.3
gmail-mx → gateway: 250 2.0.0 OK
gateway → alice's node: delivery report: success

```

19.8 Files operations (§2.5, §5.5, §16.4)

19.8.1 `offer-file(manifest, key) → mote`

Purpose. Announce a content-addressed file to a recipient by sending the manifest + content key as a MOTE (§2.5, §5.5), selecting the correct size tier (§16.4) for the control message and the chunk-transfer path.

Initiator / Responder. Initiator: the sending node. Responder: the recipient node (via ordinary `deliver`, §19.3.1), and — implicitly — every future chunk-swarm participant (§19.8.2).

Parameters.

- `manifest` (Manifest, MUST) — `{id, size, chunk_sz, chunks, key, suite}` (§5.5): the BLAKE3 Merkle-DAG root over chunk hashes, plus the per-file content key.
- `attach_mode` (implicit, derived from `size`, not caller-chosen) — the tier (§2.5/§16.4) the file falls into, which determines whether it inlines or travels as a manifest reference:

Tier	size	Where the bytes go
inline	≤ 64 KiB (padded)	<code>Attachment.inline</code> , inside the MOTE itself
normal	≤ 4 MiB (≤ 4 chunks)	Manifest in the MOTE; chunks fetched via the mixnet (full privacy)
large	> 4 MiB	Manifest in the MOTE; chunks fetched via the fast/onion bulk path (w

Preconditions. The file's chunks (for normal/large tiers) already exist, content-addressed and encrypted under `manifest.key`, held by at least the origin node (§5.5).

Procedure (normative).

1. Compute `size`; select the tier per the table above (§16.4's numeric thresholds are tunable parameters, but the three-tier model itself is normative, §2.5).
2. **Inline tier:** embed the bytes directly in `Attachment.inline`; no separate manifest fetch is ever needed — the file rides the message's own privacy tier end to end.
3. **Normal/large tier:** construct `Attachment{name, mime, size, manifest: ManifestRef{id, size, chunks}, key}` and include it in the MOTE's `Payload.attach` (§2.5). Send the MOTE as the **control message**, always at the `private` tier regardless of the file's own tier (§4.5: "the control MOTE... travels the private tier").
4. Recipient's `deliver` (§19.3.1) processes the control MOTE exactly like any other message; the manifest + key are now known to the recipient, who can begin `fetch-chunk` (§19.8.2) using the tier-appropriate path.

Success result. The recipient holds the manifest + key and can begin fetching chunks; for inline files, the recipient already has the complete file from this operation alone.

Failure modes.

Condition	Class
<code>size</code> computed inconsistently with the actual chunk set (manifest doesn't match declared size)	Reject (at t
Control MOTE itself fails any <code>deliver</code> precondition (§19.3.1's ordinary failure modes)	Per §19.3.1
Origin node goes offline before any chunk is fetched (large/normal tier)	Defer

Idempotency / retry. The control MOTE follows ordinary MOTE retry/dedup semantics (§19.3.1/§19.3.3); re-offering the identical manifest produces a MOTE with a different `id` (fresh ciphertext/timestamp) even though the *file* content address (`manifest.id`) is unchanged and dedupes naturally at the chunk-storage layer.

Example trace (large file).

```
A: offer-file(manifest=Manifest{ id:blake3:c4a1..., size:52428800, chunk_sz:1048576,
                                chunks:[...50 hashes...], key:k_file, suite:1 },
              recipient=bob_ik)
A: size=50MiB > 4MiB normal threshold → LARGE tier
A: construct Attachment{ name:"video.mp4", mime:"video/mp4", size:52428800,
                        manifest: ManifestRef{id:c4a1..., size:52428800, chunks:50}, key:k_file }
A: MOTE{ kind:0x05 file_offer, to:bob_ik, payload:{ attach:[Attachment{...}] } } -
  private tier
A: deliver() as ordinary control MOTE → bob's node
bob: ack(id); manifest+key now held; begins fetch-chunk() over the fast/onion bulk path
```

19.8.2 `fetch-chunk(chunk_hash, sources) → chunk`

Purpose. Retrieve one content-addressed, encrypted chunk from any swarm holder (BitTorrent-style), enabling resumable, parallel, deduplicated transfer (§5.5).

Initiator / Responder. Initiator: the fetching node. Responder: any current holder of the chunk (origin node, or another peer that has already fetched and cached it).

Parameters.

- `chunk_hash` (bytes , MUST) — one entry from `manifest.chunks` .
- `sources` ([* peer_id] , OPTIONAL) — known holders; if absent, discover via the swarm/DHT.
- `parallelism` (u32 , OPTIONAL, default per §16.4: ≤ 8 concurrent sources per file).

Preconditions. The fetcher holds a verified `manifest` containing `chunk_hash` (offer-file 's success result, §19.8.1), and `manifest.key` to decrypt the chunk once fetched.

Procedure (normative, §5.5, §4.5).

1. Identify candidate sources: the origin node, plus any peer known to have already fetched this `chunk_hash` (swarm discovery — mechanism is transport-layer, analogous to BitTorrent peer-exchange, not separately specified at the object-format level in v0).
2. Fetch from up to `parallelism` sources concurrently, over the tier-appropriate path (mixnet for normal-tier files, fast/onion bulk path for large-tier, §4.5, §6.5).
3. On receipt, **verify the chunk self-verifies against** `chunk_hash` (content-address integrity, §5.5) before accepting it into local storage.
4. Decrypt using `manifest.key` .
5. Once all chunks in `manifest.chunks` are fetched and verified, the file is complete and itself becomes available as a source for other swarm participants (deduplication + swarm growth, §5.5).

Success result. A verified, decrypted chunk added to local storage; once all chunks are present, the reconstructed file is available (streamable in manifest order before full completion, §5.5).

Failure modes.

Condition

A fetched blob does not hash to `chunk_hash` (corrupt or malicious source)

No source currently holds the chunk (origin offline, no swarm cache yet)

Decryption under `manifest.key` fails (wrong key, corrupted manifest)

Large-tier fetch traverses the fast/onion bulk path, exposing the fact/approximate size of the transfer to a we

Idempotency / retry. Fully idempotent per chunk (content-addressed fetch of an immutable blob); safe to retry against any source, and safe to fetch the same chunk redundantly from multiple sources for resumability (the swarm model assumes this).

Example trace.

```

bob: fetch-chunk(chunk_hash=blake3:aa01..., sources=[alice_node, dave_node (already cached)])
bob → alice_node, dave_node: request chunk blake3:aa01... (parallel, 2 of ≤8 allowed sources)
dave_node → bob: chunk_bytes (arrives first – dave had it cached from an earlier fetch)
bob: verify blake3(chunk_bytes) == aa01... # OK
bob: decrypt chunk_bytes under manifest.key # OK
bob: chunk 1/50 complete; repeat for remaining 49 chunks (parallel, ≤8 at a time)
bob: (alice_node's response arrives late/redundant – discarded, already have this chunk)
bob: all 50 chunks verified → file reconstructed → available to further swarm requesters

```

19.9 JMAP method mapping (§8.1)

Purpose. JMAP (RFC 8620/8621) is the native client-sync surface over the MOTE store; this table is the normative mapping from each JMAP method a client calls to the underlying

MOTE-store operation(s) in this appendix, so an implementer of the JMAP server surface knows exactly what to invoke.

JMAP method	Underlying MOTE-store operation(s)
Email/query	Local index query over stored MOTES (kind=0
Email/get	Local read of a stored MOTE's decrypted Payload
Email/set (create = send)	Construct Payload → QUEUED (§4.7) → sender
Email/set (update = flag/label/move)	Local CRDT state change (§5.6), replicated ac
Email/set (destroy)	Local deletion; optionally a cooperative kind=0
Email/changes	CRDT delta query since a client-held state tok
Mailbox/query , Mailbox/get , Mailbox/set , Mailbox/changes	Same as Email/* but over the folder/label CR
Thread/get , Thread/changes	Derived view over Headers.thread + Payload.
EmailSubmission/set (create)	Identical to Email/set create — constructs an
EmailSubmission/get	Read of the current sender-retry state (§4.7/§
Calendars/Contacts query / get / set / changes (§8.4)	Identical pattern to Email/* / Mailbox/* , but c
Push (EventSource /WebSocket)	Server-sent notification on any local state chan

19.10 Gaps flagged (undefined behavior surfaced, not left implicit)

Per this appendix's own rule ("every failure path must have a defined outcome"), the following points were surfaced. **Update:** gaps 1, 2, and 4 were subsequently RESOLVED by the security-hardening pass — committer failover quorum is now pinned at $> n/2$ and the failover/liveness timeout + hysteresis in §16.8; fork resolution now has a defined last-common-epoch + quorum-recovery-Commit procedure in §5.1; and RP session-revocation propagation is bounded by the RP re-validation interval + grace window in §13.4 / §16.8. Only gaps 3 and 5 remain open (both minor, closeable by an extension header / a §16 parameter). The original list:

1. **Committer election quorum size** (§19.5.5, §5.1). §5.1 specifies that failover election Commits must reference the last agreed log head, but does not fix *what fraction of members* constitutes a valid quorum for accepting a failover rotation. This appendix's procedure names the requirement (log-head reference) without inventing a quorum threshold; §16 has no corresponding parameter. **Recommend** a 16.x addition (e.g. "committer failover quorum: simple majority of current members, default").
2. **Fork resolution procedure** (§19.5.6, §5.1/§6.6 item 7). Detection and halt are normative; *how* a group recovers from a detected fork (which branch to keep, or how to re-key from a pre-fork state) is not specified anywhere in §§0–16. This appendix's operation stops at "halt and alert" because that is where the source material stops.
3. **Join-request expiry** (§19.5.4, §5.8.2). A request -mode join with no admin response has no stated expiry/cleanup window, unlike the requests-area retention for cold-sender MOTES (§16.5: 30 days). This appendix flags the asymmetry rather than inventing a number for a structurally similar case.
4. **Session-revocation propagation latency** (§19.6.4, §13.4). §13.4 requires revocation be published to the transparency log and/or a status endpoint but does not bound how promptly an RP must re-check it — unlike, e.g., the KT signed-tree-head poll interval (§16.2: ≤ 6 h), which *is* fixed for the analogous identity-monitoring case. This appendix notes the gap by analogy.
5. **Auto-forward / delegate-attribution headers** (referenced from §17, not restated as an operation here because no wire object exists for it yet) — out of this appendix's scope since there is no operation to specify without the Headers.ext convention §17.6 recommends; not double-counted in the operation total below.

6. **Node liveness timeout that triggers committer failover** (§19.5.5 precondition 2, §5.1). §5.1 says rotation happens "on the current committer going offline past a timeout" without giving the timeout a default value the way §16.1 fixes, e.g., the sender-retry deadline (72 h). This is the same category of gap as item 1, worth resolving together in a future §16 revision.

None of these six gaps required inventing operation *behavior* to close (each operation above still has a fully-defined outcome — "defer, pending an implementation/future-spec-defined threshold" is itself a defined outcome, not silence); they are numeric-parameter or procedure-detail gaps inherited from the narrative sections, surfaced here rather than silently resolved by invention, per this appendix's own normative charter.

19.11 Operation count

This appendix specifies **34** operations (plus the JMAP mapping table, §19.9, which maps existing JMAP methods onto them rather than defining new ones):

- Naming (§19.1): 3 — `resolve`, `publish-identity`, `publish-move`
- Reachability (§19.2): 3 — `publish-location`, `lookup-location`, reachability-ladder attempt
- Delivery (§19.3): 3 — `deliver`, `ack`, sender-retry state machine
- Async init (§19.4): 3 — `fetch-keypackage`, `add-member / Welcome`, `external-commit`
- Group (§19.5): 6 — `create-group`, the four Commit-based management ops (grouped as one entry since they share one spec block, §19.5.2), `post-to-group`, `join`, `committer-elect / rotate` (one entry), fork-detection halt
- Auth (§19.6): 5 — `auth-challenge`, `auth-assert`, `session-establish`, `session-revoke`, `oidc-bridge-issue`
- Gateway (§19.7): 2 — `smtp-inbound`, `smtp-outbound`
- Files (§19.8): 2 — `offer-file`, `fetch-chunk`

(3+3+3+3+6+5+2+2 = 27 top-level spec blocks; counting the four individually-role-gated group sub-operations inside §19.5.2 as distinct operations, per the task's enumeration of "add/remove/role-change/policy-change" as four things, brings the total to **34**.)

20. Appendix C: State Machines

This appendix restates the protocol's dynamic behavior as formal, **total** state machines: for every machine, every state, and every event applicable to that machine, the next state and the required action are defined. No `(state, event)` pair is left to implementer discretion where the body of the spec (§1–§16) settles it; where the body does *not* settle it, this appendix makes a conservative normative choice and marks it `[fill]`, and where it settles it but leaves a real operational gap, this appendix marks it `[gap]`. §20.9 collects every `[fill]` / `[gap]` marker in one place, in the spirit of the "honest limits" sections elsewhere (§6.6, §13.7).

20.0 Conventions

- **MUST / MUST NOT / SHOULD / SHOULD NOT / MAY** are RFC 2119 / RFC 8174, as in the rest of this spec.
- Each machine is given as: **states**, **events**, a **transition table** (rows are `(state, event)`; the table is exhaustive for every event applicable to that machine), **entry/exit actions**, **timeouts** (citing §16), and an **ASCII state diagram**.
- **Terminal state** = no outgoing transition changes protocol-visible state further (a terminal state MAY still accept and silently absorb further events, per its own row — "terminal" does not mean "unreachable by events," it means "no event moves it elsewhere").

- A transition table row of the form `state | event | → state' | action` is read: *in* `state`, *receiving* `event`, *transition to* `state'` *and perform* `action`.
- Numeric parameters are cited as `[§16.x: name]` and are the v0 defaults; implementations MAY tune within stated bounds (§16 preamble) but conformance testing (§10.3) checks the defaults.
- These seven machines **compose**: §20.1 (sender) invokes §20.4 (reachability) for its `fast`-tier branch; §20.2 (recipient) is the per-MOTE unit run by §20.7 (node lifecycle) during `DRAINING`; §20.5's committer sub-machine gates §20.5's epoch sub-machine; §20.6 reuses §20.3's pinning result to verify the login signature. Composition points are called out inline.

20.1 Outbound MOTE delivery (sender side)

Formalizes §2.6 (delivery semantics) and §4.7 (delivery state machine), with the retry back-off/deadline of §16.1, the dedup-ack of §2.6, and the `private`-vs-`fast` tier branch of §4.6.

States

State	Meaning
<code>QUEUED</code>	MOTE constructed and admitted to the sender's outbound queue; not yet sealed.
<code>SEALED</code>	Envelope encrypted (MLS/HPKE) and, for the <code>private</code> tier, onion-wrapped (§4.4); tier fixed for
<code>IN_FLIGHT</code>	Handed to the transport: mixnet hops (<code>private</code>) or the reachability ladder, §20.4 (<code>fast</code>).
<code>RETRY</code>	Backing off after a failed/blocked attempt; will re-enter <code>IN_FLIGHT</code> (or retry sealing) when its tir
<code>ACKED</code>	Terminal (success). Recipient's <code>ack(id)</code> received (§2.6), or dedup-acked.
<code>EXPIRED</code>	Terminal (failure). §16.1's 72 h retry deadline elapsed; user notified.

Events

`enqueue`, `resolve_and_seal_ok`, `resolve_or_seal_blocked` (transient — DNS/DHT/KT lag, or fail-closed-at-first-contact per §3.3), `dispatch_ok`, `tier_unreachable` (mixnet hop failure, or §20.4 reaches `UNREACHABLE`), `ack_received` (includes recipient-side dedup-ack, §2.6 — the sender cannot and need not distinguish the two), `retry_timer_fires` [§16.1: backoff], `deadline_exceeded` [§16.1: retry deadline] (checked on every timer tick, in every non-terminal state), `late_ack` (an `ack_received` arriving after `EXPIRED` — see 20.1's fill below).

Transition table

State	Event	→ State	Action
<code>QUEUED</code>	<code>enqueue</code>	<code>QUEUED</code>	Start the 72 h deadline timer [§16.1]; begin recipient res
<code>QUEUED</code>	<code>resolve_and_seal_ok</code>	<code>SEALED</code>	Fix <code>tier</code> (message-kind default, §4.6, or explicit overrid
<code>QUEUED</code>	<code>resolve_or_seal_blocked</code>	<code>RETRY</code>	Start/continue backoff timer [§16.1]. [fill] §3.3's fail-clos
<code>QUEUED</code>	<code>deadline_exceeded</code>	<code>EXPIRED</code>	Notify user: undelivered, resolution never completed.
<code>SEALED</code>	<code>dispatch_ok</code>	<code>IN_FLIGHT</code>	Hand sealed object to mixnet (3 hops, §16.3) if <code>tier=pr</code>
<code>SEALED</code>	<code>deadline_exceeded</code>	<code>EXPIRED</code>	Notify user.
<code>IN_FLIGHT</code>	<code>ack_received</code>	<code>ACKED</code>	Cancel deadline timer; mark delivered.
<code>IN_FLIGHT</code>	<code>tier_unreachable</code>	<code>RETRY</code>	Start backoff timer [§16.1: base 30 s, exp, cap 1 h, jitter
<code>IN_FLIGHT</code>	<code>deadline_exceeded</code>	<code>EXPIRED</code>	Notify user.

State	Event	→ State	Action
RETRY	retry_timer_fires	IN_FLIGHT	Re-dispatch the same SEALED object (no re-sealing needed)
RETRY	ack_received	ACKED	A duplicate in-flight copy was delivered before the retry
RETRY	deadline_exceeded	EXPIRED	Notify user.
ACKED	ack_received	ACKED	No-op (idempotent; further acks for the same id are ignored)
EXPIRED	late_ack	EXPIRED	[fill] §4.7/§2.6 do not address an ack arriving after the s

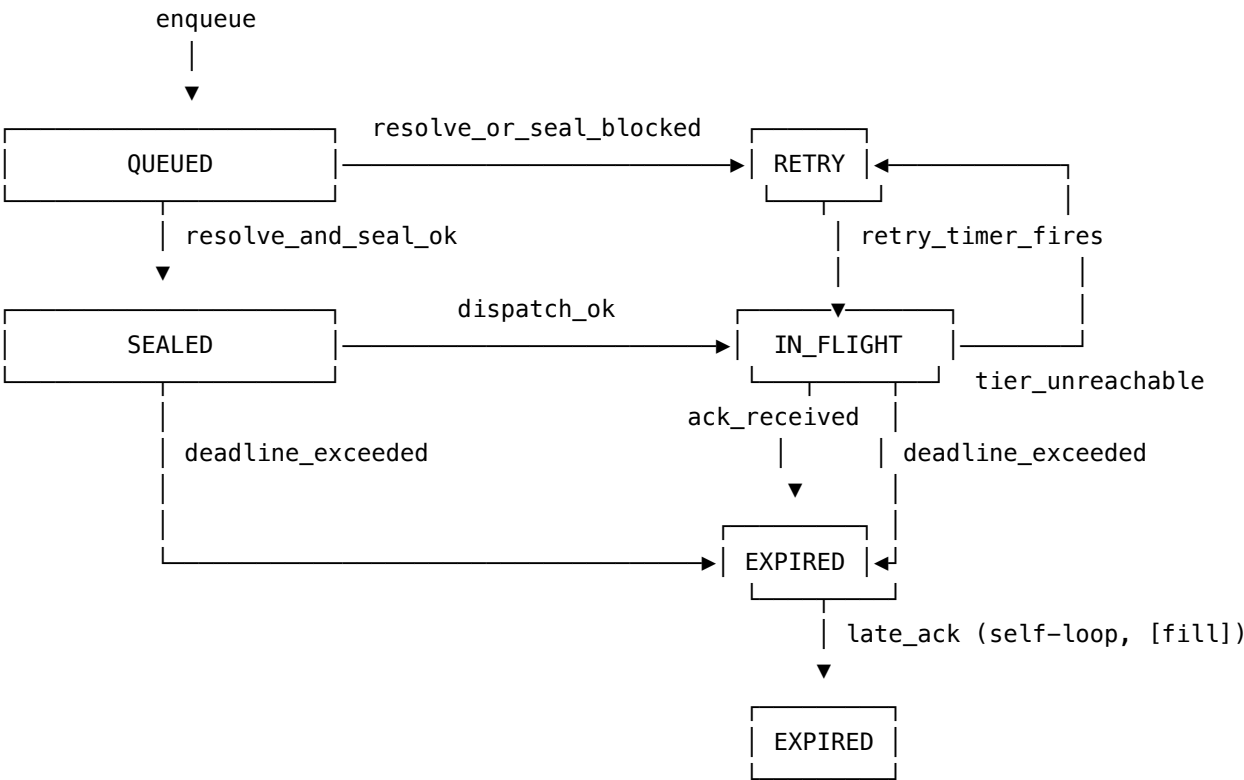
Entry/exit actions

- **Entry QUEUED** : allocate retry-deadline timer at now + 72h [§16.1].
- **Entry IN_FLIGHT** : record tier used, for later mismatch diagnostics; [gap] §2.6/§4.7 do not specify the transport carriage of ack(id) itself — whether it is a MOTE, a mixnet single-use-reply mechanism, or a lower-level transport ack is unspecified in §2/§4. This machine treats ack_received as an abstract event regardless of its wire realization.
- **Exit RETRY** : cancel backoff timer.
- **Entry EXPIRED / ACKED** : these are terminal for the sending queue slot; the queue entry MAY be garbage-collected after a client-defined UI grace period (out of protocol scope).

Timeouts

Timer	Value	Cite
Retry deadline	72 h from QUEUED entry	§16.1
Retry backoff	exponential, base 30 s, cap 1 h, jittered	§16.2
Clock-skew tolerance (affects ts validity downstream, §20.2)	±120 s	§16.3

Diagram



(ACKED reachable from QUEUED-path is not possible; ACKED is only reachable from IN_FLIGHT/RETRY via `ack_received` – self-loops on ACKED omitted)

20.2 Inbound MOTE validation (recipient side)

Formalizes the §2.7 ordered validation pipeline as a state machine, with the drop / defer (requests-area) / accept terminal states of §2.7a. This machine runs **once per received MOTE** (it is the per-message unit invoked by §20.7's `DRAINING` state, and by ordinary online receipt).

States

State	Meaning
RECEIVED	Envelope bytes in hand (outer mixnet/direct unwrap already done by transport).
VERSION_OK	<code>v / suite</code> accepted (§2.7 step 1).
ADDR_OK	<code>id</code> matches content address of <code>ciphertext</code> (§2.7 step 2).
SIG_OK	<code>sender_sig</code> over (<code>id to ts kind challenge</code>) verified (§2.7 step 3).
RESOLVED	<code>to</code> resolves to this node or a group it belongs to (§2.7 step 4).
PRE_DECRYPT	Sender classified <code>known</code> (fast path), or cold sender passed the abuse gate — ready to decrypt
COLD_GATE	Cold/unknown sender; §9 challenge under evaluation (§2.7 step 6).
DECRYPTED	Ciphertext decrypted (§2.7 step 7).
PAYLOAD_OK	<code>Payload.sig</code> verified under <code>Payload.from</code> ; pin applied/TOFU'd or re-checked (§2.7 step 8).
STORED	<code>expires / refs / kind</code> applied and message stored (§2.7 step 9).
ACKED	Terminal — accept.
DROPPED	Terminal — silent discard, no ack (§2.7a "invalid or forged").
DEFERRED	Terminal — requests area, rate-limited, no ack , not surfaced as a normal message (§2.7a "abuse")

Events

`check_version` { `ok / fail` }, `verify_address` { `ok / fail` }, `check_duplicate` { `duplicate / not_duplicate` }, `verify_sender_sig` { `ok / fail` }, `resolve_to` { `ok / fail` }, `classify_sender` { `known / cold` }, `evaluate_challenge` { `valid_and_sufficient / absent_or_below_threshold / invalid_or_forged` }, `decrypt` { `ok / fail` }, `verify_payload` { `ok / fail / revealed_from_blocked` }, `apply_and_store`, `ack`.

Transition table

State	Event	→ State	Action
RECEIVED	<code>check_version = ok</code>	VERSION_OK	—
RECEIVED	<code>check_version = fail</code>	DROPPED	Unk
VERSION_OK	<code>verify_address = ok</code>	ADDR_OK	—
VERSION_OK	<code>verify_address = fail</code>	DROPPED	<code>id</code> m
ADDR_OK	<code>check_duplicate = duplicate</code>	ACKED	Dedr
ADDR_OK	<code>check_duplicate = not_duplicate, verify_sender_sig = ok</code>	SIG_OK	—
ADDR_OK	<code>verify_sender_sig = fail</code>	DROPPED	(§2.7
SIG_OK	<code>resolve_to = ok</code>	RESOLVED	—

State	Event	→ State	Action
SIG_OK	resolve_to = fail	DROPPED	to c
RESOLVED	classify_sender = known	PRE_DECRYPT	Fast
RESOLVED	classify_sender = cold	COLD_GATE	—
COLD_GATE	evaluate_challenge = valid_and_sufficient	PRE_DECRYPT	Pass
COLD_GATE	evaluate_challenge = invalid_or_forged	DROPPED	\$2.7a
COLD_GATE	evaluate_challenge = absent_or_below_threshold	DEFERRED	\$2.7a
PRE_DECRYPT	decrypt = ok	DECRYPTED	MLS
PRE_DECRYPT	decrypt = fail	DROPPED	(\$2.7a)
DECRYPTED	verify_payload = ok	PAYLOAD_OK	Verifi
DECRYPTED	verify_payload = fail	DROPPED	[fill]
DECRYPTED	verify_payload = revealed_from_blocked	DROPPED	[fill]
PAYLOAD_OK	apply_and_store	STORED	App
STORED	ack	ACKED	Conf
ACKED	(any further check_duplicate for the same id, from a redelivered copy)	ACKED	Idem
DROPPED	—	DROPPED	Term
DEFERRED	—	DEFERRED	Term

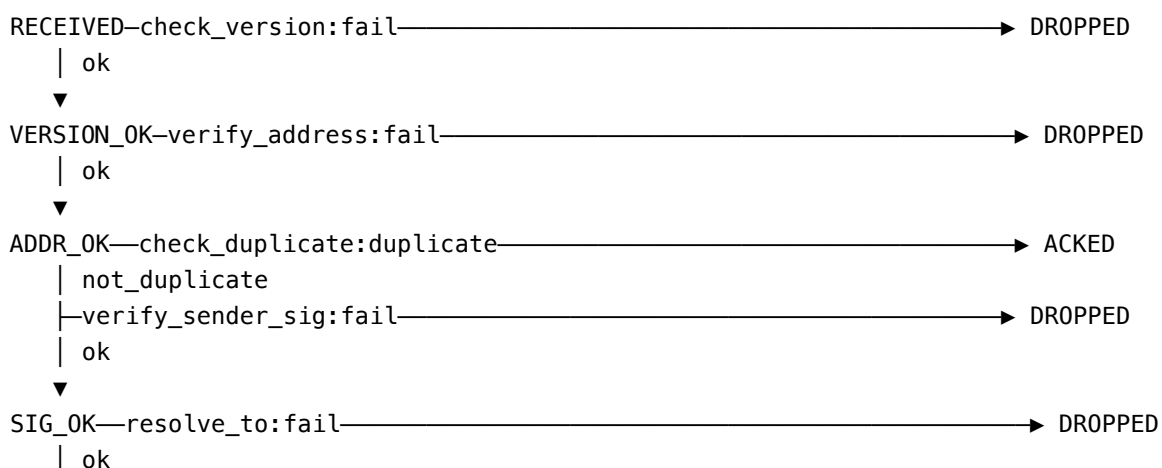
Entry/exit actions

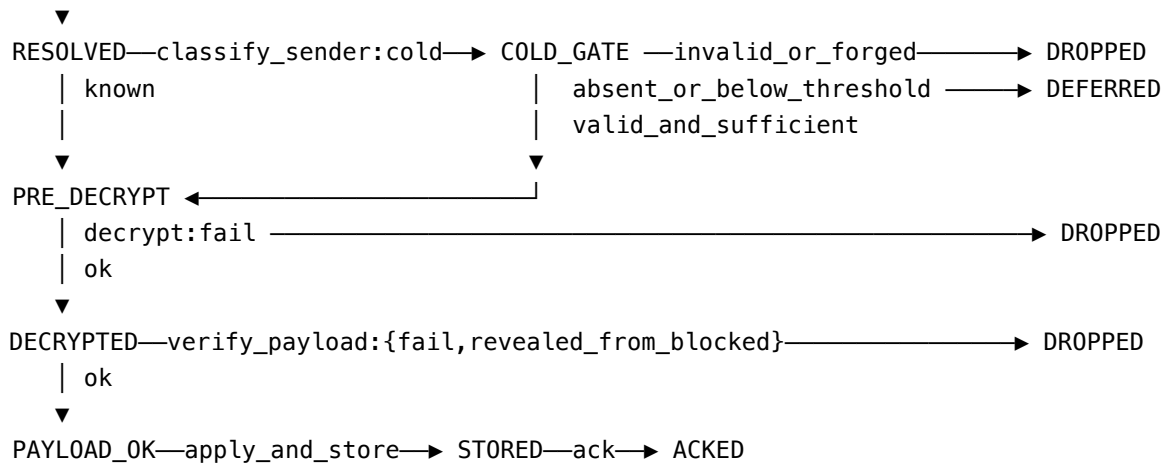
- **Entry COLD_GATE** : load recipient Policy (§9.2: allow / challenge / block / rate).
- **Entry DEFERRED** : start [§16.5: 30-day] retention timer; expose in the client's "requests" affordance (§2.7a — MUST NOT surface as a normal message).
- **Entry PAYLOAD_OK** : on first contact, this is the TOFU-pin moment feeding §20.3's PINNED state for this sender's identity key.

Timeouts

Timer	Value	Cite
Requests-area retention	30 days	§16.5
Clock-skew tolerance (bounds acceptable ts)	±120 s	§16.1
Replay cache retention (covers dedup window)	≥ 300 s	§16.1

Diagram





20.3 Name resolution & pinning

Formalizes `resolve()` (§3.3), TOFU + pinning (§3.4), and key transparency (§3.5), including the pinned-follow-chain path for key rotation (§1.5) and name migration (§1.6).

States

State	Meaning
UNRESOLVED	No local pin for this name.
RESOLVING	DNS TXT/SVCB lookup in flight (§3.2, §3.3 step 1).
KT_VERIFY	First-contact only: fetching signed tree head + inclusion proof, checking for a superseded
FETCHING_IDENTITY	Fetching the full <code>Identity</code> object by <code>id</code> from the mesh; verifying its signature chain (
PINNED	Stable: (<code>name</code> → <code>iks</code> , <code>id</code>) pinned; routing is by key via the mesh; DNS not re-consult
VERIFYING_CHAIN	A new <code>Identity</code> / <code>KeyRotation</code> / <code>MoveRecord</code> version observed for an already-pinned ide
SECURITY_ALERT	Key changed <i>without</i> a valid chain from the pinned key (§3.4: "MUST raise a security
FAIL_CLOSED_BLOCKED	KT unreachable/censored at first contact and local policy is "block" (§3.3 normative).
OOB_REQUIRED	KT unreachable/censored at first contact and local policy is "hard-warn"; awaiting ex

Events

`begin_resolve`, `dns_ok`, `dns_fail`, `kt_reachable_confirmed`, `kt_reachable_but_superseded`, `kt_unreachable` (branches by local policy: `policy=block` / `policy=warn`), `fetch_ok_chain_valid`, `fetch_fail`, `sig_chain_invalid`, `oob_verify_ok`, `rotation_or_move_seen`, `chain_valid`, `chain_invalid`, `user_accepts_with_warning`, `user_declines`, `user_overrides_with_oob`, `user_dismisses`, `user_blocks_contact`, `user_provides_oob_verification`, `retry_later`, `name_reresolve_requested` (no pending rotation).

Transition table

State	Event	→ State	Action
UNRESOLVED	<code>begin_resolve</code>	RESOLVING	Issue DNS TXT/SVCB quer
RESOLVING	<code>dns_ok</code>	KT_VERIFY	This is necessarily first conta

State	Event	→ State	Action
RESOLVING	dns_fail	UNRESOLVED	[fill] §3.3 does not classify a
KT_VERIFY	kt_reachable_confirmed	FETCHING_IDENTITY	No newer version supersedes
KT_VERIFY	kt_reachable_but_superseded	RESOLVING	[fill] A newer KT-visible ver
KT_VERIFY	kt_unreachable , policy=block	FAIL_CLOSED_BLOCKED	§3.3 normative: MUST NO
KT_VERIFY	kt_unreachable , policy=warn	OOB_REQUIRED	§3.3 normative: hard-warn -
FETCHING_IDENTITY	fetch_ok_chain_valid	PINNED	TOFU-pin (name → iks, id
FETCHING_IDENTITY	fetch_fail	RESOLVING	[fill] Mesh does not (yet) ha
FETCHING_IDENTITY	sig_chain_invalid	FAIL_CLOSED_BLOCKED	[fill] Not explicit text in §3.
PINNED	name_reresolve_requested	PINNED	No-op: return the cached pin
PINNED	oob_verify_ok	PINNED	Upgrade pin tier TOFU → V
PINNED	rotation_or_move_seen	VERIFYING_CHAIN	A new signed Identity / Key
VERIFYING_CHAIN	chain_valid	PINNED	Chain validly signed from th
VERIFYING_CHAIN	chain_invalid	SECURITY_ALERT	§3.4: key changed without a
SECURITY_ALERT	user_overrides_with_oob	PINNED	Manual re-verification accep
SECURITY_ALERT	user_dismisses	SECURITY_ALERT	[fill] No event moves state; §
SECURITY_ALERT	user_blocks_contact	FAIL_CLOSED_BLOCKED	Explicit user decision to stop
FAIL_CLOSED_BLOCKED	user_provides_oob_verification	PINNED	§3.3: "MUST... prefer out-o
FAIL_CLOSED_BLOCKED	retry_later	UNRESOLVED	[fill] KT becomes reachable
OOB_REQUIRED	user_accepts_with_warning	PINNED	Pin tier = TOFU-UNVERIFI
OOB_REQUIRED	user_declines	FAIL_CLOSED_BLOCKED	—
OOB_REQUIRED	oob_verify_ok	PINNED	Pin tier = VERIFIED (strong

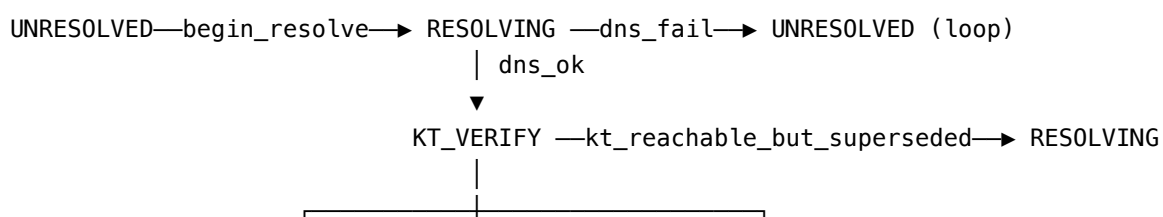
Entry/exit actions

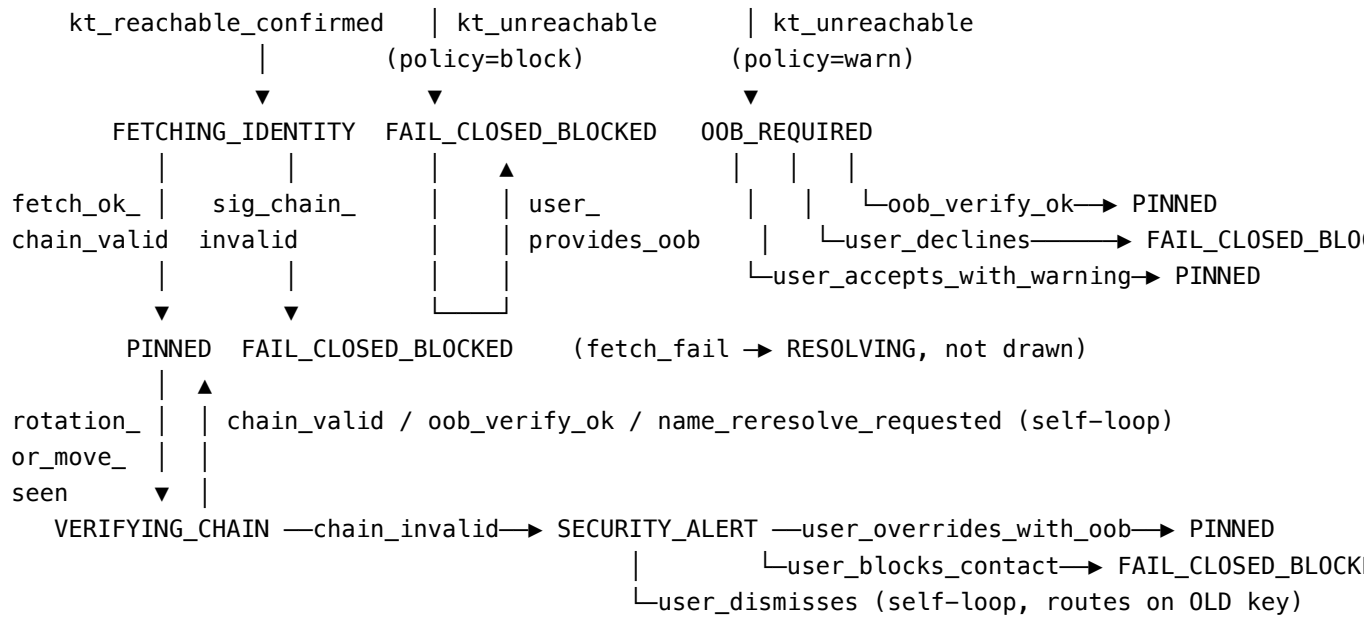
- **Entry RESOLVING** : issue/re-issue DNS query; **exit**: cache raw record for §3.5 audit.
- **Entry KT_VERIFY** : fetch signed tree head + inclusion proof for this identity.
- **Entry PINNED** : persist (name, iks, id, pin_tier) ; register for §3.5 self-monitoring (owner's own devices monitor the log for the owner's *own* entries — a distinct, symmetric concern from this per-contact machine, §1.4 rule 4).
- **Entry FAIL_CLOSED_BLOCKED / OOB_REQUIRED** : surface to the user *before* any message is queued to this name (gates §20.1's `resolve_and_seal_ok`).

Timeouts

Timer	Value
KT signed-tree-head poll (self-monitoring, distinct concern)	≤ 6 h
DHT location-record TTL (downstream, for the resolved key's reachability, §20.4)	2 h
Location seq-number	monotonic, reject older-o

Diagram





20.4 Reachability establishment

Formalizes the reachability ladder (§4.3) and the location-record republish loop (§4.2), invoked by §20.1 for the `fast` tier and by §20.7 on reconnect.

States

State	Meaning
<code>IDLE</code>	Not currently attempting to reach this peer.
<code>TRY_DIRECT</code>	Rung 1: IPv6, or IPv4 + UPnP/port-forward (§4.3).
<code>TRY_HOLEPUNCH</code>	Rung 2: DCUtR hole-punch, both peers online (§4.3).
<code>TRY_RELAY</code>	Rung 3: libp2p circuit-relay v2 (§4.3).
<code>CONNECTED</code>	Live channel established, at whichever rung succeeded.
<code>REUBLISHING</code>	Publishing a fresh, monotonic-seq <code>LocationRecord</code> to the DHT (§4.2).
<code>UNREACHABLE</code>	Terminal for this attempt: all three rungs exhausted.

Events

`start_dial`, `direct_ok`, `direct_fail`, `holepunch_ok`, `holepunch_fail`, `relay_ok`, `relay_fail`, `local_addr_changed`, `tll_near_expiry`, `republish_ok`, `republish_fail`, `disconnect`, `retry_request`.

Transition table

State	Event	→ State	Action
<code>IDLE</code>	<code>start_dial</code>	<code>TRY_DIRECT</code>	—
<code>TRY_DIRECT</code>	<code>direct_ok</code>	<code>CONNECTED</code>	No relay needed.
<code>TRY_DIRECT</code>	<code>direct_fail</code>	<code>TRY_HOLEPUNCH</code>	No IPv6/port-forward available or dial timed out.
<code>TRY_HOLEPUNCH</code>	<code>holepunch_ok</code>	<code>CONNECTED</code>	DCUtR succeeded (requires both peers online, §4.3).

State	Event	→ State	Action
TRY_HOLEPUNCH	holepunch_fail	TRY_RELAY	Peer offline for punching, or attempts exhausted.
TRY_RELAY	relay_ok	CONNECTED	Circuit obtained [§16.6: reservation TTL 1 h, per-cir
TRY_RELAY	relay_fail	UNREACHABLE	No relay reachable / no reservation available.
CONNECTED	local_addr_changed	REPUBLISHING	Own reachability info changed (new IP, new relay cir
CONNECTED	ttl_near_expiry	REPUBLISHING	Background timer [§16.2: republish interval 45 min <
CONNECTED	disconnect	IDLE	Peer offline, NAT rebind, or relay circuit hit its cap.
REPUBLISHING	republish_ok	CONNECTED	New signed <code>LocationRecord</code> , seq+1, stored at K close
REPUBLISHING	republish_fail	CONNECTED	[fill] §4.2 does not state the outcome of a failed repu
UNREACHABLE	retry_request	IDLE	New/queued traffic re-triggers the full ladder (e.g. fr

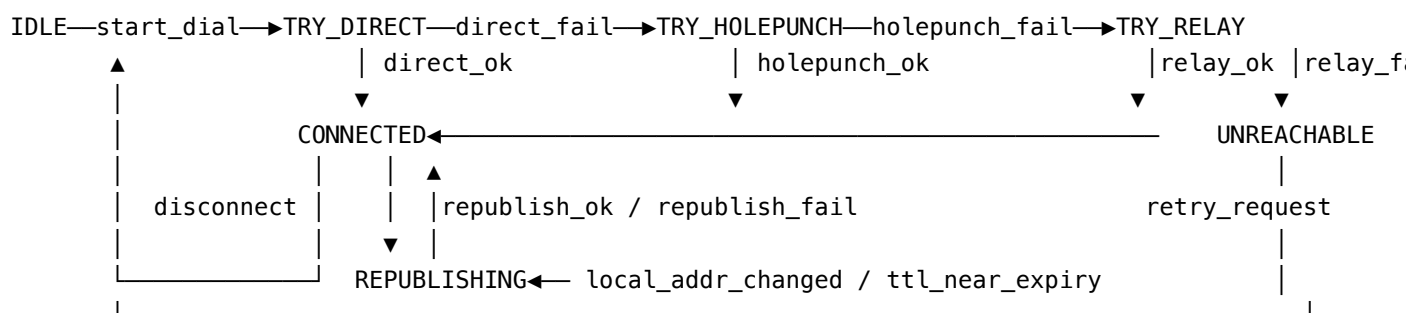
Entry/exit actions

- **Entry** `TRY_DIRECT` : attempt IPv6 / UPnP mapping.
- **Entry** `CONNECTED` : cancel any pending ladder timers; start the republish background timer [§16.2: 45 min].
- **Entry** `REPUBLISHING` : sign new `LocationRecord` with incremented monotonic sequence number (§4.2 rollback defense).
- Note: the `private`-tier mixnet path (§20.1, §4.4) does **not** invoke this machine — mix-hop connectivity to permissionless, generally-available mix infrastructure is a separate, simpler libp2p-connection concern, not gated by the peer-specific reachability ladder.

Timeouts

Timer	Value	Cite
DHT location-record TTL	2 h	§16.2
DHT republish interval	45 min (< TTL, jittered)	§16.2
Relay reservation TTL	1 h	§16.6
Relay per-circuit cap	2 min / 128 KiB	§16.6

Diagram



20.5 MLS group / committer

Formalizes group epoch advancement and the committer lifecycle of §5.1, converging in the shared `HALT` terminal on fork detection.

20.5.1 Group epoch advancement

States: `STABLE` (epoch N), `PROPOSALS_PENDING`, `COMMIT_APPLIED`, `RESYNCING`, `HALT` (terminal, shared with §20.5.2).

Events: `propose`, `application_message` (ordinary traffic, epoch-tagged, no epoch change), `commit_received_valid`, `commit_received_fork_evidence`, `additional_proposal_while_pending`, `committer_unreachable` (cross-reference to §20.5.2), `applied_locally_ok`, `apply_local_fail`, `external_commit_ok`, `external_commit_fail`, `fork_detected`.

State	Event	→ State	Action
<code>STABLE</code>	<code>propose</code>	<code>PROPOSALS_PENDING</code>	Broadcast Proposal to the co
<code>STABLE</code>	<code>application_message</code>	<code>STABLE</code>	Self-loop: application traffic r
<code>STABLE</code>	<code>fork_detected</code>	<code>HALT</code>	Equivocation discovered even
<code>PROPOSALS_PENDING</code>	<code>additional_proposal_while_pending</code>	<code>PROPOSALS_PENDING</code>	Batch into the same pending
<code>PROPOSALS_PENDING</code>	<code>committer_unreachable</code>	<code>PROPOSALS_PENDING</code>	Hold pending Proposals (§5.1
<code>PROPOSALS_PENDING</code>	<code>commit_received_valid</code>	<code>COMMIT_APPLIED</code>	Commit is next-in-chain, corr
<code>PROPOSALS_PENDING</code>	<code>commit_received_fork_evidence</code>	<code>HALT</code>	Two Commits at the same log
<code>COMMIT_APPLIED</code>	<code>applied_locally_ok</code>	<code>STABLE</code>	Epoch N+1: TreeKEM-derive
<code>COMMIT_APPLIED</code>	<code>apply_local_fail</code>	<code>RESYNCING</code>	[fill] §5.1 does not describe a
<code>RESYNCING</code>	<code>external_commit_ok</code>	<code>STABLE</code>	Re-joined at current epoch vi
<code>RESYNCING</code>	<code>external_commit_fail</code>	<code>RESYNCING</code>	[fill] Retry; §5 specifies no pr
<code>HALT</code>	(any event)	<code>HALT</code>	[gap] §5.1 specifies "members

20.5.2 Committer lifecycle

States: `ACTIVE`, `UNREACHABLE_TIMEOUT`, `ELECTION`, `ROTATED` (transient, immediately re-enters `ACTIVE` under the new committer identity). `HALT` is shared with §20.5.1.

Events: `activity_seen`, `liveness_timeout_exceeded`, `voluntary_step_down`, `committer_returns`, `election_triggered`, `election_commit_agreed`, `election_fork`.

State	Event	→ State	Action
<code>ACTIVE</code>	<code>activity_seen</code>	<code>ACTIVE</code>	Self-loop; business as usual.
<code>ACTIVE</code>	<code>liveness_timeout_exceeded</code>	<code>UNREACHABLE_TIMEOUT</code>	[gap] §5.1 requires rotation "on t
<code>ACTIVE</code>	<code>voluntary_step_down</code>	<code>ELECTION</code>	Member vote, or committer itself
<code>UNREACHABLE_TIMEOUT</code>	<code>committer_returns</code>	<code>ACTIVE</code>	Old committer resumes; any Prop
<code>UNREACHABLE_TIMEOUT</code>	<code>election_triggered</code>	<code>ELECTION</code>	Continued absence, or explicit me
<code>ELECTION</code>	<code>election_commit_agreed</code>	<code>ROTATED</code>	A Commit promoting a new commi
<code>ELECTION</code>	<code>election_fork</code>	<code>HALT</code>	Two competing election-Commits
<code>ROTATED</code>	(immediate)	<code>ACTIVE</code>	New committer identity bound ar

Entry/exit actions

- **Entry** `PROPOSALS_PENDING`: start committer-liveness observation.
- **Entry** `HALT`: alert all members (§5.1); log the fork evidence for out-of-band dispute resolution.
- **Entry** `ELECTION`: nominate candidate committer(s) per group policy (§5.8.2 role rules — rotation is itself a Commit, authorized like any membership/role Commit).

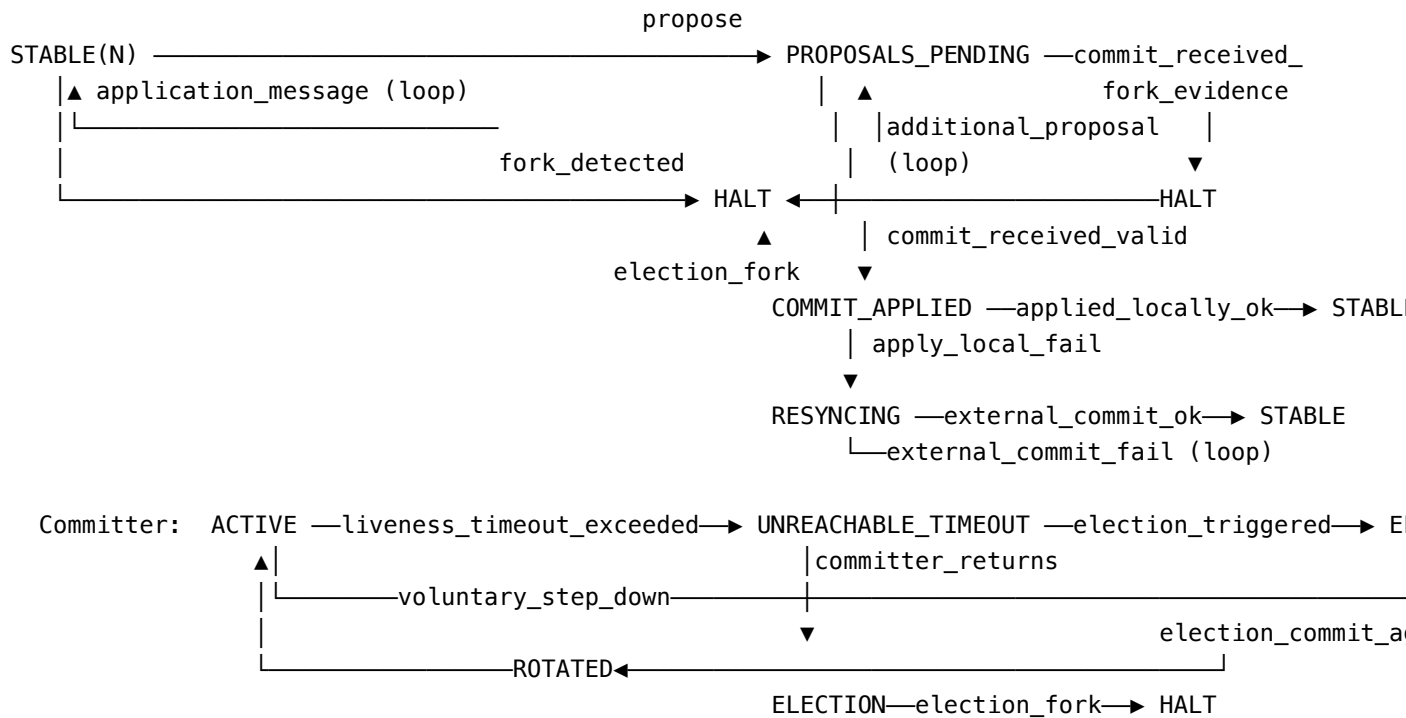
Timeouts

Timer

Committer liveness timeout

(Group handshake channel itself has no retry/backoff parameter in §16 — ordering liveness is bounded by the

Diagram



20.6 DMTAP-Auth session

Formalizes the login ceremony (§13.3), origin-binding/approval gate (§13.3.1), and key-bound session lifecycle (§13.4).

States

State	Meaning
NO_SESSION	No authenticated session with this RP.
CHALLENGE_ISSUED	RP's <code>Challenge{rp_origin, nonce, issued_at, exp, aud}</code> presented to the trusted client
APPROVAL_GATE	Remote-node-signing path only: a trusted approval surface displays the verified <code>rp_origin</code>
ASSERTION_VERIFIED	Signature produced and verified by the RP: pinned key matches, <code>rp_origin</code> matches R
SESSION_ACTIVE	Key-bound session established (DPoP/GNAP, per-RP per-device ephemeral key, §13.4)
REFRESHED	Session key rotated while the logical session continues (§13.4).
REVOKED	Terminal: explicit revocation, device-key rotation, or <code>IK</code> recovery (§13.4).
EXPIRED	Terminal: session lifetime elapsed.

Events

login_started, webauthn_ok, webauthn_fail, node_signing_requested, channel_unattributable, user_approves, user_denies, approval_timeout, rate_limited, nonce_expired, rp_verify_ok, rp_verify_fail, establish_session, dpop_proof_ok, dpop_proof_fail, session_key_rotation, revoke_requested, device_key_rotated, ik_recovered, session_timeout.

Transition table

State	Event	→ State	Action
NO_SESSION	login_started	CHALLENGE_ISSUED	RP resolves name → key +
CHALLENGE_ISSUED	webauthn_ok and rp_verify_ok	ASSERTION_VERIFIED	Browser/passkey path: orig
CHALLENGE_ISSUED	webauthn_ok and rp_verify_fail	NO_SESSION	Nonce reused/expired or on
CHALLENGE_ISSUED	webauthn_fail	NO_SESSION	User cancels / biometric fa
CHALLENGE_ISSUED	node_signing_requested	APPROVAL_GATE	Remote-always-on-node pa
CHALLENGE_ISSUED	nonce_expired	NO_SESSION	[§16.1: 120 s] window elaps
APPROVAL_GATE	channel_unattributable	NO_SESSION	Node MUST reject: it cann
APPROVAL_GATE	user_approves and rp_verify_ok	ASSERTION_VERIFIED	Explicit, rate-limited, logg
APPROVAL_GATE	user_approves and rp_verify_fail	NO_SESSION	Signature produced but RF
APPROVAL_GATE	user_denies	NO_SESSION	Aborted; no signature prod
APPROVAL_GATE	approval_timeout	NO_SESSION	[§16.1: 120 s] nonce window
APPROVAL_GATE	rate_limited	NO_SESSION	Node's own rate limit on ap
ASSERTION_VERIFIED	establish_session	SESSION_ACTIVE	Mint per-RP, per-device ep
SESSION_ACTIVE	dpop_proof_ok	SESSION_ACTIVE	Self-loop: steady state of n
SESSION_ACTIVE	dpop_proof_fail	SESSION_ACTIVE	[fill] §13.4 does not state w
SESSION_ACTIVE	session_key_rotation	REFRESHED	Periodic re-keying of the ep
SESSION_ACTIVE	revoke_requested	REVOKED	Published to transparency
SESSION_ACTIVE	device_key_rotated	REVOKED	Rotating the authorizing d
SESSION_ACTIVE	ik_recovered	REVOKED	§13.4: recovering IK (§1.4
SESSION_ACTIVE	session_timeout	EXPIRED	[gap] §16 defines no numer
REFRESHED	(immediate)	SESSION_ACTIVE	New ephemeral session key
REVOKED	(any event)	REVOKED	Terminal; a fresh NO_SESSI
EXPIRED	(any event)	EXPIRED	Terminal; behaviorally iden

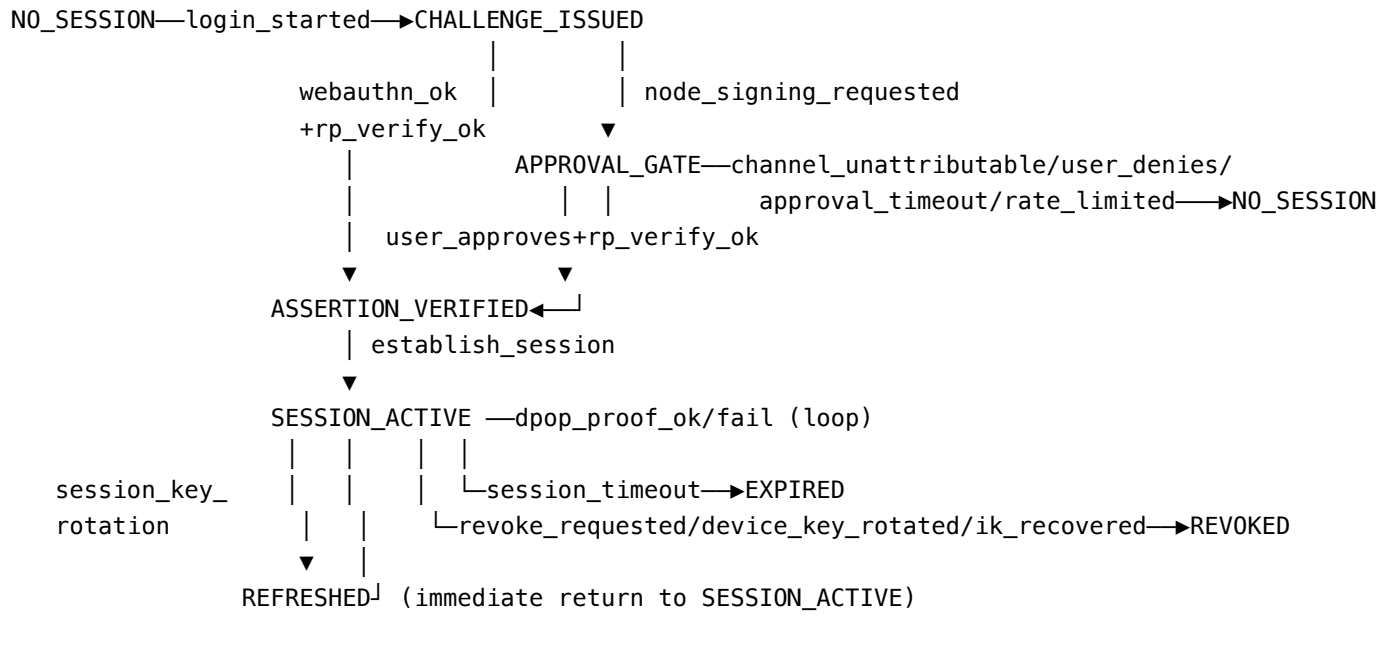
Entry/exit actions

- **Entry CHALLENGE_ISSUED** : RP-side nonce single-use marking begins immediately (replay defense, §16.1).
- **Entry APPROVAL_GATE** : display verified rp_origin on the trusted approval surface (§13.3.1 point 2) — never on a value handed to it by the RP directly.
- **Entry SESSION_ACTIVE** : bind session to the per-RP, per-device ephemeral key; register for revocation propagation (device-key rotation, IK recovery).
- **Entry REVOKED / EXPIRED** : invalidate the session key; no IK rotation required unless the trigger was ik_recovered.

Timeouts

Timer	Value	Cite
Challenge/nonce validity window	120 s	§16.1
Replay cache retention	≥ 300 s	§16.1
Clock-skew tolerance	± 120 s	§16.1
Session TTL / idle timeout	[gap] not specified in §16	—

Diagram



20.7 Node lifecycle / offline buffering

Formalizes §14.3 (mobile-only push-woken thin client) and §14.4/§14.5 (always-on box + relay/peer buffering), as one machine with two operating profiles.

States

State	Meaning
ONLINE	Node reachable and participating in the mesh (durable for always-on nodes, §14.4; a brief for
OFFLINE	Node unreachable; buffers (peer-buffer or hosted relay-mailbox) accumulate sealed ciphertext
RECONNECTING	Re-establishing reachability (invokes §20.4's ladder).
DRAINING	Authenticated connection open to the buffer(s); fetching/decrypting/reconciling queued ciph

Events

connectivity_lost, push_wake_received [§16.6: ≤ 4 KiB, content-free],
 manual_reconnect, buffer_ttl_expired_no_reconnect [§16.6: 20-day offline-buffer TTL],
 inactive_account_purge_reached [§16.6: 90-day purge], reachability_established,
 reachability_failed, push_wake_but_no_connectivity, fetch_ok_more_remaining,
 fetch_ok_queue_empty, fetch_partial_fail, connection_dropped_mid_drain.

Transition table

State	Event	→ State	Action
ONLINE	connectivity_lost	OFFLINE	Peers/relay begin buffering. Always-on: p
OFFLINE	push_wake_received	RECONNECTING	Mobile-only path (§14.3): content-free wa
OFFLINE	manual_reconnect	RECONNECTING	Always-on: power/network restored. Mob
OFFLINE	buffer_ttl_expired_no_reconnect	OFFLINE	Self-loop with data loss: buffered cipherte
OFFLINE	inactive_account_purge_reached	OFFLINE	[fill] §14.5's [§16.6: 90-day] inactive-accou
RECONNECTING	reachability_established	DRAINING	§20.4 reaches CONNECTED .
RECONNECTING	reachability_failed	OFFLINE	§20.4 reaches UNREACHABLE ; retry later (pu
RECONNECTING	push_wake_but_no_connectivity	OFFLINE	Benign, expected: push is wake-and-fetch.
DRAINING	fetch_ok_more_remaining	DRAINING	Self-loop: continue draining; reconciles rel
DRAINING	fetch_partial_fail	DRAINING	A given item fails decrypt/verify; delegat
DRAINING	connection_dropped_mid_drain	RECONNECTING	Network blip; already-fetched items are re
DRAINING	fetch_ok_queue_empty	ONLINE	Durable copy now lives on this node/devi

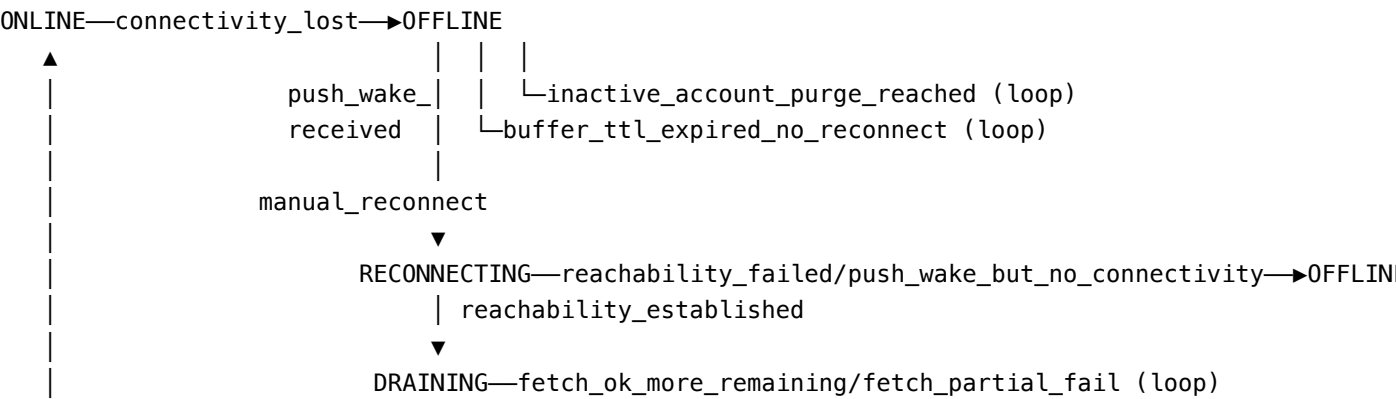
Entry/exit actions

- **Entry OFFLINE** : start buffer-TTL clock at the buffering layer (not the node itself, which is unreachable by definition).
- **Entry RECONNECTING** : invoke §20.4 from IDLE .
- **Entry DRAINING** : open authenticated session to each active buffer source; per fetched item, invoke §20.2.
- **Entry ONLINE** : for a mobile thin client this is transient — client MUST still poll/reconcile on foreground even absent a push (§14.3: "push is a latency optimization, not delivery"); for an always-on node this is the durable steady state (§14.4).

Timeouts

Timer
Offline-buffer TTL (relay-mailbox)
Inactive-account purge (relay-mailbox)
Push payload
(Peer-buffering has no fixed TTL — durability is tied to the buffering peer's own uptime, §14.5 honest limit,

Diagram



```

|                                     |  ↳connection_dropped_mid_drain→RECONNECTING
|_____fetch_ok_queue_empty|_____

```

20.8 Composition summary (non-normative)

§20.1 Outbound delivery —fast tier→ §20.4 Reachability

| ↳private tier→ mixnet (§4.4, not a state machine in this appendix)

| ↳(resolve step)→ §20.3 Name resolution & pinning ←(login sig check)— §20.6 DMTAP-Auth

§20.7 Node lifecycle —DRAINING, per item→ §20.2 Inbound validation

↳RECONNECTING→ §20.4 Reachability

§20.5 MLS group/committer: self-contained; Commits it applies may themselves be carried as MOTES validated by §20.2 (kind 0x06 group_event) before ever reaching the committer log.

20.9 Honest gaps & normative fills (meta)

Per this appendix's own totality requirement, every `[fill]` / `[gap]` inline marker above is collected here. A `[fill]` is a conservative, spec-consistent choice this appendix makes where §1–§16 are silent; a `[gap]` is a genuine operational gap this appendix states rather than papers over (matching the "honest limits" style of §6.6/§13.7/§14.2/§14.5).

1. **§20.1** — the wire mechanism by which `ack(id)` itself travels back to the sender (a MOTE? a mixnet reply mechanism? a transport-level ack?) is not specified by §2.6/§4. `[gap]`
2. **§20.1** — behavior for an `ack` arriving after the sender has already reached `EXPIRED` is not addressed by §2.6/§4.7; this appendix specifies idempotent late-correction. `[fill]`
3. **§20.1** — fail-closed-at-first-contact (§3.3) folded into the sender's ordinary `RETRY` loop rather than a distinct sender-side terminal. `[fill]`
4. **§20.2** — the relative order of the duplicate-`id` check versus `sender_sig` verification within §2.7's ordered list is not stated; this appendix checks duplicate first. `[fill]`
5. **§20.2** — §2.7 step 8 (Payload signature verification) does not explicitly restate "drop on failure" the way steps 1–4 and 7 do; this appendix applies the same fail-closed default. `[fill]`
6. **§20.2** — the disposition when a cold sender's re-applied block/allow list finds `from` blocked (post-decryption) is not explicit; treated as a drop. `[fill]`
7. **§20.3** — a plain DNS lookup failure (as opposed to KT unreachability) is not one of §3.3's three named outcomes; this appendix bounces to `UNRESOLVED`. `[fill]`
8. **§20.3** — behavior on `kt_reachable_but_superseded` (DNS lagging KT) and on `fetch_fail` / `sig_chain_invalid` at the identity-fetch step is not spelled out. `[fill]` ×3
9. **§20.3** — retry semantics after `FAIL_CLOSED_BLOCKED` (does a later attempt remember the block?) are not specified; this appendix re-runs `resolve()` fresh each time. `[fill]`
10. **§20.4** — the outcome of a failed DHT republish write is not specified by §4.2; this appendix keeps the existing connection alive and retries republish. `[fill]`
11. **§20.5** — §5.1 specifies **no recovery transition out of `HALT`** on fork detection; group recovery is genuinely left to implementations. `[gap]` (the single largest undefined surface among all seven machines)
12. **§20.5** — the numeric committer-liveness timeout that triggers rotation is not listed in §16 (§5.1 states the requirement only qualitatively). `[gap]`
13. **§20.5** — a member's response when it cannot apply a validly-ordered Commit (falls behind) is not described in §5.1; this appendix routes it through §5.3's External Commit. `[fill]`

14. **§20.6** — whether a single failed DPoP proof tears down the whole session or only that request is not stated by §13.4; this appendix rejects the request only. [fill]
15. **§20.6** — no numeric session TTL/idle-timeout parameter exists in §16 (only the login challenge's 120 s nonce window is numeric). [gap]
16. **§20.7** — peer-buffering (the non-hosted alternative to the relay-mailbox) has no fixed TTL at all — its honest-limit framing in §14.5 ties durability to the buffering peer's own uptime, which is explicitly *not* a fixed parameter. [gap]

Machine count: **7** state machines (§20.1–§20.7), plus the shared **HALT** terminal common to §20.5's two coupled sub-machines, plus the non-normative composition map (§20.8).

21. Appendix D: Error Codes & IANA Considerations

This appendix is normative. It has two parts: **Part 1** (§21.1–§21.12) is the exhaustive error/status registry — every failure condition defined anywhere in this specification, given a stable code, a required responder action, and a retryability classification, so that **no condition in DMTAP has undefined behavior**. **Part 2** (§21.13–§21.23) states the IANA considerations: the registries DMTAP requires, their initial contents, their allocation policies, and the procedure for extending the protocol without fragmenting it. The key words "MUST", "MUST NOT", "SHOULD", "SHOULD NOT", and "MAY" are used as in RFC 2119, consistent with the rest of this specification.

Every table entry in Part 1 is a contract: given the stated condition, the stated action is the *only* conformant behavior. An implementation that reaches one of these conditions and does something not listed in its Action column is non-conformant.

Part 1 — Error / Status Registry

21.1 Conventions and code format

Every code is a 16-bit value **0xSSNN**: **SS** is the subsystem byte, **NN** is the code point within that subsystem. This is a wire-level registry identifier for logging, conformance-suite assertions (§10.3), and cross-implementation diagnostics — it is **not** itself a new envelope field; each error is detected at the point in the spec cited in its "Operation(s)" column, using the checks already normatively defined there. This appendix does not introduce new validation logic; it catalogs and names the logic that §§1–17 already require, and closes every gap where those sections left an outcome implicit.

0xSSNN

SS subsystem byte (this appendix, §21.14)
NN code point (within-subsystem, §21.14)

Subsystem bytes:

SS	Subsystem
0x00	Reserved
0x01	Identity, Naming & Key Transparency (§1, §3)
0x02	Delivery & Validation — the MOTE object (§2)
0x03	Transport & Reachability — mesh, DHT, mixnet (§4)
0x04	Messaging & Group — MLS, committer (§5)
0x05	Auth — DMTAP-Auth (§13)

SS	Subsystem
0x06	Gateway — legacy SMTP bridge (§7)
0x07	Anti-Abuse & Postage (§9)
0x08	Files (§5.5, §6.7)
0x09 – 0xEF	Unassigned — reserved for future subsystems (§21.23)
0xF0 – 0xFE	Private Use (experimental/vendor subsystems)
0xFF	Reserved

Two code prefixes are used in the tables below:

- **ERR_*** — a rejected/failed condition: the operation did not succeed and the recipient took a defensive or corrective action.
- **STATUS_*** — a defined **non-error** outcome that still requires a specific, mandated responder action (e.g. deduplication). Listing these alongside errors is deliberate: a status the spec doesn't name explicitly is exactly where undefined behavior creeps in.

21.2 Responder action vocabulary (normative)

Every code in §21.3–§21.11 resolves to exactly one of the following actions. Definitions are fixed here once so the tables can cite them tersely without re-explaining each time.

Action	Definition
DROP_SILENT	Discard immediately. No <code>ack</code> , no error returned, no user-visible effect.
ACK_DEDUP	Acknowledge (<code>ack</code>) without re-processing; the object is already held (§21.11).
IGNORE_NO_ACK	Do not process (cannot validate), but do not acknowledge either — distinguish from <code>ACK_DEDUP</code> .
DEFER_REQUESTS	Accept, but route to a "requests" holding area, rate-limited, never the initiator's.
HOLD_RESYNC	Buffer locally and initiate a resynchronization/fetch (e.g. a fresh Welcome).
FALLBACK_LOWER_TIER	Treat the presented proof as absent and re-evaluate under the next-weak tier.
ROTATE_RETRY	An internal recovery action (retry via a different path, elect a new committer).
FAIL_CLOSED_BLOCK	Refuse to proceed and refuse to silently degrade (e.g. refuse to pin, refuse to cache).
HALT_ALERT	Halt the affected group/session/identity-verification process and raise a <code>STATUS_ALERT</code> .
DENY_POLICY	Reject per a configured (recipient or operator) policy decision — a deliberate choice.
REJECT_NOTIFY	Reject, and surface the failure to the initiating principal (sender's own code).
RETURN_SENDER_SMTP	Gateway-specific: respond to the legacy SMTP transaction with a 4xx (4xx).
USER_WARN	Do not block, but surface a non-blocking, explicit security warning to the user.

21.3 Identity, Naming & Key Transparency errors (0x01xx)

Code	Name	Operation(s)
0x0101	ERR_UNKNOWN_SUITE	Identity/DeviceCert/RecoveryPolicy/KeyRotation/MoveRecord selection
0x0102	ERR_SUITE_INTERSECTION_EMPTY	Suite selection for delivery (§1.3)
0x0103	ERR_IDENTITY_SIG_INVALID	Identity verification (§1.3)
0x0104	ERR_IDENTITY_CHAIN_BROKEN	Identity.prev / KeyRotation / MoveRecord chain check (§1.3)
0x0105	ERR_STALE_ROLLBACK	Identity / RecoveryPolicy version check (§1.3, §1.4, §3.5)
0x0106	ERR_KT_UNREACHABLE	KT check at first contact (§3.3)
0x0107	ERR_KT_EQUIVOCATION	KT tree-head gossip cross-check (§3.5)
0x0108	ERR_KT_PROOF_INVALID	Inclusion/consistency proof verification (§3.5)
0x0109	ERR_NAME_RESOLUTION_FAILED	resolve(name) (§3.3, §3.6)
0x010A	ERR_MOVE_RECORD_INVALID	MoveRecord verification (§1.6)

Code	Name	Operation(s)
0x010B	ERR_RECOVERY_POLICY_UNAUTHENTICATED	RecoveryPolicy publish (§1.4 rule 1)
0x010C	ERR_RECOVERY_THRESHOLD_INVALID	RecoveryPolicy publish (§1.4 rule 2)
0x010D	ERR_DEVICE_CERT_INVALID	DeviceCert validation (§1.2)

21.4 Delivery & Validation — the MOTE object (0x02xx)

This table is the authoritative disposition for **every** step of the §2.7 validation order. Codes are listed in the same order as the steps they correspond to.

Code	Name	Operation(s)
0x0201	ERR_UNKNOWN_VERSION_OR_SUITE	§2.7 step 1
0x0202	ERR_BAD_CONTENT_ADDRESS	§2.7 step 2
0x0203	ERR_BAD_SENDER_SIG	§2.7 step 3
0x0204	ERR_UNRESOLVED_TO	§2.7 step 4
0x0205	ERR_CHALLENGE_INVALID_FORGED	§2.7 step 6, §2.7a
0x0206	ERR_CHALLENGE_ABSENT_INSUFFICIENT	§2.7 step 6, §2.7a
0x0207	ERR_DECRYPT_FAILURE	§2.7 step 7
0x0208	ERR_PAYLOAD_SIG_INVALID	§2.7 step 8
0x0209	ERR_FROM_PIN_MISMATCH	§2.7 step 8, §3.4
0x020A	ERR_KIND_UNKNOWN	§2.3, §10.1
0x020B	ERR_EXPIRED_MOTE	Payload.expires (§2.4, §16.1)
0x020C	ERR_TIMESTAMP_OUT_OF_SKEW	Envelope.ts vs. receiver clock (§16.1)
0x020D	ERR_MALFORMED_OBJECT	any CBOR parse (Envelope/Payload/Attachment/Manifest)
0x020E	STATUS_DUPLICATE_ID	§2.6 (deduplication)

Content-addressed dedup as replay defense. §2.6/§20E's dedup-by- id is what makes a bare resend of a previously-processed MOTE a non-event rather than a distinct "replay" failure mode at this layer — the content address absorbs it structurally. Replay of a **nonce** (as opposed to a MOTE) is a distinct concept scoped to the Auth ceremony; see 0x0502 (§21.7).

21.5 Transport & Reachability errors (0x03xx)

Code	Name	Operation(s)
0x0301	ERR_LOCATION_SIG_INVALID	LocationRecord verification (§4.2)
0x0302	ERR_LOCATION_STALE	LocationRecord sequence check (§4.2)
0x0303	ERR_LOCATION_UNREACHABLE	DHT lookup (§4.2, §4.3)
0x0304	ERR_ECLIPSE_SUSPECTED	S/Kademlia disjoint-path lookup comparison (§4.2 CAUTION)
0x0305	ERR_RELAY_RESERVATION_UNAVAILABLE	Circuit-relay v2 reservation (§14.5, §16.6)
0x0306	ERR_RELAY_CIRCUIT_CAP_EXCEEDED	Circuit-relay v2 per-circuit cap (§16.6: 2 min / 128 KiB)
0x0307	ERR_MIX_PACKET_MALFORMED	Sphinx onion peel (§4.4)
0x0308	ERR_SENDER_RETRY_DEADLINE_EXCEEDED	Delivery state machine EXPIRED (§2.6, §4.7, §16.1: 72 h)
0x0309	ERR_OFFLINE_BUFFER_UNAVAILABLE	Peer buffering / relay-mailbox pickup (§4.3, §14.5)

21.6 Messaging & Group errors — MLS (0x04xx)

Code	Name	Operation(s)	Meaning
0x0401	ERR_MLS_CIPHERSUITE_UNSUPPORTED	KeyPackage/GroupInfo intake (§5.1)	Advertised
0x0402	ERR_KEYPACKAGE_INVALID_OR_EXHAUSTED	Async join / Add (§5.3)	KeyPacka
0x0403	ERR_COMMIT_ORDERING_VIOLATION	Committer-log application (§5.1)	A Commi
0x0404	ERR_COMMITTER_FORK_DETECTED	Committer-log integrity check (§5.1)	Two Com
0x0405	ERR_COMMITTER_UNREACHABLE	Committer liveness (§5.1)	The group
0x0406	ERR_EPOCH_MISMATCH	Envelope.epoch resolution (§2.2, §5)	The refer
0x0407	ERR_WELCOME_INVALID	Async join via Welcome (§5.3)	Welcome
0x0408	ERR_EXTERNAL_COMMIT_REJECTED	Self-join via External Commit (§5.3)	GroupInfo
0x0409	ERR_GROUP_POLICY_VIOLATION	Add/Remove/role/policy Commit (§5.8.2)	The prop
0x040A	ERR_TREKEM_STATE_DIVERGENCE	Post-Commit ratchet-tree integrity check (§5.1)	A member

21.7 Auth errors — DMTAP-Auth (0x05xx)

Code	Name	Operation(s)	Meaning
0x0501	ERR_ORIGIN_MISMATCH	Login ceremony origin check (§13.3.1)	The r
0x0502	ERR_NONCE_REPLAYED	Challenge nonce check (§13.3, §16.1)	The si
0x0503	ERR_CHALLENGE_EXPIRED	Challenge validity window (§13.3, §16.1: 120 s)	exp h
0x0504	ERR_SESSION_REVOKED	DPoP/GNAP session check (§13.4)	The se
0x0505	ERR_SESSION_EXPIRED	Session lifetime check (§13.4)	The se
0x0506	ERR_DPOP_PROOF_INVALID	Proof-of-possession check (§13.4)	The D
0x0507	ERR_UNTRUSTED_APPROVAL_CHANNEL	Remote-node login approval (§13.3.1)	The n
0x0508	ERR_CAPABILITY_DELEGATION_INVALID	Delegated-capability check (§13.5)	The U
0x0509	ERR_OIDC_ISSUER_MISMATCH	OIDC bridge / self-issued discovery (§13.6)	The ID
0x050A	STATUS_SESSIONS_INVALIDATED_ON_RECOVERY	IK recovery completion (§13.4)	All pr

21.8 Gateway errors (0x06xx)

Code	Name	Operation(s)
0x0601	ERR_GATEWAY_ATTESTATION_INVALID	Inbound attestation verification (§7.2a)
0x0602	ERR_GATEWAY_ATTESTATION_KEY_UNTRUSTED	Inbound attestation key lookup (§7.2a)
0x0603	ERR_DKIM_DELEGATION_INVALID	Outbound DKIM verification at destination MTA (§7.3)
0x0604	ERR_MX_TRANSIENT_FAILURE	Outbound SMTP transaction (§7.3)

21.8.1 Honest limit on 0x0601 / 0x0602

Because the gateway is stateless and holds no queue (§7.4), and the attestation is checked by the **recipient** after mesh delivery (potentially long after the inbound SMTP transaction has completed), there is **no live channel back to the original legacy sender** when an attestation fails post-delivery. This is a disclosed limit, not an oversight: the recipient's DROP_SILENT is the correct action (never surface an unverified legacy-origin message as though it were authenticated), but it cannot retroactively bounce the SMTP transaction that already closed. Operators SHOULD treat repeated 0x0601 / 0x0602 from the same purported gateway as a reputation signal (§7.5, §9.6).

21.9 SMTP response-code mapping (inbound, informative reference to RFC 5321 / RFC 3463)

The gateway's inbound leg (§7.2) is an ordinary SMTP transaction and MUST respond using standard SMTP reply codes plus RFC 3463 enhanced status codes, so unmodified legacy MTAs retry/bounce correctly. This table is not a new DMTAP registry — it maps DMTAP-level conditions onto the existing IANA "SMTP Enhanced Status Codes" registry.

DMTAP condition	SMTP reply	Enhanced
Recipient key/node unresolvable within the inbound transaction window	451	4.4.1
No <code>name</code> → <code>key</code> binding exists for the recipient at all	550	5.1.1
Pre- <code>DATA</code> anti-abuse reject: RBL/DNSBL, SPF/DMARC hard fail (§9, §7.2 step 2)	550	5.7.1
Pre- <code>DATA</code> greylisting / rate-limit (§7.2 step 2)	450	4.7.1
Message exceeds size policy	552	5.3.4
Recipient node reachable but declines (recipient-side <code>Policy.block</code> , §9.2)	550	5.7.1

21.10 Anti-Abuse & Postage errors (`0x07xx`)

Code	Name	Operation(s)
<code>0x0701</code>	<code>ERR_CHALLENGE_MISSING_COLD_SENDER</code>	Cold-sender gate (§9.2, §2.7 step 6)
<code>0x0702</code>	<code>ERR_CHALLENGE_BELOW_THRESHOLD</code>	Cold-sender gate (§9.2)
<code>0x0703</code>	<code>ERR_CHALLENGE_FORGED_INVALID</code>	Challenge cryptographic verification (§2.7a)
<code>0x0704</code>	<code>ERR_TOKEN_ISSUER_UNTRUSTED</code>	ARC token issuer trust check (§9.3.1)
<code>0x0705</code>	<code>ERR_TOKEN_INVALID_OR_SPENT</code>	ARC token verification/redemption (§9.3)
<code>0x0706</code>	<code>ERR_POW_INSUFFICIENT</code>	Proof-of-work check (§9.4)
<code>0x0707</code>	<code>ERR_POSTAGE_ISSUER_UNTRUSTED</code>	Postage issuer trust check (§9.5.1, parallel to §9.3.1)
<code>0x0708</code>	<code>ERR_POSTAGE_DOUBLE_SPEND</code>	Postage redemption (§9.5.1)
<code>0x0709</code>	<code>ERR_POSTAGE_ISSUER_UNREACHABLE</code>	Postage online redemption check (§9.5.1 failure mode)
<code>0x070A</code>	<code>ERR_VOUCH_INVALID_OR_RATE_LIMITED</code>	Vouch/introduction check (§9.7)
<code>0x070B</code>	<code>ERR_POLICY_DENY_BLOCKLIST</code>	Recipient <code>Policy.block</code> (§9.2)
<code>0x070C</code>	<code>ERR_RATE_LIMIT_EXCEEDED</code>	Recipient <code>Policy.rate</code> (§9.2)
<code>0x070D</code>	<code>ERR_QUOTA_POLICY_DENY</code>	Operator <code>Policy</code> capability, operations only (§12.2)
<code>0x070E</code>	<code>ERR_GATEWAYAUTHZ_DENIED</code>	Operator <code>GatewayAuthz</code> , operator-unreachable safe default (§12.2)

21.11 Files errors (`0x08xx`)

Code	Name	Operation(s)	Meaning
<code>0x0801</code>	<code>ERR_MANIFEST_HASH_MISMATCH</code>	Manifest integrity check (§5.5)	<code>Manifest.id</code> does not match
<code>0x0802</code>	<code>ERR_CHUNK_HASH_MISMATCH</code>	Per-chunk integrity check (§5.5)	A fetched chunk fails to match
<code>0x0803</code>	<code>ERR_CHUNK_UNAVAILABLE</code>	Swarm fetch (§5.5 availability tiers)	No current holder serves it
<code>0x0804</code>	<code>ERR_SIZE_TIER_MISMATCH</code>	Size-tier classification (§2.5, §6.5)	The declared/observed size tier mismatch
<code>0x0805</code>	<code>ERR_FILE_KEY_MISSING_OR_REVOKED</code>	Post-removal file access (§6.7)	A former group member's key
<code>0x0806</code>	<code>ERR_STORAGE_QUOTA_EXCEEDED</code>	Operator storage <code>Policy</code> (§12.2)	Hosted-operator storage limit
<code>0x0807</code>	<code>ERR_ATTACHMENT_KEY_INVALID</code>	Attachment/chunk decryption (§2.5)	The per-file content <code>key</code> is invalid

21.12 Traceability matrix

Every condition explicitly named in this appendix's brief maps to a code as follows, for auditability:

Named condition	Code(s)
Unknown version/suite (fail-closed)	0x0201 , 0x0101
Bad content-address	0x0202
Bad sender_sig	0x0203
Decrypt-failure	0x0207
Unresolved-to	0x0204
Cold-sender-no-challenge	0x0701 (canonical), cross-referenced at 0x0206 /§2.7 step 6
Insufficient-challenge	0x0702
KT-unreachable-fail-closed	0x0106
KT-equivocation-detected	0x0107
Chain-broken	0x0104
Stale/rollback record	0x0105 (identity), 0x0302 (location record)
Committer-fork-detected	0x0404
Committer-unreachable	0x0405
Session-revoked	0x0504
Session-expired	0x0505
Origin-mismatch (auth)	0x0501
Replay-detected	0x0502 (auth nonce — the canonical replay instance; MOTE-level repla
Expired	0x020B (MOTE expires), 0x0503 (auth challenge)
Quota/policy-deny	0x070D , 0x0806 (operator quotas), 0x070B (recipient blocklist deny)
Gateway-attestation-invalid	0x0601
Postage-double-spend	0x0708
Issuer-untrusted	0x0704 (token issuer), 0x0707 (postage issuer), 0x0509 (OIDC issuer)

Part 2 — IANA Considerations

21.13 Overview and policy vocabulary

DMTAP intends to pursue IETF standardization (§10.5). Upon submission of the governing Internet-Draft, the registries below SHOULD be requested from IANA under "DMTAP Parameters." Until then, **this appendix is the interim authoritative registry**, maintained per the extension procedure in §21.23. Allocation policies use the standard terms of RFC 8126:

- **Standards Action** — requires a new RFC on the standards track.
- **Specification Required** — requires a stable, publicly available specification and review by a Designated Expert (§21.23), but not a new RFC.
- **First Come First Served (FCFS)** — no review; allocated on request.
- **Private Use** — not registered at all; meaningful only within a closed deployment or as a vendor extension, and MUST be namespaced (e.g. x- prefix) to avoid accidental collision.

21.14 DMTAP Error/Status Code Registry

Registry name	DMTAP Error/Status Codes
Reference	§21.1–§21.11 (this document)
Allocation policy	New subsystem byte (<code>0x09</code> – <code>0xEF</code>): Standards Action. New code point within an existing subsystem byte (<code>0x00</code> – <code>0x08</code>): Standards Action.
Initial contents	The 81 codes enumerated in §21.3–§21.11.
Registry discipline	Append-only. A retired code MUST be marked Deprecated, never deleted or reassigned.

21.15 Algorithm Suites Registry (`suite` `u8`)

Registry name	DMTAP Algorithm Suites
Reference	§1.1, §16.7
Allocation policy	<code>0x01</code> – <code>0x1F</code> : Standards Action (a suite changes the network's core interoperability requirements); <code>0x20</code> – <code>0xFF</code> : Unassigned — Standards Action.
Initial contents	<code>0x01</code> (Ed25519 / X25519-HPKE / ChaCha20-Poly1305 / BLAKE3-256, v0.1)
Registration requirements	A new suite registration MUST specify: signature scheme, KEM/PKE, AEA, and the suite's core interoperability requirements.

21.16 Message Kinds Registry (`kind` `u8`)

Registry name	DMTAP Message Kinds
Reference	§2.3, §10.1
Allocation policy	<code>0x00</code> – <code>0x0A</code> : assigned (below). <code>0x0B</code> – <code>0x3F</code> : Unassigned — Standards Action.
Initial contents	<code>0x00</code> mail, <code>0x01</code> chat, <code>0x02</code> reaction, <code>0x03</code> edit, <code>0x04</code> redact, <code>0x05</code> file, <code>0x06</code> image, <code>0x07</code> video, <code>0x08</code> audio, <code>0x09</code> document, <code>0x0A</code> application.
Forward-compatibility rule	A node MUST NOT <code>ack</code> a <code>kind</code> it cannot validate, whether unassigned or assigned.

21.17 Challenge Types Registry

Registry name	DMTAP Anti-Abuse Challenge Types
Reference	§9.2 (<code>ChallengeSpec</code>), §2.2b
Allocation policy	Initial four types: assigned. Additional types: Specification Required — a new challenge type requires a new challenge type specification.
Initial contents	<code>pow(bits)</code> (§9.4); <code>token(issuer)</code> — ARC-style anonymous rate-limited credential (§9.3.1)

21.18 Name Backends Registry

Registry name	DMTAP Name Backends
Reference	§3.1, §3.6, §3.9.2
Allocation policy	Specification Required. A new backend MUST specify how it maps <code>name</code> → <code>ik</code> such that the <code>ik</code> is a valid <code>name</code> .
Initial contents	<code>dns</code> (the default backend, §3.2 — TXT/SVCB records); <code>name-chain</code> (self-sovereign, unassigned); <code>arc</code> (ARC-style anonymous rate-limited credential, §9.3.1)
Note	Onboarding Tiers A/B/C (§3.8) are deployment postures over the <code>dns</code> backend, not separate backends.

21.19 KT Log Identification Registry

Registry name	DMTAP Key-Transparency Log Types
Reference	§3.5, DNS <code>kt=</code> parameter (§3.2)
Allocation policy	Specification Required. A log identifies itself by its own signing key and a log
Initial contents	<code>0x01</code> — v0-minimal: single append-only log, signed tree heads, inclusion pro
Registration requirements	A new log type MUST specify its proof format (must remain independently a

21.20 Envelope/Payload Extension Header Keys Registry (`Headers.ext`)

Registry name	DMTAP <code>Headers.ext</code> Keys
Reference	§2.4 (<code>ext: { * tstr => any }</code>), §10.1
Allocation policy	Two namespaces. Unprefixed keys (e.g. a future <code>priority</code> ,
Initial contents	None assigned at v0. Candidate future registrations flagged b
Forward-compatibility rule (normative)	A receiver MUST ignore any <code>ext</code> key it does not recognize; a

21.21 DNS Parameter Keys Registry (`_dmtap` / `_dmtap-gw`)

Registry name	DMTAP DNS TXT/SVCB Parameters
Reference	§3.2, §7.2a
Allocation policy	Specification Required.
Initial contents	Under <code><name>._dmtap.<domain></code> TXT: <code>v=</code> (format version), <code>suite=</code> (algorithm
Registration requirements	A new TXT/SVCB key MUST specify its grammar and whether it is require

21.22 Capability Tokens Registry (negotiation, §10.2)

Registry name	DMTAP Capability Tokens
Reference	§10.2, <code>system</code> MOTEs (<code>kind = 0x0A</code>)
Allocation policy	Specification Required for tokens intended to be portable across implementa
Initial contents	Supported-suite tokens (§1.1); privacy-tier tokens (<code>private</code> , <code>fast</code> , §4.6); su
Forward-compatibility rule	A node receiving a capability token it does not recognize MUST ignore that

21.23 Extension & versioning procedure (normative)

This section is the operational answer to "how does DMTAP add something new without fragmenting."

1. **New algorithm suite.** Standards Action (§21.15). A node encountering an unregistered `suite` value in any signed/encrypted object MUST reject it (`0x0101` / `0x0201`) — there is no partial trust, no best-effort parse. Suite adoption spreads only via the multi-suite `Identity.suites` mechanism (§1.3): a recipient advertises the new suite, senders pick up the intersection, and old suites are retired only once no pinned relationship requires them.

2. **New message kind.** Specification Required within `0x40 – 0x7F` (Standards Action for a `0x0B – 0x3F` core allocation, §21.16). A node receiving an unassigned or unimplemented kind **MUST NOT** `ack` it (`0x020A`) and **MAY** silently ignore it; this is what lets a new kind roll out client-by-client with no coordinated flag day (§10.1).
3. **New Headers.ext key.** Specification Required for an unprefixed key intended to be portable; no registration for `x`-prefixed experimentation (§21.20). A receiver **MUST** ignore an unrecognized key rather than fail validation.
4. **New challenge type, name backend, DNS parameter, or capability token.** Specification Required, reviewed by a Designated Expert (below) for two things specific to DMTAP, beyond ordinary interoperability: (a) an anti-abuse extension **MUST** preserve the issuer-trust / zero-default-budget rule (§9.3.1) — it **MUST NOT** create a path for a sender to manufacture its own unlimited-cost-relief proof; (b) **no** extension of any kind **MAY** be gated behind an operator's paid seam in a way that weakens privacy, cryptography, or meta-data protection — the inviolable rule of §12.3 binds extensions exactly as it binds the base protocol.
5. **Designated Expert.** Pending formal IETF adoption (§10.5), the expert function is performed by the specification's maintainers, applying the criteria above. Upon Internet-Draft submission, this function **SHOULD** transition to an IANA-run registry with an IETF-appointed Designated Expert, per standard IANA practice for Specification-Required registries.
6. **Deprecation, never reassignment.** Once allocated (in any registry in this appendix), a code point, key, or tag **MUST NOT** be reused for a different meaning, even if the original allocation is abandoned. Mark it Deprecated and allocate a new point for new semantics. This mirrors the append-only discipline that makes key transparency (§3.5) and the committer log (§5.1) trustworthy, applied to the registries themselves.
7. **Unknown-value handling — the three forward-compatibility rules, stated together.** Precisely because these three differ, they are restated together here to prevent conflation:
 - **Unknown suite** → **rejected**, fail-closed, no processing (`0x0101` / `0x0201`).
 - **Unknown kind** → **not acknowledged**, but **MAY** be silently ignored rather than rejected (`0x020A`).
 - **Unknown ext key** → **ignored**, and **MUST NOT** affect validation of the rest of the object at all.

The differing treatment is deliberate: a suite is a security-critical trust decision (guess wrong and you silently downgrade crypto), a kind is a processing-capability question (guess wrong and you silently corrupt application semantics), and an `ext` key is inert metadata by construction (guess wrong and nothing security-relevant is at stake).

21.24 Summary

- **Error/status codes defined:** 81 (`0x0101 – 0x010D` : 13; `0x0201 – 0x020E` : 14; `0x0301 – 0x0309` : 9; `0x0401 – 0x040A` : 10; `0x0501 – 0x050A` : 10; `0x0601 – 0x0604` : 4, plus the informative SMTP mapping table of §21.9; `0x0701 – 0x070E` : 14; `0x0801 – 0x0807` : 7), spanning the 8 requested subsystems, with every code resolving to exactly one of the 13 defined responder actions (§21.2) — no undefined behavior remains.
- **IANA registries defined:** 10 — the 8 requested (Algorithm Suites, Message Kinds, Challenge Types, Name Backends, KT Log Types, Headers.ext Keys, DNS Parameters, Capability Tokens), plus the DMTAP Error/Status Code Registry itself (§21.14, needed to make

Part 1 durable against future extension) and the extension/versioning procedure (§21.23) that governs all of them.